Guida di riferimento ai contratti

Indice

1 Libreria system **3**
1.1 standard.asm 5
1.1.3 m_BASIC_Preamble 7
1.1.4 m_Swap8 9
1.1.5 m_Push_AXY 11
1.1.6 m_Pull_AXY 13
1.1.7 m_Push_XY 15
1.1.8 m_Pull_XY 17
1.1.9 m_SeV 19
1.1.10 m_CCf 21
1.1.11 m_Skip1 23
1.1.12 m_Skip2 25
1.1.13 m_Delay_5n1 27
1.2 6510_tools.asm 29
1.2.3 m_Disable_BASIC 31
1.2.4 m_Enable_BASIC 33
1.2.5 m_Disable_Kernal 35
1.2.6 m_Enable_Kernal 37
1.2.7 m_Disable_IO 39
1.2.8 m_Enable_IO 41
1.3 basic_tools.asm 43
1.3.3 m_Exit_To_BASIC 45
1.3.4 LOMEM 47
1.3.5 HIMEM 49
1.4 branch.asm 51
1.4.3 m_Bra 53
1.4.4 m_Brn 55
1.4.5 m_Bhi 57
1.4.6 m_Bhs 59
1.4.7 m_Blo 61
1.4.8 m_Bls 63
1.4.9 m_Bgt 65
1.4.10 m_Bge 67
1.4.11 m_Blt 69
1.4.12 m_Ble 71
1.4.13 m_L_Bra 73

1.4.14 m_L_Brn 75
1.4.15 m_L_Bhi 77
1.4.16 m_L_Bhs 79
1.4.17 m_L_Blo 81
1.4.18 m_L_Bls 83
1.4.19 m_L_Bgt 85
1.4.20 m_L_Bge 87
1.4.21 m_L_Blt 89
1.4.22 m_L_Ble 91
1.4.23 m_L_Bcc 93
1.4.24 m_L_Bcs 95
1.4.25 m_L_Beq 97
1.4.26 m_L_Bne 99
1.4.27 m_L_Bpl 101
1.4.28 m_L_Bmi 103
1.4.29 m_L_Bvc 105
1.4.30 m_L_Bvs 107
1.5 nmi_tools.asm 109
1.5.3 NMI_HIBASE 111
1.5.4 NMI_RESTORE 113
1.6 print_tools.asm 115
1.6.2 Note d'uso 115
1.6.3 m_At 117
1.6.4 PLOT_CHAR 119
1.6.5 COLOR_CHAR 121
1.6.6 PRINT_MSG 123
1.6.7 PRINT_NTH 125
1.6.8 PRINT_RAW 127
1.6.9 PRINT_IMM 129
1.6.10 m_Put_ECM_Char 131
1.7 raster_tools.asm 133
1.7.3 m_Init_Raster_System 135
1.7.4 m_Start_Raster_IRQ 137
1.7.5 m_Enter_Raster_IRQ 139
1.7.6 m_Next_Raster_IRQ 141
1.7.7 m_Set_Rasterline 143
1.7.8 m_Raster_Grip 145
1.7.9 m_Start_Unstable_IRQ 147

1.7.10 m_Enter_Unstable_IRQ 149
1.7.11 m_Next_Unstable_IRQ 151
1.7.12 m_Start_Stable_IRQ 153
1.7.13 m_Enter_Stable_IRQ 155
1.7.14 m_Next_Stable_IRQ 157
1.8 vic_ii_tools.asm 159
1.8.3 m_VIC_Bitmap_On 161
1.8.4 m_VIC_Bitmap_Off 163
1.8.5 m_VIC_Ext_Color_On 165
1.8.6 m_VIC_Ext_Color_Off 167
1.8.7 m_VIC_Multicolor_On 169
1.8.8 m_VIC_Multicolor_Off 171
1.8.9 VIC_PALETTE 173
1.8.10 VIC_BANK 175
1.8.11 VIC_SCREEN 177
1.8.12 VIC_CHARGEN 179
1.8.13 VIC_BITMAP 181
1.8.14 VIC_COPY_CHARGEN 183
1.8.15 COMPUTE_CELL_ADDRESS 185
1.8.16 GET_SCREEN_CELL 187
1.8.17 GET_COLOR_CELL 189
2 Libreria fp 191
2.1 fp.asm 193
2.1.2 Note d'uso 198
2.1.3 m_Store_FAC_in_Scratch 203
2.1.4 m_Store_ARG_in_Scratch 205
2.1.5 m_Load_FAC_from_Scratch 207
2.1.6 m_Load_ARG_from_Scratch 209
2.1.7 m_Add_FAC_to_ARG 211
2.1.8 m_Add_FAC_to_Mem 213
2.1.9 m_Add_FAC_to_Ptr 215
2.1.10 m_Subtract_FAC_from_ARG 217
2.1.11 m_Subtract_FAC_from_Mem 219
2.1.12 m_Subtract_FAC_from_Ptr 221
2.1.13 m_Multiply_FAC_by_ARG 223
2.1.14 m_Multiply_FAC_by_Mem 225
2.1.15 m_Multiply_FAC_by_Ptr 227
2.1.16 m_Divide_ARG_by_FAC 229

2.1.17 m_Divide_Mem_by_FAC 231
2.1.18 m_Divide_Ptr_by_FAC 233
2.1.19 m_AND_FAC_with_ARG 235
2.1.20 m_AND_FAC_with_Mem 237
2.1.21 m_AND_FAC_with_Ptr 239
2.1.22 m_OR_FAC_with_ARG 241
2.1.23 m_OR_FAC_with_Mem 243
2.1.24 m_OR_FAC_with_Ptr 245
2.1.25 m_NOT_FAC 247
2.1.26 m_NOT_ARG 249
2.1.27 m_NOT_Mem 251
2.1.28 m_NOT_Ptr 253
2.1.29 m_Compare_FAC_to_ARG 255
2.1.30 m_Compare_FAC_to_Mem 257
2.1.31 m_Compare_FAC_to_Ptr 259
2.1.32 m_Load_FAC_with 261
2.1.33 m_Load_ARG_with 263
2.1.34 m_Load_FAC_with_0 265
2.1.35 m_Load_FAC_with_0_25 267
2.1.36 m_Load_FAC_with_0_5 269
2.1.37 m_Load_FAC_with_1 271
2.1.38 m_Load_FAC_with_MINUS_1 273
2.1.39 m_Load_FAC_with_2 275
2.1.40 m_Load_FAC_with_10 277
2.1.41 m_Load_FAC_with_0_1 279
2.1.42 m_Load_FAC_with_PI4 281
2.1.43 m_Load_FAC_with_PI2 283
2.1.44 m_Load_FAC_with_PI 285
2.1.45 m_Load_FAC_with_2PI 287
2.1.46 m_Load_FAC_with_PI180 289
2.1.47 m_Load_FAC_with_PI200 291
2.1.48 m_Load_FAC_with_180PI 293
2.1.49 m_Load_FAC_with_200PI 295
2.1.50 m_Load_FAC_with_SQR2 297
2.1.51 m_Load_FAC_with_SQR3 299
2.1.52 m_Load_FAC_with_e 301
2.1.53 m_Load_FAC_with_LOG2 303
2.1.54 m_Load_FAC_with_LOG10 305

2.1.55 m_Load_FAC_with_eps 307
2.1.56 m_Load_FAC_with_MINR 309
2.1.57 m_Load_FAC_with_MAXR 311
2.1.58 m_Load_ARG_with_0 313
2.1.59 m_Load_ARG_with_0_25 315
2.1.60 m_Load_ARG_with_0_5 317
2.1.61 m_Load_ARG_with_1 319
2.1.62 m_Load_ARG_with_MINUS_1 321
2.1.63 m_Load_ARG_with_2 323
2.1.64 m_Load_ARG_with_10 325
2.1.65 m_Load_ARG_with_0_1 327
2.1.66 m_Load_ARG_with_PI4 329
2.1.67 m_Load_ARG_with_PI2 331
2.1.68 m_Load_ARG_with_PI 333
2.1.69 m_Load_ARG_with_2PI 335
2.1.70 m_Load_ARG_with_PI180 337
2.1.71 m_Load_ARG_with_PI200 339
2.1.72 m_Load_ARG_with_180PI 341
2.1.73 m_Load_ARG_with_200PI 343
2.1.74 m_Load_ARG_with_SQR2 345
2.1.75 m_Load_ARG_with_SQR3 347
2.1.76 m_Load_ARG_with_e 349
2.1.77 m_Load_ARG_with_LOG2 351
2.1.78 m_Load_ARG_with_LOG10 353
2.1.79 m_Load_ARG_with_eps 355
2.1.80 m_Load_ARG_with_MINR 357
2.1.81 m_Load_ARG_with_MAXR 359
2.1.82 FP_ERROR_HANDLER 361
2.1.83 m_Load_FAC_with_UINT8 363
2.1.84 m_Load_FAC_with_UINT8_Mem 365
2.1.85 m_Load_FAC_with_UINT8_Ptr 367
2.1.86 m_Load_ARG_with_UINT8 369
2.1.87 m_Load_ARG_with_UINT8_Mem 371
2.1.88 m_Load_ARG_with_UINT8_Ptr 373
2.1.89 m_Load_FAC_with_INT8 375
2.1.90 m_Load_FAC_with_INT8_Mem 377
2.1.91 m_Load_FAC_with_INT8_Ptr 379
2.1.92 m_Load_ARG_with_INT8 381

2.1.93 m_Load_ARG_with_INT8_Mem 383
2.1.94 m_Load_ARG_with_INT8_Ptr 385
2.1.95 m_Load_FAC_with_UINT16 387
2.1.96 m_Load_FAC_with_UINT16_Mem 389
2.1.97 m_Load_FAC_with_UINT16_Ptr 391
2.1.98 m_Load_ARG_with_UINT16 393
2.1.99 m_Load_ARG_with_UINT16_Mem 395
2.1.100 m_Load_ARG_with_UINT16_Ptr 397
2.1.101 m_Load_FAC_with_INT16 399
2.1.102 m_Load_FAC_with_INT16_Mem 401
2.1.103 m_Load_FAC_with_INT16_Ptr 403
2.1.104 m_Load_ARG_with_INT16 405
2.1.105 m_Load_ARG_with_INT16_Mem 407
2.1.106 m_Load_ARG_with_INT16_Ptr 409
2.1.107 m_Load_FAC_with_UINT24 411
2.1.108 m_Load_FAC_with_UINT24_Mem 413
2.1.109 m_Load_FAC_with_UINT24_Ptr 415
2.1.110 m_Load_ARG_with_UINT24 417
2.1.111 m_Load_ARG_with_UINT24_Mem 419
2.1.112 m_Load_ARG_with_UINT24_Ptr 421
2.1.113 m_Load_FAC_with_INT24 423
2.1.114 m_Load_FAC_with_INT24_Mem 425
2.1.115 m_Load_FAC_with_INT24_Ptr 427
2.1.116 m_Load_ARG_with_INT24 429
2.1.117 m_Load_ARG_with_INT24_Mem 431
2.1.118 m_Load_ARG_with_INT24_Ptr 433
2.1.119 m_Load_FAC_with_UINT32 435
2.1.120 m_Load_FAC_with_UINT32_Mem 437
2.1.121 m_Load_FAC_with_UINT32_Ptr 439
2.1.122 m_Load_ARG_with_UINT32 441
2.1.123 m_Load_ARG_with_UINT32_Mem 443
2.1.124 m_Load_ARG_with_UINT32_Ptr 445
2.1.125 m_Load_FAC_with_INT32 447
2.1.126 m_Load_FAC_with_INT32_Mem 449
2.1.127 m_Load_FAC_with_INT32_Ptr 451
2.1.128 m_Load_ARG_with_INT32 453
2.1.129 m_Load_ARG_with_INT32_Mem 455
2.1.130 m_Load_ARG_with_INT32_Ptr 457

2.1.131 m_Convert_FAC_into_INT32 459
2.1.132 m_Convert_ARG_into_INT32 461
2.1.133 m_FAC_to_String 463
2.1.134 m_ARG_to_String 465
2.1.135 m_String_to_FAC 467
2.1.136 m_String_to_ARG 469
2.1.137 m_Print_FAC 471
2.1.138 m_Print_ARG 473
2.1.139 m_Load_FAC_from_Mem 475
2.1.140 m_Load_FAC_from_Ptr 477
2.1.141 m_Load_ARG_from_Mem 479
2.1.142 m_Load_ARG_from_Ptr 481
2.1.143 m_Store_FAC_to_Mem 483
2.1.144 m_Store_FAC_to_Ptr 485
2.1.145 m_Store_ARG_to_Mem 487
2.1.146 m_Store_ARG_to_Ptr 489
2.1.147 m_Transfer_FAC_to_ARG 491
2.1.148 m_Transfer_ARG_to_FAC 493
2.1.149 m_Swap_FAC_and_ARG 495
2.1.150 m_Poly_Mem 497
2.1.151 m_Poly_Ptr 499
2.1.152 m_Odd_Poly_Mem 501
2.1.153 m_Odd_Poly_Ptr 503
2.1.154 m_Power_ARG_to_FAC 505
2.1.155 m_Power_Mem_to_FAC 507
2.1.156 m_Power_Ptr_to_FAC 509
2.1.157 m_SQR_FAC 511
2.1.158 m_SQR_ARG 513
2.1.159 m_SQR_Mem 515
2.1.160 m_SQR_Ptr 517
2.1.161 m_Load_FAC_with_RND 519
2.1.162 m_Randomize 521
2.1.163 m_LOG_FAC 523
2.1.164 m_LOG_ARG 525
2.1.165 m_LOG_Mem 527
2.1.166 m_LOG_Ptr 529
2.1.167 m_EXP_FAC 531
2.1.168 m_EXP_ARG 533

2.1.169 m_EXP_Mem 535
2.1.170 m_EXP_Ptr 537
2.1.171 m_SIN_FAC 539
2.1.172 m_SIN_ARG 541
2.1.173 m_SIN_Mem 543
2.1.174 m_SIN_Ptr 545
2.1.175 m_COS_FAC 547
2.1.176 m_COS_ARG 549
2.1.177 m_COS_Mem 551
2.1.178 m_COS_Ptr 553
2.1.179 m_TAN_FAC 555
2.1.180 m_TAN_ARG 557
2.1.181 m_TAN_Mem 559
2.1.182 m_TAN_Ptr 561
2.1.183 m_ATN_FAC 563
2.1.184 m_ATN_ARG 565
2.1.185 m_ATN_Mem 567
2.1.186 m_ATN_Ptr 569
2.1.187 m_Negate_FAC 571
2.1.188 m_Negate_ARG 573
2.1.189 m_Negate_Mem 575
2.1.190 m_Negate_Ptr 577
2.1.191 m_ABS_FAC 579
2.1.192 m_ABS_ARG 581
2.1.193 m_ABS_Mem 583
2.1.194 m_ABS_Ptr 585
2.1.195 m_Sign_FAC 587
2.1.196 m_Sign_ARG 589
2.1.197 m_Sign_Mem 591
2.1.198 m_Sign_Ptr 593
2.1.199 m_SGN_FAC 595
2.1.200 m_SGN_ARG 597
2.1.201 m_SGN_Mem 599
2.1.202 m_SGN_Ptr 601
2.1.203 m_INT_FAC 603
2.1.204 m_INT_ARG 605
2.1.205 m_INT_Mem 607
2.1.206 m_INT_Ptr 609

2.1.207 m_Multiply_FAC_by_2 611
2.1.208 m_Multiply_ARG_by_2 613
2.1.209 m_Multiply_Mem_by_2 615
2.1.210 m_Multiply_Ptr_by_2 617
2.1.211 m_Divide_FAC_by_2 619
2.1.212 m_Divide_ARG_by_2 621
2.1.213 m_Divide_Mem_by_2 623
2.1.214 m_Divide_Ptr_by_2 625
3 Libreria io 627
3.1 chars.asm 629
3.1.2 Note d'uso 629
3.1.3 m_Scan_Exceptions 635
3.1.4 m_Apply_Table 637
3.1.5 ASC_SCR 639
3.1.6 ASC_SCR_C 641
3.1.7 ASC_PET 643
3.1.8 ASC_PET_C 645
3.1.9 ASC_ECM 647
3.1.10 ASC_ECM_C 649
3.1.11 ECM_SCR 651
3.1.12 ECM_SCR_C 653
3.1.13 ECM_PET 655
3.1.14 ECM_PET_C 657
3.1.15 ECM_ASC 659
3.1.16 ECM_ASC_C 661
3.1.17 PET_SCR 663
3.1.18 PET_SCR_C 665
3.1.19 PET_ASC 667
3.1.20 PET_ASC_C 669
3.1.21 PET_ECM 671
3.1.22 PET_ECM_C 673
3.1.23 SCR_PET 675
3.1.24 SCR_PET_C 677
3.1.25 SCR_ASC 679
3.1.26 SCR_ASC_C 681
3.1.27 SCR_ECM 683
3.1.28 SCR_ECM_C 685
3.2 disk_tools.asm 687

3.2.3 DISK_STATUS 689
3.2.4 DECODE_DISK_STATUS 691
3.2.5 DISK_DIRECTORY 693
3.2.6 DISK_COMMAND 695
3.3 keyboard_tools.asm 697
3.3.2 Note d'uso 697
3.3.3 KEYSCAN 701
4 Libreria math 703
4.1 math8.asm 705
4.1.3 m_Neg 707
4.1.4 m_Not 709
4.1.5 m_AslS 711
4.1.6 m_AsrS 713
4.1.7 m_Rl8 715
4.1.8 m_Rr8 717
4.2 math16.asm 719
4.2.3 m_Asl16 721
4.2.4 m_Add16 723
4.2.5 m_Sub16 725
4.2.6 m_Neg16 727
4.2.7 m_Not16 729
4.2.8 m_And16 731
4.2.9 m_Ora16 733
4.2.10 m_Eor16 735
4.2.11 m_AslS16 737
4.2.12 m_Lsr16 739
4.2.13 m_AsrS16 741
4.2.14 m_Rol16 743
4.2.15 m_Rl16 745
4.2.16 m_Ror16 747
4.2.17 m_Rr16 749
4.3 math24.asm 751
4.3.3 m_Asl24 753
4.3.4 m_Add24 755
4.3.5 m_Sub24 757
4.3.6 m_Neg24 759
4.3.7 m_Not24 761
4.3.8 m_And24 763

4.3.9 m_Ora24 765
4.3.10 m_Eor24 767
4.3.11 m_AslS24 769
4.3.12 m_Lsr24 771
4.3.13 m_AsrS24 773
4.3.14 m_Rol24 775
4.3.15 m_Rl24 777
4.3.16 m_Ror24 779
4.3.17 m_Rr24 781
4.4 math32.asm 783
4.4.3 m_Asl32 785
4.4.4 m_Add32 787
4.4.5 m_Sub32 789
4.4.6 m_Neg32 791
4.4.7 m_Not32 793
4.4.8 m_And32 795
4.4.9 m_Ora32 797
4.4.10 m_Eor32 799
4.4.11 m_AslS32 801
4.4.12 m_Lsr32 803
4.4.13 m_AsrS32 805
4.4.14 m_Rol32 807
4.4.15 m_Rl32 809
4.4.16 m_Ror32 811
4.4.17 m_Rr32 813
4.5 math_div.asm 815
4.5.2 Note d'uso 816
4.5.3 m_Div_Negate 819
4.5.4 m_Div_Roll 821
4.5.5 m_Div_Core_Byte 823
4.5.6 m_Div_Core_Multi 825
4.5.7 m_Div_Core_8x8 827
4.5.8 m_Div_Core_16x8 829
4.5.9 m_Div_Core_24x8 831
4.5.10 m_Div_Core_32x8 833
4.5.11 m_Div_Core_16x16 835
4.5.12 m_Div_Core_24x16 837
4.5.13 m_Div_Core_32x16 839

4.5.14 m_Div_Core_24x24 841
4.5.15 m_Div_Core_32x24 843
4.5.16 m_Div_Core_32x32 845
4.5.17 DIV_8X8 847
4.5.18 SDIV_8X8 849
4.5.19 DIV_16X8 851
4.5.20 SDIV_16X8 853
4.5.21 DIV_24X8 855
4.5.22 SDIV_24X8 857
4.5.23 DIV_32X8 859
4.5.24 SDIV_32X8 861
4.5.25 DIV_16X16 863
4.5.26 SDIV_16X16 865
4.5.27 DIV_24X16 867
4.5.28 SDIV_24X16 869
4.5.29 DIV_32X16 871
4.5.30 SDIV_32X16 873
4.5.31 DIV_24X24 875
4.5.32 SDIV_24X24 877
4.5.33 DIV_32X24 879
4.5.34 SDIV_32X24 881
4.5.35 DIV_32X32 883
4.5.36 SDIV_32X32 885
4.6 math_mult.asm 887
4.6.2 Note d'uso 888
4.6.3 m_Mul_Negate 891
4.6.4 m_Mul_Stage 893
4.6.5 m_Mul_Core 895
4.6.6 m_Mul_Core_8x8 897
4.6.7 m_Mul_Core_16x8 899
4.6.8 m_Mul_Core_24x8 901
4.6.9 m_Mul_Core_32x8 903
4.6.10 m_Mul_Core_16x16 905
4.6.11 m_Mul_Core_24x16 907
4.6.12 m_Mul_Core_32x16 909
4.6.13 m_Mul_Core_24x24 911
4.6.14 m_Mul_Core_32x24 913
4.6.15 m_Mul_Core_32x32 915

4.6.16 MUL_8X8 917
4.6.17 SMUL_8X8 919
4.6.18 MUL_16X8 921
4.6.19 SMUL_16X8 923
4.6.20 MUL_24X8 925
4.6.21 SMUL_24X8 927
4.6.22 MUL_32X8 929
4.6.23 SMUL_32X8 931
4.6.24 MUL_16X16 933
4.6.25 SMUL_16X16 935
4.6.26 MUL_24X16 937
4.6.27 SMUL_24X16 939
4.6.28 MUL_32X16 941
4.6.29 SMUL_32X16 943
4.6.30 MUL_24X24 945
4.6.31 SMUL_24X24 947
4.6.32 MUL_32X24 949
4.6.33 SMUL_32X24 951
4.6.34 MUL_32X32 953
4.6.35 SMUL_32X32 955
5 Libreria mem 957
5.1 mem16.asm 959
5.1.3 m_Mov16 961
5.1.4 m_Swap16 963
5.1.5 m_CmpU16 965
5.1.6 m_CmpS16 967
5.1.7 m_Inc16 969
5.1.8 m_Dec16 971
5.2 mem24.asm 973
5.2.3 m_Mov24 975
5.2.4 m_Swap24 977
5.2.5 m_CmpU24 979
5.2.6 m_CmpS24 981
5.2.7 m_Inc24 983
5.2.8 m_Dec24 985
5.3 mem32.asm 987
5.3.3 m_Mov32 989
5.3.4 m_Swap32 991

5.3.5 m_CmpU32 993
5.3.6 m_CmpS32 995
5.3.7 m_Inc32 997
5.3.8 m_Dec32 999
6 Libreria c64 1001
6.1 errors.asm 1003
6.2 kernal.asm 1005
6.3 keyboard.asm 1007
6.4 symbols.asm 1011
7 Libreria chip 1021
7.1 cia.asm 1023
7.2 rec.asm 1025
7.3 sid.asm 1027
7.4 vdc.asm 1029
7.5 vic.asm 1031
7.6 vic_ii.asm 1033

Libreria

system

1 Libreria system

Modulo	Descrizione
`standard.asm`	Macro utili per la programmazione.
`6510_tools.asm`	Macro per la configurazione dei banchi di memoria del 6510.
`basic_tools.asm`	Utilità per la manipolazione dell'area di memoria dedicata al BASIC ed ai programmi ivi contenuti.
`branch.asm`	Macro per salti condizionati relativi e con flag di oVerflow.
`nmi_tools.asm`	Utilità per la gestione delle interruzioni non mascherabili.
`print_tools.asm`	Utilità per la stampa di caratteri, messaggi e tabelle di messaggi con supporto per il modo a Colore di Sfondo Esteso.
`raster_tools.asm`	Macro di utilità per la definizione di interruzioni di raster concatenate, sia stabili che instabili. Non adatte per codici iper-ottimizzati. Supportano tutte le versioni del chip VIC-II, sia PAL che NTSC.
`vic_ii_tools.asm`	Utilità per la gestione dei modi grafici offerti dal chip VIC-II.

1.1 standard.asm

1.1.1 Descrizione

Macro utili per la programmazione.

Utilità	Descrizione
`m_BASIC_Preamble`	Inserisce una riga BASIC con SYS
`m_Swap8`	Scambia due valori a 8 bit
`m_Push_AXY`	Salva .A, .X e .Y sullo stack
`m_Pull_AXY`	Recupera .A, .X e .Y dallo stack
`m_Push_XY`	Salva .X e .Y sullo stack
`m_Pull_XY`	Recupera .X e .Y dallo stack
`m_SeV`	Imposta il flag oVerflow
`m_CCf`	Complementa il flag Carry
`m_Skip1`	Maschera l'istruzione seguente (1 byte)
`m_Skip2`	Maschera l'istruzione seguente (2 byte)
`m_Delay_5n1`	Consuma 5n+1 cicli a vuoto

1.1.2 Note d'uso

Nessuna nota in questo modulo.

1.1.3 m_BASIC_Preamble

```
; ================================================================================
; MACRO: m_BASIC_Preamble
; --------------------------------------------------------------------------------
; Descrizione:  Inserisce una riga di BASIC con un comando SYS ed,
;               opzionalmente, un commento REM.
;
; Input:
;   - .line_num_ (2 byte, Macro) : Numero di riga
;   - .label_ (2 byte, Macro)    : Etichetta del punto di ingresso
;   - .comment_ (stringa, Macro) : (opzionale) Commento aggiunto come REM
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.1.4 m_Swap8

```
; ================================================================================
; MACRO: m_Swap8
; --------------------------------------------------------------------------------
; Descrizione:  Scambia i valori contenuti in due locazioni di memoria a 8 bit,
;               usando .X e .Y.
;
; Input:
;   - .addr1_ (1 o 2 byte, Macro) : Indirizzo del primo valore
;   - .addr2_ (1 o 2 byte, Macro) : Indirizzo del secondo valore
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .X, .Y, .P (Negative, Zero)
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.1.5 m_Push_AXY

```
; ================================================================================
; MACRO: m_Push_AXY
; --------------------------------------------------------------------------------
; Descrizione:  Copia i registri .A, .X e .Y sullo stack.
;
; Input:
;   - Nessuno
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Decrementato di 3 byte
;
; Registri Modificati:  .A, .P (Negative, Zero)
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.1.6 m_Pull_AXY

```
; ================================================================================
; MACRO: m_Pull_AXY
; --------------------------------------------------------------------------------
; Descrizione:  Recupera i registri .A, .X e .Y dallo stack.
;
; Input:
;   - Nessuno
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Incrementato di 3 byte
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, Zero)
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.1.7 m_Push_XY

```
; ================================================================================
; MACRO: m_Push_XY
; --------------------------------------------------------------------------------
; Descrizione:  Copia i registri .X e .Y sullo stack, riservando prima una
;               posizione per .A, che m_Pull_XY userà per ripristinarlo.
;
; Input:
;   - Nessuno
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Decrementato di 3 byte
;
; Registri Modificati:  .A, .P (Negative, Zero)
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.1.8 m_Pull_XY

```
; ================================================================================
; MACRO: m_Pull_XY
; --------------------------------------------------------------------------------
; Descrizione:  Recupera i registri .X e .Y dallo stack, lasciando invariato .A.
;
; Input:
;   - Nessuno
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Incrementato di 3 byte
;
; Registri Modificati:  .X, .Y, .P (Negative, Zero)
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

1.1.9 m_SeV

```
; ================================================================================
; MACRO: m_SeV
; --------------------------------------------------------------------------------
; Descrizione:  Imposta il flag oVerflow senza alterare gli altri flag: spreca
;               cicli ed istruzioni.
;
; Input:
;   - Nessuno
;
; Output:
;   - Flag oVerflow (1 bit, .P) : sempre 1
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (oVerflow)
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.1.10 m_CCf

```
; ================================================================================
; MACRO: m_CCf
; --------------------------------------------------------------------------------
; Descrizione:  Complementa il flag Carry senza alterare gli altri flag: spreca
;               cicli ed istruzioni.
;
; Input:
;   - Nessuno
;
; Output:
;   - Flag Carry (1 bit, .P) : 0 = prima era 1; 1 = prima era 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Carry)
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.1.11 m_Skip1

```
; ================================================================================
; MACRO: m_Skip1
; --------------------------------------------------------------------------------
; Descrizione:  Inserisce l'istruzione illegale NOP #imm (2 cicli) in modo da
;               mascherare l'istruzione da un byte che segue, senza alterare i
;               flag. Il byte emesso è l'opcode di NOP: il primo byte
;               dell'istruzione seguente ne diventa l'operando e non
;               viene eseguito.
;
; Input:
;   - Nessuno
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.1.12 m_Skip2

```
; ================================================================================
; MACRO: m_Skip2
; --------------------------------------------------------------------------------
; Descrizione:  Inserisce l'istruzione illegale NOP abs (4 cicli) in modo da
;               mascherare l'istruzione da due byte che segue, senza alterare i
;               flag. Il byte emesso è l'opcode di NOP: i due byte
;               dell'istruzione seguente ne diventano l'operando e non
;               vengono eseguiti.
;
; Input:
;   - Nessuno
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.1.13 m_Delay_5n1

```
; ================================================================================
; MACRO: m_Delay_5n1
; --------------------------------------------------------------------------------
; Descrizione:  Consuma un numero di cicli pari a 5n+1 con un ciclo di attesa su
;               .X o .Y.
;
; Input:
;   - .reg_ (stringa, Macro) : Registro contatore: ".X" oppure ".Y"
;   - .n_ (1 byte, Macro)    : Parametro n di 5n+1 (n = 0 equivale a n = 256)
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .reg_ (.X o .Y), .P (Negative, Zero)
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.2 6510_tools.asm

1.2.1 Descrizione

Macro per la configurazione dei banchi di memoria del 6510.

Utilità	Descrizione
`m_Disable_BASIC`	Disabilita la ROM del BASIC
`m_Enable_BASIC`	Riabilita la ROM del BASIC
`m_Disable_Kernal`	Disabilita la ROM del Kernal
`m_Enable_Kernal`	Riabilita la ROM del Kernal
`m_Disable_IO`	Disabilita l'I/O (rende visibile CHARGEN)
`m_Enable_IO`	Riabilita l'I/O

1.2.2 Note d'uso

Nessuna nota in questo modulo.

1.2.3 m_Disable_BASIC

```
; ================================================================================
; MACRO: m_Disable_BASIC
; --------------------------------------------------------------------------------
; Descrizione:  Disabilita la ROM del BASIC, rendendo visibile il banco di
;               memoria LORAM ($A000-$BFFF).
;
; Input:
;   - Nessuno
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), R6510
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

1.2.4 m_Enable_BASIC

```
; ================================================================================
; MACRO: m_Enable_BASIC
; --------------------------------------------------------------------------------
; Descrizione:  Riabilita la ROM del BASIC, rendendo invisibile il banco di
;               memoria LORAM ($A000-$BFFF).
;
; Input:
;   - Nessuno
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), R6510
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

1.2.5 m_Disable_Kernal

```
; ============================================================================
; MACRO: m_Disable_Kernal
; ----------------------------------------------------------------------------
; Descrizione:  Disabilita la ROM del Kernal, rendendo visibile il banco di
;               memoria HIRAM ($E000-$FFFF). ATTENZIONE: gli interrupt
;               saranno disabilitati.
;
; Input:
;   - Nessuno
;
; Output:
;   - Flag Interrupt (1 bit, .P) : sempre 1
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Interrupt, Zero), R6510
; Dipendenze Esterne:   <c64/symbols.asm>
; ============================================================================
```

1.2.6 m_Enable_Kernal

```
; ============================================================================
; MACRO: m_Enable_Kernal
; ----------------------------------------------------------------------------
; Descrizione:  Riabilita la ROM del Kernal, rendendo invisibile il banco di
;               memoria HIRAM ($E000-$FFFF). ATTENZIONE: gli interrupt
;               saranno riabilitati.
;
; Input:
;   - Nessuno
;
; Output:
;   - Flag Interrupt (1 bit, .P) : sempre 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Interrupt, Zero), R6510
; Dipendenze Esterne:   <c64/symbols.asm>
; ============================================================================
```

1.2.7 m_Disable_IO

```
; ================================================================================
; MACRO: m_Disable_IO
; --------------------------------------------------------------------------------
; Descrizione:  Disabilita le interruzioni del timer di sistema e gli indirizzi
;               di I/O, rendendo visibile il banco di memoria
;               CHARGEN ($D000-$DFFF).
;
; Input:
;   - Nessuno
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), R6510, CIAICR
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <chip/cia.asm>
; ================================================================================
```

1.2.8 m_Enable_IO

```
; ================================================================================
; MACRO: m_Enable_IO
; --------------------------------------------------------------------------------
; Descrizione:  Riabilita gli indirizzi di I/O, rendendo invisibile il banco di
;               memoria CHARGEN ($D000-$DFFF), e le interruzioni del timer
;               di sistema.
;
; Input:
;   - Nessuno
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), R6510, CIAICR
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <chip/cia.asm>
; ================================================================================
```

1.3 basic_tools.asm

1.3.1 Descrizione

Utilità per la manipolazione dell'area di memoria dedicata al BASIC ed ai programmi ivi contenuti.

Utilità	Descrizione
`m_Exit_To_BASIC`	Esce al prompt del BASIC
`LOMEM`	Sposta l'inizio della memoria BASIC
`HIMEM`	Sposta la fine della memoria BASIC

1.3.2 Note d'uso

Nessuna nota in questo modulo.

1.3.3 m_Exit_To_BASIC

```
; ================================================================================
; MACRO: m_Exit_To_BASIC
; --------------------------------------------------------------------------------
; Descrizione:  Termina un programma in linguaggio macchina e torna al prompt
;               del BASIC, passando per il gestore degli errori.
;
; Input:
;   - Nessuno
;
; Output:
;   - .PC (2 byte, CPU) : Indirizzo contenuto nel vettore IERROR
;
; Variazione Stack (.S):
;   - Azzerato
;
; Registri Modificati:  .X, .P (Negative, Zero), .S
; Dipendenze Esterne:   <c64/errors.asm>
;                       <c64/symbols.asm>
; ================================================================================
```

1.3.4 LOMEM

```
; ================================================================================
; SUBROUTINE: LOMEM
; --------------------------------------------------------------------------------
; Descrizione:  Sposta l'inizio della memoria dedicata al BASIC aggiornando la
;               variabile di sistema TXTTAB. La subroutine richiede un puntatore
;               precedente all'indirizzo che conterrà il primo carattere del
;               testo del programma. Al termine i puntatori del BASIC
;               conterranno il valore del puntatore incrementato di 1.
;               L'esecuzione ritorna al BASIC.
;
; Input:
;   - .X (1 byte, CPU)       : Indirizzo del puntatore alla locazione iniziale
;                              dell'area BASIC meno 1
;
; Output:
;   - puntatore (2 byte, ZP) : Incrementato di 1
;   - TXTTAB (2 byte, ZP)    : Nuovo inizio dell'area BASIC
;   - .PC (2 byte, CPU)      : Ritorno al BASIC
;
; Variazione Stack (.S):
;   - Azzerato
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), .S,
;                       TXTTAB, VARTAB, ARYTAB, STREND, FRETOP, TXTPTR
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

1.3.5 HIMEM

```
; ================================================================================
; SUBROUTINE: HIMEM
; --------------------------------------------------------------------------------
; Descrizione:  Sposta la fine della memoria dedicata al BASIC aggiornando la
;               variabile di sistema MEMSIZ. L'esecuzione ritorna al BASIC.
;
; Input:
;   - .X (1 byte, CPU)    : Indirizzo del puntatore alla locazione finale
;                           dell'area BASIC
;
; Output:
;   - MEMSIZ (2 byte, ZP) : Nuova fine dell'area BASIC
;   - .PC (2 byte, CPU)   : Ritorno al BASIC
;
; Variazione Stack (.S):
;   - Azzerato
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), .S,
;                       MEMSIZ, FRETOP, ARYTAB, STREND
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

1.4 branch.asm

1.4.1 Descrizione

Macro per salti condizionati relativi e con flag di oVerflow.

Utilità	Descrizione
`m_Bra`	Salta sempre
`m_Brn`	Salta mai
`m_Bhi`	Salta se più alto (.A > M senza segno)
`m_Bhs`	Salta se più alto o uguale (.A >= M senza segno)
`m_Blo`	Salta se più basso (.A < M senza segno)
`m_Bls`	Salta se più basso o uguale (.A <= M senza segno)
`m_Bgt`	Salta se maggiore (.A > M con segno)
`m_Bge`	Salta se maggiore o uguale (.A >= M con segno)
`m_Blt`	Salta se minore (.A < M con segno)
`m_Ble`	Salta se minore o uguale (.A <= M con segno)
`m_L_Bra`	Salta lungo sempre
`m_L_Brn`	Salta lungo mai
`m_L_Bhi`	Salta lungo se più alto (senza segno)
`m_L_Bhs`	Salta lungo se più alto o uguale (senza segno)
`m_L_Blo`	Salta lungo se più basso (senza segno)
`m_L_Bls`	Salta lungo se più basso o uguale (senza segno)
`m_L_Bgt`	Salta lungo se maggiore (con segno)
`m_L_Bge`	Salta lungo se maggiore o uguale (con segno)
`m_L_Blt`	Salta lungo se minore (con segno)
`m_L_Ble`	Salta lungo se minore o uguale (con segno)
`m_L_Bcc`	Salta lungo se Carry = 0
`m_L_Bcs`	Salta lungo se Carry = 1
`m_L_Beq`	Salta lungo se Zero = 1 (Equal)
`m_L_Bne`	Salta lungo se Zero = 0 (Not Equal)
`m_L_Bpl`	Salta lungo se Negative = 0 (Plus)
`m_L_Bmi`	Salta lungo se Negative = 1 (Minus)
`m_L_Bvc`	Salta lungo se oVerflow = 0 (Clear)
`m_L_Bvs`	Salta lungo se oVerflow = 1 (Set)

1.4.2 Note d'uso

Nessuna nota in questo modulo.

1.4.3 m_Bra

```
; ================================================================================
; MACRO: m_Bra
; --------------------------------------------------------------------------------
; Descrizione:  Salta sempre.
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo di destinazione
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.4.4 m_Brn

```
; ================================================================================
; MACRO: m_Brn
; --------------------------------------------------------------------------------
; Descrizione:  Salta mai.
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo (ignorato)
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.4.5 m_Bhi

```
; ================================================================================
; MACRO: m_Bhi
; --------------------------------------------------------------------------------
; Descrizione:  Salta se più alto (.A > M senza segno).
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo di destinazione
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.4.6 m_Bhs

```
; ================================================================================
; MACRO: m_Bhs
; --------------------------------------------------------------------------------
; Descrizione:  Salta se più alto o uguale (.A >= M senza segno).
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo di destinazione
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.4.7 m_Blo

```
; ================================================================================
; MACRO: m_Blo
; --------------------------------------------------------------------------------
; Descrizione:  Salta se più basso (.A < M senza segno).
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo di destinazione
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.4.8 m_Bls

```
; ================================================================================
; MACRO: m_Bls
; --------------------------------------------------------------------------------
; Descrizione:  Salta se più basso o uguale (.A <= M senza segno).
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo di destinazione
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.4.9 m_Bgt

```
; ================================================================================
; MACRO: m_Bgt
; --------------------------------------------------------------------------------
; Descrizione:  Salta se maggiore (.A > M con segno).
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo di destinazione
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.4.10 m_Bge

```
; ================================================================================
; MACRO: m_Bge
; --------------------------------------------------------------------------------
; Descrizione:  Salta se maggiore o uguale (.A >= M con segno).
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo di destinazione
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.4.11 m_Blt

```
; ================================================================================
; MACRO: m_Blt
; --------------------------------------------------------------------------------
; Descrizione:  Salta se minore (.A < M con segno).
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo di destinazione
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.4.12 m_Ble

```
; ================================================================================
; MACRO: m_Ble
; --------------------------------------------------------------------------------
; Descrizione:  Salta se minore o uguale (.A <= M con segno).
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo di destinazione
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.4.13 m_L_Bra

```
; ================================================================================
; MACRO: m_L_Bra
; --------------------------------------------------------------------------------
; Descrizione:  Salta lungo sempre.
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo di destinazione
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.4.14 m_L_Brn

```
; ================================================================================
; MACRO: m_L_Brn
; --------------------------------------------------------------------------------
; Descrizione:  Salta lungo mai.
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo (ignorato)
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.4.15 m_L_Bhi

```
; ================================================================================
; MACRO: m_L_Bhi
; --------------------------------------------------------------------------------
; Descrizione:  Salta lungo se più alto (senza segno).
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo di destinazione
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.4.16 m_L_Bhs

```
; ================================================================================
; MACRO: m_L_Bhs
; --------------------------------------------------------------------------------
; Descrizione:  Salta lungo se più alto o uguale (senza segno).
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo di destinazione
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.4.17 m_L_Blo

```
; ================================================================================
; MACRO: m_L_Blo
; --------------------------------------------------------------------------------
; Descrizione:  Salta lungo se più basso (senza segno).
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo di destinazione
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.4.18 m_L_Bls

```
; ================================================================================
; MACRO: m_L_Bls
; --------------------------------------------------------------------------------
; Descrizione:  Salta lungo se più basso o uguale (senza segno).
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo di destinazione
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.4.19 m_L_Bgt

```
; ================================================================================
; MACRO: m_L_Bgt
; --------------------------------------------------------------------------------
; Descrizione:  Salta lungo se maggiore (con segno).
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo di destinazione
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.4.20 m_L_Bge

```
; ================================================================================
; MACRO: m_L_Bge
; --------------------------------------------------------------------------------
; Descrizione:  Salta lungo se maggiore o uguale (con segno).
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo di destinazione
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.4.21 m_L_Blt

```
; ================================================================================
; MACRO: m_L_Blt
; --------------------------------------------------------------------------------
; Descrizione:  Salta lungo se minore (con segno).
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo di destinazione
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.4.22 m_L_Ble

```
; ===========================================================================
; MACRO: m_L_Ble
; ---------------------------------------------------------------------------
; Descrizione:  Salta lungo se minore o uguale (con segno).
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo di destinazione
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ===========================================================================
```

1.4.23 m_L_Bcc

```
; ================================================================================
; MACRO: m_L_Bcc
; --------------------------------------------------------------------------------
; Descrizione:  Salta lungo se Carry = 0.
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo di destinazione
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.4.24 m_L_Bcs

```
; ===========================================================================
; MACRO: m_L_Bcs
; ---------------------------------------------------------------------------
; Descrizione:  Salta lungo se Carry = 1.
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo di destinazione
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ===========================================================================
```

1.4.25 m_L_Beq

```
; ================================================================================
; MACRO: m_L_Beq
; --------------------------------------------------------------------------------
; Descrizione:  Salta lungo se Zero = 1 (Equal).
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo di destinazione
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.4.26 m_L_Bne

```
; ================================================================================
; MACRO: m_L_Bne
; --------------------------------------------------------------------------------
; Descrizione:  Salta lungo se Zero = 0 (Not Equal).
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo di destinazione
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.4.27 m_L_Bpl

```
; ===========================================================================
; MACRO: m_L_Bpl
; ---------------------------------------------------------------------------
; Descrizione:  Salta lungo se Negative = 0 (Plus).
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo di destinazione
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ===========================================================================
```

1.4.28 m_L_Bmi

```
; ===========================================================================
; MACRO: m_L_Bmi
; ---------------------------------------------------------------------------
; Descrizione:  Salta lungo se Negative = 1 (Minus).
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo di destinazione
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ===========================================================================
```

1.4.29 m_L_Bvc

```
; ================================================================================
; MACRO: m_L_Bvc
; --------------------------------------------------------------------------------
; Descrizione:  Salta lungo se oVerflow = 0 (Clear).
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo di destinazione
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.4.30 m_L_Bvs

```
; ================================================================================
; MACRO: m_L_Bvs
; --------------------------------------------------------------------------------
; Descrizione:  Salta lungo se oVerflow = 1 (Set).
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo di destinazione
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  Nessuno
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.5 nmi_tools.asm

1.5.1 Descrizione

Utilità per la gestione delle interruzioni non mascherabili.

Utilità	Descrizione
`NMI_HIBASE`	Resetta HIBASE dopo RUN/STOP + RESTORE
`NMI_RESTORE`	Ripristina il vettore NMI originale

1.5.2 Note d'uso

Nessuna nota in questo modulo.

1.5.3 NMI_HIBASE

```
; ============================================================================
; SUBROUTINE: NMI_HIBASE
; ----------------------------------------------------------------------------
; Descrizione:  Dopo un RUN/STOP + RESTORE il VIC-II viene riconfigurato con i
;               valori di default, ma la variabile di sistema HIBASE non viene
;               reinizializzata. Questa subroutine risolve il problema
;               inserendosi nella subroutine di interrupt NMI.
;
; Input:
;   - Nessuno
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), NMIVEC
; Dipendenze Esterne:   <c64/symbols.asm>
; ============================================================================
```

1.5.4 NMI_RESTORE

```
; ============================================================================
; SUBROUTINE: NMI_RESTORE
; ----------------------------------------------------------------------------
; Descrizione:  Ripristina il vettore NMI originale, annullando le modifiche
;               apportate da NMI_HIBASE.
;
; Input:
;   - Nessuno
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), NMIVEC
; Dipendenze Esterne:   <c64/symbols.asm>
; ============================================================================
```

1.6 print_tools.asm

1.6.1 Descrizione

Utilità per la stampa di caratteri, messaggi e tabelle di messaggi con supporto per il modo a Colore di Sfondo Esteso.

Utilità	Descrizione
m_At	Posiziona il cursore sullo schermo
PLOT_CHAR	Memorizza un carattere nella memoria schermo
COLOR_CHAR	Cambia il colore di un carattere
PRINT_MSG	Stampa una tabella di messaggi
PRINT_NTH	Stampa un messaggio estratto da una tabella
PRINT_RAW	Stampa un messaggio puntato da .A / .Y
PRINT_IMM	Stampa il messaggio che segue la chiamata
m_Put_ECM_Char	Invia un carattere gestendo il modo ECM

1.6.2 Note d'uso

```
; NOTA #01
; --------
; Le subroutine PRINT_MSG e PRINT_NTH lavorano su elenchi di messaggi
; organizzati in tavole. Ciascun messaggio è organizzato nel seguente formato:
;
;   !text <riga>, <colonna>, "messaggio", $00
;
; <riga> e <colonna> sono due byte che indicano la posizione sullo schermo
; alla quale andrà stampato il messaggio.
;
; Se <riga> == $FF e <colonna> != $FF il messaggio è stampato alla posizione
; corrente del cursore.
;
; Se <riga> == $FF e <colonna> == $FF la tavola è terminata.
```

```
; NOTA #02
; --------
; Le subroutine PRINT_* possono essere personalizzate con una propria subroutine
; di stampa modificando il valore del punto di configurazione __PUT_CHAR.
;
; Esempio 1: Driver di stampa in modo ECM.
;
; __PUT_CHAR:
;   +m_Put_ECM_Char               ; Stampa il carattere in modo ECM
;
; .Exit___PUT_CHAR:
;   rts
;
; Esempio 2: Driver di stampa in modo ECM a doppia spaziatura.
;
; __PUT_CHAR:
;   +m_Put_ECM_Char               ; Stampa il carattere in modo ECM
;   lda #$1D                      ; Codice PETSCII per spostare il cursore
;                                 ; a destra di una posizione
;   jsr CHROUT
;
; .Exit___PUT_CHAR:
;   rts
;
; Esempio 3: Driver di stampa in verticale.
;
; __PUT_CHAR:
;   jsr CHROUT                    ; Stampa il carattere
;
;   lda #$11                      ; Codice PETSCII per spostare il cursore
;                                 ; in basso di una posizione
;   jsr CHROUT
;   lda #$9D                      ; Codice PETSCII per spostare il cursore
;                                 ; a sinistra di una posizione
;   jsr CHROUT
;
; .Exit___PUT_CHAR:
;   rts
```

```
; NOTA #03
; --------
; Esempio di messaggio che utilizza i codici ECM_BG[0-3]:
;
;   !text $FF, $00, ECM_BG0, "Hello, ", ECM_BG1, "world!", ECM_BG0, $00
;
; L'ultimo ECM_BG0 serve a ripristinare il colore di sfondo iniziale
; altrimenti eventuali messaggi seguenti sarebbero stampati con il colore di
; sfondo ECM_BG1.
```

1.6.3 m_At

```
; ================================================================================
; MACRO: m_At
; --------------------------------------------------------------------------------
; Descrizione:  Posiziona il cursore sulla riga e sulla colonna specificata
;               dello schermo corrente, usando la chiamata PLOT del Kernal.
;
; Input:
;   - .row_ (1 byte, Macro) : Riga (0-24)
;   - .col_ (1 byte, Macro) : Colonna (0-39)
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/kernal.asm>
; ================================================================================
```

1.6.4 PLOT_CHAR

```
; ================================================================================
; SUBROUTINE: PLOT_CHAR
; --------------------------------------------------------------------------------
; Descrizione:  Scrive un carattere sullo schermo alla riga e colonna
;               specificate e supporta in modo trasparente il modo Colore di
;               Sfondo Esteso. Se questo modo è attivo il codice Schermo del
;               carattere è automaticamente riportato all'intervallo 0-63 e
;               viene scritto utilizzando il registro colore che è codificato
;               in __BG_COL:
;                 __BG_COL = 0: Usa il colore memorizzato in BGCOL0
;                 __BG_COL = 1: Usa il colore memorizzato in BGCOL1
;                 __BG_COL = 2: Usa il colore memorizzato in BGCOL2
;                 __BG_COL = 3: Usa il colore memorizzato in BGCOL3
;               Richiede __USE_COMPUTE_CELL_ADDRESS = 1.
;
; Input:
;   - .A (1 byte, CPU)      : Codice Schermo del carattere
;   - .X (1 byte, CPU)      : Colonna della cella (0-39)
;   - .Y (1 byte, CPU)      : Riga della cella (0-24)
;   - __BG_COL (1 byte, ZP) : Selettore del registro di sfondo (0-3),
;                             solo in ECM
;
; Output:
;   - Cella (1 byte, RAM)   : Cella dello schermo alla riga e colonna indicate
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .P (Negative, oVerflow, Zero, Carry), INDEX2
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <chip/vic_ii.asm>
;                       <system/standard.asm>
;                       <system/vic_ii_tools.asm>
; ================================================================================
```

1.6.5 COLOR_CHAR

```
; ================================================================================
; SUBROUTINE: COLOR_CHAR
; --------------------------------------------------------------------------------
; Descrizione:  Imposta il colore di primo piano del carattere localizzato alla
;               colonna ed alla riga specificati.
;               Richiede __USE_COMPUTE_CELL_ADDRESS = 1.
;
; Input:
;   - .A (1 byte, CPU)    : Codice del colore (0-15)
;   - .X (1 byte, CPU)    : Colonna della cella (0-39)
;   - .Y (1 byte, CPU)    : Riga della cella (0-24)
;
; Output:
;   - Cella (1 byte, RAM) : Cella della memoria colore indicata da .X e .Y
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .P (Negative, oVerflow, Zero, Carry), INDEX2
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <system/standard.asm>
;                       <system/vic_ii_tools.asm>
; ================================================================================
```

1.6.6 PRINT_MSG

```
; ================================================================================
; SUBROUTINE: PRINT_MSG
; --------------------------------------------------------------------------------
; Descrizione:  PRINT_MSG percorre attraverso chiamate successive una tavola di
;               messaggi [NOTA #01]. L'indirizzo iniziale della tavola è dato da
;               un puntatore in ZP e l'indirizzo del puntatore è fornito in .A:
;               alla prima chiamata viene stampato il primo messaggio: in caso
;               di successo il flag di Carry è uguale a zero ed il puntatore in
;               ZP incrementato all'inizio del messaggio successivo. Il
;               raggiungimento della fine della tavola è segnalato da Carry = 1:
;               successive chiamate non avranno effetto finché il puntatore non
;               viene reinizializzato all'indirizzo iniziale della tavola.
;
; Input:
;   - .A (1 byte, CPU)       : Indirizzo in ZP del puntatore alla tavola
;   - Puntatore (2 byte, ZP) : Indirizzo del prossimo messaggio da stampare
;
; Output:
;   - Puntatore (2 byte, ZP) : Messaggio successivo o marcatore di fine tavola
;   - Flag Carry (1 bit, .P) : 0 = messaggio stampato; 1 = tavola terminata
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       puntatore in ZP
; Dipendenze Esterne:   <c64/kernal.asm>
;                       <system/standard.asm>
; ================================================================================
```

1.6.7 PRINT_NTH

```
; ===========================================================================
; SUBROUTINE: PRINT_NTH
; ---------------------------------------------------------------------------
; Descrizione:  PRINT_NTH stampa un messaggio singolo estraendolo da una tavola
;               di messaggi [NOTA #01]. L'indirizzo iniziale della tavola è dato
;               da un puntatore in ZP e l'indirizzo del puntatore è fornito in
;               .A mentre il numero d'ordine del messaggio è fornito in .Y (il
;               primo messaggio ha indice 0).
;               Richiede __USE_PRINT_MSG = 1.
;
; Input:
;   - .A (1 byte, CPU)       : Indirizzo in ZP del puntatore alla tavola
;   - .Y (1 byte, CPU)       : Indice del messaggio (0 = il primo)
;   - Puntatore (2 byte, ZP) : Indirizzo iniziale della tavola
;
; Output:
;   - Flag Carry (1 bit, .P) : 0 = messaggio stampato; 1 = indice oltre la fine
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <system/standard.asm>
; ===========================================================================
```

1.6.8 PRINT_RAW

```
; ================================================================================
; SUBROUTINE: PRINT_RAW
; --------------------------------------------------------------------------------
; Descrizione:  La subroutine PRINT_RAW stampa una stringa di testo il cui
;               indirizzo è puntato da .YA: è pensata come sostituto della
;               subroutine di sistema STROUT in quanto non altera il contenuto
;               di FAC o ARG. Per mantenere la compatibilità con STROUT questa è
;               l'unica subroutine di stampa che non supporta i due byte di
;               posizionamento sullo schermo che precedono il messaggio.
;
; Input:
;   - .A (1 byte, CPU)  : Byte basso dell'indirizzo della stringa
;   - .Y (1 byte, CPU)  : Byte alto dell'indirizzo della stringa
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry), INDEX2
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <mem/mem16.asm>
; ================================================================================
```

1.6.9 PRINT_IMM

```
; ================================================================================
; SUBROUTINE: PRINT_IMM
; --------------------------------------------------------------------------------
; Descrizione:  La subroutine PRINT_IMM stampa un messaggio (formattato come un
;               elemento di una tavola di messaggi [NOTA #01]) inserito nel
;               codice sorgente immediatamente dopo la sua chiamata:
;                 jsr PRINT_IMM
;                 !text $FF, $00, "Hello, world!", $0D, $00
;
; Input:
;   - Messaggio (n byte, RAM) : Dati dopo la chiamata (vedi [NOTA #01])
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), INDEX2
; Dipendenze Esterne:   <c64/kernal.asm>
;                       <c64/symbols.asm>
;                       <mem/mem16.asm>
;                       <system/branch.asm>
; ================================================================================
```

1.6.10 m_Put_ECM_Char

```
; ============================================================================
; MACRO: m_Put_ECM_Char
; ----------------------------------------------------------------------------
; Descrizione:  Questa macro non è pensata per essere usata a sé stante, ma come
;               mattone da costruzione per un proprio driver di stampa da
;               utilizzare nel punto di configurazione __PUT_CHAR [NOTA #02]:
;               infatti la macro non termina con un RTS ma con un salto alla
;               fine del suo codice in modo da poter essere concatenata con
;               altre istruzioni [NOTA #02]. Nello specifico questa macro
;               implementa la logica per stampare a schermo un carattere PETSCII
;               in modo ECM con il colore di sfondo desiderato. La macro si
;               aspetta in .A il codice PETSCII del carattere da stampare e
;               restituisce .A, .X e .Y invariati.
;
;               I codici di controllo (0-31 e 128-159) sono trattati così: 0,
;               18 ({RVS} ON) e 146 ({RVS} OFF) sono ignorati, perché il modo
;               reverse è deciso da __BG_COL; quattro codici PETSCII non
;               utilizzati dal sistema operativo (1-4) sono reinterpretati come
;               codici di controllo che, come gli altri, hanno effetto fino al
;               loro cambiamento: essi sono ECM_BG[0-3] e selezionano il
;               registro del VIC-II contenente il colore di sfondo da
;               utilizzare, senza stampare nulla [NOTA #03]; tutti gli altri
;               sono inviati a CHROUT senza modifiche.
;
;               Gli altri codici PETSCII sono stampabili. Il codice è
;               normalizzato, cioè portato nell'intervallo 32-95 in cui il modo
;               ECM ha i suoi glifi, sostituendo i 3 bit più significativi
;               secondo la tabella .NORM_TABLE (con la macro m_Apply_Table di
;               <io/chars.asm>). Non è una transcodifica nel senso di
;               <io/chars.asm>, che conserva il glifo: in modo ECM il VIC-II
;               ignora i due bit più significativi del codice Schermo, quindi
;               la normalizzazione dà il codice il cui glifo è quello che
;               l'hardware mostrerebbe comunque. Per esempio il pigreco (255) è
;               stampato come la freccia in su: il suo codice 255 diventa 94, e
;               questo caso, che la tabella non copre, è trattato a parte con
;               la macro m_Scan_Exceptions. Poi il bit #0 di __BG_COL è copiato
;               nel bit #7 del codice normalizzato e il bit #1 di __BG_COL
;               decide il modo reverse (RVS = 0 normale, 1 inverso); il codice
;               è infine inviato a CHROUT. __BG_COL non cambia.
;
; Input:
;   - .A (1 byte, CPU)      : Codice PETSCII da stampare
;   - __BG_COL (1 byte, ZP) : Selettore del colore di sfondo (0-3)
;
; Output:
;   - __BG_COL (1 byte, ZP) : Nuovo selettore (se il codice è ECM_BG[0-3])
;   - RVS (1 byte, ZP)      : Modo reverse (bit #1 di __BG_COL), se il codice è
;                             stampabile
;
; Variazione Stack (.S):
;   - Stabile
;
```

segue...

1.6.10 m_Put_ECM_Char (continua)

```
; Registri Modificati:  .P (Negative, oVerflow, Zero, Carry), __BG_COL, RVS
; Dipendenze Esterne:   <c64/kernal.asm>
;                       <c64/symbols.asm>
;                       <io/chars.asm>
;                       <system/branch.asm>
; ================================================================================
```

1.7 raster_tools.asm

1.7.1 Descrizione

Macro di utilità per la definizione di interruzioni di raster concatenate, sia stabili che instabili. Non adatte per codici iper-ottimizzati. Supportano tutte le versioni del chip VIC-II, sia PAL che NTSC.

Utilità	Descrizione
`m_Init_Raster_System`	Prepara il sistema ai Raster IRQ
`m_Start_Raster_IRQ`	Avvia una catena di Raster IRQ
`m_Enter_Raster_IRQ`	Entra in un gestore Raster IRQ
`m_Next_Raster_IRQ`	Prepara il prossimo Raster IRQ, esce
`m_Set_Rasterline`	Imposta la linea di raster di confronto
`m_Raster_Grip`	Compensa i cicli extra del VIC-II NTSC
`m_Start_Unstable_IRQ`	Avvia una catena di IRQ instabili
`m_Enter_Unstable_IRQ`	Entra in un gestore IRQ instabile
`m_Next_Unstable_IRQ`	Prepara il prossimo IRQ instabile, esce
`m_Start_Stable_IRQ`	Avvia una catena di IRQ stabili
`m_Enter_Stable_IRQ`	Entra in un gestore IRQ stabile
`m_Next_Stable_IRQ`	Prepara il prossimo IRQ stabile, esce

1.7.2 Note d'uso

Nessuna nota in questo modulo.

1.7.3 m_Init_Raster_System

```
; ============================================================================
; MACRO: m_Init_Raster_System
; ----------------------------------------------------------------------------
; Descrizione:  Disabilita tutte le sorgenti di interruzioni legate ai chip CIA
;               ed attiva UNICAMENTE le interruzioni dovute al raster. Dato che
;               tutte le altre interruzioni sono disabilitate, la gestione della
;               tastiera e dell'I/O seriale sono demandate alla responsabilità
;               del programmatore.
;               La macro determina anche il tipo di chip VIC-II installato sul
;               computer sul quale sta girando: un codice identificativo viene
;               conservato nella locazione rappresentata dal punto di
;               configurazione __VIC_MODEL. Il significato del codice è
;               il seguente:
;
;                 $00 = PAL       63 cicli per linea
;                 $40 = Old NTSC  64 cicli per linea
;                 $C0 = New NTSC  65 cicli per linea
;
;               La macro m_Raster_Grip fa uso di questa informazione per
;               iniettare nel codice 1 o 2 cicli extra di temporizzazione in
;               base alla versione rilevata del chip VIC-II.
;
; Input:
;   - Nessuno
;
; Output:
;   - __VIC_MODEL (1 byte, ZP)   : Modello di VIC-II (vedi Descrizione)
;   - Flag Interrupt (1 bit, .P) : sempre 1
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y,
;                       .P (Negative, oVerflow, Interrupt, Zero, Carry),
;                       __VIC_MODEL, CIAICR, CI2ICR, VICIRQ, IRQMSK,
;                       R6510 (se __DISABLE_KERNAL = 1)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <chip/cia.asm>
;                       <chip/vic_ii.asm>
;                       <system/6510_tools.asm>
;                       <system/standard.asm>
; ============================================================================
```

1.7.4 m_Start_Raster_IRQ

```
; ============================================================================
; MACRO: m_Start_Raster_IRQ
; ----------------------------------------------------------------------------
; Descrizione:  Questa macro inizia una catena di interruzioni di raster: il
;               primo interrupt della catena può essere sia di tipo stabile che
;               instabile. Questa macro si limita solo a richiamare le macro
;               m_Start_Stable_IRQ e m_Start_Unstable_IRQ che si occupano
;               dell'effettiva realizzazione dell'interruzione.
;
; Input:
;   - .stable_ (stringa, Macro)     : "STABLE" = stabile, altro = instabile
;   - .rasterline_ (2 byte, Macro)  : Linea di raster desiderata
;   - .irq_handler_ (2 byte, Macro) : Indirizzo del gestore dell'interruzione
;
; Output:
;   - Flag Interrupt (1 bit, .P)    : sempre 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Interrupt, Zero), RASTER, SCROLY,
;                       IRBVEC (se __DISABLE_KERNAL = 1),
;                       CINV (se __DISABLE_KERNAL = 0)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <chip/vic_ii.asm>
; ============================================================================
```

1.7.5 m_Enter_Raster_IRQ

```
; ================================================================================
; MACRO: m_Enter_Raster_IRQ
; --------------------------------------------------------------------------------
; Descrizione:  La macro va obbligatoriamente invocata come prima istruzione del
;               gestore delle interruzioni di raster. Questa macro si limita
;               solo a richiamare le macro m_Enter_Stable_IRQ e
;               m_Enter_Unstable_IRQ che si occupano
;               dell'effettiva realizzazione.
;
; Input:
;   - .stable_ (stringa, Macro)    : "STABLE" = stabile, altro = instabile
;   - .rasterline_ (2 byte, Macro) : Linea di raster (solo se stabile)
;
; Output:
;   - Flag Interrupt (1 bit, .P)   : 0 = stabile; 1 = instabile
;
; Variazione Stack (.S):
;   - Decrementato di 3 byte
;
; Registri Modificati:  .A, .X, .Y,
;                       .P (Negative, oVerflow, Interrupt, Zero, Carry), RASTER,
;                       SCROLY, VICIRQ, IRBVEC (se __DISABLE_KERNAL = 1),
;                       CINV (se __DISABLE_KERNAL = 0)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <chip/vic_ii.asm>
;                       <system/standard.asm>
; ================================================================================
```

1.7.6 m_Next_Raster_IRQ

```
; ============================================================================
; MACRO: m_Next_Raster_IRQ
; ----------------------------------------------------------------------------
; Descrizione:  La macro va obbligatoriamente invocata come ultima istruzione
;               del gestore delle interruzioni di raster. Questa macro si limita
;               solo a richiamare le macro m_Next_Stable_IRQ e
;               m_Next_Unstable_IRQ che si occupano dell'effettiva
;               realizzazione. Affinché la catena di interruzioni di raster sia
;               ben formata è responsabilità del programmatore che l'ultimo
;               anello della catena richiami il primo gestore, quello definito
;               dalla macro m_Start_Raster_IRQ. Il parametro .fast_exit_ è
;               ignorato se __DISABLE_KERNAL = 1.
;
; Input:
;   - .stable_ (stringa, Macro)     : "STABLE" = stabile, altro = instabile
;   - .rasterline_ (2 byte, Macro)  : Linea di raster del prossimo IRQ
;   - .irq_handler_ (2 byte, Macro) : Indirizzo del gestore del prossimo IRQ
;   - .fast_exit_ (stringa, Macro)  : "FAST" = IRQMIN, altro = IRQHND
;
; Output:
;   - .PC (2 byte, CPU)             : Uscita con rti o salto a IRQMIN/IRQHND
;
; Variazione Stack (.S):
;   - Incrementato di 6 byte
;
; Registri Modificati:  .A, .P (Negative, Interrupt, Zero), RASTER, SCROLY,
;                       IRBVEC (se __DISABLE_KERNAL = 1),
;                       CINV (se __DISABLE_KERNAL = 0)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <chip/vic_ii.asm>
; ============================================================================
```

1.7.7 m_Set_Rasterline

```
; ============================================================================
; MACRO: m_Set_Rasterline
; ----------------------------------------------------------------------------
; Descrizione:  Macro di supporto per il settaggio del registro RASTER del chip
;               VIC-II. La macro viene eseguita in tempo costante (16 cicli) e
;               gestisce correttamente il settaggio del bit #9.
;
; Input:
;   - .rasterline_ (2 byte, Macro) : Linea di raster (9 bit, costante)
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), RASTER, SCROLY
; Dipendenze Esterne:   <chip/vic_ii.asm>
; ============================================================================
```

1.7.8 m_Raster_Grip

```
; ================================================================================
; MACRO: m_Raster_Grip
; --------------------------------------------------------------------------------
; Descrizione:  Macro per la sincronizzazione del raster in maniera indipendente
;               dall'hardware: siccome le 3 versioni esistenti del chip VIC-II
;               (PAL, Old NTSC e New NTSC) producono una linea di raster in 63,
;               64 e 65 cicli rispettivamente, è necessario che il codice di
;               sincronizzazione consumi 1 o 2 cicli di ritardo in più qualora
;               l'hardware lo imponga. È pertanto obbligatorio che questa macro
;               venga eseguita solo dopo l'esecuzione della macro
;               m_Init_Raster_System che si occuperà dell'identificazione del
;               chip VIC-II.
;               La macro consuma 7 cicli su hardware PAL, 8 cicli su hardware
;               Old NTSC e 9 cicli su hardware New NTSC. Nei calcoli di
;               temporizzazione il programmatore dovrà sempre assumere che la
;               linea di raster venga disegnata in 63 cicli (PAL) e che
;               m_Raster_Grip duri 7 cicli: l'implementazione interna garantirà
;               la coerenza dei calcoli in caso di hardware NTSC.
;
; Input:
;   - __VIC_MODEL (1 byte, ZP) : Modello VIC-II (vedi m_Init_Raster_System)
;
; Output:
;   - Nessuno
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .P (Negative, oVerflow, Zero)
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

1.7.9 m_Start_Unstable_IRQ

```
; ================================================================================
; MACRO: m_Start_Unstable_IRQ
; --------------------------------------------------------------------------------
; Descrizione:  La macro inizializza il vettore IRQ, la cui locazione differisce
;               in dipendenza dallo stato di attivazione del Kernal, con
;               l'indirizzo di partenza del gestore delle interruzioni poi
;               imposta la linea di raster alla quale far scattare
;               l'interruzione e ripristina il flag di Interrupt.
;
; Input:
;   - .rasterline_ (2 byte, Macro)  : Linea di raster desiderata
;   - .irq_handler_ (2 byte, Macro) : Indirizzo del gestore dell'interruzione
;
; Output:
;   - Flag Interrupt (1 bit, .P)    : sempre 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Interrupt, Zero), RASTER, SCROLY,
;                       IRBVEC (se __DISABLE_KERNAL = 1),
;                       CINV (se __DISABLE_KERNAL = 0)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <chip/vic_ii.asm>
; ================================================================================
```

1.7.10 m_Enter_Unstable_IRQ

```
; ============================================================================
; MACRO: m_Enter_Unstable_IRQ
; ----------------------------------------------------------------------------
; Descrizione:  La macro implementa le buone pratiche da seguire all'esecuzione
;               di un gestore delle interruzioni: se il Kernal è disabilitato
;               provvede a salvare i registri .A, .X e .Y sullo stack (se il
;               Kernal è abilitato questa operazione viene svolta in automatico)
;               e poi altera il registro VICIRQ per segnalare l'avvenuto
;               riconoscimento dell'interruzione.
;
; Input:
;   - Nessuno
;
; Output:
;   - Flag Interrupt (1 bit, .P) : sempre 1
;
; Variazione Stack (.S):
;   - Decrementato di 3 byte
;
; Registri Modificati:  .A (se __DISABLE_KERNAL = 1),
;                       .P (Negative, Zero, Carry), VICIRQ
; Dipendenze Esterne:   <chip/vic_ii.asm>
; ============================================================================
```

1.7.11 m_Next_Unstable_IRQ

```
; ================================================================================
; MACRO: m_Next_Unstable_IRQ
; --------------------------------------------------------------------------------
; Descrizione:  La macro implementa le buone pratiche da seguire all'uscita di
;               un gestore delle interruzioni: il vettore IRQ (tenendo conto
;               dello stato di attivazione del Kernal) viene impostato
;               all'indirizzo del prossimo anello della catena, idem per la
;               linea di raster alla quale dovrà essere eseguito e poi si
;               provvede a ripristinare lo stack: se il Kernal è abilitato si
;               salta a IRQMIN (nel caso si desideri un'uscita "fast") o a
;               IRQHND (se si desiderà un'uscita "slow" che scansioni la
;               tastiera, aggiorni i timer, eccetera). Se invece il Kernal è
;               disabilitato non vi sono alternative: l'uscita sarà sempre
;               "fast" ed il ripristino sarà realizzato mediante apposito
;               codice in-line.
;
; Input:
;   - .rasterline_ (2 byte, Macro)  : Linea di raster del prossimo IRQ
;   - .irq_handler_ (2 byte, Macro) : Indirizzo del gestore del prossimo IRQ
;   - .fast_exit_ (stringa, Macro)  : "FAST" = IRQMIN, altro = IRQHND
;
; Output:
;   - .PC (2 byte, CPU)             : Uscita con rti o salto a IRQMIN/IRQHND
;
; Variazione Stack (.S):
;   - Incrementato di 6 byte
;
; Registri Modificati:  .A, .P (Negative, Interrupt, Zero), RASTER, SCROLY,
;                       IRBVEC (se __DISABLE_KERNAL = 1),
;                       CINV (se __DISABLE_KERNAL = 0)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <chip/vic_ii.asm>
; ================================================================================
```

1.7.12 m_Start_Stable_IRQ

```
; ================================================================================
; MACRO: m_Start_Stable_IRQ
; --------------------------------------------------------------------------------
; Descrizione:  La macro inizializza il vettore IRQ, la cui locazione differisce
;               in dipendenza dallo stato di attivazione del Kernal, con
;               l'indirizzo di partenza del gestore delle interruzioni poi
;               imposta la linea di raster (vedi sotto) alla quale far scattare
;               l'interruzione e ripristina il flag di Interrupt.
;               Per ottenere la stabilizzazione dell'interruzione di raster si
;               utilizza la tecnica del doppio interrupt: questa macro serve ad
;               impostare con congruo anticipo (3 linee di raster) il primo
;               dei due.
;
;               IMPORTANTE: la macro presuppone che (.rasterline_-3) non
;               corrisponda ad una "badline". In caso contrario i calcoli di
;               temporizzazione su cui si basa il processo di sincronizzazione
;               non saranno validi, con effetti imprevedibili.
;
; Input:
;   - .rasterline_ (2 byte, Macro)  : Linea di raster desiderata
;   - .irq_handler_ (2 byte, Macro) : Indirizzo del gestore dell'interruzione
;
; Output:
;   - Flag Interrupt (1 bit, .P)    : sempre 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Interrupt, Zero), RASTER, SCROLY,
;                       IRBVEC (se __DISABLE_KERNAL = 1),
;                       CINV (se __DISABLE_KERNAL = 0)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <chip/vic_ii.asm>
; ================================================================================
```

1.7.13 m_Enter_Stable_IRQ

```
; ================================================================================
; MACRO: m_Enter_Stable_IRQ
; --------------------------------------------------------------------------------
; Descrizione:  La macro implementa la seconda parte della tecnica di
;               stabilizzazione del raster mediante doppio interrupt.
;               Per prima cosa si effettuano le operazioni di riconoscimento del
;               primo interrupt secondo le buone pratiche descritte in
;               m_Enter_Unstable_IRQ impostando il secondo interrupt in modo che
;               scatti a .rasterline_-1 (il codice di sincronizzazione parte
;               dall'etichetta @Sync); le operazioni differiscono in base allo
;               stato di attivazione del Kernal.
;               Il tempo rimanente viene impiegato per salvare il valore di .S,
;               dopodiché si consumano cicli in attesa che scatti il secondo
;               interrupt mentre si eseguono istruzioni di durata nota.
;               Il secondo interrupt procede a bruciare cicli (il numero dipende
;               dallo stato di attivazione del Kernal) per eseguire
;               un'operazione di comparazione esattamente al ciclo 59 o 60.
;               Un'operazione di salto fittizia brucerà 2 oppure 3 cicli a
;               seconda del risultato della comparazione garantendo che
;               l'istruzione successiva venga eseguita con certezza al ciclo 3
;               della linea di raster .rasterline_, stabilizzando così
;               il raster!
;
;               IMPORTANTE: la macro presuppone che (.rasterline_-1) non
;               corrisponda ad una "badline". In caso contrario i calcoli di
;               temporizzazione su cui si basa il processo di sincronizzazione
;               non saranno validi, con effetti imprevedibili.
;
;               ATTENZIONE: codice automodificante indicato con (!!!)
;
; Input:
;   - .rasterline_ (2 byte, Macro) : Linea di raster desiderata
;   - __VIC_MODEL (1 byte, ZP)     : Modello VIC-II (vedi m_Init_Raster_System)
;
; Output:
;   - Flag Interrupt (1 bit, .P)   : sempre 0
;
; Variazione Stack (.S):
;   - Decrementato di 3 byte
;
; Registri Modificati:  .A, .X, .Y,
;                       .P (Negative, oVerflow, Interrupt, Zero, Carry), RASTER,
;                       SCROLY, VICIRQ, IRBVEC (se __DISABLE_KERNAL = 1),
;                       CINV (se __DISABLE_KERNAL = 0)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <chip/vic_ii.asm>
;                       <system/standard.asm>
; ================================================================================
```

1.7.14 m_Next_Stable_IRQ

```
; ================================================================================
; MACRO: m_Next_Stable_IRQ
; --------------------------------------------------------------------------------
; Descrizione:  La macro implementa le buone pratiche da seguire all'uscita di
;               un gestore delle interruzioni: il vettore IRQ (tenendo conto
;               dello stato di attivazione del Kernal) viene impostato
;               all'indirizzo del prossimo anello della catena, idem per la
;               linea di raster alla quale dovrà essere eseguito e poi si
;               provvede a ripristinare lo stack: se il Kernal è abilitato si
;               salta a IRQMIN (nel caso si desideri un'uscita "fast") o a
;               IRQHND (se si desiderà un'uscita "slow" che scansioni la
;               tastiera, aggiorni i timer, eccetera). Se invece il Kernal è
;               disabilitato non vi sono alternative: l'uscita sarà sempre
;               "fast" ed il ripristino sarà realizzato mediante apposito
;               codice in-line.
;               Siccome la macro presuppone che il prossimo anello della catena
;               sarà ancora un IRQ stabile, la prossima linea di raster sarà
;               impostata a .rasterline_-3, coerentemente con quanto fatto
;               da m_Start_Stable_IRQ.
;
; Input:
;   - .rasterline_ (2 byte, Macro)  : Linea di raster del prossimo IRQ
;   - .irq_handler_ (2 byte, Macro) : Indirizzo del gestore del prossimo IRQ
;   - .fast_exit_ (stringa, Macro)  : "FAST" = IRQMIN, altro = IRQHND
;
; Output:
;   - .PC (2 byte, CPU)             : Uscita con rti o salto a IRQMIN/IRQHND
;
; Variazione Stack (.S):
;   - Incrementato di 6 byte
;
; Registri Modificati:  .A, .P (Negative, Interrupt, Zero), RASTER, SCROLY,
;                       IRBVEC (se __DISABLE_KERNAL = 1),
;                       CINV (se __DISABLE_KERNAL = 0)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <chip/vic_ii.asm>
; ================================================================================
```

1.8 vic_ii_tools.asm

1.8.1 Descrizione

Utilità per la gestione dei modi grafici offerti dal chip VIC-II.

Utilità	Descrizione
`m_VIC_Bitmap_On`	Abilita il modo bitmap
`m_VIC_Bitmap_Off`	Disabilita il modo bitmap
`m_VIC_Ext_Color_On`	Abilita il modo ECM
`m_VIC_Ext_Color_Off`	Disabilita il modo ECM
`m_VIC_Multicolor_On`	Abilita il modo multicolore
`m_VIC_Multicolor_Off`	Disabilita il modo multicolore
`VIC_PALETTE`	Imposta i registri colore del VIC-II
`VIC_BANK`	Seleziona il banco di memoria del VIC-II
`VIC_SCREEN`	Posiziona la memoria schermo
`VIC_CHARGEN`	Posiziona il generatore di caratteri
`VIC_BITMAP`	Posiziona la memoria ad alta risoluzione
`VIC_COPY_CHARGEN`	Sposta il generatore di caratteri in RAM
`COMPUTE_CELL_ADDRESS`	Calcola l'indirizzo di una cella video
`GET_SCREEN_CELL`	Legge una cella della memoria schermo
`GET_COLOR_CELL`	Legge una cella della memoria colore

1.8.2 Note d'uso

Nessuna nota in questo modulo.

1.8.3 m_VIC_Bitmap_On

```
; ============================================================================
; MACRO: m_VIC_Bitmap_On
; ----------------------------------------------------------------------------
; Descrizione:  Abilita la modalità grafica bitmap impostando il bit #5
;               di SCROLY.
;
; Input:
;   - Nessuno
;
; Output:
;   - SCROLY (1 byte, I/O) : Bit #5 a 1
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), SCROLY
; Dipendenze Esterne:   <chip/vic_ii.asm>
; ============================================================================
```

1.8.4 m_VIC_Bitmap_Off

```
; ============================================================================
; MACRO: m_VIC_Bitmap_Off
; ----------------------------------------------------------------------------
; Descrizione:  Disabilita la modalità grafica bitmap azzerando il bit #5
;               di SCROLY.
;
; Input:
;   - Nessuno
;
; Output:
;   - SCROLY (1 byte, I/O) : Bit #5 a 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), SCROLY
; Dipendenze Esterne:   <chip/vic_ii.asm>
; ============================================================================
```

1.8.5 m_VIC_Ext_Color_On

```
; ============================================================================
; MACRO: m_VIC_Ext_Color_On
; ----------------------------------------------------------------------------
; Descrizione:  Abilita il modo Colore di Sfondo Esteso (ECM) impostando il
;               bit #6 di SCROLY.
;
; Input:
;   - Nessuno
;
; Output:
;   - SCROLY (1 byte, I/O) : Bit #6 a 1
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), SCROLY
; Dipendenze Esterne:   <chip/vic_ii.asm>
; ============================================================================
```

1.8.6 m_VIC_Ext_Color_Off

```
; ================================================================================
; MACRO: m_VIC_Ext_Color_Off
; --------------------------------------------------------------------------------
; Descrizione:  Disabilita il modo Colore di Sfondo Esteso (ECM) azzerando il
;               bit #6 di SCROLY.
;
; Input:
;   - Nessuno
;
; Output:
;   - SCROLY (1 byte, I/O) : Bit #6 a 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), SCROLY
; Dipendenze Esterne:   <chip/vic_ii.asm>
; ================================================================================
```

1.8.7 m_VIC_Multicolor_On

```
; ============================================================================
; MACRO: m_VIC_Multicolor_On
; ----------------------------------------------------------------------------
; Descrizione:  Abilita la modalità grafica multicolore impostando il bit #4
;               di SCROLX.
;
; Input:
;   - Nessuno
;
; Output:
;   - SCROLX (1 byte, I/O) : Bit #4 a 1
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), SCROLX
; Dipendenze Esterne:   <chip/vic_ii.asm>
; ============================================================================
```

1.8.8 m_VIC_Multicolor_Off

```
; ============================================================================
; MACRO: m_VIC_Multicolor_Off
; ----------------------------------------------------------------------------
; Descrizione:  Disabilita la modalità grafica multicolore azzerando il bit #4
;               di SCROLX.
;
; Input:
;   - Nessuno
;
; Output:
;   - SCROLX (1 byte, I/O) : Bit #4 a 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), SCROLX
; Dipendenze Esterne:   <chip/vic_ii.asm>
; ============================================================================
```

1.8.9 VIC_PALETTE

```
; ============================================================================
; SUBROUTINE: VIC_PALETTE
; ----------------------------------------------------------------------------
; Descrizione:  Imposta i colori di bordo, sfondo, carattere, multicolore/colore
;               esteso e multicolore/sprite in un'unica operazione. L'indirizzo
;               in .XA punta ad un blocco di memoria che contiene in successione
;               i valori da assegnare ai registri EXTCOL, BGCOL0, BGCOL1,
;               BGCOL2, BGCOL3, SPMC0, SPMC1 e COLOR. Se il valore è
;               VIC_TRANSPARENT il registro resta immutato.
;
; Input:
;   - .A (1 byte, CPU)     : Indirizzo del blocco di memoria (byte basso)
;   - .X (1 byte, CPU)     : Indirizzo del blocco di memoria (byte alto)
;   - blocco (8 byte, RAM) : Valori per EXTCOL, BGCOL0, BGCOL1, BGCOL2, BGCOL3,
;                            SPMC0, SPMC1 e COLOR, in quest'ordine
;
; Output:
;   - EXTCOL (1 byte, I/O) : Colore del bordo (se non è VIC_TRANSPARENT)
;   - BGCOL0 (1 byte, I/O) : Colore dello sfondo (se non è VIC_TRANSPARENT)
;   - BGCOL1 (1 byte, I/O) : Colore di sfondo 1 (se non è VIC_TRANSPARENT)
;   - BGCOL2 (1 byte, I/O) : Colore di sfondo 2 (se non è VIC_TRANSPARENT)
;   - BGCOL3 (1 byte, I/O) : Colore di sfondo 3 (se non è VIC_TRANSPARENT)
;   - SPMC0 (1 byte, I/O)  : Colore comune degli sprite multicolori 0 (se non
;                            è VIC_TRANSPARENT)
;   - SPMC1 (1 byte, I/O)  : Colore comune degli sprite multicolori 1 (se non
;                            è VIC_TRANSPARENT)
;   - COLOR (1 byte, RAM)  : Colore dei caratteri (se non è VIC_TRANSPARENT)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, Zero), INDEX2, EXTCOL, BGCOL0,
;                       BGCOL1, BGCOL2, BGCOL3, SPMC0, SPMC1, COLOR
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <chip/vic_ii.asm>
; ============================================================================
```

1.8.10 VIC_BANK

```
; ================================================================================
; SUBROUTINE: VIC_BANK
; --------------------------------------------------------------------------------
; Descrizione:  Imposta o recupera l'indirizzo iniziale del banco di memoria da
;               16 kB visibile dal chip VIC-II.
;                 C = 0: .A contiene la pagina iniziale del banco da
;                        selezionare, che può essere $00, $40, $80 oppure
;                        $C0. Valori differenti vengono allineati al limite
;                        inferiore.
;                 C = 1: Il valore della pagina iniziale del banco di memoria
;                        viene restituito in .A.
;
; Input:
;   - .A (1 byte, CPU)       : Pagina iniziale del banco (solo se C = 0)
;   - Flag Carry (1 bit, .P) : 0 = imposta; 1 = legge
;
; Output:
;   - .A (1 byte, CPU)       : Pagina iniziale del banco (solo se C = 1)
;   - C2DDRA (1 byte, I/O)   : Porte di selezione in uscita (solo se C = 0)
;   - CI2PRA (1 byte, I/O)   : Selezione del banco (solo se C = 0)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero, Carry), C2DDRA, CI2PRA
; Dipendenze Esterne:   <chip/cia.asm>
;                       <system/branch.asm>
; ================================================================================
```

1.8.11 VIC_SCREEN

```
; ============================================================================
; SUBROUTINE: VIC_SCREEN
; ----------------------------------------------------------------------------
; Descrizione:  Imposta o recupera l'indirizzo iniziale della memoria schermo.
;                 C = 0: .A contiene la pagina iniziale della memoria schermo,
;                        che può variare da $00 a $FC ad intervalli di $04.
;                        Valori differenti vengono allineati al limite
;                        inferiore.
;                 C = 1: Il valore della pagina iniziale della memoria schermo
;                        viene restituito in .A.
;               Richiede __USE_VIC_BANK = 1.
;
; Input:
;   - .A (1 byte, CPU)       : Pagina iniziale della memoria schermo (C = 0)
;   - Flag Carry (1 bit, .P) : 0 = imposta; 1 = legge
;
; Output:
;   - .A (1 byte, CPU)       : Pagina iniziale della memoria schermo (C = 1)
;   - HIBASE (1 byte, RAM)   : Pagina per l'editor del BASIC (solo se C = 0)
;   - VMCSB (1 byte, I/O)    : Posizione della memoria schermo (solo se C = 0)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero, Carry), HIBASE, VMCSB
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <chip/vic_ii.asm>
;                       <system/branch.asm>
; ============================================================================
```

1.8.12 VIC_CHARGEN

```
; ============================================================================
; SUBROUTINE: VIC_CHARGEN
; ----------------------------------------------------------------------------
; Descrizione:  Imposta o recupera l'indirizzo iniziale del generatore
;               di caratteri.
;                 C = 0: .A contiene la pagina iniziale del generatore di
;                        caratteri, che può variare da $00 a $F8 ad intervalli
;                        di $08. Valori differenti vengono allineati al limite
;                        inferiore.
;                 C = 1: Il valore della pagina iniziale del generatore di
;                        caratteri viene restituito in .A.
;               Richiede __USE_VIC_BANK = 1.
;
; Input:
;   - .A (1 byte, CPU)       : Pagina iniziale del generatore di caratteri (solo
;                              se C = 0)
;   - Flag Carry (1 bit, .P) : 0 = imposta; 1 = legge
;
; Output:
;   - .A (1 byte, CPU)       : Pagina iniziale del generatore di caratteri (solo
;                              se C = 1)
;   - VMCSB (1 byte, I/O)    : Posizione del generatore (solo se C = 0)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero, Carry), VMCSB
; Dipendenze Esterne:   <chip/vic_ii.asm>
;                       <system/branch.asm>
; ============================================================================
```

1.8.13 VIC_BITMAP

```
; ============================================================================
; SUBROUTINE: VIC_BITMAP
; ----------------------------------------------------------------------------
; Descrizione:  Imposta o recupera l'indirizzo iniziale della memoria ad
;               alta risoluzione.
;                 C = 0: .A contiene la pagina iniziale della memoria ad alta
;                        risoluzione, che può variare da $00 a $E0 ad intervalli
;                        di $20. Valori differenti vengono allineati al limite
;                        inferiore.
;                 C = 1: Il valore della pagina iniziale della memoria ad alta
;                        risoluzione viene restituito in .A.
;               Richiede __USE_VIC_BANK = 1.
;
; Input:
;   - .A (1 byte, CPU)       : Pagina iniziale della memoria bitmap (C = 0)
;   - Flag Carry (1 bit, .P) : 0 = imposta; 1 = legge
;
; Output:
;   - .A (1 byte, CPU)       : Pagina iniziale della memoria bitmap (C = 1)
;   - VMCSB (1 byte, I/O)    : Posizione della memoria bitmap (solo se C = 0)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero, Carry), VMCSB
; Dipendenze Esterne:   <chip/vic_ii.asm>
;                       <system/branch.asm>
; ============================================================================
```

1.8.14 VIC_COPY_CHARGEN

```
; ============================================================================
; SUBROUTINE: VIC_COPY_CHARGEN
; ----------------------------------------------------------------------------
; Descrizione:  Sposta il generatore di caratteri dalla ROM alla RAM, nella
;               posizione definita dalla configurazione del chip VIC-II
;               attualmente in uso.
;               Richiede __USE_VIC_CHARGEN = 1.
;
; Input:
;   - Nessuno
;
; Output:
;   - generatore (4096 byte, RAM) : Copia del generatore di caratteri della ROM
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, Zero, Carry), INDEX, INDEX2,
;                       R6510, CIAICR
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <system/6510_tools.asm>
; ============================================================================
```

1.8.15 COMPUTE_CELL_ADDRESS

```
; ================================================================================
; SUBROUTINE: COMPUTE_CELL_ADDRESS
; --------------------------------------------------------------------------------
; Descrizione:  Calcola in modo veloce, mediante tavole, la cella di memoria
;               corrispondente ad una posizione dello schermo espressa in
;               notazione cartesiana: origine in alto a sinistra, asse x verso
;               destra, asse y verso il basso. Se .A = 0 restituisce in INDEX2
;               l'offset rispetto all'origine (0; 0), altrimenti la posizione
;               assoluta per uno schermo che inizia alla pagina contenuta in .A:
;               .A = (HIBASE) per lo schermo corrente, .A = >COLRAM per la
;               memoria colore.
;
; Input:
;   - .A (1 byte, CPU)    : Pagina base dello schermo (0 = offset dall'origine)
;   - .X (1 byte, CPU)    : Colonna della cella (0-39)
;   - .Y (1 byte, CPU)    : Riga della cella (0-24)
;
; Output:
;   - INDEX2 (2 byte, ZP) : Indirizzo della cella (offset se .A = 0)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, oVerflow, Zero, Carry), INDEX2
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

1.8.16 GET_SCREEN_CELL

```
; ================================================================================
; SUBROUTINE: GET_SCREEN_CELL
; --------------------------------------------------------------------------------
; Descrizione:  Restituisce il codice Schermo del carattere posizionato alla
;               colonna .X ed alla riga .Y dello schermo corrente.
;               Richiede __USE_COMPUTE_CELL_ADDRESS = 1.
;
; Input:
;   - .X (1 byte, CPU)    : Colonna della cella (0-39)
;   - .Y (1 byte, CPU)    : Riga della cella (0-24)
;
; Output:
;   - .A (1 byte, CPU)    : Codice Schermo del carattere
;   - INDEX2 (2 byte, ZP) : Indirizzo della cella
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry), INDEX2
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

1.8.17 GET_COLOR_CELL

```
; ================================================================================
; SUBROUTINE: GET_COLOR_CELL
; --------------------------------------------------------------------------------
; Descrizione:  Restituisce il codice del colore del carattere posizionato alla
;               colonna .X ed alla riga .Y dello schermo corrente.
;               Richiede __USE_COMPUTE_CELL_ADDRESS = 1.
;
; Input:
;   - .X (1 byte, CPU)    : Colonna della cella (0-39)
;   - .Y (1 byte, CPU)    : Riga della cella (0-24)
;
; Output:
;   - .A (1 byte, CPU)    : Codice del colore (0-15)
;   - INDEX2 (2 byte, ZP) : Indirizzo della cella
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry), INDEX2
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

Libreria

fp

2 Libreria fp

Modulo	Descrizione
`fp.asm`	Macro per l'aritmetica in virgola mobile nel formato MBF del BASIC Commodore, basate sulle routine della ROM.

2.1 fp.asm

2.1.1 Descrizione

Macro per l'aritmetica in virgola mobile nel formato MBF del BASIC Commodore, basate sulle routine della ROM.

Utilità	Descrizione
`m_Store_FAC_in_Scratch`	Salva FAC nell'area di appoggio
`m_Store_ARG_in_Scratch`	Salva ARG nell'area di appoggio
`m_Load_FAC_from_Scratch`	Ripristina FAC dall'area di appoggio
`m_Load_ARG_from_Scratch`	Ripristina ARG dall'area di appoggio
`m_Add_FAC_to_ARG`	Somma ARG a FAC
`m_Add_FAC_to_Mem`	Somma da indirizzo a FAC
`m_Add_FAC_to_Ptr`	Somma da puntatore a FAC
`m_Subtract_FAC_from_ARG`	Sottrae FAC da ARG
`m_Subtract_FAC_from_Mem`	Sottrae FAC da indirizzo
`m_Subtract_FAC_from_Ptr`	Sottrae FAC da puntatore
`m_Multiply_FAC_by_ARG`	Moltiplica FAC per ARG
`m_Multiply_FAC_by_Mem`	Moltiplica FAC per indirizzo
`m_Multiply_FAC_by_Ptr`	Moltiplica FAC per puntatore
`m_Divide_ARG_by_FAC`	Divide ARG per FAC
`m_Divide_Mem_by_FAC`	Divide indirizzo per FAC
`m_Divide_Ptr_by_FAC`	Divide puntatore per FAC
`m_AND_FAC_with_ARG`	AND tra FAC e ARG
`m_AND_FAC_with_Mem`	AND tra FAC e indirizzo
`m_AND_FAC_with_Ptr`	AND tra FAC e puntatore
`m_OR_FAC_with_ARG`	OR tra FAC e ARG
`m_OR_FAC_with_Mem`	OR tra FAC e indirizzo
`m_OR_FAC_with_Ptr`	OR tra FAC e puntatore
`m_NOT_FAC`	NOT di FAC
`m_NOT_ARG`	NOT di ARG, sul posto
`m_NOT_Mem`	NOT da indirizzo
`m_NOT_Ptr`	NOT da puntatore
`m_Compare_FAC_to_ARG`	Confronta FAC con ARG
`m_Compare_FAC_to_Mem`	Confronta FAC con indirizzo
`m_Compare_FAC_to_Ptr`	Confronta FAC con puntatore
`m_Load_FAC_with`	Carica FAC con 5 byte MBF
`m_Load_ARG_with`	Carica ARG con 5 byte MBF
`m_Load_FAC_with_0`	Carica FAC con 0

Utilità	Descrizione
m_Load_FAC_with_0_25	Carica FAC con 0.25
m_Load_FAC_with_0_5	Carica FAC con 0.5
m_Load_FAC_with_1	Carica FAC con 1
m_Load_FAC_with_MINUS_1	Carica FAC con -1
m_Load_FAC_with_2	Carica FAC con 2
m_Load_FAC_with_10	Carica FAC con 10
m_Load_FAC_with_0_1	Carica FAC con 0.1
m_Load_FAC_with_PI4	Carica FAC con π/4
m_Load_FAC_with_PI2	Carica FAC con π/2
m_Load_FAC_with_PI	Carica FAC con π
m_Load_FAC_with_2PI	Carica FAC con 2π
m_Load_FAC_with_PI180	Carica FAC con π/180
m_Load_FAC_with_PI200	Carica FAC con π/200
m_Load_FAC_with_180PI	Carica FAC con 180/π
m_Load_FAC_with_200PI	Carica FAC con 200/π
m_Load_FAC_with_SQR2	Carica FAC con √2
m_Load_FAC_with_SQR3	Carica FAC con √3
m_Load_FAC_with_e	Carica FAC con e
m_Load_FAC_with_LOG2	Carica FAC con ln 2
m_Load_FAC_with_LOG10	Carica FAC con ln 10
m_Load_FAC_with_eps	Carica FAC con eps
m_Load_FAC_with_MINR	Carica FAC con MINR
m_Load_FAC_with_MAXR	Carica FAC con MAXR
m_Load_ARG_with_0	Carica ARG con 0
m_Load_ARG_with_0_25	Carica ARG con 0.25
m_Load_ARG_with_0_5	Carica ARG con 0.5
m_Load_ARG_with_1	Carica ARG con 1
m_Load_ARG_with_MINUS_1	Carica ARG con -1
m_Load_ARG_with_2	Carica ARG con 2
m_Load_ARG_with_10	Carica ARG con 10
m_Load_ARG_with_0_1	Carica ARG con 0.1
m_Load_ARG_with_PI4	Carica ARG con π/4
m_Load_ARG_with_PI2	Carica ARG con π/2
m_Load_ARG_with_PI	Carica ARG con π
m_Load_ARG_with_2PI	Carica ARG con 2π
m_Load_ARG_with_PI180	Carica ARG con π/180
m_Load_ARG_with_PI200	Carica ARG con π/200

Utilità	Descrizione
m_Load_ARG_with_180PI	Carica ARG con 180/π
m_Load_ARG_with_200PI	Carica ARG con 200/π
m_Load_ARG_with_SQR2	Carica ARG con √2
m_Load_ARG_with_SQR3	Carica ARG con √3
m_Load_ARG_with_e	Carica ARG con e
m_Load_ARG_with_LOG2	Carica ARG con ln 2
m_Load_ARG_with_LOG10	Carica ARG con ln 10
m_Load_ARG_with_eps	Carica ARG con eps
m_Load_ARG_with_MINR	Carica ARG con MINR
m_Load_ARG_with_MAXR	Carica ARG con MAXR
FP_ERROR_HANDLER	Gestore delle tre condizioni d'errore
m_Load_FAC_with_UINT8	Carica FAC con UINT8 costante
m_Load_FAC_with_UINT8_Mem	Carica FAC con UINT8 da indirizzo
m_Load_FAC_with_UINT8_Ptr	Carica FAC con UINT8 da puntatore
m_Load_ARG_with_UINT8	Carica ARG con UINT8 costante
m_Load_ARG_with_UINT8_Mem	Carica ARG con UINT8 da indirizzo
m_Load_ARG_with_UINT8_Ptr	Carica ARG con UINT8 da puntatore
m_Load_FAC_with_INT8	Carica FAC con INT8 costante
m_Load_FAC_with_INT8_Mem	Carica FAC con INT8 da indirizzo
m_Load_FAC_with_INT8_Ptr	Carica FAC con INT8 da puntatore
m_Load_ARG_with_INT8	Carica ARG con INT8 costante
m_Load_ARG_with_INT8_Mem	Carica ARG con INT8 da indirizzo
m_Load_ARG_with_INT8_Ptr	Carica ARG con INT8 da puntatore
m_Load_FAC_with_UINT16	Carica FAC con UINT16 costante
m_Load_FAC_with_UINT16_Mem	Carica FAC con UINT16 da indirizzo
m_Load_FAC_with_UINT16_Ptr	Carica FAC con UINT16 da puntatore
m_Load_ARG_with_UINT16	Carica ARG con UINT16 costante
m_Load_ARG_with_UINT16_Mem	Carica ARG con UINT16 da indirizzo
m_Load_ARG_with_UINT16_Ptr	Carica ARG con UINT16 da puntatore
m_Load_FAC_with_INT16	Carica FAC con INT16 costante
m_Load_FAC_with_INT16_Mem	Carica FAC con INT16 da indirizzo
m_Load_FAC_with_INT16_Ptr	Carica FAC con INT16 da puntatore
m_Load_ARG_with_INT16	Carica ARG con INT16 costante
m_Load_ARG_with_INT16_Mem	Carica ARG con INT16 da indirizzo
m_Load_ARG_with_INT16_Ptr	Carica ARG con INT16 da puntatore
m_Load_FAC_with_UINT24	Carica FAC con UINT24 costante
m_Load_FAC_with_UINT24_Mem	Carica FAC con UINT24 da indirizzo

Utilità	Descrizione
m_Load_FAC_with_UINT24_Ptr	Carica FAC con UINT24 da puntatore
m_Load_ARG_with_UINT24	Carica ARG con UINT24 costante
m_Load_ARG_with_UINT24_Mem	Carica ARG con UINT24 da indirizzo
m_Load_ARG_with_UINT24_Ptr	Carica ARG con UINT24 da puntatore
m_Load_FAC_with_INT24	Carica FAC con INT24 costante
m_Load_FAC_with_INT24_Mem	Carica FAC con INT24 da indirizzo
m_Load_FAC_with_INT24_Ptr	Carica FAC con INT24 da puntatore
m_Load_ARG_with_INT24	Carica ARG con INT24 costante
m_Load_ARG_with_INT24_Mem	Carica ARG con INT24 da indirizzo
m_Load_ARG_with_INT24_Ptr	Carica ARG con INT24 da puntatore
m_Load_FAC_with_UINT32	Carica FAC con UINT32 costante
m_Load_FAC_with_UINT32_Mem	Carica FAC con UINT32 da indirizzo
m_Load_FAC_with_UINT32_Ptr	Carica FAC con UINT32 da puntatore
m_Load_ARG_with_UINT32	Carica ARG con UINT32 costante
m_Load_ARG_with_UINT32_Mem	Carica ARG con UINT32 da indirizzo
m_Load_ARG_with_UINT32_Ptr	Carica ARG con UINT32 da puntatore
m_Load_FAC_with_INT32	Carica FAC con INT32 costante
m_Load_FAC_with_INT32_Mem	Carica FAC con INT32 da indirizzo
m_Load_FAC_with_INT32_Ptr	Carica FAC con INT32 da puntatore
m_Load_ARG_with_INT32	Carica ARG con INT32 costante
m_Load_ARG_with_INT32_Mem	Carica ARG con INT32 da indirizzo
m_Load_ARG_with_INT32_Ptr	Carica ARG con INT32 da puntatore
m_Convert_FAC_into_INT32	Converte FAC in INT32
m_Convert_ARG_into_INT32	Converte ARG in INT32
m_FAC_to_String	Da FAC a stringa
m_ARG_to_String	Da ARG a stringa
m_String_to_FAC	Da stringa a FAC
m_String_to_ARG	Da stringa a ARG
m_Print_FAC	Stampa FAC
m_Print_ARG	Stampa ARG
m_Load_FAC_from_Mem	Carica FAC da un indirizzo assoluto
m_Load_FAC_from_Ptr	Carica FAC da un indirizzo puntato
m_Load_ARG_from_Mem	Carica ARG da un indirizzo assoluto
m_Load_ARG_from_Ptr	Carica ARG da un indirizzo puntato
m_Store_FAC_to_Mem	Scrive FAC a un indirizzo assoluto
m_Store_FAC_to_Ptr	Scrive FAC a un indirizzo puntato
m_Store_ARG_to_Mem	Scrive ARG a un indirizzo assoluto

Utilità	Descrizione
m_Store_ARG_to_Ptr	Scrive ARG a un indirizzo puntato
m_Transfer_FAC_to_ARG	Copia FAC in ARG
m_Transfer_ARG_to_FAC	Copia ARG in FAC
m_Swap_FAC_and_ARG	Scambia FAC e ARG
m_Poly_Mem	Polinomio, tabella a indirizzo
m_Poly_Ptr	Polinomio, tabella da puntatore
m_Odd_Poly_Mem	Polinomio dispari, tabella a indirizzo
m_Odd_Poly_Ptr	Polinomio dispari, tabella da puntatore
m_Power_ARG_to_FAC	ARG elevato a FAC
m_Power_Mem_to_FAC	Indirizzo elevato a FAC
m_Power_Ptr_to_FAC	Puntatore elevato a FAC
m_SQR_FAC	Radice quadrata di FAC
m_SQR_ARG	Radice quadrata di ARG
m_SQR_Mem	Radice da indirizzo
m_SQR_Ptr	Radice da puntatore
m_Load_FAC_with_RND	Numero casuale in FAC
m_Randomize	Inizializza il generatore
m_LOG_FAC	Logaritmo di FAC
m_LOG_ARG	Logaritmo di ARG
m_LOG_Mem	Logaritmo da indirizzo
m_LOG_Ptr	Logaritmo da puntatore
m_EXP_FAC	Esponenziale di FAC
m_EXP_ARG	Esponenziale di ARG
m_EXP_Mem	Esponenziale da indirizzo
m_EXP_Ptr	Esponenziale da puntatore
m_SIN_FAC	Seno di FAC
m_SIN_ARG	Seno di ARG
m_SIN_Mem	Seno da indirizzo
m_SIN_Ptr	Seno da puntatore
m_COS_FAC	Coseno di FAC
m_COS_ARG	Coseno di ARG
m_COS_Mem	Coseno da indirizzo
m_COS_Ptr	Coseno da puntatore
m_TAN_FAC	Tangente di FAC
m_TAN_ARG	Tangente di ARG
m_TAN_Mem	Tangente da indirizzo
m_TAN_Ptr	Tangente da puntatore

Utilità	Descrizione
m_ATN_FAC	Arcotangente di FAC
m_ATN_ARG	Arcotangente di ARG
m_ATN_Mem	Arcotangente da indirizzo
m_ATN_Ptr	Arcotangente da puntatore
m_Negate_FAC	Cambia il segno di FAC
m_Negate_ARG	Cambia il segno di ARG
m_Negate_Mem	Nega da indirizzo, in FAC
m_Negate_Ptr	Nega da puntatore, in FAC
m_ABS_FAC	Valore assoluto di FAC
m_ABS_ARG	Valore assoluto di ARG
m_ABS_Mem	Valore assoluto da indirizzo
m_ABS_Ptr	Valore assoluto da puntatore
m_Sign_FAC	Segno di FAC in .A
m_Sign_ARG	Segno di ARG in .A
m_Sign_Mem	Segno da indirizzo, in .A
m_Sign_Ptr	Segno da puntatore, in .A
m_SGN_FAC	FAC diventa il suo segno
m_SGN_ARG	ARG diventa il suo segno
m_SGN_Mem	Segno da indirizzo, in FAC
m_SGN_Ptr	Segno da puntatore, in FAC
m_INT_FAC	Arrotonda FAC per difetto
m_INT_ARG	Arrotonda ARG per difetto
m_INT_Mem	Arrotonda da indirizzo, in FAC
m_INT_Ptr	Arrotonda da puntatore, in FAC
m_Multiply_FAC_by_2	Moltiplica FAC per 2
m_Multiply_ARG_by_2	Moltiplica ARG per 2
m_Multiply_Mem_by_2	Per 2 da indirizzo, in FAC
m_Multiply_Ptr_by_2	Per 2 da puntatore, in FAC
m_Divide_FAC_by_2	Divide FAC per 2
m_Divide_ARG_by_2	Divide ARG per 2
m_Divide_Mem_by_2	Diviso 2 da indirizzo, in FAC
m_Divide_Ptr_by_2	Diviso 2 da puntatore, in FAC

2.1.2 Note d'uso

```
; NOTA #01
; --------
; I numeri in memoria sono codificati in due formati diversi:
;
; MICROSOFT BINARY FORMAT (MBF)
; -----------------------------
;
;     Byte 0   Byte 1   Byte 2   Byte 3   Byte 4
;   |76543210|76543210|76543210|76543210|76543210|
;   |Esp.+$80|S<------------Mantissa------------>|
;
; Poiché la mantissa è memorizzata in forma normalizzata, il bit #7 del
; byte 1 è per definizione sempre "1". Sfruttando questo fatto, il bit #7
; del byte 1 codifica in realtà il segno del numero: "+" = 0 e "-" = 1, con
; il vantaggio di una rappresentazione che occupa meno memoria.
;
; FORMATO FLOATING POINT ACCUMULATOR (FAC)
; ----------------------------------------
;
; Quando il numero viene memorizzato in un FAC, o trasferito in esso, la
; rappresentazione viene espansa in una forma più adatta alla
; manipolazione numerica:
;
;     Byte 0   Byte 1   Byte 2   Byte 3   Byte 4   Byte 5
;   |76543210|76543210|76543210|76543210|76543210|76543210|
;   |Esp.+$80|<------------Mantissa------------->|S<inut.>|
;
; In questa rappresentazione il bit #7 del byte 1 è ripristinato a "1",
; mentre il bit #7 del byte aggiuntivo codifica il segno: "+" = 0, "-" = 1.
```

```
; NOTA #02
; --------
; Il registro ARISGN contiene $00 se i segni di FAC e ARG sono uguali e un
; valore diverso da $00 se sono diversi, ma questo è garantito solo se ARG
; viene aggiornato DOPO FAC. Per semplificare il compito del programmatore, le
; macro della libreria aggiornano automaticamente il registro ARISGN ogni
; volta che FAC viene modificato.
```

```
; NOTA #03
; --------
; Tutto ciò che esce da FAC viene arrotondato in base al valore del
; registro FACOV.
```

```
; NOTA #04
; --------
; Le librerie math e fp usano le stesse celle di pagina zero. NUMWRK ($57-$60)
; è l'area di lavoro di math_mult, di math8 (NUMWRK+0) e della ROM; anche
; _SCRATCH_2, che contiene la copia di ARG, coincide con NUMWRK. In
; moltiplicazione le celle arrivano fino ai primi due byte di FAC ($61-$62).
; Un dato lasciato in quelle celle da una libreria viene distrutto dall'altra:
; ogni chiamata va considerata autonoma e il suo risultato va copiato altrove
; prima di chiamare l'altra libreria.
```

```
; NOTA #05
; --------
; Le macro chiamano routine della ROM BASIC ($A000-$BFFF) e della sua parte
; residente nella ROM KERNAL ($E000-$E4AC). Entrambe le ROM devono essere
; visibili: LORAM e HIRAM (bit #0 e #1 di $01) devono valere 1, come nella
; configurazione standard ($01 = $37).
```

```
; NOTA #06
; --------
; Le routine della ROM alterano variabili vitali del BASIC. _SCRATCH_1
; ($3D-$43) copre OLDTXT, DATLIN, DATPTR e il byte basso di INPPTR: dopo l'uso
; di fp il comando CONT non è più affidabile e READ e INPUT ripartono da
; puntatori alterati. Vengono inoltre modificati NUMWRK, FAC, ARG, le celle di
; lavoro della ROM e, nelle conversioni da stringa, TXTPTR ($7A/$7B). Un
; programma in LM che non torna al BASIC non è interessato; uno che ci torna
; deve considerare persi questi valori.
```

```
; NOTA #07
; --------
; Le routine della ROM possono segnalare tre errori: ?ILLEGAL QUANTITY ($0E),
; ?OVERFLOW ($0F) e ?DIVISION BY ZERO ($14). La ROM non ritorna al chiamante:
; salta a IERROR tramite il vettore ($0300) con il codice in .X, e il BASIC
; stampa il messaggio e torna a READY.
;
; Con __USE_FP_ERROR_HANDLER = 1 ogni macro a rischio salva .S in
; __FP_SAVED_SP (ZP, $06) prima della chiamata. Se il chiamante fa puntare il
; vettore IERROR a FP_ERROR_HANDLER, il gestore ripristina .S e ritorna subito
; dopo il jsr della macro, con Carry = 1 e il codice d'errore in .X.
; Carry = 0 indica che non c'è stato errore. L'installazione del vettore è
; compito del chiamante.
```

```
; NOTA #08
; --------
; La moltiplicazione in virgola mobile della ROM contiene un bug noto, che
; interessa quasi tutti i BASIC Commodore a 8 bit (non il BASIC 7.0 del C128).
; Per certe coppie di operandi il risultato è sbagliato di pochi bit, e può
; dipendere dall'ordine: con V = 1 + 255/2^31, V*1-V dà 0 mentre 1*V-V non dà
; 0. La causa è in MLTPLY ($BA59): quando un byte della mantissa di ARG è
; zero, la routine salta a MULSHF ($B983), che somma senza controllare il
; Carry e può scorrere di 9 bit invece di 8.
;
; Il bug riguarda ogni macro che moltiplica, direttamente o attraverso la ROM:
; m_Multiply_FAC_by_ARG/_Mem/_Ptr, le conversioni in stringa e le stampe, i
; polinomi, la potenza e SQR, LOG ed EXP, le funzioni trigonometriche e
; m_Load_FAC_with_RND (con FAC positivo).
;
; Fonte: https://www.c64-wiki.com/wiki/Multiply_bug
;
; Nella sola moltiplicazione si può ridurre il problema scambiando i due
; operandi: su 20000 coppie casuali lo scambio corregge circa il 94% dei casi
; in cui un ordine è sbagliato, ma non tutti, perché in 300 casi sono
; sbagliati entrambi. Dopo lo scambio ARISGN va ricalcolato (FACSGN xor
; ARGSGN). Per le altre funzioni il rimedio non si applica: l'ordine delle
; moltiplicazioni interne non dipende dal chiamante.
```

```
; NOTA #09
; --------
; SQR calcola la radice tramite LOG ed EXP, quindi ne eredita i polinomi e il
; bug della moltiplicazione (NOTA #08). Il wiki dichiara un accordo col valore
; vero fino a 30 bit (25 bit per pochi ingressi). Nelle nostre misure, su
; 1500 ingressi casuali, la mediana è 33.0 bit e il minimo 31.0 (la mantissa
; ha 32 bit). SQR(0) = 0; un argomento negativo dà ?ILLEGAL QUANTITY.
;
; Fonte: https://www.c64-wiki.com/wiki/SQR
```

```
; NOTA #10
; --------
; RND dipende dal segno di FAC. Con FAC > 0 restituisce il numero successivo
; della sequenza (seme x 11879546 + 3.927677739E-8, poi i byte della mantissa
; vengono scambiati). Con FAC < 0 FAC diventa il nuovo seme, e lo stesso seme
; dà sempre la stessa sequenza: utile per collaudi e debug. Il risultato di
; quella chiamata è il seme stesso, quindi con un seme piccolo è un numero
; molto piccolo (RND(-1) = 2.99E-8). Con FAC = 0 il seme viene ricavato dai
; timer A e dall'orologio TOD del CIA #1 ($DC04, $DC05, $DC08, $DC09): questi
; registri non sono inizializzati all'avvio e alcuni lavorano in BCD, quindi
; non assumono tutti i valori possibili e lo spazio dei semi si riduce.
;
; m_Randomize usa come seme -TI: TI ha 24 bit, quindi 8 bit della mantissa a
; 32 bit sono sempre zero. Per aggiungere casualità, il numero ottenuto (tra 0
; e 1) viene poi negato e usato per reinizializzare il generatore.
;
; Secondo il wiki l'implementazione può degenerare in sequenze molto corte
; (nel caso peggiore 723 valori distinti); questo non è stato verificato qui.
;
; Fonte: https://www.c64-wiki.com/wiki/RND
```

```
; NOTA #11
; --------
; LOG(V) dà ?ILLEGAL QUANTITY per V <= 0; LOG(1) = 0. Il wiki dichiara un
; accordo col valore vero fino a 29 bit (24 per pochi ingressi), con
; peggioramento vicino a V = 1. Nelle nostre misure su 1500 ingressi casuali
; la mediana è 31.9 bit e il minimo 22.8 (26 ingressi sotto 27 bit). I casi
; peggiori sono vicini a 1: LOG(0.99) dà 24.4 bit, LOG(1.01) 26.9, LOG(1.1)
; 27.7. L'errore assoluto resta piccolo, ma il risultato è piccolo, perciò
; cresce l'errore relativo. Il bug della moltiplicazione (NOTA #08) non è la
; causa: con la ROM corretta i valori sono identici. La potenza (m_Power_*)
; passa da LOG ed EXP e ne eredita i limiti.
;
; Fonte: https://www.c64-wiki.com/wiki/LOG
```

```
; NOTA #12
; --------
; EXP(V) dà ?OVERFLOW per V oltre circa 88.03 (il wiki indica 89.03, ma
; EXP(89.02) dà già errore; EXP(88) = 1.65E38). Per argomenti molto negativi
; il risultato è 0 senza errore (provati -89 e -100). Il wiki dichiara un
; accordo col valore vero fino a 30 bit (26 per pochi ingressi); nelle nostre
; misure su 1500 ingressi casuali la mediana è 33.0 bit e il minimo 30.9.
; EXP(0) = 1. La potenza (m_Power_*) passa da LOG ed EXP: stessi limiti.
;
; Fonte: https://www.c64-wiki.com/wiki/EXP
```

```
; NOTA #13
; --------
; SIN riduce l'argomento a un quarto di periodo e valuta un polinomio. Il wiki
; dichiara un accordo col valore vero fino a 30 bit tra circa -2.6 e +2.6, con
; peggioramento oltre per l'imprecisione di PI (NOTA #17) e per pochi ingressi
; fino a 24 bit. Nelle nostre misure su 1500 ingressi casuali la mediana è
; 32.1 bit e il minimo 27.5. Per argomenti piccoli l'errore relativo sale: da
; 4E-11 per 0.04 a 3.4E-9 per 1E-4 (da 34 a 28 bit), perché i coefficienti del
; polinomio non sono esatti. Un argomento fuori dai limiti della virgola
; mobile dà ?OVERFLOW; SIN stessa non ne ha altri.
;
; Fonte: https://www.c64-wiki.com/wiki/SIN
```

```
; NOTA #14
; --------
; COS(V) è calcolata come SIN(V + PI/2): valgono quindi i limiti della NOTA
; #13 e il bug della moltiplicazione (NOTA #08). Il wiki dichiara un accordo
; col valore vero fino a 29 bit, con pochi ingressi a 24 e peggioramento
; vicino ai multipli dispari di PI/2. Nelle nostre misure su 1500 ingressi
; casuali la mediana è 31.8 bit e il minimo 27.7. La simmetria non è esatta:
; COS(0.9) - COS(-0.9) = -2 x 2^-32 (il wiki riporta 3 x 2^-32).
;
; Fonte: https://www.c64-wiki.com/wiki/COS
```

```
; NOTA #15
; --------
; TAN(V) è calcolata come SIN(V)/COS(V): eredita i limiti di entrambe e perde
; precisione dove le perdono loro. Il wiki dichiara un accordo di circa 29 bit
; (25 per pochi ingressi). Nelle nostre misure su 1500 ingressi casuali la
; mediana è 32.2 bit e il minimo 27.3. Se COS vale 0 la divisione dà ?DIVISION
; BY ZERO (il wiki cita TAN(PI/2)); questo è l'unico errore di TAN oltre a
; ?OVERFLOW. TAN(PI) non è esattamente 0: con PI arrotondato a 3.14159265359
; abbiamo misurato 0, il wiki riporta -7.3E-10.
;
; Fonte: https://www.c64-wiki.com/wiki/TAN
```

```
; NOTA #16
; --------
; ATN usa la simmetria dispari e, per |V| > 1, l'identità PI/2 - ATN(1/V); per
; 0 <= V <= 1 valuta un polinomio. Il wiki dichiara un accordo col valore vero
; fino a 30 bit (25 per pochi ingressi). Nelle nostre misure su 1500 ingressi
; casuali la mediana è 32.1 bit e il minimo 30.6. ATN non dà errori a parte
; ?OVERFLOW per argomenti fuori dai limiti della virgola mobile.
;
; Fonte: https://www.c64-wiki.com/wiki/ATN
```

```
; NOTA #17
; --------
; La costante PI della ROM (5 byte $82 $49 $0F $DA $A1 a $AEA8) vale
; 3.1415926525 invece di 3.1415926536: l'ultimo byte $A1 è meno preciso del
; valore più vicino, che si ottiene con $A2 (errore -1.2E-10 invece di
; -1.05E-9). 2*PI e PI/2, usate da SIN, COS e TAN, hanno invece l'ultimo byte
; $A2. Di conseguenza ATN(1)*4 dà 3.1415926525 (non 3.1415266 come riporta il
; wiki, che è sbagliato). Per usare il valore più vicino a pi greco si può
; caricare una propria costante ($82 $49 $0F $DA $A2).
;
; Fonte: https://www.c64-wiki.com/wiki/Pi
```

2.1.3 m_Store_FAC_in_Scratch

```
; ============================================================================
; MACRO: m_Store_FAC_in_Scratch
; ----------------------------------------------------------------------------
; Descrizione:  Copia in _SCRATCH_1 il contenuto di FAC, un registro alla volta.
;               Uso interno della libreria.
;
; Input:
;   - FAC (7 byte, ZP)        : Registri da FACEXP a FACOV
;
; Output:
;   - _SCRATCH_1 (7 byte, ZP) : Copia di FAC
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), _SCRATCH_1
; Dipendenze Esterne:   <c64/symbols.asm>
; ============================================================================
```

2.1.4 m_Store_ARG_in_Scratch

```
; ================================================================================
; MACRO: m_Store_ARG_in_Scratch
; --------------------------------------------------------------------------------
; Descrizione:  Copia in _SCRATCH_2 il contenuto di ARG, un registro alla volta.
;               Uso interno della libreria.
;
; Input:
;   - ARG (6 byte, ZP)        : Registri da ARGEXP a ARGSGN
;
; Output:
;   - _SCRATCH_2 (6 byte, ZP) : Copia di ARG
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), _SCRATCH_2
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.5 m_Load_FAC_from_Scratch

```
; ================================================================================
; MACRO: m_Load_FAC_from_Scratch
; --------------------------------------------------------------------------------
; Descrizione:  Copia il contenuto di _SCRATCH_1 in FAC, un registro alla volta.
;               Uso interno della libreria.
;
; Input:
;   - _SCRATCH_1 (7 byte, ZP) : Copia di FAC
;
; Output:
;   - FAC (7 byte, ZP)        : Registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.6 m_Load_ARG_from_Scratch

```
; ================================================================================
; MACRO: m_Load_ARG_from_Scratch
; --------------------------------------------------------------------------------
; Descrizione:  Copia il contenuto di _SCRATCH_2 in ARG, un registro alla volta.
;               Uso interno della libreria.
;
; Input:
;   - _SCRATCH_2 (6 byte, ZP) : Copia di ARG
;
; Output:
;   - ARG (6 byte, ZP)        : Registri da ARGEXP a ARGSGN
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), ARG
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.7 m_Add_FAC_to_ARG

```
; ============================================================================
; MACRO: m_Add_FAC_to_ARG
; ----------------------------------------------------------------------------
; Descrizione:  Somma FAC e un numero in formato MBF che si trova in ARG e
;               lascia il risultato in FAC: FAC = FAC + ARG.
;
;               Calcola il segno del risultato in ARISGN (bit #7 = 0 se i segni
;               di FAC e ARG sono uguali, 1 se sono diversi) e chiama la routine
;               FADDT della ROM del BASIC, che richiede in .A l'esponente di
;               FAC: se è zero, FAC è considerato zero.
;
;               La routine FADDT della ROM del BASIC può alterare ARG.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in _SCRATCH_2 prima
;               della chiamata e lo ripristina dopo; se vale 0 il contenuto di
;               ARG non è garantito dopo la chiamata.
;
;               Se il risultato è troppo grande per il formato MBF la ROM
;               segnala OVERFLOW ERROR e non ritorna al chiamante; se è troppo
;               piccolo il risultato è zero. Per la gestione degli errori vedi
;               [NOTA #07 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - FAC (7 byte, ZP)               : Registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Registri da ARGEXP a ARGSGN
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Risultato, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - ARISGN (1 byte, ZP)            : Segno aritmetico, bit #7
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, _SCRATCH_2 (se __PRESERVE_ARG = 1),
;                       __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_common.asm>
; ============================================================================
```

2.1.8 m_Add_FAC_to_Mem

```
; ============================================================================
; MACRO: m_Add_FAC_to_Mem
; ----------------------------------------------------------------------------
; Descrizione:  Somma FAC e un numero in formato MBF che si trova a un indirizzo
;               assoluto e lascia il risultato in FAC: FAC = FAC + (.addr_).
;
;               Punta .A (LSB) e .Y (MSB) all'indirizzo del numero e chiama la
;               routine FADD della ROM del BASIC, che lo carica in ARG con
;               CONUPK e aggiorna ARISGN. La routine FADD altera anche INDEX: il
;               suo contenuto non è garantito dopo la chiamata.
;
;               La routine FADD della ROM del BASIC altera sempre ARG.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in _SCRATCH_2 prima
;               della chiamata e lo ripristina dopo; se vale 0 il contenuto di
;               ARG non è garantito dopo la chiamata.
;
;               Se il risultato è troppo grande per il formato MBF la ROM
;               segnala OVERFLOW ERROR e non ritorna al chiamante; se è troppo
;               piccolo il risultato è zero. Per la gestione degli errori vedi
;               [NOTA #07 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - FAC (7 byte, ZP)               : Registri da FACEXP a FACOV
;   - .addr_ (1 o 2 byte, Macro)     : Indirizzo del numero
;   - (.addr_) (5 byte, RAM)         : Numero in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Risultato, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - ARISGN (1 byte, ZP)            : Segno aritmetico, bit #7
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, _SCRATCH_2 (se __PRESERVE_ARG = 1),
;                       __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_common.asm>
; ============================================================================
```

2.1.9 m_Add_FAC_to_Ptr

```
; ============================================================================
; MACRO: m_Add_FAC_to_Ptr
; ----------------------------------------------------------------------------
; Descrizione:  Somma FAC e un numero in formato MBF che si trova all'indirizzo
;               a cui punta un puntatore e lascia il risultato in FAC:
;               FAC = FAC + ((.ptr_)).
;
;               Punta .A (LSB) e .Y (MSB) all'indirizzo del numero e chiama la
;               routine FADD della ROM del BASIC, che lo carica in ARG con
;               CONUPK e aggiorna ARISGN. La routine FADD altera anche INDEX: il
;               suo contenuto non è garantito dopo la chiamata.
;
;               La routine FADD della ROM del BASIC altera sempre ARG.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in _SCRATCH_2 prima
;               della chiamata e lo ripristina dopo; se vale 0 il contenuto di
;               ARG non è garantito dopo la chiamata.
;
;               Se il risultato è troppo grande per il formato MBF la ROM
;               segnala OVERFLOW ERROR e non ritorna al chiamante; se è troppo
;               piccolo il risultato è zero. Per la gestione degli errori vedi
;               [NOTA #07 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - FAC (7 byte, ZP)               : Registri da FACEXP a FACOV
;   - .ptr_ (1 o 2 byte, Macro)      : Indirizzo del puntatore
;   - (.ptr_) (2 byte, RAM)          : Indirizzo del numero (LSB, MSB)
;   - ((.ptr_)) (5 byte, RAM)        : Numero in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Risultato, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - ARISGN (1 byte, ZP)            : Segno aritmetico, bit #7
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, _SCRATCH_2 (se __PRESERVE_ARG = 1),
;                       __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_common.asm>
; ============================================================================
```

2.1.10 m_Subtract_FAC_from_ARG

```
; ================================================================================
; MACRO: m_Subtract_FAC_from_ARG
; --------------------------------------------------------------------------------
; Descrizione:  Sottrae FAC da un numero in formato MBF che si trova in ARG e
;               lascia il risultato in FAC: FAC = ARG - FAC.
;
;               Chiama la routine FSUBT della ROM del BASIC, che inverte il
;               segno di FAC, calcola ARISGN e tratta da sé il caso di FAC
;               uguale a zero.
;
;               La routine FSUBT della ROM del BASIC può alterare ARG.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in _SCRATCH_2 prima
;               della chiamata e lo ripristina dopo; se vale 0 il contenuto di
;               ARG non è garantito dopo la chiamata.
;
;               Se il risultato è troppo grande per il formato MBF la ROM
;               segnala OVERFLOW ERROR e non ritorna al chiamante; se è troppo
;               piccolo il risultato è zero. Per la gestione degli errori vedi
;               [NOTA #07 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - FAC (7 byte, ZP)               : Registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Registri da ARGEXP a ARGSGN
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Risultato, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - ARISGN (1 byte, ZP)            : Segno aritmetico, bit #7
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, _SCRATCH_2 (se __PRESERVE_ARG = 1),
;                       __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_common.asm>
; ================================================================================
```

2.1.11 m_Subtract_FAC_from_Mem

```
; ============================================================================
; MACRO: m_Subtract_FAC_from_Mem
; ----------------------------------------------------------------------------
; Descrizione:  Sottrae FAC da un numero in formato MBF che si trova a un
;               indirizzo assoluto e lascia il risultato in FAC:
;               FAC = (.addr_) - FAC.
;
;               Punta .A (LSB) e .Y (MSB) all'indirizzo del numero e chiama la
;               routine FSUB della ROM del BASIC, che lo carica in ARG con
;               CONUPK e aggiorna ARISGN. La routine FSUB altera anche INDEX: il
;               suo contenuto non è garantito dopo la chiamata.
;
;               La routine FSUB della ROM del BASIC altera sempre ARG.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in _SCRATCH_2 prima
;               della chiamata e lo ripristina dopo; se vale 0 il contenuto di
;               ARG non è garantito dopo la chiamata.
;
;               Se il risultato è troppo grande per il formato MBF la ROM
;               segnala OVERFLOW ERROR e non ritorna al chiamante; se è troppo
;               piccolo il risultato è zero. Per la gestione degli errori vedi
;               [NOTA #07 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - FAC (7 byte, ZP)               : Registri da FACEXP a FACOV
;   - .addr_ (1 o 2 byte, Macro)     : Indirizzo del numero
;   - (.addr_) (5 byte, RAM)         : Numero in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Risultato, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - ARISGN (1 byte, ZP)            : Segno aritmetico, bit #7
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, _SCRATCH_2 (se __PRESERVE_ARG = 1),
;                       __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_common.asm>
; ============================================================================
```

2.1.12 m_Subtract_FAC_from_Ptr

```
; ================================================================================
; MACRO: m_Subtract_FAC_from_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Sottrae FAC da un numero in formato MBF che si trova
;               all'indirizzo a cui punta un puntatore e lascia il risultato in
;               FAC: FAC = ((.ptr_)) - FAC.
;
;               Punta .A (LSB) e .Y (MSB) all'indirizzo del numero e chiama la
;               routine FSUB della ROM del BASIC, che lo carica in ARG con
;               CONUPK e aggiorna ARISGN. La routine FSUB altera anche INDEX: il
;               suo contenuto non è garantito dopo la chiamata.
;
;               La routine FSUB della ROM del BASIC altera sempre ARG.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in _SCRATCH_2 prima
;               della chiamata e lo ripristina dopo; se vale 0 il contenuto di
;               ARG non è garantito dopo la chiamata.
;
;               Se il risultato è troppo grande per il formato MBF la ROM
;               segnala OVERFLOW ERROR e non ritorna al chiamante; se è troppo
;               piccolo il risultato è zero. Per la gestione degli errori vedi
;               [NOTA #07 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - FAC (7 byte, ZP)               : Registri da FACEXP a FACOV
;   - .ptr_ (1 o 2 byte, Macro)      : Indirizzo del puntatore
;   - (.ptr_) (2 byte, RAM)          : Indirizzo del numero (LSB, MSB)
;   - ((.ptr_)) (5 byte, RAM)        : Numero in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Risultato, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - ARISGN (1 byte, ZP)            : Segno aritmetico, bit #7
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, _SCRATCH_2 (se __PRESERVE_ARG = 1),
;                       __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_common.asm>
; ================================================================================
```

2.1.13 m_Multiply_FAC_by_ARG

```
; ============================================================================
; MACRO: m_Multiply_FAC_by_ARG
; ----------------------------------------------------------------------------
; Descrizione:  Moltiplica FAC per un numero in formato MBF che si trova in ARG
;               e lascia il risultato in FAC: FAC = FAC x ARG.
;
;               Calcola il segno del risultato in ARISGN (bit #7 = 0 se i segni
;               di FAC e ARG sono uguali, 1 se sono diversi) e chiama la routine
;               FMULTT della ROM del BASIC, che richiede in .A l'esponente di
;               FAC: se è zero, FAC è considerato zero.
;
;               La routine FMULTT della ROM del BASIC non altera ARG, quindi la
;               macro non ha bisogno di preservarlo e l'interruttore
;               __PRESERVE_ARG non ha effetto.
;
;               Se il risultato è troppo grande per il formato MBF la ROM
;               segnala OVERFLOW ERROR e non ritorna al chiamante; se è troppo
;               piccolo il risultato è zero. Per la gestione degli errori vedi
;               [NOTA #07 in <fp/fp.asm>].
;
;               Per l'accuratezza del risultato vedi [NOTA #08 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - FAC (7 byte, ZP)           : Registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)           : Registri da ARGEXP a ARGSGN
;
; Output:
;   - FAC (7 byte, ZP)           : Risultato, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)           : Invariato
;   - ARISGN (1 byte, ZP)        : Segno aritmetico, bit #7
;   - __FP_SAVED_SP (1 byte, ZP) : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)          : Ritorno al chiamante o, se la ROM segnala un
;                                  errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
; ============================================================================
```

2.1.14 m_Multiply_FAC_by_Mem

```
; ============================================================================
; MACRO: m_Multiply_FAC_by_Mem
; ----------------------------------------------------------------------------
; Descrizione:  Moltiplica FAC per un numero in formato MBF che si trova a un
;               indirizzo assoluto e lascia il risultato in FAC:
;               FAC = FAC x (.addr_).
;
;               Punta .A (LSB) e .Y (MSB) all'indirizzo del numero e chiama la
;               routine FMULT della ROM del BASIC, che lo carica in ARG con
;               CONUPK e aggiorna ARISGN. La routine FMULT altera anche INDEX:
;               il suo contenuto non è garantito dopo la chiamata.
;
;               La routine FMULT della ROM del BASIC altera sempre ARG.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in _SCRATCH_2 prima
;               della chiamata e lo ripristina dopo; se vale 0 il contenuto di
;               ARG non è garantito dopo la chiamata.
;
;               Se il risultato è troppo grande per il formato MBF la ROM
;               segnala OVERFLOW ERROR e non ritorna al chiamante; se è troppo
;               piccolo il risultato è zero. Per la gestione degli errori vedi
;               [NOTA #07 in <fp/fp.asm>].
;
;               Per l'accuratezza del risultato vedi [NOTA #08 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - FAC (7 byte, ZP)               : Registri da FACEXP a FACOV
;   - .addr_ (1 o 2 byte, Macro)     : Indirizzo del numero
;   - (.addr_) (5 byte, RAM)         : Numero in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Risultato, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - ARISGN (1 byte, ZP)            : Segno aritmetico, bit #7
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, _SCRATCH_2 (se __PRESERVE_ARG = 1),
;                       __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_common.asm>
```

segue...

2.1.14 m_Multiply_FAC_by_Mem (continua)

```
; ================================================================================
```

2.1.15 m_Multiply_FAC_by_Ptr

```
; ============================================================================
; MACRO: m_Multiply_FAC_by_Ptr
; ----------------------------------------------------------------------------
; Descrizione:  Moltiplica FAC per un numero in formato MBF che si trova
;               all'indirizzo a cui punta un puntatore e lascia il risultato in
;               FAC: FAC = FAC x ((.ptr_)).
;
;               Punta .A (LSB) e .Y (MSB) all'indirizzo del numero e chiama la
;               routine FMULT della ROM del BASIC, che lo carica in ARG con
;               CONUPK e aggiorna ARISGN. La routine FMULT altera anche INDEX:
;               il suo contenuto non è garantito dopo la chiamata.
;
;               La routine FMULT della ROM del BASIC altera sempre ARG.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in _SCRATCH_2 prima
;               della chiamata e lo ripristina dopo; se vale 0 il contenuto di
;               ARG non è garantito dopo la chiamata.
;
;               Se il risultato è troppo grande per il formato MBF la ROM
;               segnala OVERFLOW ERROR e non ritorna al chiamante; se è troppo
;               piccolo il risultato è zero. Per la gestione degli errori vedi
;               [NOTA #07 in <fp/fp.asm>].
;
;               Per l'accuratezza del risultato vedi [NOTA #08 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - FAC (7 byte, ZP)               : Registri da FACEXP a FACOV
;   - .ptr_ (1 o 2 byte, Macro)      : Indirizzo del puntatore
;   - (.ptr_) (2 byte, RAM)          : Indirizzo del numero (LSB, MSB)
;   - ((.ptr_)) (5 byte, RAM)        : Numero in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Risultato, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - ARISGN (1 byte, ZP)            : Segno aritmetico, bit #7
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, _SCRATCH_2 (se __PRESERVE_ARG = 1),
;                       __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
```

segue...

2.1.15 m_Multiply_FAC_by_Ptr (continua)

```
;                       <fp/fp_common.asm>
; ============================================================================
```

2.1.16 m_Divide_ARG_by_FAC

```
; ============================================================================
; MACRO: m_Divide_ARG_by_FAC
; ----------------------------------------------------------------------------
; Descrizione:  Divide un numero in formato MBF che si trova in ARG per FAC e
;               lascia il risultato in FAC: FAC = ARG / FAC.
;
;               Calcola il segno del risultato in ARISGN (bit #7 = 0 se i segni
;               di FAC e ARG sono uguali, 1 se sono diversi) e chiama la routine
;               FDIVT della ROM del BASIC, che richiede in .A l'esponente di
;               FAC: se è zero, FAC è considerato zero.
;
;               La routine FDIVT della ROM del BASIC altera sempre ARG.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in _SCRATCH_2 prima
;               della chiamata e lo ripristina dopo; se vale 0 il contenuto di
;               ARG non è garantito dopo la chiamata.
;
;               Se FAC è zero la ROM segnala DIVISION BY ZERO ERROR e non
;               ritorna al chiamante.
;
;               Se il risultato è troppo grande per il formato MBF la ROM
;               segnala OVERFLOW ERROR e non ritorna al chiamante; se è troppo
;               piccolo il risultato è zero. Per la gestione degli errori vedi
;               [NOTA #07 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - FAC (7 byte, ZP)              : Registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)              : Registri da ARGEXP a ARGSGN
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)              : Risultato, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)              : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - ARISGN (1 byte, ZP)           : Segno aritmetico, bit #7
;   - __FP_SAVED_SP (1 byte, ZP)    : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)             : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, _SCRATCH_2 (se __PRESERVE_ARG = 1),
;                       __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_common.asm>
; ============================================================================
```

2.1.17 m_Divide_Mem_by_FAC

```
; ============================================================================
; MACRO: m_Divide_Mem_by_FAC
; ----------------------------------------------------------------------------
; Descrizione:  Divide un numero in formato MBF che si trova a un indirizzo
;               assoluto per FAC e lascia il risultato in FAC:
;               FAC = (.addr_) / FAC.
;
;               Punta .A (LSB) e .Y (MSB) all'indirizzo del numero e chiama la
;               routine FDIV della ROM del BASIC, che lo carica in ARG con
;               CONUPK e aggiorna ARISGN. La routine FDIV altera anche INDEX: il
;               suo contenuto non è garantito dopo la chiamata.
;
;               La routine FDIV della ROM del BASIC altera sempre ARG.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in _SCRATCH_2 prima
;               della chiamata e lo ripristina dopo; se vale 0 il contenuto di
;               ARG non è garantito dopo la chiamata.
;
;               Se FAC è zero la ROM segnala DIVISION BY ZERO ERROR e non
;               ritorna al chiamante.
;
;               Se il risultato è troppo grande per il formato MBF la ROM
;               segnala OVERFLOW ERROR e non ritorna al chiamante; se è troppo
;               piccolo il risultato è zero. Per la gestione degli errori vedi
;               [NOTA #07 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - FAC (7 byte, ZP)               : Registri da FACEXP a FACOV
;   - .addr_ (1 o 2 byte, Macro)     : Indirizzo del numero
;   - (.addr_) (5 byte, RAM)         : Numero in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Risultato, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - ARISGN (1 byte, ZP)            : Segno aritmetico, bit #7
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, _SCRATCH_2 (se __PRESERVE_ARG = 1),
;                       __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
```

segue...

2.1.17 m_Divide_Mem_by_FAC (continua)

```
;                       <fp/fp_common.asm>
; ============================================================================
```

2.1.18 m_Divide_Ptr_by_FAC

```
; ================================================================================
; MACRO: m_Divide_Ptr_by_FAC
; --------------------------------------------------------------------------------
; Descrizione:  Divide un numero in formato MBF che si trova all'indirizzo a cui
;               punta un puntatore per FAC e lascia il risultato in FAC:
;               FAC = ((.ptr_)) / FAC.
;
;               Punta .A (LSB) e .Y (MSB) all'indirizzo del numero e chiama la
;               routine FDIV della ROM del BASIC, che lo carica in ARG con
;               CONUPK e aggiorna ARISGN. La routine FDIV altera anche INDEX: il
;               suo contenuto non è garantito dopo la chiamata.
;
;               La routine FDIV della ROM del BASIC altera sempre ARG.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in _SCRATCH_2 prima
;               della chiamata e lo ripristina dopo; se vale 0 il contenuto di
;               ARG non è garantito dopo la chiamata.
;
;               Se FAC è zero la ROM segnala DIVISION BY ZERO ERROR e non
;               ritorna al chiamante.
;
;               Se il risultato è troppo grande per il formato MBF la ROM
;               segnala OVERFLOW ERROR e non ritorna al chiamante; se è troppo
;               piccolo il risultato è zero. Per la gestione degli errori vedi
;               [NOTA #07 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - FAC (7 byte, ZP)               : Registri da FACEXP a FACOV
;   - .ptr_ (1 o 2 byte, Macro)      : Indirizzo del puntatore
;   - (.ptr_) (2 byte, RAM)          : Indirizzo del numero (LSB, MSB)
;   - ((.ptr_)) (5 byte, RAM)        : Numero in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Risultato, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - ARISGN (1 byte, ZP)            : Segno aritmetico, bit #7
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, _SCRATCH_2 (se __PRESERVE_ARG = 1),
;                       __FP_SAVED_SP
```

segue...

2.1.18 m_Divide_Ptr_by_FAC (continua)

```
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_common.asm>
; ================================================================================
```

2.1.19 m_AND_FAC_with_ARG

```
; ============================================================================
; MACRO: m_AND_FAC_with_ARG
; ----------------------------------------------------------------------------
; Descrizione:  Esegue un AND bit a bit tra FAC e il numero in formato MBF che è
;               in ARG e lascia il risultato in FAC: FAC = FAC AND ARG.
;
;               Entrambi i numeri sono convertiti in interi a 16 bit con segno,
;               arrotondati per difetto (-1.5 diventa -2); il risultato è
;               riconvertito in virgola mobile.
;
;               Chiama la routine ANDOP della ROM del BASIC. Se FAC o l'altro
;               numero è fuori dall'intervallo da -32768 a 32767, la ROM segnala
;               ILLEGAL QUANTITY ERROR e non ritorna al chiamante: vedi
;               [NOTA #07 in <fp/fp.asm>].
;
;               La ROM altera queste celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07-$08, $0B e $0D (VALTYP, posto
;               a zero).
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - FAC (7 byte, ZP)           : Registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)           : Registri da ARGEXP a ARGSGN
;
; Output:
;   - FAC (7 byte, ZP)           : Risultato, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)           : Invariato
;   - __FP_SAVED_SP (1 byte, ZP) : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)          : Ritorno al chiamante o, se la ROM segnala un
;                                  errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
; ============================================================================
```

2.1.20 m_AND_FAC_with_Mem

```
; ================================================================================
; MACRO: m_AND_FAC_with_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Esegue un AND bit a bit tra FAC e il numero in formato MBF che
;               si trova a un indirizzo assoluto e lascia il risultato in FAC:
;               FAC = FAC AND (.addr_).
;
;               Entrambi i numeri sono convertiti in interi a 16 bit con segno,
;               arrotondati per difetto (-1.5 diventa -2); il risultato è
;               riconvertito in virgola mobile.
;
;               Carica prima il numero in ARG con m_Load_ARG_from_Mem, che
;               aggiorna ARISGN e altera INDEX.
;
;               Chiama la routine ANDOP della ROM del BASIC. Se FAC o l'altro
;               numero è fuori dall'intervallo da -32768 a 32767, la ROM segnala
;               ILLEGAL QUANTITY ERROR e non ritorna al chiamante: vedi
;               [NOTA #07 in <fp/fp.asm>].
;
;               La ROM altera queste celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07-$08, $0B e $0D (VALTYP, posto
;               a zero).
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in _SCRATCH_2 prima di
;               caricare il numero e lo ripristina dopo; se vale 0 ARG contiene
;               il numero caricato. Un ARG uguale a zero (ARGEXP = 0) torna con
;               ARGHO = $80, ed è ancora zero.
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - FAC (7 byte, ZP)               : Registri da FACEXP a FACOV
;   - .addr_ (1 o 2 byte, Macro)     : Indirizzo del numero
;   - (.addr_) (5 byte, RAM)         : Numero in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Risultato, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti contiene il numero
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, _SCRATCH_2 (se __PRESERVE_ARG = 1),
;                       __FP_SAVED_SP
```

segue...

2.1.20 m_AND_FAC_with_Mem (continua)

```
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_common.asm>
;                       <fp/fp_memory.asm>
; ==============================================================================
```

2.1.21 m_AND_FAC_with_Ptr

```
; ================================================================================
; MACRO: m_AND_FAC_with_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Esegue un AND bit a bit tra FAC e il numero in formato MBF che
;               si trova all'indirizzo a cui punta un puntatore e lascia il
;               risultato in FAC: FAC = FAC AND ((.ptr_)).
;
;               Entrambi i numeri sono convertiti in interi a 16 bit con segno,
;               arrotondati per difetto (-1.5 diventa -2); il risultato è
;               riconvertito in virgola mobile.
;
;               Carica prima il numero in ARG con m_Load_ARG_from_Ptr, che
;               aggiorna ARISGN e altera INDEX.
;
;               Chiama la routine ANDOP della ROM del BASIC. Se FAC o l'altro
;               numero è fuori dall'intervallo da -32768 a 32767, la ROM segnala
;               ILLEGAL QUANTITY ERROR e non ritorna al chiamante: vedi
;               [NOTA #07 in <fp/fp.asm>].
;
;               La ROM altera queste celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07-$08, $0B e $0D (VALTYP, posto
;               a zero).
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in _SCRATCH_2 prima di
;               caricare il numero e lo ripristina dopo; se vale 0 ARG contiene
;               il numero caricato. Un ARG uguale a zero (ARGEXP = 0) torna con
;               ARGHO = $80, ed è ancora zero.
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - FAC (7 byte, ZP)               : Registri da FACEXP a FACOV
;   - .ptr_ (1 o 2 byte, Macro)      : Indirizzo del puntatore
;   - (.ptr_) (2 byte, RAM)          : Indirizzo del numero (LSB, MSB)
;   - ((.ptr_)) (5 byte, RAM)        : Numero in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Risultato, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti contiene il numero
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, _SCRATCH_2 (se __PRESERVE_ARG = 1),
```

segue...

2.1.21 m_AND_FAC_with_Ptr (continua)

```
;                         __FP_SAVED_SP
; Dipendenze Esterne:     <c64/symbols.asm>
;                         <fp/fp_common.asm>
;                         <fp/fp_memory.asm>
; ============================================================================
```

2.1.22 m_OR_FAC_with_ARG

```
; ================================================================================
; MACRO: m_OR_FAC_with_ARG
; --------------------------------------------------------------------------------
; Descrizione:  Esegue un OR bit a bit tra FAC e il numero in formato MBF che è
;               in ARG e lascia il risultato in FAC: FAC = FAC OR ARG.
;
;               Entrambi i numeri sono convertiti in interi a 16 bit con segno,
;               arrotondati per difetto (-1.5 diventa -2); il risultato è
;               riconvertito in virgola mobile.
;
;               Chiama la routine OROP della ROM del BASIC. Se FAC o l'altro
;               numero è fuori dall'intervallo da -32768 a 32767, la ROM segnala
;               ILLEGAL QUANTITY ERROR e non ritorna al chiamante: vedi
;               [NOTA #07 in <fp/fp.asm>].
;
;               La ROM altera queste celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07-$08, $0B e $0D (VALTYP, posto
;               a zero).
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - FAC (7 byte, ZP)           : Registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)           : Registri da ARGEXP a ARGSGN
;
; Output:
;   - FAC (7 byte, ZP)           : Risultato, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)           : Invariato
;   - __FP_SAVED_SP (1 byte, ZP) : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)          : Ritorno al chiamante o, se la ROM segnala un
;                                  errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.23 m_OR_FAC_with_Mem

```
; ================================================================================
; MACRO: m_OR_FAC_with_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Esegue un OR bit a bit tra FAC e il numero in formato MBF che si
;               trova a un indirizzo assoluto e lascia il risultato in FAC: FAC
;               = FAC OR (.addr_).
;
;               Entrambi i numeri sono convertiti in interi a 16 bit con segno,
;               arrotondati per difetto (-1.5 diventa -2); il risultato è
;               riconvertito in virgola mobile.
;
;               Carica prima il numero in ARG con m_Load_ARG_from_Mem, che
;               aggiorna ARISGN e altera INDEX.
;
;               Chiama la routine OROP della ROM del BASIC. Se FAC o l'altro
;               numero è fuori dall'intervallo da -32768 a 32767, la ROM segnala
;               ILLEGAL QUANTITY ERROR e non ritorna al chiamante: vedi
;               [NOTA #07 in <fp/fp.asm>].
;
;               La ROM altera queste celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07-$08, $0B e $0D (VALTYP, posto
;               a zero).
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in _SCRATCH_2 prima di
;               caricare il numero e lo ripristina dopo; se vale 0 ARG contiene
;               il numero caricato. Un ARG uguale a zero (ARGEXP = 0) torna con
;               ARGHO = $80, ed è ancora zero.
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - FAC (7 byte, ZP)               : Registri da FACEXP a FACOV
;   - .addr_ (1 o 2 byte, Macro)     : Indirizzo del numero
;   - (.addr_) (5 byte, RAM)         : Numero in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Risultato, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti contiene il numero
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, _SCRATCH_2 (se __PRESERVE_ARG = 1),
;                       __FP_SAVED_SP
```

segue...

2.1.23 m_OR_FAC_with_Mem (continua)

```
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_common.asm>
;                       <fp/fp_memory.asm>
; ==============================================================================
```

2.1.24 m_OR_FAC_with_Ptr

```
; ================================================================================
; MACRO: m_OR_FAC_with_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Esegue un OR bit a bit tra FAC e il numero in formato MBF che si
;               trova all'indirizzo a cui punta un puntatore e lascia il
;               risultato in FAC: FAC = FAC OR ((.ptr_)).
;
;               Entrambi i numeri sono convertiti in interi a 16 bit con segno,
;               arrotondati per difetto (-1.5 diventa -2); il risultato è
;               riconvertito in virgola mobile.
;
;               Carica prima il numero in ARG con m_Load_ARG_from_Ptr, che
;               aggiorna ARISGN e altera INDEX.
;
;               Chiama la routine OROP della ROM del BASIC. Se FAC o l'altro
;               numero è fuori dall'intervallo da -32768 a 32767, la ROM segnala
;               ILLEGAL QUANTITY ERROR e non ritorna al chiamante: vedi
;               [NOTA #07 in <fp/fp.asm>].
;
;               La ROM altera queste celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07-$08, $0B e $0D (VALTYP, posto
;               a zero).
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in _SCRATCH_2 prima di
;               caricare il numero e lo ripristina dopo; se vale 0 ARG contiene
;               il numero caricato. Un ARG uguale a zero (ARGEXP = 0) torna con
;               ARGHO = $80, ed è ancora zero.
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - FAC (7 byte, ZP)               : Registri da FACEXP a FACOV
;   - .ptr_ (1 o 2 byte, Macro)      : Indirizzo del puntatore
;   - (.ptr_) (2 byte, RAM)          : Indirizzo del numero (LSB, MSB)
;   - ((.ptr_)) (5 byte, RAM)        : Numero in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Risultato, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti contiene il numero
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, _SCRATCH_2 (se __PRESERVE_ARG = 1),
```

segue...

2.1.24 m_OR_FAC_with_Ptr (continua)

```
;                         __FP_SAVED_SP
; Dipendenze Esterne:     <c64/symbols.asm>
;                         <fp/fp_common.asm>
;                         <fp/fp_memory.asm>
; ============================================================================
```

2.1.25 m_NOT_FAC

```
; ============================================================================
; MACRO: m_NOT_FAC
; ----------------------------------------------------------------------------
; Descrizione:  Esegue un NOT bit a bit su FAC e lascia il risultato in FAC: FAC
;               = NOT FAC.
;
;               Il numero è convertito in un intero a 16 bit con segno,
;               arrotondato per difetto (-1.5 diventa -2); il risultato, che
;               vale -n-1, è riconvertito in virgola mobile.
;
;               Chiama la routine NOTOP della ROM del BASIC. Se il numero è
;               fuori dall'intervallo da -32768 a 32767, la ROM segnala ILLEGAL
;               QUANTITY ERROR e non ritorna al chiamante: vedi
;               [NOTA #07 in <fp/fp.asm>].
;
;               La ROM altera la cella di lavoro $0D (VALTYP), che viene posta
;               a zero.
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - FAC (7 byte, ZP)           : Registri da FACEXP a FACOV
;
; Output:
;   - FAC (7 byte, ZP)           : Risultato, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)           : Invariato
;   - __FP_SAVED_SP (1 byte, ZP) : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)          : Ritorno al chiamante o, se la ROM segnala un
;                                  errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
; ============================================================================
```

2.1.26 m_NOT_ARG

```
; ============================================================================
; MACRO: m_NOT_ARG
; ----------------------------------------------------------------------------
; Descrizione:  Esegue un NOT bit a bit su ARG e lascia il risultato in ARG,
;               senza alterare FAC: ARG = NOT ARG. È l'unica macro di questo
;               gruppo che lavora sul posto, e non lascia il risultato in FAC.
;
;               Scambia FAC e ARG con m_Swap_FAC_and_ARG, esegue il NOT su FAC e
;               li scambia di nuovo.
;
;               Il numero è convertito in un intero a 16 bit con segno,
;               arrotondato per difetto (-1.5 diventa -2); il risultato, che
;               vale -n-1, è riconvertito in virgola mobile.
;
;               Chiama la routine NOTOP della ROM del BASIC. Se il numero
;               è fuori dall'intervallo da -32768 a 32767, la ROM segnala
;               ILLEGAL QUANTITY ERROR e non ritorna al chiamante: vedi
;               [NOTA #07 in <fp/fp.asm>]. In questo caso FAC e ARG
;               restano scambiati.
;
;               La ROM altera la cella di lavoro $0D (VALTYP), che viene posta
;               a zero.
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - ARG (6 byte, ZP)           : Registri da ARGEXP a ARGSGN
;
; Output:
;   - ARG (6 byte, ZP)           : Risultato, registri da ARGEXP a ARGSGN
;   - FAC (7 byte, ZP)           : Invariato
;   - __FP_SAVED_SP (1 byte, ZP) : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)          : Ritorno al chiamante o, se la ROM segnala un
;                                  errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       _SCRATCH_2, __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_common.asm>
;                       <fp/fp_memory.asm>
; ============================================================================
```

2.1.27 m_NOT_Mem

```
; ============================================================================
; MACRO: m_NOT_Mem
; ----------------------------------------------------------------------------
; Descrizione:  Esegue un NOT bit a bit sul numero in formato MBF che si trova a
;               un indirizzo assoluto e lascia il risultato in FAC, che perde il
;               suo vecchio contenuto.
;
;               Carica prima il numero in FAC con m_Load_FAC_from_Mem, che
;               altera INDEX.
;
;               Il numero è convertito in un intero a 16 bit con segno,
;               arrotondato per difetto (-1.5 diventa -2); il risultato, che
;               vale -n-1, è riconvertito in virgola mobile.
;
;               Chiama la routine NOTOP della ROM del BASIC. Se il numero è
;               fuori dall'intervallo da -32768 a 32767, la ROM segnala ILLEGAL
;               QUANTITY ERROR e non ritorna al chiamante: vedi
;               [NOTA #07 in <fp/fp.asm>].
;
;               La ROM altera la cella di lavoro $0D (VALTYP), che viene posta
;               a zero.
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - .addr_ (1 o 2 byte, Macro) : Indirizzo del numero
;   - (.addr_) (5 byte, RAM)     : Numero in formato MBF
;
; Output:
;   - FAC (7 byte, ZP)           : Risultato, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)           : Invariato
;   - __FP_SAVED_SP (1 byte, ZP) : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)          : Ritorno al chiamante o, se la ROM segnala un
;                                  errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), INDEX,
;                       __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ============================================================================
```

2.1.28 m_NOT_Ptr

```
; ============================================================================
; MACRO: m_NOT_Ptr
; ----------------------------------------------------------------------------
; Descrizione:  Esegue un NOT bit a bit sul numero in formato MBF che si trova
;               all'indirizzo a cui punta un puntatore e lascia il risultato in
;               FAC, che perde il suo vecchio contenuto.
;
;               Carica prima il numero in FAC con m_Load_FAC_from_Ptr, che
;               altera INDEX.
;
;               Il numero è convertito in un intero a 16 bit con segno,
;               arrotondato per difetto (-1.5 diventa -2); il risultato, che
;               vale -n-1, è riconvertito in virgola mobile.
;
;               Chiama la routine NOTOP della ROM del BASIC. Se il numero è
;               fuori dall'intervallo da -32768 a 32767, la ROM segnala ILLEGAL
;               QUANTITY ERROR e non ritorna al chiamante: vedi
;               [NOTA #07 in <fp/fp.asm>].
;
;               La ROM altera la cella di lavoro $0D (VALTYP), che viene posta
;               a zero.
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - .ptr_ (1 o 2 byte, Macro)  : Indirizzo del puntatore
;   - (.ptr_) (2 byte, RAM)      : Indirizzo del numero (LSB, MSB)
;   - ((.ptr_)) (5 byte, RAM)    : Numero in formato MBF
;
; Output:
;   - FAC (7 byte, ZP)           : Risultato, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)           : Invariato
;   - __FP_SAVED_SP (1 byte, ZP) : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)          : Ritorno al chiamante o, se la ROM segnala un
;                                  errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), INDEX,
;                       __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ============================================================================
```

fp_compare.asm

2.1.29 m_Compare_FAC_to_ARG

```
; ============================================================================
; MACRO: m_Compare_FAC_to_ARG
; ----------------------------------------------------------------------------
; Descrizione:  Confronta FAC con il numero in virgola mobile di ARG.
;
;               FCOMP non lavora con ARG, che è in formato non impacchettato:
;               la macro lo scrive in formato MBF in 5 byte a partire da
;               STACK ($0100), con m_Store_ARG_to_Mem, e punta FCOMP a
;               quell'indirizzo. Il contenuto di STACK non è garantito dopo
;               la chiamata.
;
;               Chiama la routine FCOMP della ROM del BASIC, che altera INDEX2:
;               il suo contenuto non è garantito dopo la chiamata.
;
;               Il risultato è in .A: $00 se FAC = numero, $01 se FAC > numero,
;               $FF se FAC < numero. Anche i flag Negative e Zero riflettono .A;
;               il Carry vale 1 se FAC < numero e 0 se FAC > numero, mentre non
;               è garantito se sono uguali.
;
;               Il confronto è esatto su tutti i bit della mantissa, FACOV
;               compreso. FAC e ARG non cambiano; due zeri sono uguali,
;               qualunque sia il loro segno.
;
; Input:
;   - FAC (7 byte, ZP)       : Registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)       : Registri da ARGEXP a ARGSGN
;
; Output:
;   - .A (1 byte, CPU)       : $00 = uguali; $01 = FAC maggiore; $FF = FAC
;                              minore
;   - Flag Carry (1 bit, .P) : 0 = FAC maggiore; 1 = FAC minore (non garantito
;                              se uguali)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       INDEX2, STACK (5 byte)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ============================================================================
```

2.1.30 m_Compare_FAC_to_Mem

```
; ============================================================================
; MACRO: m_Compare_FAC_to_Mem
; ----------------------------------------------------------------------------
; Descrizione:  Confronta FAC con il numero in formato MBF che si trova a un
;               indirizzo assoluto.
;
;               Chiama la routine FCOMP della ROM del BASIC, che altera INDEX2:
;               il suo contenuto non è garantito dopo la chiamata.
;
;               Il risultato è in .A: $00 se FAC = numero, $01 se FAC > numero,
;               $FF se FAC < numero. Anche i flag Negative e Zero riflettono .A;
;               il Carry vale 1 se FAC < numero e 0 se FAC > numero, mentre non
;               è garantito se sono uguali.
;
;               Il confronto è esatto su tutti i bit della mantissa, FACOV
;               compreso. FAC e ARG non cambiano; due zeri sono uguali,
;               qualunque sia il loro segno.
;
; Input:
;   - FAC (7 byte, ZP)           : Registri da FACEXP a FACOV
;   - .addr_ (1 o 2 byte, Macro) : Indirizzo del numero
;   - (.addr_) (5 byte, RAM)     : Numero in formato MBF
;
; Output:
;   - .A (1 byte, CPU)           : $00 = uguali; $01 = FAC maggiore; $FF = FAC
;                                  minore
;   - Flag Carry (1 bit, .P)     : 0 = FAC maggiore; 1 = FAC minore (non
;                                  garantito se uguali)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), INDEX2
; Dipendenze Esterne:   <c64/symbols.asm>
; ============================================================================
```

2.1.31 m_Compare_FAC_to_Ptr

```
; ============================================================================
; MACRO: m_Compare_FAC_to_Ptr
; ----------------------------------------------------------------------------
; Descrizione:  Confronta FAC con il numero in formato MBF che si trova
;               all'indirizzo a cui punta un puntatore.
;
;               Chiama la routine FCOMP della ROM del BASIC, che altera INDEX2:
;               il suo contenuto non è garantito dopo la chiamata.
;
;               Il risultato è in .A: $00 se FAC = numero, $01 se FAC > numero,
;               $FF se FAC < numero. Anche i flag Negative e Zero riflettono .A;
;               il Carry vale 1 se FAC < numero e 0 se FAC > numero, mentre non
;               è garantito se sono uguali.
;
;               Il confronto è esatto su tutti i bit della mantissa, FACOV
;               compreso. FAC e ARG non cambiano; due zeri sono uguali,
;               qualunque sia il loro segno.
;
; Input:
;   - FAC (7 byte, ZP)          : Registri da FACEXP a FACOV
;   - .ptr_ (1 o 2 byte, Macro) : Indirizzo del puntatore
;   - (.ptr_) (2 byte, RAM)     : Indirizzo del numero (LSB, MSB)
;   - ((.ptr_)) (5 byte, RAM)   : Numero in formato MBF
;
; Output:
;   - .A (1 byte, CPU)          : $00 = uguali; $01 = FAC maggiore; $FF = FAC
;                                 minore
;   - Flag Carry (1 bit, .P)    : 0 = FAC maggiore; 1 = FAC minore (non
;                                 garantito se uguali)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), INDEX2
; Dipendenze Esterne:   <c64/symbols.asm>
; ============================================================================
```

2.1.32 m_Load_FAC_with

```
; ============================================================================
; MACRO: m_Load_FAC_with
; ----------------------------------------------------------------------------
; Descrizione:  Carica FAC con un numero in virgola mobile in formato MBF a
;               precisione estesa (5 byte), scrivendo ogni byte con istruzioni
;               lda e sta, senza chiamare la ROM del BASIC. Il valore caricato
;               in FAC è sempre considerato arrotondato: FACOV viene azzerato.
;
;               Quando un byte è uguale al precedente la macro omette il
;               caricamento in .A. Gli operandi devono essere definiti prima
;               dell'invocazione, perché la macro li confronta per scegliere
;               il ramo.
;
; Input:
;   - .exp_ (1 byte, Macro) : Esponente
;   - .ho_ (1 byte, Macro)  : MSB della mantissa, con il segno nel bit #7
;   - .moh_ (1 byte, Macro) : Secondo byte della mantissa
;   - .mo_ (1 byte, Macro)  : Terzo byte della mantissa
;   - .lo_ (1 byte, Macro)  : LSB della mantissa
;
; Output:
;   - FAC (7 byte, ZP)      : Numero caricato, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ============================================================================
```

2.1.33 m_Load_ARG_with

```
; ============================================================================
; MACRO: m_Load_ARG_with
; ----------------------------------------------------------------------------
; Descrizione:  Carica ARG con un numero in virgola mobile in formato MBF a
;               precisione estesa (5 byte), scrivendo ogni byte con istruzioni
;               lda e sta, senza chiamare la ROM del BASIC.
;
;               Quando un byte è uguale al precedente la macro omette il
;               caricamento in .A. Gli operandi devono essere definiti prima
;               dell'invocazione, perché la macro li confronta per scegliere
;               il ramo.
;
; Input:
;   - .exp_ (1 byte, Macro) : Esponente
;   - .ho_ (1 byte, Macro)  : MSB della mantissa, con il segno nel bit #7
;   - .moh_ (1 byte, Macro) : Secondo byte della mantissa
;   - .mo_ (1 byte, Macro)  : Terzo byte della mantissa
;   - .lo_ (1 byte, Macro)  : LSB della mantissa
;
; Output:
;   - ARG (6 byte, ZP)      : Numero caricato, registri da ARGEXP a ARGSGN
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), ARG
; Dipendenze Esterne:   <c64/symbols.asm>
; ============================================================================
```

2.1.34 m_Load_FAC_with_0

```
; ================================================================================
; MACRO: m_Load_FAC_with_0
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con la costante 0. FACOV viene azzerato.
;
;               Scrive i 5 byte della costante ($00, $00, $00, $00, $00) con
;               m_Load_FAC_with, senza chiamare la ROM del BASIC.
;
;               FACEXP vale zero, quindi il valore è zero per la ROM del BASIC;
;               il MSB della mantissa vale $80 (il bit implicito).
;
; Input:
;   - Nessuno
;
; Output:
;   - FAC (7 byte, ZP) : Costante caricata, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.35 m_Load_FAC_with_0_25

```
; ================================================================================
; MACRO: m_Load_FAC_with_0_25
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con la costante 0.25. FACOV viene azzerato.
;
;               Legge la costante da FR4 ($E2EA) nella ROM del BASIC con
;               m_Load_FAC_from_Mem, che altera anche INDEX: il suo contenuto
;               non è garantito dopo la chiamata.
;
; Input:
;   - Nessuno
;
; Output:
;   - FAC (7 byte, ZP) : Costante caricata, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.36 m_Load_FAC_with_0_5

```
; ================================================================================
; MACRO: m_Load_FAC_with_0_5
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con la costante 0.5. FACOV viene azzerato.
;
;               Legge la costante da FHALF ($BF11) nella ROM del BASIC con
;               m_Load_FAC_from_Mem, che altera anche INDEX: il suo contenuto
;               non è garantito dopo la chiamata.
;
; Input:
;   - Nessuno
;
; Output:
;   - FAC (7 byte, ZP) : Costante caricata, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.37 m_Load_FAC_with_1

```
; ================================================================================
; MACRO: m_Load_FAC_with_1
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con la costante 1. FACOV viene azzerato.
;
;               Legge la costante da FONE ($B9BC) nella ROM del BASIC con
;               m_Load_FAC_from_Mem, che altera anche INDEX: il suo contenuto
;               non è garantito dopo la chiamata.
;
; Input:
;   - Nessuno
;
; Output:
;   - FAC (7 byte, ZP) : Costante caricata, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.38 m_Load_FAC_with_MINUS_1

```
; ================================================================================
; MACRO: m_Load_FAC_with_MINUS_1
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con la costante -1. FACOV viene azzerato.
;
;               Scrive i 5 byte della costante ($81, $80, $00, $00, $00) con
;               m_Load_FAC_with, senza chiamare la ROM del BASIC.
;
; Input:
;   - Nessuno
;
; Output:
;   - FAC (7 byte, ZP) : Costante caricata, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.39 m_Load_FAC_with_2

```
; ================================================================================
; MACRO: m_Load_FAC_with_2
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con la costante 2. FACOV viene azzerato.
;
;               Scrive i 5 byte della costante ($82, $00, $00, $00, $00) con
;               m_Load_FAC_with, senza chiamare la ROM del BASIC.
;
; Input:
;   - Nessuno
;
; Output:
;   - FAC (7 byte, ZP) : Costante caricata, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.40 m_Load_FAC_with_10

```
; ================================================================================
; MACRO: m_Load_FAC_with_10
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con la costante 10. FACOV viene azzerato.
;
;               Legge la costante da TENC ($BAF9) nella ROM del BASIC con
;               m_Load_FAC_from_Mem, che altera anche INDEX: il suo contenuto
;               non è garantito dopo la chiamata.
;
; Input:
;   - Nessuno
;
; Output:
;   - FAC (7 byte, ZP) : Costante caricata, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.41 m_Load_FAC_with_0_1

```
; ================================================================================
; MACRO: m_Load_FAC_with_0_1
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con la costante 0.1. FACOV viene azzerato.
;
;               Scrive i 5 byte della costante ($7D, $4C, $CC, $CC, $CD) con
;               m_Load_FAC_with, senza chiamare la ROM del BASIC.
;
; Input:
;   - Nessuno
;
; Output:
;   - FAC (7 byte, ZP) : Costante caricata, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.42 m_Load_FAC_with_PI4

```
; ============================================================================
; MACRO: m_Load_FAC_with_PI4
; ----------------------------------------------------------------------------
; Descrizione:  Carica FAC con la costante π/4. FACOV viene azzerato.
;
;               Carica π/2 dalla ROM del BASIC con m_Load_FAC_from_Mem, che
;               altera anche INDEX: il suo contenuto non è garantito dopo
;               la chiamata. Poi decrementa l'esponente di FAC, che dimezza
;               il valore.
;
; Input:
;   - Nessuno
;
; Output:
;   - FAC (7 byte, ZP) : Costante caricata, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry), FACEXP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ============================================================================
```

2.1.43 m_Load_FAC_with_PI2

```
; ================================================================================
; MACRO: m_Load_FAC_with_PI2
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con la costante π/2. FACOV viene azzerato.
;
;               Legge la costante da PI2 ($E2E0) nella ROM del BASIC con
;               m_Load_FAC_from_Mem, che altera anche INDEX: il suo contenuto
;               non è garantito dopo la chiamata.
;
; Input:
;   - Nessuno
;
; Output:
;   - FAC (7 byte, ZP) : Costante caricata, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.44 m_Load_FAC_with_PI

```
; ================================================================================
; MACRO: m_Load_FAC_with_PI
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con la costante π. FACOV viene azzerato.
;
;               Legge la costante da PIVAL ($AEA8) nella ROM del BASIC con
;               m_Load_FAC_from_Mem, che altera anche INDEX: il suo contenuto
;               non è garantito dopo la chiamata.
;
;               Il valore della ROM non è il più vicino a π:
;               vedi [NOTA #17 in <fp/fp.asm>].
;
; Input:
;   - Nessuno
;
; Output:
;   - FAC (7 byte, ZP) : Costante caricata, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.45 m_Load_FAC_with_2PI

```
; ================================================================================
; MACRO: m_Load_FAC_with_2PI
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con la costante 2π. FACOV viene azzerato.
;
;               Legge la costante da TWOPI ($E2E5) nella ROM del BASIC con
;               m_Load_FAC_from_Mem, che altera anche INDEX: il suo contenuto
;               non è garantito dopo la chiamata.
;
; Input:
;   - Nessuno
;
; Output:
;   - FAC (7 byte, ZP) : Costante caricata, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.46 m_Load_FAC_with_PI180

```
; ================================================================================
; MACRO: m_Load_FAC_with_PI180
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con la costante π/180. FACOV viene azzerato.
;
;               Scrive i 5 byte della costante ($7B, $0E, $FA, $35, $13) con
;               m_Load_FAC_with, senza chiamare la ROM del BASIC.
;
; Input:
;   - Nessuno
;
; Output:
;   - FAC (7 byte, ZP) : Costante caricata, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.47 m_Load_FAC_with_PI200

```
; ================================================================================
; MACRO: m_Load_FAC_with_PI200
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con la costante π/200. FACOV viene azzerato.
;
;               Scrive i 5 byte della costante ($7B, $00, $AD, $FC, $91) con
;               m_Load_FAC_with, senza chiamare la ROM del BASIC.
;
; Input:
;   - Nessuno
;
; Output:
;   - FAC (7 byte, ZP) : Costante caricata, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.48 m_Load_FAC_with_180PI

```
; ================================================================================
; MACRO: m_Load_FAC_with_180PI
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con la costante 180/π. FACOV viene azzerato.
;
;               Scrive i 5 byte della costante ($86, $65, $2E, $E0, $D3) con
;               m_Load_FAC_with, senza chiamare la ROM del BASIC.
;
; Input:
;   - Nessuno
;
; Output:
;   - FAC (7 byte, ZP) : Costante caricata, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.49 m_Load_FAC_with_200PI

```
; ================================================================================
; MACRO: m_Load_FAC_with_200PI
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con la costante 200/π. FACOV viene azzerato.
;
;               Scrive i 5 byte della costante ($86, $7E, $A5, $DD, $5C) con
;               m_Load_FAC_with, senza chiamare la ROM del BASIC.
;
; Input:
;   - Nessuno
;
; Output:
;   - FAC (7 byte, ZP) : Costante caricata, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.50 m_Load_FAC_with_SQR2

```
; ================================================================================
; MACRO: m_Load_FAC_with_SQR2
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con la costante √2. FACOV viene azzerato.
;
;               Legge la costante da FSQR2 ($B9DB) nella ROM del BASIC con
;               m_Load_FAC_from_Mem, che altera anche INDEX: il suo contenuto
;               non è garantito dopo la chiamata.
;
; Input:
;   - Nessuno
;
; Output:
;   - FAC (7 byte, ZP) : Costante caricata, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.51 m_Load_FAC_with_SQR3

```
; ================================================================================
; MACRO: m_Load_FAC_with_SQR3
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con la costante √3. FACOV viene azzerato.
;
;               Scrive i 5 byte della costante ($81, $5D, $B3, $D7, $43) con
;               m_Load_FAC_with, senza chiamare la ROM del BASIC.
;
; Input:
;   - Nessuno
;
; Output:
;   - FAC (7 byte, ZP) : Costante caricata, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.52 m_Load_FAC_with_e

```
; ================================================================================
; MACRO: m_Load_FAC_with_e
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con la costante e. FACOV viene azzerato.
;
;               Scrive i 5 byte della costante ($82, $2D, $F8, $54, $59) con
;               m_Load_FAC_with, senza chiamare la ROM del BASIC.
;
; Input:
;   - Nessuno
;
; Output:
;   - FAC (7 byte, ZP) : Costante caricata, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.53 m_Load_FAC_with_LOG2

```
; ================================================================================
; MACRO: m_Load_FAC_with_LOG2
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con la costante ln 2. FACOV viene azzerato.
;
;               Legge la costante da FLOG2 ($B9E5) nella ROM del BASIC con
;               m_Load_FAC_from_Mem, che altera anche INDEX: il suo contenuto
;               non è garantito dopo la chiamata.
;
; Input:
;   - Nessuno
;
; Output:
;   - FAC (7 byte, ZP) : Costante caricata, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.54 m_Load_FAC_with_LOG10

```
; ================================================================================
; MACRO: m_Load_FAC_with_LOG10
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con la costante ln 10. FACOV viene azzerato.
;
;               Scrive i 5 byte della costante ($82, $13, $5D, $8D, $DE) con
;               m_Load_FAC_with, senza chiamare la ROM del BASIC.
;
; Input:
;   - Nessuno
;
; Output:
;   - FAC (7 byte, ZP) : Costante caricata, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.55 m_Load_FAC_with_eps

```
; ================================================================================
; MACRO: m_Load_FAC_with_eps
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con l'epsilon di macchina. FACOV viene azzerato.
;
;               Scrive i 5 byte della costante ($62, $00, $00, $00, $00) con
;               m_Load_FAC_with, senza chiamare la ROM del BASIC.
;
;               Vale 4.65661287*10^-10 ed è la differenza tra 1 e il numero in
;               virgola mobile successivo.
;
; Input:
;   - Nessuno
;
; Output:
;   - FAC (7 byte, ZP) : Costante caricata, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.56 m_Load_FAC_with_MINR

```
; ================================================================================
; MACRO: m_Load_FAC_with_MINR
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con il più piccolo numero rappresentabile (MINR).
;               FACOV viene azzerato.
;
;               Scrive i 5 byte della costante ($01, $00, $00, $00, $00) con
;               m_Load_FAC_with, senza chiamare la ROM del BASIC.
;
;               Vale 2.93873588*10^-39 ed è il più piccolo numero diverso da
;               zero rappresentabile in formato MBF.
;
; Input:
;   - Nessuno
;
; Output:
;   - FAC (7 byte, ZP) : Costante caricata, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.57 m_Load_FAC_with_MAXR

```
; ================================================================================
; MACRO: m_Load_FAC_with_MAXR
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con il più grande numero rappresentabile (MAXR).
;               FACOV viene azzerato.
;
;               Scrive i 5 byte della costante ($FF, $7F, $FF, $FF, $FF) con
;               m_Load_FAC_with, senza chiamare la ROM del BASIC.
;
;               Vale 1.70141183*10^38 ed è il più grande numero rappresentabile
;               in formato MBF.
;
; Input:
;   - Nessuno
;
; Output:
;   - FAC (7 byte, ZP) : Costante caricata, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.58 m_Load_ARG_with_0

```
; ================================================================================
; MACRO: m_Load_ARG_with_0
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con la costante 0. FAC non cambia.
;
;               Scrive i 5 byte della costante ($00, $00, $00, $00, $00) con
;               m_Load_ARG_with, senza chiamare la ROM del BASIC.
;
;               FACEXP vale zero, quindi il valore è zero per la ROM del BASIC;
;               il MSB della mantissa vale $80 (il bit implicito).
;
; Input:
;   - Nessuno
;
; Output:
;   - ARG (6 byte, ZP) : Costante caricata, registri da ARGEXP a ARGSGN
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), ARG
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.59 m_Load_ARG_with_0_25

```
; ================================================================================
; MACRO: m_Load_ARG_with_0_25
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con la costante 0.25. FAC non cambia.
;
;               Legge la costante da FR4 ($E2EA) nella ROM del BASIC con
;               m_Load_ARG_from_Mem, che altera anche INDEX: il suo contenuto
;               non è garantito dopo la chiamata.
;
;               La routine CONUPK della ROM aggiorna anche ARISGN: il bit #7
;               vale 1 se i segni di FAC e di ARG sono diversi.
;
; Input:
;   - Nessuno
;
; Output:
;   - ARG (6 byte, ZP)    : Costante caricata, registri da ARGEXP a ARGSGN
;   - ARISGN (1 byte, ZP) : Bit #7: 0 = segni uguali; 1 = segni diversi
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.60 m_Load_ARG_with_0_5

```
; ================================================================================
; MACRO: m_Load_ARG_with_0_5
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con la costante 0.5. FAC non cambia.
;
;               Legge la costante da FHALF ($BF11) nella ROM del BASIC con
;               m_Load_ARG_from_Mem, che altera anche INDEX: il suo contenuto
;               non è garantito dopo la chiamata.
;
;               La routine CONUPK della ROM aggiorna anche ARISGN: il bit #7
;               vale 1 se i segni di FAC e di ARG sono diversi.
;
; Input:
;   - Nessuno
;
; Output:
;   - ARG (6 byte, ZP)    : Costante caricata, registri da ARGEXP a ARGSGN
;   - ARISGN (1 byte, ZP) : Bit #7: 0 = segni uguali; 1 = segni diversi
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.61 m_Load_ARG_with_1

```
; ================================================================================
; MACRO: m_Load_ARG_with_1
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con la costante 1. FAC non cambia.
;
;               Legge la costante da FONE ($B9BC) nella ROM del BASIC con
;               m_Load_ARG_from_Mem, che altera anche INDEX: il suo contenuto
;               non è garantito dopo la chiamata.
;
;               La routine CONUPK della ROM aggiorna anche ARISGN: il bit #7
;               vale 1 se i segni di FAC e di ARG sono diversi.
;
; Input:
;   - Nessuno
;
; Output:
;   - ARG (6 byte, ZP)    : Costante caricata, registri da ARGEXP a ARGSGN
;   - ARISGN (1 byte, ZP) : Bit #7: 0 = segni uguali; 1 = segni diversi
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.62 m_Load_ARG_with_MINUS_1

```
; ================================================================================
; MACRO: m_Load_ARG_with_MINUS_1
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con la costante -1. FAC non cambia.
;
;               Scrive i 5 byte della costante ($81, $80, $00, $00, $00) con
;               m_Load_ARG_with, senza chiamare la ROM del BASIC.
;
; Input:
;   - Nessuno
;
; Output:
;   - ARG (6 byte, ZP) : Costante caricata, registri da ARGEXP a ARGSGN
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), ARG
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.63 m_Load_ARG_with_2

```
; ================================================================================
; MACRO: m_Load_ARG_with_2
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con la costante 2. FAC non cambia.
;
;               Scrive i 5 byte della costante ($82, $00, $00, $00, $00) con
;               m_Load_ARG_with, senza chiamare la ROM del BASIC.
;
; Input:
;   - Nessuno
;
; Output:
;   - ARG (6 byte, ZP) : Costante caricata, registri da ARGEXP a ARGSGN
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), ARG
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.64 m_Load_ARG_with_10

```
; ================================================================================
; MACRO: m_Load_ARG_with_10
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con la costante 10. FAC non cambia.
;
;               Legge la costante da TENC ($BAF9) nella ROM del BASIC con
;               m_Load_ARG_from_Mem, che altera anche INDEX: il suo contenuto
;               non è garantito dopo la chiamata.
;
;               La routine CONUPK della ROM aggiorna anche ARISGN: il bit #7
;               vale 1 se i segni di FAC e di ARG sono diversi.
;
; Input:
;   - Nessuno
;
; Output:
;   - ARG (6 byte, ZP)    : Costante caricata, registri da ARGEXP a ARGSGN
;   - ARISGN (1 byte, ZP) : Bit #7: 0 = segni uguali; 1 = segni diversi
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.65 m_Load_ARG_with_0_1

```
; ================================================================================
; MACRO: m_Load_ARG_with_0_1
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con la costante 0.1. FAC non cambia.
;
;               Scrive i 5 byte della costante ($7D, $4C, $CC, $CC, $CD) con
;               m_Load_ARG_with, senza chiamare la ROM del BASIC.
;
; Input:
;   - Nessuno
;
; Output:
;   - ARG (6 byte, ZP) : Costante caricata, registri da ARGEXP a ARGSGN
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), ARG
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.66 m_Load_ARG_with_PI4

```
; ================================================================================
; MACRO: m_Load_ARG_with_PI4
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con la costante π/4. FAC non cambia.
;
;               Carica π/2 dalla ROM del BASIC con m_Load_ARG_from_Mem, che
;               altera anche INDEX: il suo contenuto non è garantito dopo
;               la chiamata. Poi decrementa l'esponente di ARG, che dimezza
;               il valore.
;
;               La routine CONUPK della ROM aggiorna anche ARISGN: il bit #7
;               vale 1 se i segni di FAC e di ARG sono diversi.
;
; Input:
;   - Nessuno
;
; Output:
;   - ARG (6 byte, ZP)    : Costante caricata, registri da ARGEXP a ARGSGN
;   - ARISGN (1 byte, ZP) : Bit #7: 0 = segni uguali; 1 = segni diversi
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry), ARGEXP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.67 m_Load_ARG_with_PI2

```
; ================================================================================
; MACRO: m_Load_ARG_with_PI2
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con la costante π/2. FAC non cambia.
;
;               Legge la costante da PI2 ($E2E0) nella ROM del BASIC con
;               m_Load_ARG_from_Mem, che altera anche INDEX: il suo contenuto
;               non è garantito dopo la chiamata.
;
;               La routine CONUPK della ROM aggiorna anche ARISGN: il bit #7
;               vale 1 se i segni di FAC e di ARG sono diversi.
;
; Input:
;   - Nessuno
;
; Output:
;   - ARG (6 byte, ZP)    : Costante caricata, registri da ARGEXP a ARGSGN
;   - ARISGN (1 byte, ZP) : Bit #7: 0 = segni uguali; 1 = segni diversi
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.68 m_Load_ARG_with_PI

```
; ============================================================================
; MACRO: m_Load_ARG_with_PI
; ----------------------------------------------------------------------------
; Descrizione:  Carica ARG con la costante π. FAC non cambia.
;
;               Legge la costante da PIVAL ($AEA8) nella ROM del BASIC con
;               m_Load_ARG_from_Mem, che altera anche INDEX: il suo contenuto
;               non è garantito dopo la chiamata.
;
;               Il valore della ROM non è il più vicino a π:
;               vedi [NOTA #17 in <fp/fp.asm>].
;
;               La routine CONUPK della ROM aggiorna anche ARISGN: il bit #7
;               vale 1 se i segni di FAC e di ARG sono diversi.
;
; Input:
;   - Nessuno
;
; Output:
;   - ARG (6 byte, ZP)    : Costante caricata, registri da ARGEXP a ARGSGN
;   - ARISGN (1 byte, ZP) : Bit #7: 0 = segni uguali; 1 = segni diversi
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ============================================================================
```

2.1.69 m_Load_ARG_with_2PI

```
; ================================================================================
; MACRO: m_Load_ARG_with_2PI
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con la costante 2π. FAC non cambia.
;
;               Legge la costante da TWOPI ($E2E5) nella ROM del BASIC con
;               m_Load_ARG_from_Mem, che altera anche INDEX: il suo contenuto
;               non è garantito dopo la chiamata.
;
;               La routine CONUPK della ROM aggiorna anche ARISGN: il bit #7
;               vale 1 se i segni di FAC e di ARG sono diversi.
;
; Input:
;   - Nessuno
;
; Output:
;   - ARG (6 byte, ZP)    : Costante caricata, registri da ARGEXP a ARGSGN
;   - ARISGN (1 byte, ZP) : Bit #7: 0 = segni uguali; 1 = segni diversi
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.70 m_Load_ARG_with_PI180

```
; ================================================================================
; MACRO: m_Load_ARG_with_PI180
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con la costante π/180. FAC non cambia.
;
;               Scrive i 5 byte della costante ($7B, $0E, $FA, $35, $13) con
;               m_Load_ARG_with, senza chiamare la ROM del BASIC.
;
; Input:
;   - Nessuno
;
; Output:
;   - ARG (6 byte, ZP) : Costante caricata, registri da ARGEXP a ARGSGN
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), ARG
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.71 m_Load_ARG_with_PI200

```
; ================================================================================
; MACRO: m_Load_ARG_with_PI200
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con la costante π/200. FAC non cambia.
;
;               Scrive i 5 byte della costante ($7B, $00, $AD, $FC, $91) con
;               m_Load_ARG_with, senza chiamare la ROM del BASIC.
;
; Input:
;   - Nessuno
;
; Output:
;   - ARG (6 byte, ZP) : Costante caricata, registri da ARGEXP a ARGSGN
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), ARG
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.72 m_Load_ARG_with_180PI

```
; ============================================================================
; MACRO: m_Load_ARG_with_180PI
; ----------------------------------------------------------------------------
; Descrizione:  Carica ARG con la costante 180/π. FAC non cambia.
;
;               Scrive i 5 byte della costante ($86, $65, $2E, $E0, $D3) con
;               m_Load_ARG_with, senza chiamare la ROM del BASIC.
;
; Input:
;   - Nessuno
;
; Output:
;   - ARG (6 byte, ZP) : Costante caricata, registri da ARGEXP a ARGSGN
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), ARG
; Dipendenze Esterne:   <c64/symbols.asm>
; ============================================================================
```

2.1.73 m_Load_ARG_with_200PI

```
; ================================================================================
; MACRO: m_Load_ARG_with_200PI
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con la costante 200/π. FAC non cambia.
;
;               Scrive i 5 byte della costante ($86, $7E, $A5, $DD, $5C) con
;               m_Load_ARG_with, senza chiamare la ROM del BASIC.
;
; Input:
;   - Nessuno
;
; Output:
;   - ARG (6 byte, ZP) : Costante caricata, registri da ARGEXP a ARGSGN
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), ARG
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.74 m_Load_ARG_with_SQR2

```
; ================================================================================
; MACRO: m_Load_ARG_with_SQR2
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con la costante √2. FAC non cambia.
;
;               Legge la costante da FSQR2 ($B9DB) nella ROM del BASIC con
;               m_Load_ARG_from_Mem, che altera anche INDEX: il suo contenuto
;               non è garantito dopo la chiamata.
;
;               La routine CONUPK della ROM aggiorna anche ARISGN: il bit #7
;               vale 1 se i segni di FAC e di ARG sono diversi.
;
; Input:
;   - Nessuno
;
; Output:
;   - ARG (6 byte, ZP)    : Costante caricata, registri da ARGEXP a ARGSGN
;   - ARISGN (1 byte, ZP) : Bit #7: 0 = segni uguali; 1 = segni diversi
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.75 m_Load_ARG_with_SQR3

```
; ============================================================================
; MACRO: m_Load_ARG_with_SQR3
; ----------------------------------------------------------------------------
; Descrizione:  Carica ARG con la costante √3. FAC non cambia.
;
;               Scrive i 5 byte della costante ($81, $5D, $B3, $D7, $43) con
;               m_Load_ARG_with, senza chiamare la ROM del BASIC.
;
; Input:
;   - Nessuno
;
; Output:
;   - ARG (6 byte, ZP) : Costante caricata, registri da ARGEXP a ARGSGN
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), ARG
; Dipendenze Esterne:   <c64/symbols.asm>
; ============================================================================
```

2.1.76 m_Load_ARG_with_e

```
; ================================================================================
; MACRO: m_Load_ARG_with_e
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con la costante e. FAC non cambia.
;
;               Scrive i 5 byte della costante ($82, $2D, $F8, $54, $59) con
;               m_Load_ARG_with, senza chiamare la ROM del BASIC.
;
; Input:
;   - Nessuno
;
; Output:
;   - ARG (6 byte, ZP) : Costante caricata, registri da ARGEXP a ARGSGN
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), ARG
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.77 m_Load_ARG_with_LOG2

```
; ================================================================================
; MACRO: m_Load_ARG_with_LOG2
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con la costante ln 2. FAC non cambia.
;
;               Legge la costante da FLOG2 ($B9E5) nella ROM del BASIC con
;               m_Load_ARG_from_Mem, che altera anche INDEX: il suo contenuto
;               non è garantito dopo la chiamata.
;
;               La routine CONUPK della ROM aggiorna anche ARISGN: il bit #7
;               vale 1 se i segni di FAC e di ARG sono diversi.
;
; Input:
;   - Nessuno
;
; Output:
;   - ARG (6 byte, ZP)    : Costante caricata, registri da ARGEXP a ARGSGN
;   - ARISGN (1 byte, ZP) : Bit #7: 0 = segni uguali; 1 = segni diversi
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.78 m_Load_ARG_with_LOG10

```
; ================================================================================
; MACRO: m_Load_ARG_with_LOG10
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con la costante ln 10. FAC non cambia.
;
;               Scrive i 5 byte della costante ($82, $13, $5D, $8D, $DE) con
;               m_Load_ARG_with, senza chiamare la ROM del BASIC.
;
; Input:
;   - Nessuno
;
; Output:
;   - ARG (6 byte, ZP) : Costante caricata, registri da ARGEXP a ARGSGN
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), ARG
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.79 m_Load_ARG_with_eps

```
; ================================================================================
; MACRO: m_Load_ARG_with_eps
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con l'epsilon di macchina. FAC non cambia.
;
;               Scrive i 5 byte della costante ($62, $00, $00, $00, $00) con
;               m_Load_ARG_with, senza chiamare la ROM del BASIC.
;
;               Vale 4.65661287*10^-10 ed è la differenza tra 1 e il numero in
;               virgola mobile successivo.
;
; Input:
;   - Nessuno
;
; Output:
;   - ARG (6 byte, ZP) : Costante caricata, registri da ARGEXP a ARGSGN
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), ARG
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.80 m_Load_ARG_with_MINR

```
; ================================================================================
; MACRO: m_Load_ARG_with_MINR
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con il più piccolo numero rappresentabile (MINR). FAC
;               non cambia.
;
;               Scrive i 5 byte della costante ($01, $00, $00, $00, $00) con
;               m_Load_ARG_with, senza chiamare la ROM del BASIC.
;
;               Vale 2.93873588*10^-39 ed è il più piccolo numero diverso da
;               zero rappresentabile in formato MBF.
;
; Input:
;   - Nessuno
;
; Output:
;   - ARG (6 byte, ZP) : Costante caricata, registri da ARGEXP a ARGSGN
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), ARG
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.81 m_Load_ARG_with_MAXR

```
; ================================================================================
; MACRO: m_Load_ARG_with_MAXR
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con il più grande numero rappresentabile (MAXR). FAC
;               non cambia.
;
;               Scrive i 5 byte della costante ($FF, $7F, $FF, $FF, $FF) con
;               m_Load_ARG_with, senza chiamare la ROM del BASIC.
;
;               Vale 1.70141183*10^38 ed è il più grande numero rappresentabile
;               in formato MBF.
;
; Input:
;   - Nessuno
;
; Output:
;   - ARG (6 byte, ZP) : Costante caricata, registri da ARGEXP a ARGSGN
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), ARG
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

fp_error.asm

2.1.82 FP_ERROR_HANDLER

```
; ================================================================================
; SUBROUTINE: FP_ERROR_HANDLER
; --------------------------------------------------------------------------------
; Descrizione:  Gestore delle condizioni d'errore che le routine in virgola
;               mobile della ROM possono segnalare. È pensato per essere
;               installato nel vettore IERROR, al posto del gestore del BASIC:
;               la ROM carica in .X il codice d'errore e salta all'indirizzo
;               contenuto in IERROR.
;
;               Le macro che possono provocare un errore salvano .S in
;               __FP_SAVED_SP prima di chiamare la ROM. Il gestore lo
;               ripristina, meno i 2 byte dell'indirizzo di ritorno della
;               chiamata, così l'rts finale ritorna alla macro subito dopo
;               la chiamata, qualunque sia la profondità dello stack nella
;               ROM. Per la macro lo stack resta quindi com'era prima della
;               chiamata. Il gestore è valido solo per gli errori segnalati
;               durante l'esecuzione di queste macro.
;
;               Le condizioni sono tre: ILLEGAL QUANTITY, OVERFLOW e
;               DIVISION BY ZERO. Per ciascuna c'è un punto di gestione
;               distinto; per il momento tutti e tre pongono a 1 il Carry e
;               terminano con rts. Con qualunque altro codice, che non
;               dovrebbe presentarsi, la subroutine pone a 0 il Carry e ritorna.
;
; Input:
;   - .X (1 byte, CPU)           : Codice d'errore (ERR_ILLEGAL_QUANTITY,
;                                  ERR_OVERFLOW o ERR_DIVISION_BY_ZERO)
;   - __FP_SAVED_SP (1 byte, ZP) : Valore di .S prima della chiamata alla ROM
;
; Output:
;   - Flag Carry (1 bit, .P)     : 0 = altro codice; 1 = condizione prevista
;   - .PC (2 byte, CPU)          : Ritorno alla macro che ha chiamato la ROM,
;                                  subito dopo la chiamata
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .P (Negative, Zero, Carry), .S
; Dipendenze Esterne:   <c64/errors.asm>
; ================================================================================
```

fp_integers.asm

2.1.83 m_Load_FAC_with_UINT8

```
; ================================================================================
; MACRO: m_Load_FAC_with_UINT8
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con un intero senza segno a 8 bit, passato come
;               costante, e azzera FACOV.
;
;               I bit del valore oltre il bit #7 sono ignorati.
;
;               Converte il numero con la routine FLOATB della ROM del BASIC,
;               che normalizza FAC e prende il segno dal Carry: la macro lo
;               imposta a 1 (numero positivo). ARG non cambia.
;
; Input:
;   - .value_ (1 byte, Macro) : Valore da caricare, senza segno
;
; Output:
;   - FAC (7 byte, ZP)        : Intero convertito, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.84 m_Load_FAC_with_UINT8_Mem

```
; ================================================================================
; MACRO: m_Load_FAC_with_UINT8_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con un intero senza segno a 8 bit, letto da un
;               indirizzo assoluto, a partire dal LSB (little-endian), e
;               azzera FACOV.
;
;               Converte il numero con la routine FLOATB della ROM del BASIC,
;               che normalizza FAC e prende il segno dal Carry: la macro lo
;               imposta a 1 (numero positivo). ARG non cambia.
;
; Input:
;   - .addr_ (1 o 2 byte, Macro) : Indirizzo dell'intero
;   - (.addr_) (1 byte, RAM)     : Intero, LSB per primo
;
; Output:
;   - FAC (7 byte, ZP)           : Intero convertito, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.85 m_Load_FAC_with_UINT8_Ptr

```
; ================================================================================
; MACRO: m_Load_FAC_with_UINT8_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con un intero senza segno a 8 bit, letto
;               dall'indirizzo indicato da un puntatore in ZP, LSB per primo, e
;               azzera FACOV.
;
;               Converte il numero con la routine FLOATB della ROM del BASIC,
;               che normalizza FAC e prende il segno dal Carry: la macro lo
;               imposta a 1 (numero positivo). ARG non cambia.
;
; Input:
;   - .ptr_ (1 byte, Macro)   : Indirizzo in ZP del puntatore
;   - (.ptr_) (2 byte, ZP)    : Indirizzo dell'intero (LSB, MSB)
;   - ((.ptr_)) (1 byte, RAM) : Intero, LSB per primo
;
; Output:
;   - FAC (7 byte, ZP)        : Intero convertito, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.86 m_Load_ARG_with_UINT8

```
; ================================================================================
; MACRO: m_Load_ARG_with_UINT8
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con un intero senza segno a 8 bit, passato
;               come costante.
;
;               I bit del valore oltre il bit #7 sono ignorati.
;
;               Scambia FAC e ARG con m_Swap_FAC_and_ARG, carica FAC con
;               la macro m_Load_FAC_with_UINT8 e li scambia di nuovo. Dopo
;               la chiamata FAC contiene il valore di prima, arrotondato,
;               e FACOV vale zero; il vecchio ARG si perde e ARISGN non
;               viene aggiornato.
;
;               Il contenuto di _SCRATCH_2 non è garantito dopo la chiamata.
;
; Input:
;   - .value_ (1 byte, Macro) : Valore da caricare, senza segno
;   - FAC (7 byte, ZP)        : Valore da conservare, registri da FACEXP a FACOV
;
; Output:
;   - ARG (6 byte, ZP)        : Intero convertito, registri da ARGEXP a ARGSGN
;   - FAC (7 byte, ZP)        : Valore di prima, arrotondato, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC,
;                       _SCRATCH_2
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.87 m_Load_ARG_with_UINT8_Mem

```
; ================================================================================
; MACRO: m_Load_ARG_with_UINT8_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con un intero senza segno a 8 bit, letto da un
;               indirizzo assoluto, a partire dal LSB (little-endian).
;
;               Scambia FAC e ARG con m_Swap_FAC_and_ARG, carica FAC con
;               la macro m_Load_FAC_with_UINT8_Mem e li scambia di nuovo.
;               Dopo la chiamata FAC contiene il valore di prima,
;               arrotondato, e FACOV vale zero; il vecchio ARG si perde e
;               ARISGN non viene aggiornato.
;
;               Il contenuto di _SCRATCH_2 non è garantito dopo la chiamata.
;
; Input:
;   - .addr_ (1 o 2 byte, Macro) : Indirizzo dell'intero
;   - (.addr_) (1 byte, RAM)     : Intero, LSB per primo
;   - FAC (7 byte, ZP)           : Valore da conservare, registri da FACEXP a
;                                  FACOV
;
; Output:
;   - ARG (6 byte, ZP)           : Intero convertito, registri da ARGEXP a
;                                  ARGSGN
;   - FAC (7 byte, ZP)           : Valore di prima, arrotondato, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC,
;                       _SCRATCH_2
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.88 m_Load_ARG_with_UINT8_Ptr

```
; ================================================================================
; MACRO: m_Load_ARG_with_UINT8_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con un intero senza segno a 8 bit, letto
;               dall'indirizzo indicato da un puntatore in ZP, LSB per primo.
;
;               Scambia FAC e ARG con m_Swap_FAC_and_ARG, carica FAC con
;               la macro m_Load_FAC_with_UINT8_Ptr e li scambia di nuovo.
;               Dopo la chiamata FAC contiene il valore di prima,
;               arrotondato, e FACOV vale zero; il vecchio ARG si perde e
;               ARISGN non viene aggiornato.
;
;               Il contenuto di _SCRATCH_2 non è garantito dopo la chiamata.
;
; Input:
;   - .ptr_ (1 byte, Macro)   : Indirizzo in ZP del puntatore
;   - (.ptr_) (2 byte, ZP)    : Indirizzo dell'intero (LSB, MSB)
;   - ((.ptr_)) (1 byte, RAM) : Intero, LSB per primo
;   - FAC (7 byte, ZP)        : Valore da conservare, registri da FACEXP a FACOV
;
; Output:
;   - ARG (6 byte, ZP)        : Intero convertito, registri da ARGEXP a ARGSGN
;   - FAC (7 byte, ZP)        : Valore di prima, arrotondato, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC,
;                       _SCRATCH_2
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.89 m_Load_FAC_with_INT8

```
; ================================================================================
; MACRO: m_Load_FAC_with_INT8
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con un intero con segno a 8 bit, passato come
;               costante, e azzera FACOV.
;
;               I bit del valore oltre il bit #7 sono ignorati.
;
;               Converte il numero con la routine FLOATB della ROM del BASIC,
;               che normalizza FAC e prende il segno dal Carry: la macro lo
;               imposta a 1 per i numeri senza segno e, per quelli con segno, lo
;               ricava dal bit #7 del MSB (complemento a due). ARG non cambia.
;
; Input:
;   - .value_ (1 byte, Macro) : Valore da caricare, con segno
;
; Output:
;   - FAC (7 byte, ZP)        : Intero convertito, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.90 m_Load_FAC_with_INT8_Mem

```
; ================================================================================
; MACRO: m_Load_FAC_with_INT8_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con un intero con segno a 8 bit, letto da un
;               indirizzo assoluto, a partire dal LSB (little-endian), e
;               azzera FACOV.
;
;               Converte il numero con la routine FLOATB della ROM del BASIC,
;               che normalizza FAC e prende il segno dal Carry: la macro lo
;               imposta a 1 per i numeri senza segno e, per quelli con segno, lo
;               ricava dal bit #7 del MSB (complemento a due). ARG non cambia.
;
; Input:
;   - .addr_ (1 o 2 byte, Macro) : Indirizzo dell'intero
;   - (.addr_) (1 byte, RAM)     : Intero, LSB per primo
;
; Output:
;   - FAC (7 byte, ZP)           : Intero convertito, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.91 m_Load_FAC_with_INT8_Ptr

```
; ================================================================================
; MACRO: m_Load_FAC_with_INT8_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con un intero con segno a 8 bit, letto dall'indirizzo
;               indicato da un puntatore in ZP, LSB per primo, e azzera FACOV.
;
;               Converte il numero con la routine FLOATB della ROM del BASIC,
;               che normalizza FAC e prende il segno dal Carry: la macro lo
;               imposta a 1 per i numeri senza segno e, per quelli con segno, lo
;               ricava dal bit #7 del MSB (complemento a due). ARG non cambia.
;
; Input:
;   - .ptr_ (1 byte, Macro)   : Indirizzo in ZP del puntatore
;   - (.ptr_) (2 byte, ZP)    : Indirizzo dell'intero (LSB, MSB)
;   - ((.ptr_)) (1 byte, RAM) : Intero, LSB per primo
;
; Output:
;   - FAC (7 byte, ZP)        : Intero convertito, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.92 m_Load_ARG_with_INT8

```
; ================================================================================
; MACRO: m_Load_ARG_with_INT8
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con un intero con segno a 8 bit, passato
;               come costante.
;
;               I bit del valore oltre il bit #7 sono ignorati.
;
;               Scambia FAC e ARG con m_Swap_FAC_and_ARG, carica FAC con
;               la macro m_Load_FAC_with_INT8 e li scambia di nuovo. Dopo
;               la chiamata FAC contiene il valore di prima, arrotondato,
;               e FACOV vale zero; il vecchio ARG si perde e ARISGN non
;               viene aggiornato.
;
;               Il contenuto di _SCRATCH_2 non è garantito dopo la chiamata.
;
; Input:
;   - .value_ (1 byte, Macro) : Valore da caricare, con segno
;   - FAC (7 byte, ZP)        : Valore da conservare, registri da FACEXP a FACOV
;
; Output:
;   - ARG (6 byte, ZP)        : Intero convertito, registri da ARGEXP a ARGSGN
;   - FAC (7 byte, ZP)        : Valore di prima, arrotondato, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC,
;                       _SCRATCH_2
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.93 m_Load_ARG_with_INT8_Mem

```
; ================================================================================
; MACRO: m_Load_ARG_with_INT8_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con un intero con segno a 8 bit, letto da un
;               indirizzo assoluto, a partire dal LSB (little-endian).
;
;               Scambia FAC e ARG con m_Swap_FAC_and_ARG, carica FAC con
;               la macro m_Load_FAC_with_INT8_Mem e li scambia di nuovo.
;               Dopo la chiamata FAC contiene il valore di prima,
;               arrotondato, e FACOV vale zero; il vecchio ARG si perde e
;               ARISGN non viene aggiornato.
;
;               Il contenuto di _SCRATCH_2 non è garantito dopo la chiamata.
;
; Input:
;   - .addr_ (1 o 2 byte, Macro) : Indirizzo dell'intero
;   - (.addr_) (1 byte, RAM)     : Intero, LSB per primo
;   - FAC (7 byte, ZP)           : Valore da conservare, registri da FACEXP a
;                                  FACOV
;
; Output:
;   - ARG (6 byte, ZP)           : Intero convertito, registri da ARGEXP a
;                                  ARGSGN
;   - FAC (7 byte, ZP)           : Valore di prima, arrotondato, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC,
;                       _SCRATCH_2
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.94 m_Load_ARG_with_INT8_Ptr

```
; ================================================================================
; MACRO: m_Load_ARG_with_INT8_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con un intero con segno a 8 bit, letto dall'indirizzo
;               indicato da un puntatore in ZP, LSB per primo.
;
;               Scambia FAC e ARG con m_Swap_FAC_and_ARG, carica FAC con
;               la macro m_Load_FAC_with_INT8_Ptr e li scambia di nuovo.
;               Dopo la chiamata FAC contiene il valore di prima,
;               arrotondato, e FACOV vale zero; il vecchio ARG si perde e
;               ARISGN non viene aggiornato.
;
;               Il contenuto di _SCRATCH_2 non è garantito dopo la chiamata.
;
; Input:
;   - .ptr_ (1 byte, Macro)   : Indirizzo in ZP del puntatore
;   - (.ptr_) (2 byte, ZP)    : Indirizzo dell'intero (LSB, MSB)
;   - ((.ptr_)) (1 byte, RAM) : Intero, LSB per primo
;   - FAC (7 byte, ZP)        : Valore da conservare, registri da FACEXP a FACOV
;
; Output:
;   - ARG (6 byte, ZP)        : Intero convertito, registri da ARGEXP a ARGSGN
;   - FAC (7 byte, ZP)        : Valore di prima, arrotondato, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC,
;                       _SCRATCH_2
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.95 m_Load_FAC_with_UINT16

```
; ================================================================================
; MACRO: m_Load_FAC_with_UINT16
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con un intero senza segno a 16 bit, passato come
;               costante, e azzera FACOV.
;
;               I bit del valore oltre il bit #15 sono ignorati.
;
;               Converte il numero con la routine FLOATB della ROM del BASIC,
;               che normalizza FAC e prende il segno dal Carry: la macro lo
;               imposta a 1 (numero positivo). ARG non cambia.
;
; Input:
;   - .value_ (2 byte, Macro) : Valore da caricare, senza segno
;
; Output:
;   - FAC (7 byte, ZP)        : Intero convertito, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.96 m_Load_FAC_with_UINT16_Mem

```
; ================================================================================
; MACRO: m_Load_FAC_with_UINT16_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con un intero senza segno a 16 bit, letto da un
;               indirizzo assoluto, a partire dal LSB (little-endian), e
;               azzera FACOV.
;
;               Converte il numero con la routine FLOATB della ROM del BASIC,
;               che normalizza FAC e prende il segno dal Carry: la macro lo
;               imposta a 1 (numero positivo). ARG non cambia.
;
; Input:
;   - .addr_ (1 o 2 byte, Macro) : Indirizzo dell'intero
;   - (.addr_) (2 byte, RAM)     : Intero, LSB per primo
;
; Output:
;   - FAC (7 byte, ZP)           : Intero convertito, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.97 m_Load_FAC_with_UINT16_Ptr

```
; ================================================================================
; MACRO: m_Load_FAC_with_UINT16_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con un intero senza segno a 16 bit, letto
;               dall'indirizzo indicato da un puntatore in ZP, LSB per primo, e
;               azzera FACOV.
;
;               Converte il numero con la routine FLOATB della ROM del BASIC,
;               che normalizza FAC e prende il segno dal Carry: la macro lo
;               imposta a 1 (numero positivo). ARG non cambia.
;
; Input:
;   - .ptr_ (1 byte, Macro)   : Indirizzo in ZP del puntatore
;   - (.ptr_) (2 byte, ZP)    : Indirizzo dell'intero (LSB, MSB)
;   - ((.ptr_)) (2 byte, RAM) : Intero, LSB per primo
;
; Output:
;   - FAC (7 byte, ZP)        : Intero convertito, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.98 m_Load_ARG_with_UINT16

```
; ================================================================================
; MACRO: m_Load_ARG_with_UINT16
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con un intero senza segno a 16 bit, passato
;               come costante.
;
;               I bit del valore oltre il bit #15 sono ignorati.
;
;               Scambia FAC e ARG con m_Swap_FAC_and_ARG, carica FAC con
;               la macro m_Load_FAC_with_UINT16 e li scambia di nuovo.
;               Dopo la chiamata FAC contiene il valore di prima,
;               arrotondato, e FACOV vale zero; il vecchio ARG si perde e
;               ARISGN non viene aggiornato.
;
;               Il contenuto di _SCRATCH_2 non è garantito dopo la chiamata.
;
; Input:
;   - .value_ (2 byte, Macro) : Valore da caricare, senza segno
;   - FAC (7 byte, ZP)        : Valore da conservare, registri da FACEXP a FACOV
;
; Output:
;   - ARG (6 byte, ZP)        : Intero convertito, registri da ARGEXP a ARGSGN
;   - FAC (7 byte, ZP)        : Valore di prima, arrotondato, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC,
;                       _SCRATCH_2
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.99 m_Load_ARG_with_UINT16_Mem

```
; ================================================================================
; MACRO: m_Load_ARG_with_UINT16_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con un intero senza segno a 16 bit, letto da un
;               indirizzo assoluto, a partire dal LSB (little-endian).
;
;               Scambia FAC e ARG con m_Swap_FAC_and_ARG, carica FAC con la
;               macro m_Load_FAC_with_UINT16_Mem e li scambia di nuovo.
;               Dopo la chiamata FAC contiene il valore di prima,
;               arrotondato, e FACOV vale zero; il vecchio ARG si perde e
;               ARISGN non viene aggiornato.
;
;               Il contenuto di _SCRATCH_2 non è garantito dopo la chiamata.
;
; Input:
;   - .addr_ (1 o 2 byte, Macro) : Indirizzo dell'intero
;   - (.addr_) (2 byte, RAM)     : Intero, LSB per primo
;   - FAC (7 byte, ZP)           : Valore da conservare, registri da FACEXP a
;                                  FACOV
;
; Output:
;   - ARG (6 byte, ZP)           : Intero convertito, registri da ARGEXP a
;                                  ARGSGN
;   - FAC (7 byte, ZP)           : Valore di prima, arrotondato, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC,
;                       _SCRATCH_2
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.100 m_Load_ARG_with_UINT16_Ptr

```
; ================================================================================
; MACRO: m_Load_ARG_with_UINT16_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con un intero senza segno a 16 bit, letto
;               dall'indirizzo indicato da un puntatore in ZP, LSB per primo.
;
;               Scambia FAC e ARG con m_Swap_FAC_and_ARG, carica FAC con la
;               macro m_Load_FAC_with_UINT16_Ptr e li scambia di nuovo.
;               Dopo la chiamata FAC contiene il valore di prima,
;               arrotondato, e FACOV vale zero; il vecchio ARG si perde e
;               ARISGN non viene aggiornato.
;
;               Il contenuto di _SCRATCH_2 non è garantito dopo la chiamata.
;
; Input:
;   - .ptr_ (1 byte, Macro)   : Indirizzo in ZP del puntatore
;   - (.ptr_) (2 byte, ZP)    : Indirizzo dell'intero (LSB, MSB)
;   - ((.ptr_)) (2 byte, RAM) : Intero, LSB per primo
;   - FAC (7 byte, ZP)        : Valore da conservare, registri da FACEXP a FACOV
;
; Output:
;   - ARG (6 byte, ZP)        : Intero convertito, registri da ARGEXP a ARGSGN
;   - FAC (7 byte, ZP)        : Valore di prima, arrotondato, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC,
;                       _SCRATCH_2
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.101 m_Load_FAC_with_INT16

```
; ================================================================================
; MACRO: m_Load_FAC_with_INT16
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con un intero con segno a 16 bit, passato come
;               costante, e azzera FACOV.
;
;               I bit del valore oltre il bit #15 sono ignorati.
;
;               Converte il numero con la routine FLOATB della ROM del BASIC,
;               che normalizza FAC e prende il segno dal Carry: la macro lo
;               imposta a 1 per i numeri senza segno e, per quelli con segno, lo
;               ricava dal bit #15 del MSB (complemento a due). ARG non cambia.
;
; Input:
;   - .value_ (2 byte, Macro) : Valore da caricare, con segno
;
; Output:
;   - FAC (7 byte, ZP)        : Intero convertito, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.102 m_Load_FAC_with_INT16_Mem

```
; ================================================================================
; MACRO: m_Load_FAC_with_INT16_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con un intero con segno a 16 bit, letto da un
;               indirizzo assoluto, a partire dal LSB (little-endian), e
;               azzera FACOV.
;
;               Converte il numero con la routine FLOATB della ROM del BASIC,
;               che normalizza FAC e prende il segno dal Carry: la macro lo
;               imposta a 1 per i numeri senza segno e, per quelli con segno, lo
;               ricava dal bit #15 del MSB (complemento a due). ARG non cambia.
;
; Input:
;   - .addr_ (1 o 2 byte, Macro) : Indirizzo dell'intero
;   - (.addr_) (2 byte, RAM)     : Intero, LSB per primo
;
; Output:
;   - FAC (7 byte, ZP)           : Intero convertito, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.103 m_Load_FAC_with_INT16_Ptr

```
; ================================================================================
; MACRO: m_Load_FAC_with_INT16_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con un intero con segno a 16 bit, letto
;               dall'indirizzo indicato da un puntatore in ZP, LSB per primo, e
;               azzera FACOV.
;
;               Converte il numero con la routine FLOATB della ROM del BASIC,
;               che normalizza FAC e prende il segno dal Carry: la macro lo
;               imposta a 1 per i numeri senza segno e, per quelli con segno, lo
;               ricava dal bit #15 del MSB (complemento a due). ARG non cambia.
;
; Input:
;   - .ptr_ (1 byte, Macro)   : Indirizzo in ZP del puntatore
;   - (.ptr_) (2 byte, ZP)    : Indirizzo dell'intero (LSB, MSB)
;   - ((.ptr_)) (2 byte, RAM) : Intero, LSB per primo
;
; Output:
;   - FAC (7 byte, ZP)        : Intero convertito, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.104 m_Load_ARG_with_INT16

```
; ================================================================================
; MACRO: m_Load_ARG_with_INT16
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con un intero con segno a 16 bit, passato
;               come costante.
;
;               I bit del valore oltre il bit #15 sono ignorati.
;
;               Scambia FAC e ARG con m_Swap_FAC_and_ARG, carica FAC con
;               la macro m_Load_FAC_with_INT16 e li scambia di nuovo. Dopo
;               la chiamata FAC contiene il valore di prima, arrotondato,
;               e FACOV vale zero; il vecchio ARG si perde e ARISGN non
;               viene aggiornato.
;
;               Il contenuto di _SCRATCH_2 non è garantito dopo la chiamata.
;
; Input:
;   - .value_ (2 byte, Macro) : Valore da caricare, con segno
;   - FAC (7 byte, ZP)        : Valore da conservare, registri da FACEXP a FACOV
;
; Output:
;   - ARG (6 byte, ZP)        : Intero convertito, registri da ARGEXP a ARGSGN
;   - FAC (7 byte, ZP)        : Valore di prima, arrotondato, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC,
;                       _SCRATCH_2
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.105 m_Load_ARG_with_INT16_Mem

```
; ================================================================================
; MACRO: m_Load_ARG_with_INT16_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con un intero con segno a 16 bit, letto da un
;               indirizzo assoluto, a partire dal LSB (little-endian).
;
;               Scambia FAC e ARG con m_Swap_FAC_and_ARG, carica FAC con
;               la macro m_Load_FAC_with_INT16_Mem e li scambia di nuovo.
;               Dopo la chiamata FAC contiene il valore di prima,
;               arrotondato, e FACOV vale zero; il vecchio ARG si perde e
;               ARISGN non viene aggiornato.
;
;               Il contenuto di _SCRATCH_2 non è garantito dopo la chiamata.
;
; Input:
;   - .addr_ (1 o 2 byte, Macro) : Indirizzo dell'intero
;   - (.addr_) (2 byte, RAM)     : Intero, LSB per primo
;   - FAC (7 byte, ZP)           : Valore da conservare, registri da FACEXP a
;                                  FACOV
;
; Output:
;   - ARG (6 byte, ZP)           : Intero convertito, registri da ARGEXP a
;                                  ARGSGN
;   - FAC (7 byte, ZP)           : Valore di prima, arrotondato, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC,
;                       _SCRATCH_2
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.106 m_Load_ARG_with_INT16_Ptr

```
; ================================================================================
; MACRO: m_Load_ARG_with_INT16_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con un intero con segno a 16 bit, letto
;               dall'indirizzo indicato da un puntatore in ZP, LSB per primo.
;
;               Scambia FAC e ARG con m_Swap_FAC_and_ARG, carica FAC con
;               la macro m_Load_FAC_with_INT16_Ptr e li scambia di nuovo.
;               Dopo la chiamata FAC contiene il valore di prima,
;               arrotondato, e FACOV vale zero; il vecchio ARG si perde e
;               ARISGN non viene aggiornato.
;
;               Il contenuto di _SCRATCH_2 non è garantito dopo la chiamata.
;
; Input:
;   - .ptr_ (1 byte, Macro)   : Indirizzo in ZP del puntatore
;   - (.ptr_) (2 byte, ZP)    : Indirizzo dell'intero (LSB, MSB)
;   - ((.ptr_)) (2 byte, RAM) : Intero, LSB per primo
;   - FAC (7 byte, ZP)        : Valore da conservare, registri da FACEXP a FACOV
;
; Output:
;   - ARG (6 byte, ZP)        : Intero convertito, registri da ARGEXP a ARGSGN
;   - FAC (7 byte, ZP)        : Valore di prima, arrotondato, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC,
;                       _SCRATCH_2
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.107 m_Load_FAC_with_UINT24

```
; ================================================================================
; MACRO: m_Load_FAC_with_UINT24
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con un intero senza segno a 24 bit, passato come
;               costante, e azzera FACOV.
;
;               I bit del valore oltre il bit #23 sono ignorati.
;
;               Converte il numero con la routine FLOATB della ROM del BASIC,
;               che normalizza FAC e prende il segno dal Carry: la macro lo
;               imposta a 1 (numero positivo). ARG non cambia.
;
; Input:
;   - .value_ (3 byte, Macro) : Valore da caricare, senza segno
;
; Output:
;   - FAC (7 byte, ZP)        : Intero convertito, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.108 m_Load_FAC_with_UINT24_Mem

```
; ================================================================================
; MACRO: m_Load_FAC_with_UINT24_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con un intero senza segno a 24 bit, letto da un
;               indirizzo assoluto, a partire dal LSB (little-endian), e
;               azzera FACOV.
;
;               Converte il numero con la routine FLOATB della ROM del BASIC,
;               che normalizza FAC e prende il segno dal Carry: la macro lo
;               imposta a 1 (numero positivo). ARG non cambia.
;
; Input:
;   - .addr_ (1 o 2 byte, Macro) : Indirizzo dell'intero
;   - (.addr_) (3 byte, RAM)     : Intero, LSB per primo
;
; Output:
;   - FAC (7 byte, ZP)           : Intero convertito, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.109 m_Load_FAC_with_UINT24_Ptr

```
; ================================================================================
; MACRO: m_Load_FAC_with_UINT24_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con un intero senza segno a 24 bit, letto
;               dall'indirizzo indicato da un puntatore in ZP, LSB per primo, e
;               azzera FACOV.
;
;               Converte il numero con la routine FLOATB della ROM del BASIC,
;               che normalizza FAC e prende il segno dal Carry: la macro lo
;               imposta a 1 (numero positivo). ARG non cambia.
;
; Input:
;   - .ptr_ (1 byte, Macro)   : Indirizzo in ZP del puntatore
;   - (.ptr_) (2 byte, ZP)    : Indirizzo dell'intero (LSB, MSB)
;   - ((.ptr_)) (3 byte, RAM) : Intero, LSB per primo
;
; Output:
;   - FAC (7 byte, ZP)        : Intero convertito, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.110 m_Load_ARG_with_UINT24

```
; ================================================================================
; MACRO: m_Load_ARG_with_UINT24
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con un intero senza segno a 24 bit, passato
;               come costante.
;
;               I bit del valore oltre il bit #23 sono ignorati.
;
;               Scambia FAC e ARG con m_Swap_FAC_and_ARG, carica FAC con
;               la macro m_Load_FAC_with_UINT24 e li scambia di nuovo.
;               Dopo la chiamata FAC contiene il valore di prima,
;               arrotondato, e FACOV vale zero; il vecchio ARG si perde e
;               ARISGN non viene aggiornato.
;
;               Il contenuto di _SCRATCH_2 non è garantito dopo la chiamata.
;
; Input:
;   - .value_ (3 byte, Macro) : Valore da caricare, senza segno
;   - FAC (7 byte, ZP)        : Valore da conservare, registri da FACEXP a FACOV
;
; Output:
;   - ARG (6 byte, ZP)        : Intero convertito, registri da ARGEXP a ARGSGN
;   - FAC (7 byte, ZP)        : Valore di prima, arrotondato, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC,
;                       _SCRATCH_2
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.111 m_Load_ARG_with_UINT24_Mem

```
; ================================================================================
; MACRO: m_Load_ARG_with_UINT24_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con un intero senza segno a 24 bit, letto da un
;               indirizzo assoluto, a partire dal LSB (little-endian).
;
;               Scambia FAC e ARG con m_Swap_FAC_and_ARG, carica FAC con la
;               macro m_Load_FAC_with_UINT24_Mem e li scambia di nuovo.
;               Dopo la chiamata FAC contiene il valore di prima,
;               arrotondato, e FACOV vale zero; il vecchio ARG si perde e
;               ARISGN non viene aggiornato.
;
;               Il contenuto di _SCRATCH_2 non è garantito dopo la chiamata.
;
; Input:
;   - .addr_ (1 o 2 byte, Macro) : Indirizzo dell'intero
;   - (.addr_) (3 byte, RAM)     : Intero, LSB per primo
;   - FAC (7 byte, ZP)           : Valore da conservare, registri da FACEXP a
;                                  FACOV
;
; Output:
;   - ARG (6 byte, ZP)           : Intero convertito, registri da ARGEXP a
;                                  ARGSGN
;   - FAC (7 byte, ZP)           : Valore di prima, arrotondato, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC,
;                       _SCRATCH_2
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.112 m_Load_ARG_with_UINT24_Ptr

```
; ================================================================================
; MACRO: m_Load_ARG_with_UINT24_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con un intero senza segno a 24 bit, letto
;               dall'indirizzo indicato da un puntatore in ZP, LSB per primo.
;
;               Scambia FAC e ARG con m_Swap_FAC_and_ARG, carica FAC con la
;               macro m_Load_FAC_with_UINT24_Ptr e li scambia di nuovo.
;               Dopo la chiamata FAC contiene il valore di prima,
;               arrotondato, e FACOV vale zero; il vecchio ARG si perde e
;               ARISGN non viene aggiornato.
;
;               Il contenuto di _SCRATCH_2 non è garantito dopo la chiamata.
;
; Input:
;   - .ptr_ (1 byte, Macro)   : Indirizzo in ZP del puntatore
;   - (.ptr_) (2 byte, ZP)    : Indirizzo dell'intero (LSB, MSB)
;   - ((.ptr_)) (3 byte, RAM) : Intero, LSB per primo
;   - FAC (7 byte, ZP)        : Valore da conservare, registri da FACEXP a FACOV
;
; Output:
;   - ARG (6 byte, ZP)        : Intero convertito, registri da ARGEXP a ARGSGN
;   - FAC (7 byte, ZP)        : Valore di prima, arrotondato, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC,
;                       _SCRATCH_2
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.113 m_Load_FAC_with_INT24

```
; ================================================================================
; MACRO: m_Load_FAC_with_INT24
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con un intero con segno a 24 bit, passato come
;               costante, e azzera FACOV.
;
;               I bit del valore oltre il bit #23 sono ignorati.
;
;               Converte il numero con la routine FLOATB della ROM del BASIC,
;               che normalizza FAC e prende il segno dal Carry: la macro lo
;               imposta a 1 per i numeri senza segno e, per quelli con segno, lo
;               ricava dal bit #23 del MSB (complemento a due). ARG non cambia.
;
; Input:
;   - .value_ (3 byte, Macro) : Valore da caricare, con segno
;
; Output:
;   - FAC (7 byte, ZP)        : Intero convertito, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.114 m_Load_FAC_with_INT24_Mem

```
; ================================================================================
; MACRO: m_Load_FAC_with_INT24_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con un intero con segno a 24 bit, letto da un
;               indirizzo assoluto, a partire dal LSB (little-endian), e
;               azzera FACOV.
;
;               Converte il numero con la routine FLOATB della ROM del BASIC,
;               che normalizza FAC e prende il segno dal Carry: la macro lo
;               imposta a 1 per i numeri senza segno e, per quelli con segno, lo
;               ricava dal bit #23 del MSB (complemento a due). ARG non cambia.
;
; Input:
;   - .addr_ (1 o 2 byte, Macro) : Indirizzo dell'intero
;   - (.addr_) (3 byte, RAM)     : Intero, LSB per primo
;
; Output:
;   - FAC (7 byte, ZP)           : Intero convertito, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.115 m_Load_FAC_with_INT24_Ptr

```
; ================================================================================
; MACRO: m_Load_FAC_with_INT24_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con un intero con segno a 24 bit, letto
;               dall'indirizzo indicato da un puntatore in ZP, LSB per primo, e
;               azzera FACOV.
;
;               Converte il numero con la routine FLOATB della ROM del BASIC,
;               che normalizza FAC e prende il segno dal Carry: la macro lo
;               imposta a 1 per i numeri senza segno e, per quelli con segno, lo
;               ricava dal bit #23 del MSB (complemento a due). ARG non cambia.
;
; Input:
;   - .ptr_ (1 byte, Macro)   : Indirizzo in ZP del puntatore
;   - (.ptr_) (2 byte, ZP)    : Indirizzo dell'intero (LSB, MSB)
;   - ((.ptr_)) (3 byte, RAM) : Intero, LSB per primo
;
; Output:
;   - FAC (7 byte, ZP)        : Intero convertito, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.116 m_Load_ARG_with_INT24

```
; ================================================================================
; MACRO: m_Load_ARG_with_INT24
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con un intero con segno a 24 bit, passato
;               come costante.
;
;               I bit del valore oltre il bit #23 sono ignorati.
;
;               Scambia FAC e ARG con m_Swap_FAC_and_ARG, carica FAC con
;               la macro m_Load_FAC_with_INT24 e li scambia di nuovo. Dopo
;               la chiamata FAC contiene il valore di prima, arrotondato,
;               e FACOV vale zero; il vecchio ARG si perde e ARISGN non
;               viene aggiornato.
;
;               Il contenuto di _SCRATCH_2 non è garantito dopo la chiamata.
;
; Input:
;   - .value_ (3 byte, Macro) : Valore da caricare, con segno
;   - FAC (7 byte, ZP)        : Valore da conservare, registri da FACEXP a FACOV
;
; Output:
;   - ARG (6 byte, ZP)        : Intero convertito, registri da ARGEXP a ARGSGN
;   - FAC (7 byte, ZP)        : Valore di prima, arrotondato, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC,
;                       _SCRATCH_2
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.117 m_Load_ARG_with_INT24_Mem

```
; ================================================================================
; MACRO: m_Load_ARG_with_INT24_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con un intero con segno a 24 bit, letto da un
;               indirizzo assoluto, a partire dal LSB (little-endian).
;
;               Scambia FAC e ARG con m_Swap_FAC_and_ARG, carica FAC con
;               la macro m_Load_FAC_with_INT24_Mem e li scambia di nuovo.
;               Dopo la chiamata FAC contiene il valore di prima,
;               arrotondato, e FACOV vale zero; il vecchio ARG si perde e
;               ARISGN non viene aggiornato.
;
;               Il contenuto di _SCRATCH_2 non è garantito dopo la chiamata.
;
; Input:
;   - .addr_ (1 o 2 byte, Macro) : Indirizzo dell'intero
;   - (.addr_) (3 byte, RAM)     : Intero, LSB per primo
;   - FAC (7 byte, ZP)           : Valore da conservare, registri da FACEXP a
;                                  FACOV
;
; Output:
;   - ARG (6 byte, ZP)           : Intero convertito, registri da ARGEXP a
;                                  ARGSGN
;   - FAC (7 byte, ZP)           : Valore di prima, arrotondato, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC,
;                       _SCRATCH_2
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.118 m_Load_ARG_with_INT24_Ptr

```
; ================================================================================
; MACRO: m_Load_ARG_with_INT24_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con un intero con segno a 24 bit, letto
;               dall'indirizzo indicato da un puntatore in ZP, LSB per primo.
;
;               Scambia FAC e ARG con m_Swap_FAC_and_ARG, carica FAC con
;               la macro m_Load_FAC_with_INT24_Ptr e li scambia di nuovo.
;               Dopo la chiamata FAC contiene il valore di prima,
;               arrotondato, e FACOV vale zero; il vecchio ARG si perde e
;               ARISGN non viene aggiornato.
;
;               Il contenuto di _SCRATCH_2 non è garantito dopo la chiamata.
;
; Input:
;   - .ptr_ (1 byte, Macro)   : Indirizzo in ZP del puntatore
;   - (.ptr_) (2 byte, ZP)    : Indirizzo dell'intero (LSB, MSB)
;   - ((.ptr_)) (3 byte, RAM) : Intero, LSB per primo
;   - FAC (7 byte, ZP)        : Valore da conservare, registri da FACEXP a FACOV
;
; Output:
;   - ARG (6 byte, ZP)        : Intero convertito, registri da ARGEXP a ARGSGN
;   - FAC (7 byte, ZP)        : Valore di prima, arrotondato, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC,
;                       _SCRATCH_2
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.119 m_Load_FAC_with_UINT32

```
; ================================================================================
; MACRO: m_Load_FAC_with_UINT32
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con un intero senza segno a 32 bit, passato come
;               costante, e azzera FACOV.
;
;               I bit del valore oltre il bit #31 sono ignorati.
;
;               Converte il numero con la routine FLOATB della ROM del BASIC,
;               che normalizza FAC e prende il segno dal Carry: la macro lo
;               imposta a 1 (numero positivo). ARG non cambia.
;
; Input:
;   - .value_ (4 byte, Macro) : Valore da caricare, senza segno
;
; Output:
;   - FAC (7 byte, ZP)        : Intero convertito, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.120 m_Load_FAC_with_UINT32_Mem

```
; ================================================================================
; MACRO: m_Load_FAC_with_UINT32_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con un intero senza segno a 32 bit, letto da un
;               indirizzo assoluto, a partire dal LSB (little-endian), e
;               azzera FACOV.
;
;               Converte il numero con la routine FLOATB della ROM del BASIC,
;               che normalizza FAC e prende il segno dal Carry: la macro lo
;               imposta a 1 (numero positivo). ARG non cambia.
;
; Input:
;   - .addr_ (1 o 2 byte, Macro) : Indirizzo dell'intero
;   - (.addr_) (4 byte, RAM)     : Intero, LSB per primo
;
; Output:
;   - FAC (7 byte, ZP)           : Intero convertito, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.121 m_Load_FAC_with_UINT32_Ptr

```
; ================================================================================
; MACRO: m_Load_FAC_with_UINT32_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con un intero senza segno a 32 bit, letto
;               dall'indirizzo indicato da un puntatore in ZP, LSB per primo, e
;               azzera FACOV.
;
;               Converte il numero con la routine FLOATB della ROM del BASIC,
;               che normalizza FAC e prende il segno dal Carry: la macro lo
;               imposta a 1 (numero positivo). ARG non cambia.
;
; Input:
;   - .ptr_ (1 byte, Macro)   : Indirizzo in ZP del puntatore
;   - (.ptr_) (2 byte, ZP)    : Indirizzo dell'intero (LSB, MSB)
;   - ((.ptr_)) (4 byte, RAM) : Intero, LSB per primo
;
; Output:
;   - FAC (7 byte, ZP)        : Intero convertito, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.122 m_Load_ARG_with_UINT32

```
; ================================================================================
; MACRO: m_Load_ARG_with_UINT32
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con un intero senza segno a 32 bit, passato
;               come costante.
;
;               I bit del valore oltre il bit #31 sono ignorati.
;
;               Scambia FAC e ARG con m_Swap_FAC_and_ARG, carica FAC con
;               la macro m_Load_FAC_with_UINT32 e li scambia di nuovo.
;               Dopo la chiamata FAC contiene il valore di prima,
;               arrotondato, e FACOV vale zero; il vecchio ARG si perde e
;               ARISGN non viene aggiornato.
;
;               Il contenuto di _SCRATCH_2 non è garantito dopo la chiamata.
;
; Input:
;   - .value_ (4 byte, Macro) : Valore da caricare, senza segno
;   - FAC (7 byte, ZP)        : Valore da conservare, registri da FACEXP a FACOV
;
; Output:
;   - ARG (6 byte, ZP)        : Intero convertito, registri da ARGEXP a ARGSGN
;   - FAC (7 byte, ZP)        : Valore di prima, arrotondato, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC,
;                       _SCRATCH_2
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.123 m_Load_ARG_with_UINT32_Mem

```
; ================================================================================
; MACRO: m_Load_ARG_with_UINT32_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con un intero senza segno a 32 bit, letto da un
;               indirizzo assoluto, a partire dal LSB (little-endian).
;
;               Scambia FAC e ARG con m_Swap_FAC_and_ARG, carica FAC con la
;               macro m_Load_FAC_with_UINT32_Mem e li scambia di nuovo.
;               Dopo la chiamata FAC contiene il valore di prima,
;               arrotondato, e FACOV vale zero; il vecchio ARG si perde e
;               ARISGN non viene aggiornato.
;
;               Il contenuto di _SCRATCH_2 non è garantito dopo la chiamata.
;
; Input:
;   - .addr_ (1 o 2 byte, Macro) : Indirizzo dell'intero
;   - (.addr_) (4 byte, RAM)     : Intero, LSB per primo
;   - FAC (7 byte, ZP)           : Valore da conservare, registri da FACEXP a
;                                  FACOV
;
; Output:
;   - ARG (6 byte, ZP)           : Intero convertito, registri da ARGEXP a
;                                  ARGSGN
;   - FAC (7 byte, ZP)           : Valore di prima, arrotondato, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC,
;                       _SCRATCH_2
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.124 m_Load_ARG_with_UINT32_Ptr

```
; ================================================================================
; MACRO: m_Load_ARG_with_UINT32_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con un intero senza segno a 32 bit, letto
;               dall'indirizzo indicato da un puntatore in ZP, LSB per primo.
;
;               Scambia FAC e ARG con m_Swap_FAC_and_ARG, carica FAC con la
;               macro m_Load_FAC_with_UINT32_Ptr e li scambia di nuovo.
;               Dopo la chiamata FAC contiene il valore di prima,
;               arrotondato, e FACOV vale zero; il vecchio ARG si perde e
;               ARISGN non viene aggiornato.
;
;               Il contenuto di _SCRATCH_2 non è garantito dopo la chiamata.
;
; Input:
;   - .ptr_ (1 byte, Macro)   : Indirizzo in ZP del puntatore
;   - (.ptr_) (2 byte, ZP)    : Indirizzo dell'intero (LSB, MSB)
;   - ((.ptr_)) (4 byte, RAM) : Intero, LSB per primo
;   - FAC (7 byte, ZP)        : Valore da conservare, registri da FACEXP a FACOV
;
; Output:
;   - ARG (6 byte, ZP)        : Intero convertito, registri da ARGEXP a ARGSGN
;   - FAC (7 byte, ZP)        : Valore di prima, arrotondato, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC,
;                       _SCRATCH_2
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.125 m_Load_FAC_with_INT32

```
; ================================================================================
; MACRO: m_Load_FAC_with_INT32
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con un intero con segno a 32 bit, passato come
;               costante, e azzera FACOV.
;
;               I bit del valore oltre il bit #31 sono ignorati.
;
;               Converte il numero con la routine FLOATB della ROM del BASIC,
;               che normalizza FAC e prende il segno dal Carry: la macro lo
;               imposta a 1 per i numeri senza segno e, per quelli con segno, lo
;               ricava dal bit #31 del MSB (complemento a due). ARG non cambia.
;
; Input:
;   - .value_ (4 byte, Macro) : Valore da caricare, con segno
;
; Output:
;   - FAC (7 byte, ZP)        : Intero convertito, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.126 m_Load_FAC_with_INT32_Mem

```
; ================================================================================
; MACRO: m_Load_FAC_with_INT32_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con un intero con segno a 32 bit, letto da un
;               indirizzo assoluto, a partire dal LSB (little-endian), e
;               azzera FACOV.
;
;               Converte il numero con la routine FLOATB della ROM del BASIC,
;               che normalizza FAC e prende il segno dal Carry: la macro lo
;               imposta a 1 per i numeri senza segno e, per quelli con segno, lo
;               ricava dal bit #31 del MSB (complemento a due). ARG non cambia.
;
; Input:
;   - .addr_ (1 o 2 byte, Macro) : Indirizzo dell'intero
;   - (.addr_) (4 byte, RAM)     : Intero, LSB per primo
;
; Output:
;   - FAC (7 byte, ZP)           : Intero convertito, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.127 m_Load_FAC_with_INT32_Ptr

```
; ================================================================================
; MACRO: m_Load_FAC_with_INT32_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con un intero con segno a 32 bit, letto
;               dall'indirizzo indicato da un puntatore in ZP, LSB per primo, e
;               azzera FACOV.
;
;               Converte il numero con la routine FLOATB della ROM del BASIC,
;               che normalizza FAC e prende il segno dal Carry: la macro lo
;               imposta a 1 per i numeri senza segno e, per quelli con segno, lo
;               ricava dal bit #31 del MSB (complemento a due). ARG non cambia.
;
; Input:
;   - .ptr_ (1 byte, Macro)   : Indirizzo in ZP del puntatore
;   - (.ptr_) (2 byte, ZP)    : Indirizzo dell'intero (LSB, MSB)
;   - ((.ptr_)) (4 byte, RAM) : Intero, LSB per primo
;
; Output:
;   - FAC (7 byte, ZP)        : Intero convertito, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.128 m_Load_ARG_with_INT32

```
; ================================================================================
; MACRO: m_Load_ARG_with_INT32
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con un intero con segno a 32 bit, passato
;               come costante.
;
;               I bit del valore oltre il bit #31 sono ignorati.
;
;               Scambia FAC e ARG con m_Swap_FAC_and_ARG, carica FAC con
;               la macro m_Load_FAC_with_INT32 e li scambia di nuovo. Dopo
;               la chiamata FAC contiene il valore di prima, arrotondato,
;               e FACOV vale zero; il vecchio ARG si perde e ARISGN non
;               viene aggiornato.
;
;               Il contenuto di _SCRATCH_2 non è garantito dopo la chiamata.
;
; Input:
;   - .value_ (4 byte, Macro) : Valore da caricare, con segno
;   - FAC (7 byte, ZP)        : Valore da conservare, registri da FACEXP a FACOV
;
; Output:
;   - ARG (6 byte, ZP)        : Intero convertito, registri da ARGEXP a ARGSGN
;   - FAC (7 byte, ZP)        : Valore di prima, arrotondato, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC,
;                       _SCRATCH_2
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.129 m_Load_ARG_with_INT32_Mem

```
; ================================================================================
; MACRO: m_Load_ARG_with_INT32_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con un intero con segno a 32 bit, letto da un
;               indirizzo assoluto, a partire dal LSB (little-endian).
;
;               Scambia FAC e ARG con m_Swap_FAC_and_ARG, carica FAC con
;               la macro m_Load_FAC_with_INT32_Mem e li scambia di nuovo.
;               Dopo la chiamata FAC contiene il valore di prima,
;               arrotondato, e FACOV vale zero; il vecchio ARG si perde e
;               ARISGN non viene aggiornato.
;
;               Il contenuto di _SCRATCH_2 non è garantito dopo la chiamata.
;
; Input:
;   - .addr_ (1 o 2 byte, Macro) : Indirizzo dell'intero
;   - (.addr_) (4 byte, RAM)     : Intero, LSB per primo
;   - FAC (7 byte, ZP)           : Valore da conservare, registri da FACEXP a
;                                  FACOV
;
; Output:
;   - ARG (6 byte, ZP)           : Intero convertito, registri da ARGEXP a
;                                  ARGSGN
;   - FAC (7 byte, ZP)           : Valore di prima, arrotondato, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC,
;                       _SCRATCH_2
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.130 m_Load_ARG_with_INT32_Ptr

```
; ================================================================================
; MACRO: m_Load_ARG_with_INT32_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con un intero con segno a 32 bit, letto
;               dall'indirizzo indicato da un puntatore in ZP, LSB per primo.
;
;               Scambia FAC e ARG con m_Swap_FAC_and_ARG, carica FAC con
;               la macro m_Load_FAC_with_INT32_Ptr e li scambia di nuovo.
;               Dopo la chiamata FAC contiene il valore di prima,
;               arrotondato, e FACOV vale zero; il vecchio ARG si perde e
;               ARISGN non viene aggiornato.
;
;               Il contenuto di _SCRATCH_2 non è garantito dopo la chiamata.
;
; Input:
;   - .ptr_ (1 byte, Macro)   : Indirizzo in ZP del puntatore
;   - (.ptr_) (2 byte, ZP)    : Indirizzo dell'intero (LSB, MSB)
;   - ((.ptr_)) (4 byte, RAM) : Intero, LSB per primo
;   - FAC (7 byte, ZP)        : Valore da conservare, registri da FACEXP a FACOV
;
; Output:
;   - ARG (6 byte, ZP)        : Intero convertito, registri da ARGEXP a ARGSGN
;   - FAC (7 byte, ZP)        : Valore di prima, arrotondato, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC,
;                       _SCRATCH_2
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.131 m_Convert_FAC_into_INT32

```
; ================================================================================
; MACRO: m_Convert_FAC_into_INT32
; --------------------------------------------------------------------------------
; Descrizione:  Converte il numero in virgola mobile di FAC in un intero a 32
;               bit con segno, arrotondato per difetto (verso -infinito, tenendo
;               conto anche di FACOV). Il numero deve stare nell'intervallo da
;               -2147483647 a 2147483647; fuori da questo intervallo il
;               risultato è indefinito.
;
;               L'intero, in complemento a due, va nei 4 byte della mantissa,
;               con il MSB in FACHO. Per ottenere un intero a 24, 16 o 8 bit, se
;               il valore ci sta, si leggono gli ultimi 3, 2 o 1 byte, cioè da
;               FACMOH, da FACMO o FACLO. FACEXP e FACSGN non cambiano, ma FAC
;               non è più un numero valido; FACOV contiene i bit scartati.
;
;               Chiama la routine QINT della ROM del BASIC, che usa la cella $68
;               (byte di overflow di FAC): deve valere zero in ingresso, e la
;               routine la azzera in uscita.
;
; Input:
;   - FAC (7 byte, ZP)   : Registri da FACEXP a FACOV
;
; Output:
;   - FACHO (4 byte, ZP) : Intero con segno, da FACHO a FACLO, MSB per primo
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.132 m_Convert_ARG_into_INT32

```
; ================================================================================
; MACRO: m_Convert_ARG_into_INT32
; --------------------------------------------------------------------------------
; Descrizione:  Converte il numero in virgola mobile di ARG in un intero a 32
;               bit con segno, arrotondato per difetto (verso -infinito). Il
;               numero deve stare nell'intervallo da -2147483647 a 2147483647;
;               fuori da questo intervallo il risultato è indefinito.
;
;               L'intero, in complemento a due, va nei 4 byte della mantissa di
;               ARG, con il MSB in ARGHO. Per ottenere un intero a 24, 16 o 8
;               bit, se il valore ci sta, si leggono gli ultimi 3, 2 o 1 byte,
;               cioè da ARGMOH, da ARGMO o ARGLO. ARGEXP e ARGSGN non cambiano,
;               ma ARG non è più un numero valido. ARISGN non viene aggiornato.
;
;               Scambia FAC e ARG con m_Swap_FAC_and_ARG, converte con la
;               routine QINT della ROM del BASIC, azzera FACOV e li scambia di
;               nuovo. Dopo la chiamata FAC contiene il valore di prima,
;               arrotondato, e FACOV vale zero. La macro azzera FACOV perché
;               QINT vi lascia i bit scartati, che lo Swap userebbe per
;               arrotondare l'intero.
;
;               QINT usa la cella $68 (byte di overflow di FAC): deve valere
;               zero in ingresso, e la routine la azzera in uscita. Il contenuto
;               di _SCRATCH_2 non è garantito dopo la chiamata.
;
; Input:
;   - ARG (6 byte, ZP)   : Registri da ARGEXP a ARGSGN
;   - FAC (7 byte, ZP)   : Valore da conservare, registri da FACEXP a FACOV
;
; Output:
;   - ARGHO (4 byte, ZP) : Intero con segno, da ARGHO a ARGLO, MSB per primo
;   - FAC (7 byte, ZP)   : Valore di prima, arrotondato, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC,
;                       _SCRATCH_2
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

fp_io.asm

2.1.133 m_FAC_to_String

```
; ================================================================================
; MACRO: m_FAC_to_String
; --------------------------------------------------------------------------------
; Descrizione:  Converte il numero in virgola mobile di FAC in una stringa e la
;               scrive in STACK.
;
;               La stringa, terminata da zero, è lunga al massimo 15 caratteri
;               più il terminatore: ha uno spazio o il segno meno davanti, e usa
;               la forma esponenziale (1E+10) quando serve. Si trova in STACK,
;               da $0100 in poi.
;
;               Chiama la routine FOUT della ROM del BASIC, che altera FAC e
;               ARG. Con m_Store_FAC_in_Scratch e m_Store_ARG_in_Scratch la
;               macro salva FAC in _SCRATCH_1 e ARG in _SCRATCH_2 prima della
;               chiamata e li ripristina dopo. Per le variabili del BASIC
;               alterate vedi [NOTA #06 in <fp/fp.asm>].
;
;               La ROM altera queste celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: INDEX, INDEX2, RESHO, $47, $56,
;               $5D-$5E e $71.
;
;               Per l'accuratezza della conversione vedi
;               [NOTA #08 in <fp/fp.asm>].
;
; Input:
;   - FAC (7 byte, ZP)     : Registri da FACEXP a FACOV
;
; Output:
;   - STACK (16 byte, RAM) : Stringa terminata da zero, da $0100
;   - FAC (7 byte, ZP)     : Invariato
;   - ARG (6 byte, ZP)     : Invariato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, _SCRATCH_1, _SCRATCH_2, STACK (16 byte)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_common.asm>
; ================================================================================
```

2.1.134 m_ARG_to_String

```
; ============================================================================
; MACRO: m_ARG_to_String
; ----------------------------------------------------------------------------
; Descrizione:  Converte il numero in virgola mobile di ARG in una stringa e la
;               scrive in STACK.
;
;               La stringa, terminata da zero, è lunga al massimo 15 caratteri
;               più il terminatore: ha uno spazio o il segno meno davanti, e usa
;               la forma esponenziale (1E+10) quando serve. Si trova in STACK,
;               da $0100 in poi.
;
;               Copia prima ARG in FAC con m_Transfer_ARG_to_FAC, poi chiama la
;               routine FOUT della ROM del BASIC, che altera FAC e ARG. Con
;               m_Store_FAC_in_Scratch e m_Store_ARG_in_Scratch la macro salva
;               FAC in _SCRATCH_1 e ARG in _SCRATCH_2 prima della copia e li
;               ripristina dopo. Per le variabili del BASIC alterate vedi
;               [NOTA #06 in <fp/fp.asm>].
;
;               La ROM altera queste celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: INDEX, INDEX2, RESHO, $47, $56,
;               $5D-$5E e $71.
;
;               Per l'accuratezza della conversione vedi
;               [NOTA #08 in <fp/fp.asm>].
;
; Input:
;   - ARG (6 byte, ZP)     : Registri da ARGEXP a ARGSGN
;
; Output:
;   - STACK (16 byte, RAM) : Stringa terminata da zero, da $0100
;   - FAC (7 byte, ZP)     : Invariato
;   - ARG (6 byte, ZP)     : Invariato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, _SCRATCH_1, _SCRATCH_2, STACK (16 byte)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_common.asm>
;                       <fp/fp_memory.asm>
; ============================================================================
```

2.1.135 m_String_to_FAC

```
; ===========================================================================
; MACRO: m_String_to_FAC
; ---------------------------------------------------------------------------
; Descrizione:  Converte una stringa in un numero in virgola mobile e lo scrive
;               in FAC, che perde il suo vecchio contenuto.
;
;               La stringa, terminata da zero, rappresenta un numero: spazi
;               iniziali, segno opzionale, cifre, punto decimale ed esponente,
;               introdotto dal carattere E (codice $45) con segno opzionale. La
;               conversione si ferma al primo carattere non valido, senza
;               errori; senza un numero il risultato è zero. Un esponente troppo
;               piccolo dà zero.
;
;               Dopo avere scritto TXTPTR, la macro chiama CHRGOT, che legge
;               il primo carattere, e FIN della ROM del BASIC. FIN altera
;               sempre ARG.
;
;               La macro scrive l'indirizzo della stringa in TXTPTR, il
;               puntatore al testo del BASIC, e la ROM lo fa avanzare fino al
;               primo carattere dopo il numero. Per le variabili del BASIC
;               alterate vedi [NOTA #06 in <fp/fp.asm>].
;
;               Se il numero è troppo grande la ROM segnala OVERFLOW ERROR e non
;               ritorna al chiamante: vedi [NOTA #07 in <fp/fp.asm>].
;
;               La ROM altera queste celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: INDEX, RESHO, $56, $5D-$60 e $67.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in _SCRATCH_2 prima
;               della conversione e lo ripristina dopo; se vale 0 il contenuto
;               di ARG non è garantito dopo la chiamata. Un ARG uguale a zero
;               (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero.
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - .addr_ (1 o 2 byte, Macro)     : Indirizzo della stringa
;   - (.addr_) (n byte, RAM)         : Stringa terminata da zero
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Numero convertito, registri da FACEXP a
;                                      FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - TXTPTR (2 byte, ZP)            : Indirizzo del primo carattere dopo il
;                                      numero
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
```

segue...

2.1.135 m_String_to_FAC (continua)

```
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, TXTPTR,
;                       _SCRATCH_2 (se __PRESERVE_ARG = 1), __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_common.asm>
; ================================================================================
```

2.1.136 m_String_to_ARG

```
; ================================================================================
; MACRO: m_String_to_ARG
; --------------------------------------------------------------------------------
; Descrizione:  Converte una stringa in un numero in virgola mobile e lo scrive
;               in ARG, che perde il suo vecchio contenuto. FAC non cambia.
;
;               La stringa, terminata da zero, rappresenta un numero: spazi
;               iniziali, segno opzionale, cifre, punto decimale ed esponente,
;               introdotto dal carattere E (codice $45) con segno opzionale. La
;               conversione si ferma al primo carattere non valido, senza
;               errori; senza un numero il risultato è zero. Un esponente troppo
;               piccolo dà zero.
;
;               Dopo avere scritto TXTPTR, la macro chiama CHRGOT, che legge il
;               primo carattere, e FIN della ROM del BASIC, che converte la
;               stringa in FAC e altera ARG; poi chiama MOVAF per copiare FAC in
;               ARG. Con m_Store_FAC_in_Scratch la macro salva FAC in _SCRATCH_1
;               prima della conversione e con m_Load_FAC_from_Scratch lo
;               ripristina dopo.
;
;               La macro scrive l'indirizzo della stringa in TXTPTR, il
;               puntatore al testo del BASIC, e la ROM lo fa avanzare fino al
;               primo carattere dopo il numero. Per le variabili del BASIC
;               alterate vedi [NOTA #06 in <fp/fp.asm>].
;
;               Se il numero è troppo grande la ROM segnala OVERFLOW ERROR e non
;               ritorna al chiamante: vedi [NOTA #07 in <fp/fp.asm>].
;
;               La ROM altera queste celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: INDEX, RESHO, $56, $5D-$60 e $67.
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - .addr_ (1 o 2 byte, Macro) : Indirizzo della stringa
;   - (.addr_) (n byte, RAM)     : Stringa terminata da zero
;
; Output:
;   - ARG (6 byte, ZP)           : Numero convertito, registri da ARGEXP a
;                                  ARGSGN
;   - FAC (7 byte, ZP)           : Invariato
;   - TXTPTR (2 byte, ZP)        : Indirizzo del primo carattere dopo il numero
;   - __FP_SAVED_SP (1 byte, ZP) : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)          : Ritorno al chiamante o, se la ROM segnala un
;                                  errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, TXTPTR, _SCRATCH_1, __FP_SAVED_SP
```

segue...

2.1.136 m_String_to_ARG (continua)

```
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_common.asm>
; ============================================================================
```

2.1.137 m_Print_FAC

```
; ================================================================================
; MACRO: m_Print_FAC
; --------------------------------------------------------------------------------
; Descrizione:  Converte il numero in virgola mobile di FAC in una stringa e
;               la stampa.
;
;               La stringa, terminata da zero, è lunga al massimo 15 caratteri
;               più il terminatore: ha uno spazio o il segno meno davanti, e usa
;               la forma esponenziale (1E+10) quando serve. Si trova in STACK,
;               da $0100 in poi.
;
;               La stringa è stampata con la subroutine __PUT_STRING, un punto
;               di personalizzazione di config.asm: per default è STROUT della
;               ROM del BASIC. Una subroutine alternativa deve stampare la
;               stringa in STACK, terminata da $00; ne riceve l'indirizzo in .A
;               (LSB) e .Y (MSB).
;
;               Chiama la routine FOUT della ROM del BASIC, che altera FAC e
;               ARG, e poi __PUT_STRING con l'indirizzo di STACK. Con
;               m_Store_FAC_in_Scratch e m_Store_ARG_in_Scratch la macro salva
;               FAC in _SCRATCH_1 e ARG in _SCRATCH_2 prima della chiamata e li
;               ripristina dopo. Per le variabili del BASIC alterate vedi
;               [NOTA #06 in <fp/fp.asm>].
;
;               La ROM altera queste celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: INDEX, INDEX2, RESHO, $47, $56,
;               $5D-$5E e $71.
;
;               La subroutine __PUT_STRING può alterare a sua volta registri e
;               celle, che non sono elencati qui.
;
;               Per l'accuratezza della conversione vedi
;               [NOTA #08 in <fp/fp.asm>].
;
; Input:
;   - FAC (7 byte, ZP)             : Registri da FACEXP a FACOV
;   - __PUT_STRING (2 byte, Macro) : Indirizzo della subroutine di stampa
;
; Output:
;   - STACK (16 byte, RAM)         : Stringa terminata da zero, da $0100
;   - FAC (7 byte, ZP)             : Invariato
;   - ARG (6 byte, ZP)             : Invariato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, _SCRATCH_1, _SCRATCH_2, STACK (16 byte)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_common.asm>
; ================================================================================
```

2.1.138 m_Print_ARG

```
; ================================================================================
; MACRO: m_Print_ARG
; --------------------------------------------------------------------------------
; Descrizione:  Converte il numero in virgola mobile di ARG in una stringa e
;               la stampa.
;
;               La stringa, terminata da zero, è lunga al massimo 15 caratteri
;               più il terminatore: ha uno spazio o il segno meno davanti, e usa
;               la forma esponenziale (1E+10) quando serve. Si trova in STACK,
;               da $0100 in poi.
;
;               La stringa è stampata con la subroutine __PUT_STRING, un punto
;               di personalizzazione di config.asm: per default è STROUT della
;               ROM del BASIC. Una subroutine alternativa deve stampare la
;               stringa in STACK, terminata da $00; ne riceve l'indirizzo in .A
;               (LSB) e .Y (MSB).
;
;               Copia prima ARG in FAC con m_Transfer_ARG_to_FAC, poi chiama la
;               routine FOUT della ROM del BASIC, che altera FAC e ARG, e poi
;               __PUT_STRING con l'indirizzo di STACK. Con
;               m_Store_FAC_in_Scratch e m_Store_ARG_in_Scratch la macro salva
;               FAC in _SCRATCH_1 e ARG in _SCRATCH_2 prima della copia e li
;               ripristina dopo. Per le variabili del BASIC alterate vedi
;               [NOTA #06 in <fp/fp.asm>].
;
;               La ROM altera queste celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: INDEX, INDEX2, RESHO, $47, $56,
;               $5D-$5E e $71.
;
;               La subroutine __PUT_STRING può alterare a sua volta registri e
;               celle, che non sono elencati qui.
;
;               Per l'accuratezza della conversione vedi
;               [NOTA #08 in <fp/fp.asm>].
;
; Input:
;   - ARG (6 byte, ZP)             : Registri da ARGEXP a ARGSGN
;   - __PUT_STRING (2 byte, Macro) : Indirizzo della subroutine di stampa
;
; Output:
;   - STACK (16 byte, RAM)         : Stringa terminata da zero, da $0100
;   - FAC (7 byte, ZP)             : Invariato
;   - ARG (6 byte, ZP)             : Invariato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, _SCRATCH_1, _SCRATCH_2, STACK (16 byte)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_common.asm>
;                       <fp/fp_memory.asm>
```

segue...

2.1.138 m_Print_ARG (continua)

```
; ================================================================================
```

fp_memory.asm

2.1.139 m_Load_FAC_from_Mem

```
; ================================================================================
; MACRO: m_Load_FAC_from_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con il numero in virgola mobile, in formato MBF (5
;               byte), che si trova a un indirizzo assoluto, e azzera FACOV.
;
;               Chiama la routine MOVFM della ROM del BASIC, che altera anche
;               INDEX: il suo contenuto non è garantito dopo la chiamata.
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo del numero da caricare
;   - (.addr_) (5 byte, RAM) : Numero in formato MBF
;
; Output:
;   - FAC (7 byte, ZP)       : Numero caricato, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.140 m_Load_FAC_from_Ptr

```
; ================================================================================
; MACRO: m_Load_FAC_from_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con il numero in virgola mobile, in formato MBF (5
;               byte), a cui punta un puntatore, e azzera FACOV.
;
;               Chiama la routine MOVFM della ROM del BASIC, che altera anche
;               INDEX: il suo contenuto non è garantito dopo la chiamata.
;
; Input:
;   - .ptr_ (1 o 2 byte, Macro) : Indirizzo del puntatore
;   - (.ptr_) (2 byte, RAM)     : Indirizzo del numero (LSB, MSB)
;   - ((.ptr_)) (5 byte, RAM)   : Numero in formato MBF
;
; Output:
;   - FAC (7 byte, ZP)          : Numero caricato, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.141 m_Load_ARG_from_Mem

```
; ================================================================================
; MACRO: m_Load_ARG_from_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con il numero in virgola mobile, in formato MBF (5
;               byte), che si trova a un indirizzo assoluto. FAC non cambia.
;
;               Chiama la routine CONUPK della ROM del BASIC, che altera anche
;               INDEX: il suo contenuto non è garantito dopo la chiamata. CONUPK
;               aggiorna ARISGN: il bit #7 vale 1 se i segni di FAC e di ARG
;               sono diversi.
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo del numero da caricare
;   - (.addr_) (5 byte, RAM) : Numero in formato MBF
;
; Output:
;   - ARG (6 byte, ZP)       : Numero caricato, registri da ARGEXP a ARGSGN
;   - ARISGN (1 byte, ZP)    : Bit #7: 0 = segni uguali; 1 = segni diversi
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.142 m_Load_ARG_from_Ptr

```
; ================================================================================
; MACRO: m_Load_ARG_from_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Carica ARG con il numero in virgola mobile, in formato MBF (5
;               byte), a cui punta un puntatore. FAC non cambia.
;
;               Chiama la routine CONUPK della ROM del BASIC, che altera anche
;               INDEX: il suo contenuto non è garantito dopo la chiamata. CONUPK
;               aggiorna ARISGN: il bit #7 vale 1 se i segni di FAC e di ARG
;               sono diversi.
;
; Input:
;   - .ptr_ (1 o 2 byte, Macro) : Indirizzo del puntatore
;   - (.ptr_) (2 byte, RAM)     : Indirizzo del numero (LSB, MSB)
;   - ((.ptr_)) (5 byte, RAM)   : Numero in formato MBF
;
; Output:
;   - ARG (6 byte, ZP)          : Numero caricato, registri da ARGEXP a ARGSGN
;   - ARISGN (1 byte, ZP)       : Bit #7: 0 = segni uguali; 1 = segni diversi
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.143 m_Store_FAC_to_Mem

```
; ================================================================================
; MACRO: m_Store_FAC_to_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Scrive FAC in formato MBF (5 byte) a un indirizzo assoluto. FAC
;               viene prima arrotondato, perché il formato in memoria non ha il
;               byte di arrotondamento: dopo la chiamata FAC contiene il valore
;               arrotondato e FACOV vale zero.
;
;               Chiama la routine MOVMF della ROM del BASIC, che altera anche
;               INDEX: il suo contenuto non è garantito dopo la chiamata.
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo di destinazione
;
; Output:
;   - .addr_ (5 byte, Macro) : Riceve FAC in formato MBF
;   - FAC (7 byte, ZP)       : Arrotondato, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.144 m_Store_FAC_to_Ptr

```
; ================================================================================
; MACRO: m_Store_FAC_to_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Scrive FAC in formato MBF (5 byte) a un indirizzo a cui punta un
;               puntatore. FAC viene prima arrotondato, perché il formato in
;               memoria non ha il byte di arrotondamento: dopo la chiamata FAC
;               contiene il valore arrotondato e FACOV vale zero.
;
;               Chiama la routine MOVMF della ROM del BASIC, che altera anche
;               INDEX: il suo contenuto non è garantito dopo la chiamata.
;
; Input:
;   - .ptr_ (1 o 2 byte, Macro) : Indirizzo del puntatore
;   - (.ptr_) (2 byte, RAM)     : Indirizzo di destinazione (LSB, MSB)
;
; Output:
;   - ((.ptr_)) (5 byte, RAM)   : Riceve FAC in formato MBF
;   - FAC (7 byte, ZP)          : Arrotondato, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.145 m_Store_ARG_to_Mem

```
; ================================================================================
; MACRO: m_Store_ARG_to_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Scrive ARG in formato MBF (5 byte) a un indirizzo assoluto,
;               senza chiamare la ROM del BASIC. Nel MSB della mantissa il
;               bit #7, che in ARG è sempre 1, è sostituito dal segno. ARG non
;               cambia e non va arrotondato, perché lo è già.
;
; Input:
;   - .addr_ (2 byte, Macro) : Indirizzo di destinazione
;   - ARG (6 byte, ZP)       : Registri da ARGEXP a ARGSGN
;
; Output:
;   - .addr_ (5 byte, Macro) : Riceve ARG in formato MBF
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero)
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.146 m_Store_ARG_to_Ptr

```
; ============================================================================
; MACRO: m_Store_ARG_to_Ptr
; ----------------------------------------------------------------------------
; Descrizione:  Scrive ARG in formato MBF (5 byte) a un indirizzo a cui punta un
;               puntatore, senza chiamare la ROM del BASIC. Nel MSB della
;               mantissa il bit #7, che in ARG è sempre 1, è sostituito dal
;               segno. ARG non cambia e non va arrotondato, perché lo è già.
;
;               L'indirizzo di destinazione passa per INDEX, il puntatore
;               temporaneo che la ROM usa per queste operazioni.
;
; Input:
;   - .ptr_ (1 o 2 byte, Macro) : Indirizzo del puntatore
;   - (.ptr_) (2 byte, RAM)     : Indirizzo di destinazione (LSB, MSB)
;   - ARG (6 byte, ZP)          : Registri da ARGEXP a ARGSGN
;
; Output:
;   - ((.ptr_)) (5 byte, RAM)   : Riceve ARG in formato MBF
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, Zero), INDEX, INDEX+1
; Dipendenze Esterne:   <c64/symbols.asm>
; ============================================================================
```

2.1.147 m_Transfer_FAC_to_ARG

```
; ================================================================================
; MACRO: m_Transfer_FAC_to_ARG
; --------------------------------------------------------------------------------
; Descrizione:  Copia FAC in ARG. FAC viene prima arrotondato, perché ARG non
;               può contenere un valore non arrotondato: dopo la chiamata FAC
;               contiene il valore arrotondato e FACOV vale zero. ARISGN non
;               viene aggiornato.
;
;               Chiama la routine MOVAF della ROM del BASIC.
;
; Input:
;   - FAC (7 byte, ZP) : Registri da FACEXP a FACOV
;
; Output:
;   - ARG (6 byte, ZP) : Copia di FAC, registri da ARGEXP a ARGSGN
;   - FAC (7 byte, ZP) : Arrotondato, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.148 m_Transfer_ARG_to_FAC

```
; ================================================================================
; MACRO: m_Transfer_ARG_to_FAC
; --------------------------------------------------------------------------------
; Descrizione:  Copia ARG in FAC e azzera FACOV. ARG e ARISGN non cambiano.
;
;               Chiama la routine MOVFA della ROM del BASIC.
;
; Input:
;   - ARG (6 byte, ZP) : Registri da ARGEXP a ARGSGN
;
; Output:
;   - FAC (7 byte, ZP) : Copia di ARG, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.149 m_Swap_FAC_and_ARG

```
; ================================================================================
; MACRO: m_Swap_FAC_and_ARG
; --------------------------------------------------------------------------------
; Descrizione:  Scambia FAC e ARG. FAC viene prima arrotondato, perché ARG non
;               può contenere un valore non arrotondato: dopo lo scambio FACOV
;               vale zero. ARISGN non viene aggiornato.
;
;               Salva ARG in _SCRATCH_2 e chiama la routine MOVAF della ROM
;               del BASIC. Il contenuto di _SCRATCH_2 non è garantito dopo
;               la chiamata.
;
; Input:
;   - FAC (7 byte, ZP) : Registri da FACEXP a FACOV
;   - ARG (6 byte, ZP) : Registri da ARGEXP a ARGSGN
;
; Output:
;   - FAC (7 byte, ZP) : Valore di ARG, FACOV azzerato
;   - ARG (6 byte, ZP) : Valore di FAC, arrotondato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .P (Negative, oVerflow, Zero, Carry), FAC, _SCRATCH
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_common.asm>
; ================================================================================
```

fp_poly.asm

2.1.150 m_Poly_Mem

```
; ================================================================================
; MACRO: m_Poly_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Calcola in FAC il valore di un polinomio di grado n nel punto x
;               che è in FAC: FAC = a_n*x^n + a_(n-1)*x^(n-1) + ... + a_1*x +
;               a_0. I coefficienti sono in una tabella a un indirizzo assoluto.
;
;               Il primo byte della tabella è n; seguono n+1 numeri in formato
;               MBF, i coefficienti da a_n ad a_0. Il valore di n deve essere
;               almeno 1: con n = 0 la ROM esegue 256 iterazioni sui byte che
;               seguono la tabella e di solito segnala OVERFLOW ERROR.
;
;               Chiama la routine POLY2 della ROM del BASIC, che usa il metodo
;               di Horner. Se un risultato intermedio è troppo grande la ROM
;               segnala OVERFLOW ERROR e non ritorna al chiamante: vedi
;               [NOTA #07 in <fp/fp.asm>].
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: INDEX, RESHO, $56, $5C-$60, $67
;               e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #08 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - FAC (7 byte, ZP)               : Punto x, registri da FACEXP a FACOV
;   - .addr_ (1 o 2 byte, Macro)     : Indirizzo della tabella
;   - (.addr_) (1 byte, RAM)         : Valore di n, almeno 1
;   - (.addr_+1) (5*n+5 byte, RAM)   : Coefficienti in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Valore del polinomio, registri da FACEXP
;                                      a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
```

segue...

2.1.150 m_Poly_Mem (continua)

```
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, STACK (5 byte, se __PRESERVE_ARG = 1),
;                       __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ============================================================================
```

2.1.151 m_Poly_Ptr

```
; ============================================================================
; MACRO: m_Poly_Ptr
; ----------------------------------------------------------------------------
; Descrizione:  Calcola in FAC il valore di un polinomio di grado n nel punto x
;               che è in FAC: FAC = a_n*x^n + a_(n-1)*x^(n-1) + ... + a_1*x +
;               a_0. I coefficienti sono in una tabella all'indirizzo a cui
;               punta un puntatore.
;
;               Il primo byte della tabella è n; seguono n+1 numeri in formato
;               MBF, i coefficienti da a_n ad a_0. Il valore di n deve essere
;               almeno 1: con n = 0 la ROM esegue 256 iterazioni sui byte che
;               seguono la tabella e di solito segnala OVERFLOW ERROR.
;
;               Chiama la routine POLY2 della ROM del BASIC, che usa il metodo
;               di Horner. Se un risultato intermedio è troppo grande la ROM
;               segnala OVERFLOW ERROR e non ritorna al chiamante: vedi
;               [NOTA #07 in <fp/fp.asm>].
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: INDEX, RESHO, $56, $5C-$60, $67
;               e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #08 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - FAC (7 byte, ZP)               : Punto x, registri da FACEXP a FACOV
;   - .ptr_ (1 o 2 byte, Macro)      : Indirizzo del puntatore
;   - (.ptr_) (2 byte, RAM)          : Indirizzo della tabella (LSB, MSB)
;   - ((.ptr_)) (1 byte, RAM)        : Valore di n, almeno 1
;   - ((.ptr_)+1) (5*n+5 byte, RAM)  : Coefficienti in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Valore del polinomio, registri da FACEXP
;                                      a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
```

segue...

2.1.151 m_Poly_Ptr (continua)

```
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, STACK (5 byte, se __PRESERVE_ARG = 1),
;                       __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.152 m_Odd_Poly_Mem

```
; ================================================================================
; MACRO: m_Odd_Poly_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Calcola in FAC il valore di un polinomio dispari di grado n, con
;               n dispari, nel punto x che è in FAC: FAC = a_n*x^n +
;               a_(n-2)*x^(n-2) + ... + a_3*x^3 + a_1*x. I coefficienti sono in
;               una tabella a un indirizzo assoluto.
;
;               Il primo byte della tabella è d = (n-1)/2; seguono d+1 numeri in
;               formato MBF, i coefficienti da a_n ad a_1. Il valore di d deve
;               essere almeno 1: con d = 0 la ROM esegue 256 iterazioni sui byte
;               che seguono la tabella e di solito segnala OVERFLOW ERROR.
;
;               Chiama la routine POLY1 della ROM del BASIC, che usa il metodo
;               di Horner. Se un risultato intermedio è troppo grande la ROM
;               segnala OVERFLOW ERROR e non ritorna al chiamante: vedi
;               [NOTA #07 in <fp/fp.asm>].
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: INDEX, RESHO, $56, NUMWRK, $67
;               e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #08 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - FAC (7 byte, ZP)               : Punto x, registri da FACEXP a FACOV
;   - .addr_ (1 o 2 byte, Macro)     : Indirizzo della tabella
;   - (.addr_) (1 byte, RAM)         : Valore di d, almeno 1
;   - (.addr_+1) (5*d+5 byte, RAM)   : Coefficienti in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Valore del polinomio, registri da FACEXP
;                                      a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
```

segue...

2.1.152 m_Odd_Poly_Mem (continua)

```
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, STACK (5 byte, se __PRESERVE_ARG = 1),
;                       __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.153 m_Odd_Poly_Ptr

```
; ================================================================================
; MACRO: m_Odd_Poly_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Calcola in FAC il valore di un polinomio dispari di grado n, con
;               n dispari, nel punto x che è in FAC: FAC = a_n*x^n +
;               a_(n-2)*x^(n-2) + ... + a_3*x^3 + a_1*x. I coefficienti sono in
;               una tabella all'indirizzo a cui punta un puntatore.
;
;               Il primo byte della tabella è d = (n-1)/2; seguono d+1 numeri in
;               formato MBF, i coefficienti da a_n ad a_1. Il valore di d deve
;               essere almeno 1: con d = 0 la ROM esegue 256 iterazioni sui byte
;               che seguono la tabella e di solito segnala OVERFLOW ERROR.
;
;               Chiama la routine POLY1 della ROM del BASIC, che usa il metodo
;               di Horner. Se un risultato intermedio è troppo grande la ROM
;               segnala OVERFLOW ERROR e non ritorna al chiamante: vedi
;               [NOTA #07 in <fp/fp.asm>].
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: INDEX, RESHO, $56, NUMWRK, $67
;               e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #08 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - FAC (7 byte, ZP)               : Punto x, registri da FACEXP a FACOV
;   - .ptr_ (1 o 2 byte, Macro)      : Indirizzo del puntatore
;   - (.ptr_) (2 byte, RAM)          : Indirizzo della tabella (LSB, MSB)
;   - ((.ptr_)) (1 byte, RAM)        : Valore di d, almeno 1
;   - ((.ptr_)+1) (5*d+5 byte, RAM)  : Coefficienti in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Valore del polinomio, registri da FACEXP
;                                      a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
```

segue...

2.1.153 m_Odd_Poly_Ptr (continua)

```
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, STACK (5 byte, se __PRESERVE_ARG = 1),
;                       __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

fp_power.asm

2.1.154 m_Power_ARG_to_FAC

```
; ================================================================================
; MACRO: m_Power_ARG_to_FAC
; --------------------------------------------------------------------------------
; Descrizione:  Eleva il numero in virgola mobile di ARG alla potenza di FAC e
;               lascia il risultato in FAC: FAC = ARG ^ FAC.
;
;               Chiama la routine FPWRT della ROM del BASIC, che calcola
;               EXP(LOG(ARG) x FAC). I casi particolari sono: se FAC è zero il
;               risultato è 1, anche con base zero; se la base è zero, e FAC non
;               lo è, il risultato è zero, anche con esponente negativo; con
;               base negativa l'esponente deve essere intero, altrimenti la ROM
;               segnala ILLEGAL QUANTITY ERROR; se il risultato è troppo grande
;               la ROM segnala OVERFLOW ERROR, se è troppo piccolo il risultato
;               è zero. Se la ROM segnala un errore non ritorna al chiamante:
;               vedi [NOTA #07 in <fp/fp.asm>].
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07, INDEX, INDEX2, RESHO, $4E-$52,
;               $56, NUMWRK, $67 e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #11 in <fp/fp.asm>],
;               [NOTA #12 in <fp/fp.asm>] e [NOTA #08 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - FAC (7 byte, ZP)              : Registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)              : Registri da ARGEXP a ARGSGN
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                     essere alterato
;
; Output:
;   - FAC (7 byte, ZP)              : Risultato, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)              : Invariato se __PRESERVE_ARG = 1;
;                                     altrimenti non garantito
;   - __FP_SAVED_SP (1 byte, ZP)    : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)             : Ritorno al chiamante o, se la ROM segnala
;                                     un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, INDEX2,
```

segue...

2.1.154 m_Power_ARG_to_FAC (continua)

```
;                       STACK (5 byte, se __PRESERVE_ARG = 1), __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.155 m_Power_Mem_to_FAC

```
; ================================================================================
; MACRO: m_Power_Mem_to_FAC
; --------------------------------------------------------------------------------
; Descrizione:  Eleva il numero in formato MBF che si trova a un indirizzo
;               assoluto alla potenza di FAC e lascia il risultato in FAC: FAC =
;               (.addr_) ^ FAC.
;
;               Carica prima il numero in ARG con m_Load_ARG_from_Mem, che
;               aggiorna ARISGN e altera INDEX.
;
;               Chiama la routine FPWRT della ROM del BASIC, che calcola
;               EXP(LOG(ARG) x FAC). I casi particolari sono: se FAC è zero il
;               risultato è 1, anche con base zero; se la base è zero, e FAC non
;               lo è, il risultato è zero, anche con esponente negativo; con
;               base negativa l'esponente deve essere intero, altrimenti la ROM
;               segnala ILLEGAL QUANTITY ERROR; se il risultato è troppo grande
;               la ROM segnala OVERFLOW ERROR, se è troppo piccolo il risultato
;               è zero. Se la ROM segnala un errore non ritorna al chiamante:
;               vedi [NOTA #07 in <fp/fp.asm>].
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07, INDEX, INDEX2, RESHO, $4E-$52,
;               $56, NUMWRK, $67 e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #11 in <fp/fp.asm>],
;               [NOTA #12 in <fp/fp.asm>] e [NOTA #08 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - FAC (7 byte, ZP)               : Registri da FACEXP a FACOV
;   - .addr_ (1 o 2 byte, Macro)     : Indirizzo del numero
;   - (.addr_) (5 byte, RAM)         : Numero in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Risultato, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
```

segue...

2.1.155 m_Power_Mem_to_FAC (continua)

```
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, INDEX2,
;                       STACK (5 byte, se __PRESERVE_ARG = 1), __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.156 m_Power_Ptr_to_FAC

```
; ================================================================================
; MACRO: m_Power_Ptr_to_FAC
; --------------------------------------------------------------------------------
; Descrizione:  Eleva il numero in formato MBF che si trova all'indirizzo a cui
;               punta un puntatore alla potenza di FAC e lascia il risultato in
;               FAC: FAC = ((.ptr_)) ^ FAC.
;
;               Carica prima il numero in ARG con m_Load_ARG_from_Ptr, che
;               aggiorna ARISGN e altera INDEX.
;
;               Chiama la routine FPWRT della ROM del BASIC, che calcola
;               EXP(LOG(ARG) x FAC). I casi particolari sono: se FAC è zero il
;               risultato è 1, anche con base zero; se la base è zero, e FAC non
;               lo è, il risultato è zero, anche con esponente negativo; con
;               base negativa l'esponente deve essere intero, altrimenti la ROM
;               segnala ILLEGAL QUANTITY ERROR; se il risultato è troppo grande
;               la ROM segnala OVERFLOW ERROR, se è troppo piccolo il risultato
;               è zero. Se la ROM segnala un errore non ritorna al chiamante:
;               vedi [NOTA #07 in <fp/fp.asm>].
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07, INDEX, INDEX2, RESHO, $4E-$52,
;               $56, NUMWRK, $67 e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #11 in <fp/fp.asm>],
;               [NOTA #12 in <fp/fp.asm>] e [NOTA #08 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - FAC (7 byte, ZP)               : Registri da FACEXP a FACOV
;   - .ptr_ (1 o 2 byte, Macro)      : Indirizzo del puntatore
;   - (.ptr_) (2 byte, RAM)          : Indirizzo del numero (LSB, MSB)
;   - ((.ptr_)) (5 byte, RAM)        : Numero in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Risultato, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
```

segue...

2.1.156 m_Power_Ptr_to_FAC (continua)

```
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, INDEX2,
;                       STACK (5 byte, se __PRESERVE_ARG = 1), __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.157 m_SQR_FAC

```
; ============================================================================
; MACRO: m_SQR_FAC
; ----------------------------------------------------------------------------
; Descrizione:  Calcola la radice quadrata del numero in virgola mobile di FAC e
;               lascia il risultato in FAC.
;
;               Chiama la routine SQR della ROM del BASIC, che copia FAC in ARG,
;               carica 0.5 in FAC e chiama FPWRT: calcola quindi ARG ^ 0.5. La
;               radice di zero è zero; se il numero è negativo la ROM segnala
;               ILLEGAL QUANTITY ERROR e non ritorna al chiamante: vedi
;               [NOTA #07 in <fp/fp.asm>].
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07, INDEX, INDEX2, RESHO, $4E-$52,
;               $56, NUMWRK, $67 e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #09 in <fp/fp.asm>] e
;               [NOTA #08 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - FAC (7 byte, ZP)               : Registri da FACEXP a FACOV
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Radice quadrata, registri da FACEXP a
;                                      FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, INDEX2,
;                       STACK (5 byte, se __PRESERVE_ARG = 1), __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ============================================================================
```

2.1.158 m_SQR_ARG

```
; ============================================================================
; MACRO: m_SQR_ARG
; ----------------------------------------------------------------------------
; Descrizione:  Calcola la radice quadrata del numero in virgola mobile di ARG e
;               lascia il risultato in FAC, che perde il suo vecchio contenuto.
;
;               Copia prima ARG in FAC con m_Transfer_ARG_to_FAC.
;
;               Chiama la routine SQR della ROM del BASIC, che copia FAC in ARG,
;               carica 0.5 in FAC e chiama FPWRT: calcola quindi ARG ^ 0.5. La
;               radice di zero è zero; se il numero è negativo la ROM segnala
;               ILLEGAL QUANTITY ERROR e non ritorna al chiamante: vedi
;               [NOTA #07 in <fp/fp.asm>].
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07, INDEX, INDEX2, RESHO, $4E-$52,
;               $56, NUMWRK, $67 e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #09 in <fp/fp.asm>] e
;               [NOTA #08 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - ARG (6 byte, ZP)               : Registri da ARGEXP a ARGSGN
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Radice quadrata, registri da FACEXP a
;                                      FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, INDEX2,
;                       STACK (5 byte, se __PRESERVE_ARG = 1), __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
```

segue...

2.1.158 m_SQR_ARG (continua)

```
;                       <fp/fp_memory.asm>
; ============================================================================
```

2.1.159 m_SQR_Mem

```
; ================================================================================
; MACRO: m_SQR_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Calcola la radice quadrata del numero in formato MBF che si
;               trova a un indirizzo assoluto e lascia il risultato in FAC, che
;               perde il suo vecchio contenuto.
;
;               Carica prima il numero in FAC con m_Load_FAC_from_Mem, che
;               altera INDEX.
;
;               Chiama la routine SQR della ROM del BASIC, che copia FAC in ARG,
;               carica 0.5 in FAC e chiama FPWRT: calcola quindi ARG ^ 0.5. La
;               radice di zero è zero; se il numero è negativo la ROM segnala
;               ILLEGAL QUANTITY ERROR e non ritorna al chiamante: vedi
;               [NOTA #07 in <fp/fp.asm>].
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07, INDEX, INDEX2, RESHO, $4E-$52,
;               $56, NUMWRK, $67 e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #09 in <fp/fp.asm>] e
;               [NOTA #08 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - .addr_ (1 o 2 byte, Macro)     : Indirizzo del numero
;   - (.addr_) (5 byte, RAM)         : Numero in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Radice quadrata, registri da FACEXP a
;                                      FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
```

segue...

2.1.159 m_SQR_Mem (continua)

```
;                       ARISGN, INDEX, INDEX2,
;                       STACK (5 byte, se __PRESERVE_ARG = 1), __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ============================================================================
```

2.1.160 m_SQR_Ptr

```
; ================================================================================
; MACRO: m_SQR_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Calcola la radice quadrata del numero in formato MBF che si
;               trova all'indirizzo a cui punta un puntatore e lascia il
;               risultato in FAC, che perde il suo vecchio contenuto.
;
;               Carica prima il numero in FAC con m_Load_FAC_from_Ptr, che
;               altera INDEX.
;
;               Chiama la routine SQR della ROM del BASIC, che copia FAC in ARG,
;               carica 0.5 in FAC e chiama FPWRT: calcola quindi ARG ^ 0.5. La
;               radice di zero è zero; se il numero è negativo la ROM segnala
;               ILLEGAL QUANTITY ERROR e non ritorna al chiamante: vedi
;               [NOTA #07 in <fp/fp.asm>].
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07, INDEX, INDEX2, RESHO, $4E-$52,
;               $56, NUMWRK, $67 e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #09 in <fp/fp.asm>] e
;               [NOTA #08 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - .ptr_ (1 o 2 byte, Macro)      : Indirizzo del puntatore
;   - (.ptr_) (2 byte, RAM)          : Indirizzo del numero (LSB, MSB)
;   - ((.ptr_)) (5 byte, RAM)        : Numero in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Radice quadrata, registri da FACEXP a
;                                      FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
```

segue...

2.1.160 m_SQR_Ptr (continua)

```
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, INDEX2,
;                       STACK (5 byte, se __PRESERVE_ARG = 1), __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

fp_random.asm

2.1.161 m_Load_FAC_with_RND

```
; ================================================================================
; MACRO: m_Load_FAC_with_RND
; --------------------------------------------------------------------------------
; Descrizione:  Genera in FAC un numero pseudocasuale in virgola mobile tra 0 e
;               1, 1 escluso, con la routine RND della ROM del BASIC. Il
;               generatore deve essere inizializzato prima: vedi m_Randomize.
;
;               Il segno di FAC prima della chiamata decide il comportamento.
;               Con FAC > 0 FAC è sostituito dal numero successivo della
;               sequenza e il suo valore non conta. Con FAC = 0 il numero deriva
;               dai registri dei timer del CIA #1. Con FAC < 0 il generatore è
;               reinizializzato con il numero che è in FAC, e FAC contiene il
;               nuovo seme: con un seme piccolo è un numero molto piccolo.
;
;               Un seme negativo genera sempre la stessa sequenza: è utile per
;               collaudo e debug. Per i limiti del caso FAC = 0 e per le altre
;               particolarità di RND vedi [NOTA #10 in <fp/fp.asm>].
;
;               La ROM altera queste celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: INDEX, RESHO e $56. Aggiorna inoltre
;               il seme, in RNDX.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in _SCRATCH_2 prima
;               della chiamata e lo ripristina dopo, perché con FAC > 0 la ROM
;               altera ARG; se vale 0 il contenuto di ARG non è garantito dopo
;               la chiamata. Un ARG uguale a zero (ARGEXP = 0) torna con ARGHO =
;               $80, ed è ancora zero.
;
;               Con FAC > 0 la ROM usa una moltiplicazione: per l'accuratezza
;               vedi [NOTA #08 in <fp/fp.asm>].
;
; Input:
;   - FAC (7 byte, ZP)              : Segno: > 0 numero successivo; = 0 seme
;                                     dal CIA #1; < 0 nuovo seme
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                     essere alterato
;
; Output:
;   - FAC (7 byte, ZP)              : Numero tra 0 e 1, registri da FACEXP a
;                                     FACOV
;   - ARG (6 byte, ZP)              : Invariato se __PRESERVE_ARG = 1;
;                                     altrimenti non garantito
;   - RNDX (5 byte, ZP)             : Seme aggiornato, da RNDX+0 a RNDX+4
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, RNDX (5 byte), _SCRATCH_2 (se
;                       __PRESERVE_ARG = 1)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_common.asm>
```

segue...

2.1.161 m_Load_FAC_with_RND (continua)

```
; ================================================================================
```

2.1.162 m_Randomize

```
; ================================================================================
; MACRO: m_Randomize
; --------------------------------------------------------------------------------
; Descrizione:  Inizializza il generatore di numeri pseudocasuali con -TI, cioè
;               con il valore dell'orologio jiffy TIME, cambiato di segno.
;               Poiché TI ha 24 bit, 8 bit del seme sarebbero sempre 0, e questo
;               limita i semi possibili.
;
;               Per aggiungere casualità, il numero positivo tra 0 e 1 che la
;               ROM genera con il primo seme è negato e usato come secondo seme:
;               così tutti i 32 bit della mantissa sono valorizzati. Alla fine
;               FAC contiene il seme in uso, un numero positivo tra 0 e 1.
;
;               La macro usa solo i rami di RND con FAC negativo, che non
;               alterano ARG: per questo non ha bisogno di preservarlo e
;               l'interruttore __PRESERVE_ARG non ha effetto. Per le
;               particolarità di RND vedi [NOTA #10 in <fp/fp.asm>].
;
;               La ROM altera la cella di lavoro INDEX, il cui contenuto non è
;               garantito dopo la chiamata. Aggiorna inoltre il seme, in RNDX.
;
; Input:
;   - TIME (3 byte, ZP) : Orologio jiffy
;
; Output:
;   - FAC (7 byte, ZP)  : Seme in uso, numero tra 0 e 1
;   - ARG (6 byte, ZP)  : Invariato
;   - RNDX (5 byte, ZP) : Nuovo seme, da RNDX+0 a RNDX+4
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), INDEX,
;                       RNDX (5 byte)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_integers.asm>
;                       <fp/fp_unary.asm>
; ================================================================================
```

2.1.163 m_LOG_FAC

```
; ============================================================================
; MACRO: m_LOG_FAC
; ----------------------------------------------------------------------------
; Descrizione:  Calcola il logaritmo naturale del numero in virgola mobile di
;               FAC e lascia il risultato in FAC.
;
;               Chiama la routine LOG della ROM del BASIC. Se il numero è zero o
;               negativo la ROM segnala ILLEGAL QUANTITY ERROR e non ritorna al
;               chiamante: vedi [NOTA #07 in <fp/fp.asm>]. LOG(1) vale zero.
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07, INDEX, RESHO, $56, NUMWRK, $67
;               e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #11 in <fp/fp.asm>] e
;               [NOTA #08 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - FAC (7 byte, ZP)               : Registri da FACEXP a FACOV
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Logaritmo, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, STACK (5 byte, se __PRESERVE_ARG = 1),
;                       __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ============================================================================
```

2.1.164 m_LOG_ARG

```
; ============================================================================
; MACRO: m_LOG_ARG
; ----------------------------------------------------------------------------
; Descrizione:  Calcola il logaritmo naturale di ARG e lascia il risultato in
;               FAC, che perde il suo vecchio contenuto.
;
;               Copia prima ARG in FAC con m_Transfer_ARG_to_FAC.
;
;               Chiama la routine LOG della ROM del BASIC. Se il numero è zero o
;               negativo la ROM segnala ILLEGAL QUANTITY ERROR e non ritorna al
;               chiamante: vedi [NOTA #07 in <fp/fp.asm>]. LOG(1) vale zero.
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07, INDEX, RESHO, $56, NUMWRK, $67
;               e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #11 in <fp/fp.asm>] e
;               [NOTA #08 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - ARG (6 byte, ZP)               : Registri da ARGEXP a ARGSGN
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Logaritmo, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, STACK (5 byte, se __PRESERVE_ARG = 1),
;                       __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ============================================================================
```

2.1.165 m_LOG_Mem

```
; ============================================================================
; MACRO: m_LOG_Mem
; ----------------------------------------------------------------------------
; Descrizione:  Calcola il logaritmo naturale del numero in formato MBF che si
;               trova a un indirizzo assoluto e lascia il risultato in FAC, che
;               perde il suo vecchio contenuto.
;
;               Carica prima il numero in FAC con m_Load_FAC_from_Mem, che
;               altera INDEX.
;
;               Chiama la routine LOG della ROM del BASIC. Se il numero è zero o
;               negativo la ROM segnala ILLEGAL QUANTITY ERROR e non ritorna al
;               chiamante: vedi [NOTA #07 in <fp/fp.asm>]. LOG(1) vale zero.
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07, INDEX, RESHO, $56, NUMWRK, $67
;               e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #11 in <fp/fp.asm>] e
;               [NOTA #08 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - .addr_ (1 o 2 byte, Macro)     : Indirizzo del numero
;   - (.addr_) (5 byte, RAM)         : Numero in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Logaritmo, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, STACK (5 byte, se __PRESERVE_ARG = 1),
;                       __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
```

segue...

2.1.165 m_LOG_Mem (continua)

```
;                       <fp/fp_memory.asm>
; ============================================================================
```

2.1.166 m_LOG_Ptr

```
; ================================================================================
; MACRO: m_LOG_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Calcola il logaritmo naturale del numero in formato MBF che si
;               trova all'indirizzo a cui punta un puntatore e lascia il
;               risultato in FAC, che perde il suo vecchio contenuto.
;
;               Carica prima il numero in FAC con m_Load_FAC_from_Ptr, che
;               altera INDEX.
;
;               Chiama la routine LOG della ROM del BASIC. Se il numero è zero o
;               negativo la ROM segnala ILLEGAL QUANTITY ERROR e non ritorna al
;               chiamante: vedi [NOTA #07 in <fp/fp.asm>]. LOG(1) vale zero.
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07, INDEX, RESHO, $56, NUMWRK, $67
;               e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #11 in <fp/fp.asm>] e
;               [NOTA #08 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - .ptr_ (1 o 2 byte, Macro)      : Indirizzo del puntatore
;   - (.ptr_) (2 byte, RAM)          : Indirizzo del numero (LSB, MSB)
;   - ((.ptr_)) (5 byte, RAM)        : Numero in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Logaritmo, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, STACK (5 byte, se __PRESERVE_ARG = 1),
;                       __FP_SAVED_SP
```

segue...

2.1.166 m_LOG_Ptr (continua)

```
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ============================================================================
```

2.1.167 m_EXP_FAC

```
; ============================================================================
; MACRO: m_EXP_FAC
; ----------------------------------------------------------------------------
; Descrizione:  Calcola e elevato al numero in virgola mobile di FAC e lascia il
;               risultato in FAC.
;
;               Chiama la routine EXP della ROM del BASIC. Se il risultato è
;               troppo grande, cioè per un numero da circa 88.03 in su, la ROM
;               segnala OVERFLOW ERROR e non ritorna al chiamante: vedi
;               [NOTA #07 in <fp/fp.asm>]; se è troppo piccolo il risultato è
;               zero. EXP(0) vale 1.
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07, INDEX, RESHO, $56, NUMWRK, $67
;               e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #12 in <fp/fp.asm>] e
;               [NOTA #08 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - FAC (7 byte, ZP)               : Registri da FACEXP a FACOV
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Risultato, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, STACK (5 byte, se __PRESERVE_ARG = 1),
;                       __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ============================================================================
```

2.1.168 m_EXP_ARG

```
; ============================================================================
; MACRO: m_EXP_ARG
; ----------------------------------------------------------------------------
; Descrizione:  Calcola e elevato a ARG e lascia il risultato in FAC, che perde
;               il suo vecchio contenuto.
;
;               Copia prima ARG in FAC con m_Transfer_ARG_to_FAC.
;
;               Chiama la routine EXP della ROM del BASIC. Se il risultato è
;               troppo grande, cioè per un numero da circa 88.03 in su, la ROM
;               segnala OVERFLOW ERROR e non ritorna al chiamante: vedi
;               [NOTA #07 in <fp/fp.asm>]; se è troppo piccolo il risultato è
;               zero. EXP(0) vale 1.
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07, INDEX, RESHO, $56, NUMWRK, $67
;               e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #12 in <fp/fp.asm>] e
;               [NOTA #08 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - ARG (6 byte, ZP)               : Registri da ARGEXP a ARGSGN
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Risultato, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, STACK (5 byte, se __PRESERVE_ARG = 1),
;                       __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
```

segue...

2.1.168 m_EXP_ARG (continua)

```
; ===========================================================================
```

2.1.169 m_EXP_Mem

```
; ============================================================================
; MACRO: m_EXP_Mem
; ----------------------------------------------------------------------------
; Descrizione:  Calcola e elevato al numero in formato MBF che si trova a un
;               indirizzo assoluto e lascia il risultato in FAC, che perde il
;               suo vecchio contenuto.
;
;               Carica prima il numero in FAC con m_Load_FAC_from_Mem, che
;               altera INDEX.
;
;               Chiama la routine EXP della ROM del BASIC. Se il risultato è
;               troppo grande, cioè per un numero da circa 88.03 in su, la ROM
;               segnala OVERFLOW ERROR e non ritorna al chiamante: vedi
;               [NOTA #07 in <fp/fp.asm>]; se è troppo piccolo il risultato è
;               zero. EXP(0) vale 1.
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07, INDEX, RESHO, $56, NUMWRK, $67
;               e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #12 in <fp/fp.asm>] e
;               [NOTA #08 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - .addr_ (1 o 2 byte, Macro)     : Indirizzo del numero
;   - (.addr_) (5 byte, RAM)         : Numero in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Risultato, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, STACK (5 byte, se __PRESERVE_ARG = 1),
```

segue...

2.1.169 m_EXP_Mem (continua)

```
;                         __FP_SAVED_SP
; Dipendenze Esterne:     <c64/symbols.asm>
;                         <fp/fp_memory.asm>
; ==============================================================================
```

2.1.170 m_EXP_Ptr

```
; ==============================================================================
; MACRO: m_EXP_Ptr
; ------------------------------------------------------------------------------
; Descrizione:  Calcola e elevato al numero in formato MBF che si trova
;               all'indirizzo a cui punta un puntatore e lascia il risultato in
;               FAC, che perde il suo vecchio contenuto.
;
;               Carica prima il numero in FAC con m_Load_FAC_from_Ptr, che
;               altera INDEX.
;
;               Chiama la routine EXP della ROM del BASIC. Se il risultato è
;               troppo grande, cioè per un numero da circa 88.03 in su, la ROM
;               segnala OVERFLOW ERROR e non ritorna al chiamante: vedi
;               [NOTA #07 in <fp/fp.asm>]; se è troppo piccolo il risultato è
;               zero. EXP(0) vale 1.
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07, INDEX, RESHO, $56, NUMWRK, $67
;               e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #12 in <fp/fp.asm>] e
;               [NOTA #08 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - .ptr_ (1 o 2 byte, Macro)      : Indirizzo del puntatore
;   - (.ptr_) (2 byte, RAM)          : Indirizzo del numero (LSB, MSB)
;   - ((.ptr_)) (5 byte, RAM)        : Numero in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Risultato, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
```

segue...

2.1.170 m_EXP_Ptr (continua)

```
;                       ARISGN, INDEX, STACK (5 byte, se __PRESERVE_ARG = 1),
;                       __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ==============================================================================
```

fp_trig.asm

2.1.171 m_SIN_FAC

```
; ================================================================================
; MACRO: m_SIN_FAC
; --------------------------------------------------------------------------------
; Descrizione:  Calcola il seno del numero in virgola mobile di FAC e lascia il
;               risultato in FAC. L'angolo è in radianti.
;
;               Chiama la routine SIN della ROM del BASIC.
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07, $12, INDEX, RESHO, $56, NUMWRK,
;               $67 e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #13 in <fp/fp.asm>] e
;               [NOTA #08 in <fp/fp.asm>].
;
; Input:
;   - FAC (7 byte, ZP)               : Registri da FACEXP a FACOV
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Seno, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, STACK (5 byte, se __PRESERVE_ARG = 1)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.172 m_SIN_ARG

```
; ================================================================================
; MACRO: m_SIN_ARG
; --------------------------------------------------------------------------------
; Descrizione:  Calcola il seno del numero in virgola mobile di ARG e lascia il
;               risultato in FAC, che perde il suo vecchio contenuto. L'angolo è
;               in radianti.
;
;               Copia prima ARG in FAC con m_Transfer_ARG_to_FAC.
;
;               Chiama la routine SIN della ROM del BASIC.
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07, $12, INDEX, RESHO, $56, NUMWRK,
;               $67 e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #13 in <fp/fp.asm>] e
;               [NOTA #08 in <fp/fp.asm>].
;
; Input:
;   - ARG (6 byte, ZP)               : Registri da ARGEXP a ARGSGN
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Seno, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, STACK (5 byte, se __PRESERVE_ARG = 1)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.173 m_SIN_Mem

```
; ================================================================================
; MACRO: m_SIN_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Calcola il seno del numero in formato MBF che si trova a un
;               indirizzo assoluto e lascia il risultato in FAC, che perde il
;               suo vecchio contenuto. L'angolo è in radianti.
;
;               Carica prima il numero in FAC con m_Load_FAC_from_Mem, che
;               altera INDEX.
;
;               Chiama la routine SIN della ROM del BASIC.
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07, $12, INDEX, RESHO, $56, NUMWRK,
;               $67 e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #13 in <fp/fp.asm>] e
;               [NOTA #08 in <fp/fp.asm>].
;
; Input:
;   - .addr_ (1 o 2 byte, Macro)     : Indirizzo del numero
;   - (.addr_) (5 byte, RAM)         : Numero in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Seno, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, STACK (5 byte, se __PRESERVE_ARG = 1)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.174 m_SIN_Ptr

```
; ============================================================================
; MACRO: m_SIN_Ptr
; ----------------------------------------------------------------------------
; Descrizione:  Calcola il seno del numero in formato MBF che si trova
;               all'indirizzo a cui punta un puntatore e lascia il risultato in
;               FAC, che perde il suo vecchio contenuto. L'angolo è in radianti.
;
;               Carica prima il numero in FAC con m_Load_FAC_from_Ptr, che
;               altera INDEX.
;
;               Chiama la routine SIN della ROM del BASIC.
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07, $12, INDEX, RESHO, $56, NUMWRK,
;               $67 e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #13 in <fp/fp.asm>] e
;               [NOTA #08 in <fp/fp.asm>].
;
; Input:
;   - .ptr_ (1 o 2 byte, Macro)      : Indirizzo del puntatore
;   - (.ptr_) (2 byte, RAM)          : Indirizzo del numero (LSB, MSB)
;   - ((.ptr_)) (5 byte, RAM)        : Numero in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Seno, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, STACK (5 byte, se __PRESERVE_ARG = 1)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ============================================================================
```

2.1.175 m_COS_FAC

```
; ============================================================================
; MACRO: m_COS_FAC
; ----------------------------------------------------------------------------
; Descrizione:  Calcola il coseno del numero in virgola mobile di FAC e lascia
;               il risultato in FAC. L'angolo è in radianti.
;
;               Chiama la routine COS della ROM del BASIC.
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07, $12, INDEX, RESHO, $56, NUMWRK,
;               $67 e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #14 in <fp/fp.asm>] e
;               [NOTA #08 in <fp/fp.asm>].
;
; Input:
;   - FAC (7 byte, ZP)               : Registri da FACEXP a FACOV
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Coseno, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, STACK (5 byte, se __PRESERVE_ARG = 1)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ============================================================================
```

2.1.176 m_COS_ARG

```
; ================================================================================
; MACRO: m_COS_ARG
; --------------------------------------------------------------------------------
; Descrizione:  Calcola il coseno del numero in virgola mobile di ARG e lascia
;               il risultato in FAC, che perde il suo vecchio contenuto.
;               L'angolo è in radianti.
;
;               Copia prima ARG in FAC con m_Transfer_ARG_to_FAC.
;
;               Chiama la routine COS della ROM del BASIC.
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07, $12, INDEX, RESHO, $56, NUMWRK,
;               $67 e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #14 in <fp/fp.asm>] e
;               [NOTA #08 in <fp/fp.asm>].
;
; Input:
;   - ARG (6 byte, ZP)               : Registri da ARGEXP a ARGSGN
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Coseno, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, STACK (5 byte, se __PRESERVE_ARG = 1)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.177 m_COS_Mem

```
; ================================================================================
; MACRO: m_COS_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Calcola il coseno del numero in formato MBF che si trova a un
;               indirizzo assoluto e lascia il risultato in FAC, che perde il
;               suo vecchio contenuto. L'angolo è in radianti.
;
;               Carica prima il numero in FAC con m_Load_FAC_from_Mem, che
;               altera INDEX.
;
;               Chiama la routine COS della ROM del BASIC.
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07, $12, INDEX, RESHO, $56, NUMWRK,
;               $67 e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #14 in <fp/fp.asm>] e
;               [NOTA #08 in <fp/fp.asm>].
;
; Input:
;   - .addr_ (1 o 2 byte, Macro)     : Indirizzo del numero
;   - (.addr_) (5 byte, RAM)         : Numero in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Coseno, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, STACK (5 byte, se __PRESERVE_ARG = 1)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.178 m_COS_Ptr

```
; ================================================================================
; MACRO: m_COS_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Calcola il coseno del numero in formato MBF che si trova
;               all'indirizzo a cui punta un puntatore e lascia il risultato in
;               FAC, che perde il suo vecchio contenuto. L'angolo è in radianti.
;
;               Carica prima il numero in FAC con m_Load_FAC_from_Ptr, che
;               altera INDEX.
;
;               Chiama la routine COS della ROM del BASIC.
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07, $12, INDEX, RESHO, $56, NUMWRK,
;               $67 e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #14 in <fp/fp.asm>] e
;               [NOTA #08 in <fp/fp.asm>].
;
; Input:
;   - .ptr_ (1 o 2 byte, Macro)      : Indirizzo del puntatore
;   - (.ptr_) (2 byte, RAM)          : Indirizzo del numero (LSB, MSB)
;   - ((.ptr_)) (5 byte, RAM)        : Numero in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Coseno, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, STACK (5 byte, se __PRESERVE_ARG = 1)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.179 m_TAN_FAC

```
; ================================================================================
; MACRO: m_TAN_FAC
; --------------------------------------------------------------------------------
; Descrizione:  Calcola il tangente del numero in virgola mobile di FAC e lascia
;               il risultato in FAC. L'angolo è in radianti.
;
;               Chiama la routine TAN della ROM del BASIC. Se il coseno è zero
;               la ROM segnala DIVISION BY ZERO ERROR e non ritorna al
;               chiamante: vedi [NOTA #07 in <fp/fp.asm>].
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07, $12, INDEX, RESHO, $4E-$52,
;               $56, NUMWRK, $67 e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #15 in <fp/fp.asm>] e
;               [NOTA #08 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - FAC (7 byte, ZP)               : Registri da FACEXP a FACOV
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Tangente, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, STACK (5 byte, se __PRESERVE_ARG = 1),
;                       __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.180 m_TAN_ARG

```
; ================================================================================
; MACRO: m_TAN_ARG
; --------------------------------------------------------------------------------
; Descrizione:  Calcola il tangente del numero in virgola mobile di ARG e lascia
;               il risultato in FAC, che perde il suo vecchio contenuto.
;               L'angolo è in radianti.
;
;               Copia prima ARG in FAC con m_Transfer_ARG_to_FAC.
;
;               Chiama la routine TAN della ROM del BASIC. Se il coseno è zero
;               la ROM segnala DIVISION BY ZERO ERROR e non ritorna al
;               chiamante: vedi [NOTA #07 in <fp/fp.asm>].
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07, $12, INDEX, RESHO, $4E-$52,
;               $56, NUMWRK, $67 e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #15 in <fp/fp.asm>] e
;               [NOTA #08 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - ARG (6 byte, ZP)               : Registri da ARGEXP a ARGSGN
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Tangente, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, STACK (5 byte, se __PRESERVE_ARG = 1),
;                       __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.181 m_TAN_Mem

```
; ============================================================================
; MACRO: m_TAN_Mem
; ----------------------------------------------------------------------------
; Descrizione:  Calcola il tangente del numero in formato MBF che si trova a un
;               indirizzo assoluto e lascia il risultato in FAC, che perde il
;               suo vecchio contenuto. L'angolo è in radianti.
;
;               Carica prima il numero in FAC con m_Load_FAC_from_Mem, che
;               altera INDEX.
;
;               Chiama la routine TAN della ROM del BASIC. Se il coseno è zero
;               la ROM segnala DIVISION BY ZERO ERROR e non ritorna al
;               chiamante: vedi [NOTA #07 in <fp/fp.asm>].
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07, $12, INDEX, RESHO, $4E-$52,
;               $56, NUMWRK, $67 e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #15 in <fp/fp.asm>] e
;               [NOTA #08 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - .addr_ (1 o 2 byte, Macro)     : Indirizzo del numero
;   - (.addr_) (5 byte, RAM)         : Numero in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Tangente, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, STACK (5 byte, se __PRESERVE_ARG = 1),
;                       __FP_SAVED_SP
; Dipendenze Esterne:   <c64/symbols.asm>
```

segue...

2.1.181 m_TAN_Mem (continua)

```
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.182 m_TAN_Ptr

```
; ================================================================================
; MACRO: m_TAN_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Calcola il tangente del numero in formato MBF che si trova
;               all'indirizzo a cui punta un puntatore e lascia il risultato in
;               FAC, che perde il suo vecchio contenuto. L'angolo è in radianti.
;
;               Carica prima il numero in FAC con m_Load_FAC_from_Ptr, che
;               altera INDEX.
;
;               Chiama la routine TAN della ROM del BASIC. Se il coseno è zero
;               la ROM segnala DIVISION BY ZERO ERROR e non ritorna al
;               chiamante: vedi [NOTA #07 in <fp/fp.asm>].
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: $07, $12, INDEX, RESHO, $4E-$52,
;               $56, NUMWRK, $67 e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #15 in <fp/fp.asm>] e
;               [NOTA #08 in <fp/fp.asm>].
;
;               Prima della chiamata alla ROM la macro salva .S in
;               __FP_SAVED_SP, a uso di FP_ERROR_HANDLER.
;
; Input:
;   - .ptr_ (1 o 2 byte, Macro)      : Indirizzo del puntatore
;   - (.ptr_) (2 byte, RAM)          : Indirizzo del numero (LSB, MSB)
;   - ((.ptr_)) (5 byte, RAM)        : Numero in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Tangente, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;   - __FP_SAVED_SP (1 byte, ZP)     : Valore di .S prima della chiamata
;   - .PC (2 byte, CPU)              : Ritorno al chiamante o, se la ROM segnala
;                                      un errore, salto al vettore IERROR
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, STACK (5 byte, se __PRESERVE_ARG = 1),
;                       __FP_SAVED_SP
```

segue...

2.1.182 m_TAN_Ptr (continua)

```
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ==============================================================================
```

2.1.183 m_ATN_FAC

```
; ============================================================================
; MACRO: m_ATN_FAC
; ----------------------------------------------------------------------------
; Descrizione:  Calcola il arcotangente del numero in virgola mobile di FAC e
;               lascia il risultato in FAC. Il risultato è un angolo in
;               radianti, tra -PI/2 e PI/2.
;
;               Chiama la routine ATN della ROM del BASIC.
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: INDEX, RESHO, $56, NUMWRK, $67
;               e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #16 in <fp/fp.asm>] e
;               [NOTA #08 in <fp/fp.asm>].
;
; Input:
;   - FAC (7 byte, ZP)               : Registri da FACEXP a FACOV
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Arcotangente, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, STACK (5 byte, se __PRESERVE_ARG = 1)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ============================================================================
```

2.1.184 m_ATN_ARG

```
; ================================================================================
; MACRO: m_ATN_ARG
; --------------------------------------------------------------------------------
; Descrizione:  Calcola il arcotangente del numero in virgola mobile di ARG e
;               lascia il risultato in FAC, che perde il suo vecchio contenuto.
;               Il risultato è un angolo in radianti, tra -PI/2 e PI/2.
;
;               Copia prima ARG in FAC con m_Transfer_ARG_to_FAC.
;
;               Chiama la routine ATN della ROM del BASIC.
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: INDEX, RESHO, $56, NUMWRK, $67
;               e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #16 in <fp/fp.asm>] e
;               [NOTA #08 in <fp/fp.asm>].
;
; Input:
;   - ARG (6 byte, ZP)               : Registri da ARGEXP a ARGSGN
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Arcotangente, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, STACK (5 byte, se __PRESERVE_ARG = 1)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.185 m_ATN_Mem

```
; ================================================================================
; MACRO: m_ATN_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Calcola il arcotangente del numero in formato MBF che si trova a
;               un indirizzo assoluto e lascia il risultato in FAC, che perde il
;               suo vecchio contenuto. Il risultato è un angolo in radianti, tra
;               -PI/2 e PI/2.
;
;               Carica prima il numero in FAC con m_Load_FAC_from_Mem, che
;               altera INDEX.
;
;               Chiama la routine ATN della ROM del BASIC.
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: INDEX, RESHO, $56, NUMWRK, $67
;               e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #16 in <fp/fp.asm>] e
;               [NOTA #08 in <fp/fp.asm>].
;
; Input:
;   - .addr_ (1 o 2 byte, Macro)     : Indirizzo del numero
;   - (.addr_) (5 byte, RAM)         : Numero in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Arcotangente, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, STACK (5 byte, se __PRESERVE_ARG = 1)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.186 m_ATN_Ptr

```
; ================================================================================
; MACRO: m_ATN_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Calcola il arcotangente del numero in formato MBF che si trova
;               all'indirizzo a cui punta un puntatore e lascia il risultato in
;               FAC, che perde il suo vecchio contenuto. Il risultato è un
;               angolo in radianti, tra -PI/2 e PI/2.
;
;               Carica prima il numero in FAC con m_Load_FAC_from_Ptr, che
;               altera INDEX.
;
;               Chiama la routine ATN della ROM del BASIC.
;
;               La ROM altera molte celle di lavoro, il cui contenuto non è
;               garantito dopo la chiamata: INDEX, RESHO, $56, NUMWRK, $67
;               e $71-$72.
;
;               Se __PRESERVE_ARG = 1 la macro salva ARG in formato MBF in STACK
;               ($0100-$0104) prima della chiamata e lo ripristina dopo, perché
;               la ROM usa NUMWRK, dove si trova _SCRATCH_2; se vale 0 il
;               contenuto di ARG non è garantito dopo la chiamata. Un ARG uguale
;               a zero (ARGEXP = 0) torna con ARGHO = $80, ed è ancora zero. Il
;               contenuto di STACK non è garantito dopo la chiamata.
;
;               Per l'accuratezza del risultato vedi [NOTA #16 in <fp/fp.asm>] e
;               [NOTA #08 in <fp/fp.asm>].
;
; Input:
;   - .ptr_ (1 o 2 byte, Macro)      : Indirizzo del puntatore
;   - (.ptr_) (2 byte, RAM)          : Indirizzo del numero (LSB, MSB)
;   - ((.ptr_)) (5 byte, RAM)        : Numero in formato MBF
;   - __PRESERVE_ARG (1 byte, Macro) : 1 = ARG viene preservato; 0 = ARG può
;                                      essere alterato
;
; Output:
;   - FAC (7 byte, ZP)               : Arcotangente, registri da FACEXP a FACOV
;   - ARG (6 byte, ZP)               : Invariato se __PRESERVE_ARG = 1;
;                                      altrimenti non garantito
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       ARISGN, INDEX, STACK (5 byte, se __PRESERVE_ARG = 1)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

fp_unary.asm

2.1.187 m_Negate_FAC

```
; ================================================================================
; MACRO: m_Negate_FAC
; --------------------------------------------------------------------------------
; Descrizione:  Cambia il segno del numero in virgola mobile di FAC.
;
;               Se FAC è zero (FACEXP = 0) il segno non cambia, come fa
;               la routine della ROM del BASIC, per non produrre uno zero
;               con segno.
;
; Input:
;   - FAC (7 byte, ZP)    : Registri da FACEXP a FACOV
;
; Output:
;   - FACSGN (1 byte, ZP) : Segno invertito, solo il bit #7 è significativo
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), FACSGN
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.188 m_Negate_ARG

```
; ================================================================================
; MACRO: m_Negate_ARG
; --------------------------------------------------------------------------------
; Descrizione:  Cambia il segno del numero in virgola mobile di ARG.
;
;               Se ARG è zero (ARGEXP = 0) il segno non cambia, come fa la
;               routine della ROM del BASIC, per non produrre uno zero con
;               segno. ARISGN non viene aggiornato.
;
; Input:
;   - ARG (6 byte, ZP)    : Registri da ARGEXP a ARGSGN
;
; Output:
;   - ARGSGN (1 byte, ZP) : Segno invertito, solo il bit #7 è significativo
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), ARGSGN
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.189 m_Negate_Mem

```
; ================================================================================
; MACRO: m_Negate_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con il numero in formato MBF che si trova a un
;               indirizzo assoluto, ne cambia il segno e lascia il risultato
;               in FAC.
;
;               La lettura usa m_Load_FAC_from_Mem, che altera anche INDEX: il
;               suo contenuto non è garantito dopo la chiamata. Il vecchio
;               contenuto di FAC si perde.
;
;               Se FAC è zero (FACEXP = 0) il segno non cambia, come fa
;               la routine della ROM del BASIC, per non produrre uno zero
;               con segno.
;
; Input:
;   - .addr_ (1 o 2 byte, Macro) : Indirizzo del numero
;   - (.addr_) (5 byte, RAM)     : Numero in formato MBF
;
; Output:
;   - FAC (7 byte, ZP)           : Numero letto, con segno invertito, registri
;                                  da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry), FACSGN
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.190 m_Negate_Ptr

```
; ================================================================================
; MACRO: m_Negate_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con il numero in formato MBF che si trova
;               all'indirizzo a cui punta un puntatore, ne cambia il segno e
;               lascia il risultato in FAC.
;
;               La lettura usa m_Load_FAC_from_Ptr, che altera anche INDEX: il
;               suo contenuto non è garantito dopo la chiamata. Il vecchio
;               contenuto di FAC si perde.
;
;               Se FAC è zero (FACEXP = 0) il segno non cambia, come fa
;               la routine della ROM del BASIC, per non produrre uno zero
;               con segno.
;
; Input:
;   - .ptr_ (1 o 2 byte, Macro) : Indirizzo del puntatore
;   - (.ptr_) (2 byte, RAM)     : Indirizzo del numero (LSB, MSB)
;   - ((.ptr_)) (5 byte, RAM)   : Numero in formato MBF
;
; Output:
;   - FAC (7 byte, ZP)          : Numero letto, con segno invertito, registri da
;                                 FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry), FACSGN
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.191 m_ABS_FAC

```
; ================================================================================
; MACRO: m_ABS_FAC
; --------------------------------------------------------------------------------
; Descrizione:  Azzera il segno del numero in virgola mobile di FAC, cioè ne
;               calcola il valore assoluto.
;
;               Basta azzerare il bit #7 del segno.
;
; Input:
;   - FAC (7 byte, ZP)    : Registri da FACEXP a FACOV
;
; Output:
;   - FACSGN (1 byte, ZP) : Bit #7 azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .P (Negative, Zero, Carry), FACSGN
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.192 m_ABS_ARG

```
; ================================================================================
; MACRO: m_ABS_ARG
; --------------------------------------------------------------------------------
; Descrizione:  Azzera il segno del numero in virgola mobile di ARG, cioè ne
;               calcola il valore assoluto.
;
;               Basta azzerare il bit #7 del segno. ARISGN non viene aggiornato.
;
; Input:
;   - ARG (6 byte, ZP)    : Registri da ARGEXP a ARGSGN
;
; Output:
;   - ARGSGN (1 byte, ZP) : Bit #7 azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .P (Negative, Zero, Carry), ARGSGN
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.193 m_ABS_Mem

```
; ================================================================================
; MACRO: m_ABS_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con il numero in formato MBF che si trova a un
;               indirizzo assoluto e ne calcola il valore assoluto, lasciando il
;               risultato in FAC.
;
;               La lettura usa m_Load_FAC_from_Mem, che altera anche INDEX: il
;               suo contenuto non è garantito dopo la chiamata. Il vecchio
;               contenuto di FAC si perde.
;
;               Basta azzerare il bit #7 del segno.
;
; Input:
;   - .addr_ (1 o 2 byte, Macro) : Indirizzo del numero
;   - (.addr_) (5 byte, RAM)     : Numero in formato MBF
;
; Output:
;   - FAC (7 byte, ZP)           : Valore assoluto, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry), FACSGN
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.194 m_ABS_Ptr

```
; ================================================================================
; MACRO: m_ABS_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con il numero in formato MBF che si trova
;               all'indirizzo a cui punta un puntatore e ne calcola il valore
;               assoluto, lasciando il risultato in FAC.
;
;               La lettura usa m_Load_FAC_from_Ptr, che altera anche INDEX: il
;               suo contenuto non è garantito dopo la chiamata. Il vecchio
;               contenuto di FAC si perde.
;
;               Basta azzerare il bit #7 del segno.
;
; Input:
;   - .ptr_ (1 o 2 byte, Macro) : Indirizzo del puntatore
;   - (.ptr_) (2 byte, RAM)     : Indirizzo del numero (LSB, MSB)
;   - ((.ptr_)) (5 byte, RAM)   : Numero in formato MBF
;
; Output:
;   - FAC (7 byte, ZP)          : Valore assoluto, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry), FACSGN
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.195 m_Sign_FAC

```
; ================================================================================
; MACRO: m_Sign_FAC
; --------------------------------------------------------------------------------
; Descrizione:  Restituisce in .A il segno del numero in virgola mobile di FAC.
;
;               Chiama la routine SIGN della ROM del BASIC.
;
;               Se il numero è zero il Carry non è garantito.
;
; Input:
;   - FAC (7 byte, ZP)       : Registri da FACEXP a FACOV
;
; Output:
;   - .A (1 byte, CPU)       : $FF = negativo; $00 = zero; $01 = positivo
;   - Flag Carry (1 bit, .P) : 0 = positivo; 1 = negativo
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.196 m_Sign_ARG

```
; ================================================================================
; MACRO: m_Sign_ARG
; --------------------------------------------------------------------------------
; Descrizione:  Restituisce in .A il segno del numero in virgola mobile di ARG.
;
;               Il codice è interno alla macro.
;
;               Se il numero è zero il Carry non è garantito.
;
; Input:
;   - ARG (6 byte, ZP)       : Registri da ARGEXP a ARGSGN
;
; Output:
;   - .A (1 byte, CPU)       : $FF = negativo; $00 = zero; $01 = positivo
;   - Flag Carry (1 bit, .P) : 0 = positivo; 1 = negativo
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.197 m_Sign_Mem

```
; ================================================================================
; MACRO: m_Sign_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con il numero in formato MBF che si trova a un
;               indirizzo assoluto e restituisce in .A il suo segno.
;
;               La lettura usa m_Load_FAC_from_Mem, che altera anche INDEX: il
;               suo contenuto non è garantito dopo la chiamata. Il vecchio
;               contenuto di FAC si perde.
;
;               Chiama la routine SIGN della ROM del BASIC.
;
;               Se il numero è zero il Carry non è garantito.
;
; Input:
;   - .addr_ (1 o 2 byte, Macro) : Indirizzo del numero
;   - (.addr_) (5 byte, RAM)     : Numero in formato MBF
;
; Output:
;   - .A (1 byte, CPU)           : $FF = negativo; $00 = zero; $01 = positivo
;   - Flag Carry (1 bit, .P)     : 0 = positivo; 1 = negativo
;   - FAC (7 byte, ZP)           : Numero letto, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.198 m_Sign_Ptr

```
; ================================================================================
; MACRO: m_Sign_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con il numero in formato MBF che si trova
;               all'indirizzo a cui punta un puntatore e restituisce in .A il
;               suo segno.
;
;               La lettura usa m_Load_FAC_from_Ptr, che altera anche INDEX: il
;               suo contenuto non è garantito dopo la chiamata. Il vecchio
;               contenuto di FAC si perde.
;
;               Chiama la routine SIGN della ROM del BASIC.
;
;               Se il numero è zero il Carry non è garantito.
;
; Input:
;   - .ptr_ (1 o 2 byte, Macro) : Indirizzo del puntatore
;   - (.ptr_) (2 byte, RAM)     : Indirizzo del numero (LSB, MSB)
;   - ((.ptr_)) (5 byte, RAM)   : Numero in formato MBF
;
; Output:
;   - .A (1 byte, CPU)          : $FF = negativo; $00 = zero; $01 = positivo
;   - Flag Carry (1 bit, .P)    : 0 = positivo; 1 = negativo
;   - FAC (7 byte, ZP)          : Numero letto, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.199 m_SGN_FAC

```
; ================================================================================
; MACRO: m_SGN_FAC
; --------------------------------------------------------------------------------
; Descrizione:  Sostituisce il numero in virgola mobile di FAC con il suo segno:
;               -1, 0 o 1.
;
;               Chiama la routine SGN della ROM del BASIC.
;
; Input:
;   - FAC (7 byte, ZP) : Registri da FACEXP a FACOV
;
; Output:
;   - FAC (7 byte, ZP) : -1, 0 o 1, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.200 m_SGN_ARG

```
; ============================================================================
; MACRO: m_SGN_ARG
; ----------------------------------------------------------------------------
; Descrizione:  Sostituisce il numero in virgola mobile di ARG con il suo segno:
;               -1, 0 o 1.
;
;               Scambia FAC e ARG con m_Swap_FAC_and_ARG, chiama la routine SGN
;               della ROM del BASIC e li scambia di nuovo. Dopo la chiamata FAC
;               contiene il valore di prima, arrotondato, e FACOV vale zero;
;               ARISGN non viene aggiornato. Il contenuto di _SCRATCH_2 non è
;               garantito dopo la chiamata.
;
; Input:
;   - ARG (6 byte, ZP) : Registri da ARGEXP a ARGSGN
;   - FAC (7 byte, ZP) : Registri da FACEXP a FACOV
;
; Output:
;   - ARG (6 byte, ZP) : -1, 0 o 1, registri da ARGEXP a ARGSGN
;   - FAC (7 byte, ZP) : Valore di prima, arrotondato, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC,
;                       _SCRATCH_2
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ============================================================================
```

2.1.201 m_SGN_Mem

```
; ================================================================================
; MACRO: m_SGN_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con il numero in formato MBF che si trova a un
;               indirizzo assoluto e lo sostituisce con il suo segno: -1, 0 o 1.
;
;               La lettura usa m_Load_FAC_from_Mem, che altera anche INDEX: il
;               suo contenuto non è garantito dopo la chiamata. Il vecchio
;               contenuto di FAC si perde.
;
;               Chiama la routine SGN della ROM del BASIC.
;
; Input:
;   - .addr_ (1 o 2 byte, Macro) : Indirizzo del numero
;   - (.addr_) (5 byte, RAM)     : Numero in formato MBF
;
; Output:
;   - FAC (7 byte, ZP)           : -1, 0 o 1, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.202 m_SGN_Ptr

```
; ================================================================================
; MACRO: m_SGN_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con il numero in formato MBF che si trova
;               all'indirizzo a cui punta un puntatore e lo sostituisce con il
;               suo segno: -1, 0 o 1.
;
;               La lettura usa m_Load_FAC_from_Ptr, che altera anche INDEX: il
;               suo contenuto non è garantito dopo la chiamata. Il vecchio
;               contenuto di FAC si perde.
;
;               Chiama la routine SGN della ROM del BASIC.
;
; Input:
;   - .ptr_ (1 o 2 byte, Macro) : Indirizzo del puntatore
;   - (.ptr_) (2 byte, RAM)     : Indirizzo del numero (LSB, MSB)
;   - ((.ptr_)) (5 byte, RAM)   : Numero in formato MBF
;
; Output:
;   - FAC (7 byte, ZP)          : -1, 0 o 1, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.203 m_INT_FAC

```
; ================================================================================
; MACRO: m_INT_FAC
; --------------------------------------------------------------------------------
; Descrizione:  Arrotonda per difetto, verso -infinito, il numero in virgola
;               mobile di FAC: 1.2 diventa 1 e -1.2 diventa -2.
;
;               I numeri con esponente maggiore o uguale a $A0, cioè di valore
;               assoluto non inferiore a 2^31, sono già interi e non cambiano.
;
;               Chiama la routine INT della ROM del BASIC.
;
;               La routine INT altera le celle $07 e $68; usa QINT, che richiede
;               $68 = 0 in ingresso e lo azzera in uscita. Il contenuto di $07
;               non è garantito dopo la chiamata.
;
; Input:
;   - FAC (7 byte, ZP) : Registri da FACEXP a FACOV
;
; Output:
;   - FAC (7 byte, ZP) : Numero arrotondato, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.204 m_INT_ARG

```
; ================================================================================
; MACRO: m_INT_ARG
; --------------------------------------------------------------------------------
; Descrizione:  Arrotonda per difetto, verso -infinito, il numero in virgola
;               mobile di ARG: 1.2 diventa 1 e -1.2 diventa -2.
;
;               I numeri con esponente maggiore o uguale a $A0, cioè di valore
;               assoluto non inferiore a 2^31, sono già interi e non cambiano.
;
;               Scambia FAC e ARG con m_Swap_FAC_and_ARG, chiama la routine INT
;               della ROM del BASIC e li scambia di nuovo. Dopo la chiamata FAC
;               contiene il valore di prima, arrotondato, e FACOV vale zero;
;               ARISGN non viene aggiornato. Il contenuto di _SCRATCH_2 non è
;               garantito dopo la chiamata.
;
;               La routine INT altera le celle $07 e $68; usa QINT, che richiede
;               $68 = 0 in ingresso e lo azzera in uscita. Il contenuto di $07
;               non è garantito dopo la chiamata.
;
; Input:
;   - ARG (6 byte, ZP) : Registri da ARGEXP a ARGSGN
;   - FAC (7 byte, ZP) : Registri da FACEXP a FACOV
;
; Output:
;   - ARG (6 byte, ZP) : Numero arrotondato, registri da ARGEXP a ARGSGN
;   - FAC (7 byte, ZP) : Valore di prima, arrotondato, FACOV azzerato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry), FAC,
;                       _SCRATCH_2
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.205 m_INT_Mem

```
; ================================================================================
; MACRO: m_INT_Mem
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con il numero in formato MBF che si trova a un
;               indirizzo assoluto e lo arrotonda per difetto, verso -infinito:
;               1.2 diventa 1 e -1.2 diventa -2. Il risultato resta in FAC.
;
;               La lettura usa m_Load_FAC_from_Mem, che altera anche INDEX: il
;               suo contenuto non è garantito dopo la chiamata. Il vecchio
;               contenuto di FAC si perde.
;
;               I numeri con esponente maggiore o uguale a $A0, cioè di valore
;               assoluto non inferiore a 2^31, sono già interi e non cambiano.
;
;               Chiama la routine INT della ROM del BASIC.
;
;               La routine INT altera le celle $07 e $68; usa QINT, che richiede
;               $68 = 0 in ingresso e lo azzera in uscita. Il contenuto di $07
;               non è garantito dopo la chiamata.
;
; Input:
;   - .addr_ (1 o 2 byte, Macro) : Indirizzo del numero
;   - (.addr_) (5 byte, RAM)     : Numero in formato MBF
;
; Output:
;   - FAC (7 byte, ZP)           : Numero arrotondato, registri da FACEXP a
;                                  FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.206 m_INT_Ptr

```
; ================================================================================
; MACRO: m_INT_Ptr
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con il numero in formato MBF che si trova
;               all'indirizzo a cui punta un puntatore e lo arrotonda per
;               difetto, verso -infinito: 1.2 diventa 1 e -1.2 diventa -2. Il
;               risultato resta in FAC.
;
;               La lettura usa m_Load_FAC_from_Ptr, che altera anche INDEX: il
;               suo contenuto non è garantito dopo la chiamata. Il vecchio
;               contenuto di FAC si perde.
;
;               I numeri con esponente maggiore o uguale a $A0, cioè di valore
;               assoluto non inferiore a 2^31, sono già interi e non cambiano.
;
;               Chiama la routine INT della ROM del BASIC.
;
;               La routine INT altera le celle $07 e $68; usa QINT, che richiede
;               $68 = 0 in ingresso e lo azzera in uscita. Il contenuto di $07
;               non è garantito dopo la chiamata.
;
; Input:
;   - .ptr_ (1 o 2 byte, Macro) : Indirizzo del puntatore
;   - (.ptr_) (2 byte, RAM)     : Indirizzo del numero (LSB, MSB)
;   - ((.ptr_)) (5 byte, RAM)   : Numero in formato MBF
;
; Output:
;   - FAC (7 byte, ZP)          : Numero arrotondato, registri da FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.207 m_Multiply_FAC_by_2

```
; ============================================================================
; MACRO: m_Multiply_FAC_by_2
; ----------------------------------------------------------------------------
; Descrizione:  Moltiplica per 2 il numero in virgola mobile di FAC,
;               incrementando l'esponente.
;
;               È molto più veloce di una moltiplicazione o di una divisione
;               vera e propria.
;
;               Se FAC è zero (FACEXP = 0) resta zero. Se l'esponente vale $FF
;               il risultato è zero, senza l'errore di overflow della ROM del
;               BASIC: il valore è troppo grande per il formato MBF.
;
; Input:
;   - FAC (7 byte, ZP)    : Registri da FACEXP a FACOV
;
; Output:
;   - FACEXP (1 byte, ZP) : Esponente incrementato, salvo se il numero è zero
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), FACEXP
; Dipendenze Esterne:   <c64/symbols.asm>
; ============================================================================
```

2.1.208 m_Multiply_ARG_by_2

```
; ================================================================================
; MACRO: m_Multiply_ARG_by_2
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplica per 2 il numero in virgola mobile di ARG,
;               incrementando l'esponente.
;
;               È molto più veloce di una moltiplicazione o di una divisione
;               vera e propria.
;
;               Se ARG è zero (ARGEXP = 0) resta zero. Se l'esponente vale $FF
;               il risultato è zero, senza l'errore di overflow della ROM del
;               BASIC: il valore è troppo grande per il formato MBF.
;
; Input:
;   - ARG (6 byte, ZP)    : Registri da ARGEXP a ARGSGN
;
; Output:
;   - ARGEXP (1 byte, ZP) : Esponente incrementato, salvo se il numero è zero
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), ARGEXP
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.209 m_Multiply_Mem_by_2

```
; ================================================================================
; MACRO: m_Multiply_Mem_by_2
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con il numero in formato MBF che si trova a un
;               indirizzo assoluto e lo moltiplica per 2, incrementando
;               l'esponente. Il risultato resta in FAC.
;
;               La lettura usa m_Load_FAC_from_Mem, che altera anche INDEX: il
;               suo contenuto non è garantito dopo la chiamata. Il vecchio
;               contenuto di FAC si perde.
;
;               È molto più veloce di una moltiplicazione o di una divisione
;               vera e propria.
;
;               Se FAC è zero (FACEXP = 0) resta zero. Se l'esponente vale $FF
;               il risultato è zero, senza l'errore di overflow della ROM del
;               BASIC: il valore è troppo grande per il formato MBF.
;
; Input:
;   - .addr_ (1 o 2 byte, Macro) : Indirizzo del numero
;   - (.addr_) (5 byte, RAM)     : Numero in formato MBF
;
; Output:
;   - FAC (7 byte, ZP)           : Numero letto moltiplicato per 2, registri da
;                                  FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry), FACEXP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.210 m_Multiply_Ptr_by_2

```
; ================================================================================
; MACRO: m_Multiply_Ptr_by_2
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con il numero in formato MBF che si trova
;               all'indirizzo a cui punta un puntatore e lo moltiplica per 2,
;               incrementando l'esponente. Il risultato resta in FAC.
;
;               La lettura usa m_Load_FAC_from_Ptr, che altera anche INDEX: il
;               suo contenuto non è garantito dopo la chiamata. Il vecchio
;               contenuto di FAC si perde.
;
;               È molto più veloce di una moltiplicazione o di una divisione
;               vera e propria.
;
;               Se FAC è zero (FACEXP = 0) resta zero. Se l'esponente vale $FF
;               il risultato è zero, senza l'errore di overflow della ROM del
;               BASIC: il valore è troppo grande per il formato MBF.
;
; Input:
;   - .ptr_ (1 o 2 byte, Macro) : Indirizzo del puntatore
;   - (.ptr_) (2 byte, RAM)     : Indirizzo del numero (LSB, MSB)
;   - ((.ptr_)) (5 byte, RAM)   : Numero in formato MBF
;
; Output:
;   - FAC (7 byte, ZP)          : Numero letto moltiplicato per 2, registri da
;                                 FACEXP a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry), FACEXP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.211 m_Divide_FAC_by_2

```
; ================================================================================
; MACRO: m_Divide_FAC_by_2
; --------------------------------------------------------------------------------
; Descrizione:  Divide per 2 il numero in virgola mobile di FAC,
;               decrementando l'esponente.
;
;               È molto più veloce di una moltiplicazione o di una divisione
;               vera e propria.
;
;               Se FAC è zero (FACEXP = 0) resta zero. Se l'esponente vale 1 il
;               risultato è zero (underflow): il valore è troppo piccolo per il
;               formato MBF.
;
; Input:
;   - FAC (7 byte, ZP)    : Registri da FACEXP a FACOV
;
; Output:
;   - FACEXP (1 byte, ZP) : Esponente decrementato, salvo se il numero è zero
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), FACEXP
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.212 m_Divide_ARG_by_2

```
; ================================================================================
; MACRO: m_Divide_ARG_by_2
; --------------------------------------------------------------------------------
; Descrizione:  Divide per 2 il numero in virgola mobile di ARG,
;               decrementando l'esponente.
;
;               È molto più veloce di una moltiplicazione o di una divisione
;               vera e propria.
;
;               Se ARG è zero (ARGEXP = 0) resta zero. Se l'esponente vale 1 il
;               risultato è zero (underflow): il valore è troppo piccolo per il
;               formato MBF.
;
; Input:
;   - ARG (6 byte, ZP)    : Registri da ARGEXP a ARGSGN
;
; Output:
;   - ARGEXP (1 byte, ZP) : Esponente decrementato, salvo se il numero è zero
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), ARGEXP
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

2.1.213 m_Divide_Mem_by_2

```
; ================================================================================
; MACRO: m_Divide_Mem_by_2
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con il numero in formato MBF che si trova a un
;               indirizzo assoluto e lo divide per 2, decrementando l'esponente.
;               Il risultato resta in FAC.
;
;               La lettura usa m_Load_FAC_from_Mem, che altera anche INDEX: il
;               suo contenuto non è garantito dopo la chiamata. Il vecchio
;               contenuto di FAC si perde.
;
;               È molto più veloce di una moltiplicazione o di una divisione
;               vera e propria.
;
;               Se FAC è zero (FACEXP = 0) resta zero. Se l'esponente vale 1 il
;               risultato è zero (underflow): il valore è troppo piccolo per il
;               formato MBF.
;
; Input:
;   - .addr_ (1 o 2 byte, Macro) : Indirizzo del numero
;   - (.addr_) (5 byte, RAM)     : Numero in formato MBF
;
; Output:
;   - FAC (7 byte, ZP)           : Numero letto diviso per 2, registri da FACEXP
;                                  a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry), FACEXP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

2.1.214 m_Divide_Ptr_by_2

```
; ================================================================================
; MACRO: m_Divide_Ptr_by_2
; --------------------------------------------------------------------------------
; Descrizione:  Carica FAC con il numero in formato MBF che si trova
;               all'indirizzo a cui punta un puntatore e lo divide per 2,
;               decrementando l'esponente. Il risultato resta in FAC.
;
;               La lettura usa m_Load_FAC_from_Ptr, che altera anche INDEX: il
;               suo contenuto non è garantito dopo la chiamata. Il vecchio
;               contenuto di FAC si perde.
;
;               È molto più veloce di una moltiplicazione o di una divisione
;               vera e propria.
;
;               Se FAC è zero (FACEXP = 0) resta zero. Se l'esponente vale 1 il
;               risultato è zero (underflow): il valore è troppo piccolo per il
;               formato MBF.
;
; Input:
;   - .ptr_ (1 o 2 byte, Macro) : Indirizzo del puntatore
;   - (.ptr_) (2 byte, RAM)     : Indirizzo del numero (LSB, MSB)
;   - ((.ptr_)) (5 byte, RAM)   : Numero in formato MBF
;
; Output:
;   - FAC (7 byte, ZP)          : Numero letto diviso per 2, registri da FACEXP
;                                 a FACOV
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, oVerflow, Zero, Carry), FACEXP
; Dipendenze Esterne:   <c64/symbols.asm>
;                       <fp/fp_memory.asm>
; ================================================================================
```

Libreria

io

3 Libreria io

Modulo	Descrizione
`chars.asm`	Transcodifica dei caratteri tra le codifiche Codice Schermo, PETSCII, ASCII a 7 bit ed ECM (modulo principale).
`disk_tools.asm`	Utilità per la gestione delle unità disco.
`keyboard_tools.asm`	Lettura diretta della tastiera dalla matrice del CIA #1, senza passare dal Kernal. Deriva da quella del gioco Platoon, accorciata e con la serie storica integrata.

3.1 chars.asm

3.1.1 Descrizione

Transcodifica dei caratteri tra le codifiche Codice Schermo, PETSCII, ASCII a 7 bit ed ECM (modulo principale).

Utilità	Descrizione
`m_Scan_Exceptions`	Cerca nella tabella delle eccezioni
`m_Apply_Table`	Sostituisce i 3 bit alti del codice
`ASC_SCR`	Da ASCII a Codice Schermo, senza controlli
`ASC_SCR_C`	Da ASCII a Codice Schermo, con controlli
`ASC_PET`	Da ASCII a PETSCII, senza controlli
`ASC_PET_C`	Da ASCII a PETSCII, con controlli
`ASC_ECM`	Da ASCII a ECM, senza controlli
`ASC_ECM_C`	Da ASCII a ECM, con controlli
`ECM_SCR`	Da ECM a Codice Schermo, senza controlli
`ECM_SCR_C`	Da ECM a Codice Schermo, con controlli
`ECM_PET`	Da ECM a PETSCII, senza controlli
`ECM_PET_C`	Da ECM a PETSCII, con controlli
`ECM_ASC`	Da ECM a ASCII, senza controlli
`ECM_ASC_C`	Da ECM a ASCII, con controlli
`PET_SCR`	Da PETSCII a Codice Schermo, senza controlli
`PET_SCR_C`	Da PETSCII a Codice Schermo, con controlli
`PET_ASC`	Da PETSCII a ASCII, senza controlli
`PET_ASC_C`	Da PETSCII a ASCII, con controlli
`PET_ECM`	Da PETSCII a ECM, senza controlli
`PET_ECM_C`	Da PETSCII a ECM, con controlli
`SCR_PET`	Da Codice Schermo a PETSCII, senza controlli
`SCR_PET_C`	Da Codice Schermo a PETSCII, con controlli
`SCR_ASC`	Da Codice Schermo a ASCII, senza controlli
`SCR_ASC_C`	Da Codice Schermo a ASCII, con controlli
`SCR_ECM`	Da Codice Schermo a ECM, senza controlli
`SCR_ECM_C`	Da Codice Schermo a ECM, con controlli

3.1.2 Note d'uso

```
; NOTA #01
; --------
; Un Commodore a 8 bit riconosce 256 codici carattere. Un codice diventa
; visibile quando il chip video lo associa a un glifo, cioè a un disegno del
; set di caratteri: i set sono due, il #1 "Maiuscolo/Grafici" e il #2
; "Minuscolo/Maiuscolo". L'aspetto del glifo non cambia il significato del
; codice, che dipende soltanto dalla codifica.
;
; I codici sono di due tipi: stampabili, che producono un glifo, e di
; controllo, che eseguono un'azione. In modo Virgolette e in modo
; Inserimento i codici di controllo perdono l'azione e mostrano un glifo.
;
; Le codifiche
; ------------
;
; La libreria converte tra quattro codifiche:
;
; - Codice Schermo: la codifica più vicina all'hardware, quella in cui sono
;   scritti i 40x25 caratteri della memoria video. Tutti i codici sono
;   stampabili, non esistono codici di controllo e a ogni codice corrisponde
;   un solo glifo. I codici da 0 a 127 sono i caratteri normali, quelli da 128
;   a 255 gli stessi caratteri in modalità inversa.
;
; - PETSCII: la codifica del BASIC e del Kernal (funzioni ASC e CHR$). Ha i
;   codici di controllo, e non ha i caratteri inversi: in modo Virgolette
;   il Kernal mostra i controlli con un glifo inverso, senza ambiguità con i
;   glifi stampabili. Alcuni codici sono duplicati: il BASIC normalizza i
;   codici da 96 a 127 in quelli da 192 a 223 e i codici da 224 a 255 in
;   quelli da 160 a 191, quindi i duplicati condividono i glifi.
;
; - ASCII a 7 bit: non è nativa del Commodore, ma serve per lo scambio con
;   altre architetture. Ha solo 128 codici, con i controlli da 0 a 31 e il 127
;   (DEL), e non ha né i caratteri inversi né i grafici del Commodore.
;
; - ECM: non è una codifica in senso stretto, ma l'interpretazione del Codice
;   Schermo quando è attivo il modo Colore di Sfondo Esteso. I due bit più
;   significativi scelgono il registro del colore di sfondo (BGCOL0-BGCOL3) e
;   gli altri sei il glifo, da 0 a 63: il glifo è nel codice, lo sfondo in
;   __BG_COL. Si tratta come una codifica perché il suo codice contiene
;   un'informazione di colore che le altre non hanno. Poiché il VIC-II ignora
;   i due bit più significativi, i glifi oltre il 63 non esistono: il pigreco
;   (codice Schermo 94) appare come la freccia in su (glifo 30), come
;   sull'hardware reale.
;
; I blocchi
; ---------
;
; I 256 codici si dividono in otto blocchi da 32, numerati da 0 a 7 (0 = 0-31,
; 1 = 32-63 e così via): il blocco è nei tre bit più significativi del codice,
; e le regole di transcodifica si esprimono meglio con i blocchi.
```

segue...

```
;
; Blocco  Codici    Codice Schermo  PETSCII          ASCII
; ------  --------  --------------  ---------------  -----------
;   0     0-31      normali         controlli        controlli
;   1     32-63     normali         stampabili       stampabili
;   2     64-95     normali         stampabili       stampabili
;   3     96-127    normali         duplicati del 6  stampabili
;   4     128-159   inversi         controlli        inesistenti
;   5     160-191   inversi         stampabili       inesistenti
;   6     192-223   inversi         stampabili       inesistenti
;   7     224-255   inversi         duplicati del 5  inesistenti
;
; In ASCII anche il codice 127 è di controllo. In ECM il blocco dice il
; registro di sfondo: 0 e 1 BGCOL0, 2 e 3 BGCOL1, 4 e 5 BGCOL2, 6 e 7 BGCOL3,
; con gli stessi glifi nei blocchi pari e nei dispari.
```

```
; NOTA #02
; --------
; Una transcodifica non può mai essere senza perdite: Codice Schermo non ha i
; controlli, PETSCII e ASCII non hanno i caratteri inversi, ASCII non ha molti
; glifi del Commodore, ECM porta nel codice un colore che le altre codifiche
; non hanno. Ciò che il codice di arrivo non può conservare si conserva
; altrove, in __BG_COL, oppure si segnala con il Carry. Le routine seguono
; cinque criteri, in quest'ordine.
;
; 1. Glifo per glifo.
;
; Una conversione conserva l'aspetto del carattere, non il codice né il
; significato: il codice di arrivo è quello che rappresenta lo stesso glifo di
; quello di partenza. I codici di controllo non si traducono nel loro effetto:
; si convertono nel glifo che mostrano.
;
; 2. Un solo codice per glifo.
;
; Quando più codici hanno lo stesso glifo si sceglie sempre quello primario,
; mai il duplicato: in arrivo su PETSCII i blocchi 5 e 6, non i blocchi 7 e 3
; (il pigreco è l'unica eccezione dichiarata).
;
; 3. Le transcodifiche si compongono.
;
; Convertire in due passi, passando per una terza codifica, dà lo stesso
; risultato della conversione diretta. Vale per i caratteri stampabili; i
; codici di controllo seguono il criterio 5.
;
; 4. Andata e ritorno.
;
; Convertire un codice e riconvertire il risultato nella codifica di partenza
; restituisce il codice originale a tre condizioni: le due conversioni sono
; perfette (Carry a 0), usano la stessa variante (entrambe senza controlli o
; entrambe con) e il codice di partenza è un codice primario. I duplicati di
; PETSCII tornano come il codice primario con lo stesso glifo (il pigreco, 126
; o 255, torna sempre come 255). Con ECM il colore di sfondo, che è in
; __BG_COL, deve restare intatto tra le due chiamate.
;
; 5. Ciò che non si conserva si dichiara.
;
; Con il Carry a 0 il carattere di arrivo ha l'aspetto di quello di partenza,
; o del sostituto grafico previsto per i caratteri ASCII che il Commodore non
; ha. Altrimenti il Carry è a 1: la conversione è imperfetta e si restituisce
; il meglio possibile (per esempio il glifo normale da stampare in modalità
; inversa), oppure è impossibile e .A vale $FF. Nei contratti ogni routine
; dice come leggere il Carry a 1.
```

```
; NOTA #03
; --------
; Otto caratteri ASCII non hanno un glifo uguale nei set di caratteri del C64:
; la barra inversa, il circonflesso, il carattere di sottolineatura, l'accento
; grave, le due graffe, la barra verticale e la tilde. Le routine li
; sostituiscono con grafici che somigliano all'originale e che hanno lo stesso
; glifo nel set #1 e nel set #2 (con lo stesso codice), in modo che la
; conversione non dipenda dal set attivo né da un set personalizzato. La
; sostituzione è reversibile: nel verso opposto il grafico torna a essere il
; carattere che sostituisce, e la conversione è perfetta (Carry a 0).
;
; ASCII  Carattere        Codice Schermo  PETSCII  Grafico che lo sostituisce
; -----  ---------------  --------------  -------  --------------------------
;  92    barra inversa    127             191      due quarti di blocco opposti
;  94    circonflesso     30              94       freccia in su
;  95    sottolineato     100             164      linea in basso
;  96    accento grave    109             173      angolo in basso a sinistra
;  123   graffa aperta    115             179      tratto con ramo a sinistra
;  124   barra verticale  93              221      tratto verticale
;  125   graffa chiusa    107             171      tratto con ramo a destra
;  126   tilde            64              192      linea a metà altezza
;
; I controlli ASCII da 0 a 31 e il DEL (127) non hanno un glifo, e anche qui
; la scelta è di progetto, non una sostituzione: ASCII non ha una convenzione.
; I controlli sono rappresentati dai caratteri inversi di Codice Schermo da
; 128 a 159 e il DEL dal codice 102, una trama a scacchiera.
```

3.1.3 m_Scan_Exceptions

```
; ================================================================================
; MACRO: m_Scan_Exceptions
; --------------------------------------------------------------------------------
; Descrizione:  Cerca il codice in .A nella tabella delle eccezioni. La tabella
;               è formata da coppie di byte (codice sorgente, codice
;               destinazione), una coppia per ogni eccezione, e la ricerca va
;               dall'ultima coppia alla prima.
;
;               Se trova il codice, carica in .A il codice di destinazione e
;               salta a .found_, con .Y che contiene la posizione della coppia
;               nella tabella (0 per la prima, 2 per la seconda, e così via). Se
;               non lo trova, prosegue con l'istruzione che segue la macro, con
;               .A invariato.
;
;               La tabella non può essere vuota e può contenere al massimo 65
;               coppie (130 byte).
;
; Input:
;   - .A (1 byte, CPU)         : Codice sorgente da cercare
;   - .table_ (2 byte, Macro)  : Indirizzo della tabella delle eccezioni
;   - .size_ (1 byte, Macro)   : Dimensione della tabella in byte (2 per
;                                coppia), da 2 a 130
;   - .found_ (2 byte, Macro)  : Indirizzo a cui saltare se il codice è nella
;                                tabella
;
; Output:
;   - .A (1 byte, CPU)         : Codice destinazione se trovato, altrimenti
;                                invariato
;   - .Y (1 byte, CPU)         : Posizione della coppia trovata. Indefinito se
;                                il codice non è nella tabella
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, Zero, Carry)
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

3.1.4 m_Apply_Table

```
; ==============================================================================
; MACRO: m_Apply_Table
; ------------------------------------------------------------------------------
; Descrizione:  Sostituisce i tre bit più significativi del codice (il suo
;               blocco, da 0 a 7) con quelli della tabella dei blocchi e lascia
;               invariati i cinque bit meno significativi.
;
;               La tabella ha 8 byte, uno per blocco: in ognuno contano solo i
;               tre bit più significativi, e gli altri cinque devono essere
;               zero.
;
; Input:
;   - .A (1 byte, CPU)         : Codice da convertire
;   - .X (1 byte, CPU)         : Copia del codice da convertire
;   - .table_ (2 byte, Macro)  : Indirizzo della tabella dei blocchi
;
; Output:
;   - .A (1 byte, CPU)         : Codice convertito
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .Y, .P (Negative, Zero, Carry)
; Dipendenze Esterne:   Nessuna
; ==============================================================================
```

chars_asc.asm

3.1.5 ASC_SCR

```
; ============================================================================
; SUBROUTINE: ASC_SCR
; ----------------------------------------------------------------------------
; Descrizione:  Converte un codice dalla codifica ASCII a 7 bit alla codifica
;               Codice Schermo, senza produrre codici di controllo. La
;               conversione è "glifo per glifo", con i codici ASCII letti come
;               li mostra il set di caratteri #2: da 65 a 90 le lettere
;               maiuscole (i codici da 65 a 90 di Codice Schermo), da 97 a 122
;               le minuscole (da 1 a 26), da 32 a 63 gli stessi caratteri.
;
;               I caratteri ASCII che il Commodore non ha sono sostituiti da
;               caratteri grafici: vedi [NOTA #03 in <io/chars.asm>].
;
;               I codici di controllo ASCII (da 0 a 31) e il DEL (127) non hanno
;               un glifo: i controlli diventano il glifo inverso del carattere
;               con il codice più 64 (da 128 a 159), e il DEL il codice 102, che
;               somiglia a una cancellazione. La conversione è perfetta: Codice
;               Schermo ha i caratteri inversi.
;
;               I codici da 128 a 255, che non sono ASCII a 7 bit, non hanno
;               equivalente: la conversione è impossibile, con il Carry a 1 e .A
;               = $FF. Nessun codice ASCII ha $FF come equivalente, e il Carry a
;               1 ha solo questo significato.
;
; Input:
;   - .A (1 byte, CPU)       : Codice carattere in codifica ASCII a 7 bit
;
; Output:
;   - .A (1 byte, CPU)       : Codice carattere in codifica Codice Schermo
;                              ($FF se la conversione è impossibile)
;   - .X (1 byte, CPU)       : Codice carattere originale
;   - Flag Carry (1 bit, .P) : 0 = conversione riuscita; 1 = conversione
;                              impossibile (.A = $FF)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, Zero, Carry)
; Dipendenze Esterne:   <io/chars_common.asm>
; ============================================================================
```

3.1.6 ASC_SCR_C

```
; ============================================================================
; SUBROUTINE: ASC_SCR_C
; ----------------------------------------------------------------------------
; Descrizione:  Converte un codice dalla codifica ASCII a 7 bit alla codifica
;               Codice Schermo, riconoscendo i codici di controllo. Come
;               ASC_SCR, ma i codici di controllo ASCII (da 0 a 31 e il DEL,
;               127) non hanno un equivalente in Codice Schermo, che non ha
;               codici di controllo: la conversione è impossibile.
;
;               Sono impossibili allo stesso modo i codici da 128 a 255, che non
;               sono ASCII a 7 bit. Il Carry a 0 dice che la conversione è
;               riuscita, con la sostituzione del glifo quando serve. Il Carry a
;               1 dice che è impossibile, con .A = $FF: nessun codice ASCII ha
;               $FF come equivalente.
;
; Input:
;   - .A (1 byte, CPU)       : Codice carattere in codifica ASCII a 7 bit
;
; Output:
;   - .A (1 byte, CPU)       : Codice carattere in codifica Codice Schermo
;                              ($FF se la conversione è impossibile)
;   - .X (1 byte, CPU)       : Codice carattere originale
;   - Flag Carry (1 bit, .P) : 0 = conversione riuscita; 1 = conversione
;                              impossibile (.A = $FF)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, Zero, Carry)
; Dipendenze Esterne:   <io/chars_common.asm>
; ============================================================================
```

3.1.7 ASC_PET

```
; ============================================================================
; SUBROUTINE: ASC_PET
; ----------------------------------------------------------------------------
; Descrizione:  Converte un codice dalla codifica ASCII a 7 bit alla codifica
;               PETSCII, senza produrre codici di controllo. La conversione è
;               "glifo per glifo", con i codici ASCII letti come li mostra il
;               set di caratteri #2: le lettere maiuscole (da 65 a 90) diventano
;               i codici da 193 a 218 e le minuscole (da 97 a 122) i codici da
;               65 a 90.
;
;               I caratteri ASCII che il Commodore non ha sono sostituiti da
;               caratteri grafici: vedi [NOTA #03 in <io/chars.asm>]. Tra i
;               codici duplicati di PETSCII la routine produce sempre quelli dei
;               blocchi originali (da 160 a 191 e da 192 a 223), mai i duplicati
;               (da 96 a 127 e da 224 a 255).
;
;               I codici di controllo ASCII (da 0 a 31) diventano i glifi che
;               Codice Schermo ha con lo stesso codice (i codici da 64 a 95 in
;               PETSCII): il 28 diventa la sterlina e il 31 la freccia a
;               sinistra: PETSCII non ha caratteri inversi, quindi il Carry è a
;               1 e il carattere va stampato in modalità inversa. Il DEL (127)
;               diventa il codice 166, il glifo che somiglia a una
;               cancellazione, con il Carry a 0.
;
;               I codici da 128 a 255, che non sono ASCII a 7 bit, non hanno
;               equivalente: la conversione è impossibile, con il Carry a 1 e .A
;               = $FF. Il Carry a 1 ha dunque due significati, che si
;               distinguono con un test su .A: se vale $FF la conversione è
;               impossibile; altrimenti .A è un carattere da stampare in
;               modalità inversa (il codice originale in .X è minore di 32).
;
; Input:
;   - .A (1 byte, CPU)       : Codice carattere in codifica ASCII a 7 bit
;
; Output:
;   - .A (1 byte, CPU)       : Codice carattere in codifica PETSCII
;                              ($FF se la conversione è impossibile)
;   - .X (1 byte, CPU)       : Codice carattere originale
;   - Flag Carry (1 bit, .P) : 0 = conversione riuscita; 1 = conversione
;                              impossibile (.A = $FF) o carattere da
;                              stampare in modalità inversa
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, Zero, Carry)
; Dipendenze Esterne:   <io/chars_common.asm>
; ============================================================================
```

3.1.8 ASC_PET_C

```
; ============================================================================
; SUBROUTINE: ASC_PET_C
; ----------------------------------------------------------------------------
; Descrizione:  Converte un codice dalla codifica ASCII a 7 bit alla codifica
;               PETSCII, riconoscendo i codici di controllo. Come ASC_PET, ma i
;               codici di controllo ASCII da 0 a 31 restano codici di controllo
;               con lo stesso codice (la conversione è perfetta; il significato
;               dei controlli non viene convertito).
;
;               Il DEL (127) non ha un controllo equivalente in PETSCII: diventa
;               il codice 166, il glifo che somiglia a una cancellazione, ma
;               essendo un controllo che diventa stampabile la conversione è
;               imperfetta, con il Carry a 1. I codici da 128 a 255, che non
;               sono ASCII a 7 bit, sono impossibili: Carry a 1 e .A = $FF. Il
;               Carry a 1 si distingue quindi con un test su .A: $FF è
;               impossibile, 166 è il DEL.
;
; Input:
;   - .A (1 byte, CPU)       : Codice carattere in codifica ASCII a 7 bit
;
; Output:
;   - .A (1 byte, CPU)       : Codice carattere in codifica PETSCII
;                              ($FF se la conversione è impossibile)
;   - .X (1 byte, CPU)       : Codice carattere originale
;   - Flag Carry (1 bit, .P) : 0 = conversione riuscita; 1 = conversione
;                              impossibile (.A = $FF) o DEL
;                              convertito in carattere (.A = 166)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, Zero, Carry)
; Dipendenze Esterne:   <io/chars_common.asm>
; ============================================================================
```

3.1.9 ASC_ECM

```
; ===========================================================================
; SUBROUTINE: ASC_ECM
; ---------------------------------------------------------------------------
; Descrizione:  Converte un codice dalla codifica ASCII a 7 bit alla pseudo-
;               codifica ECM, cioè i codici come si interpretano con il modo
;               Colore di Sfondo Esteso attivo, senza produrre codici di
;               controllo. La conversione separa le due informazioni: .A riceve
;               il codice del glifo, da 0 a 63, e il selettore del colore di
;               sfondo, da 0 a 3, va in __BG_COL. È il risultato di ASC_SCR
;               seguita da SCR_ECM: le tabelle hanno i codici Schermo.
;
;               Il colore di sfondo segue le regole di PET_ECM: BGCOL0 per i
;               codici da 32 a 63, per le lettere minuscole e per @, [, ] e ^;
;               BGCOL1 per le lettere maiuscole, per i grafici che sostituiscono
;               i caratteri ASCII che il Commodore non ha, tranne ^ (vedi [NOTA
;               #03 in <io/chars.asm>]), e per il DEL; BGCOL2 per i codici di
;               controllo da 0 a 31, che hanno il glifo inverso del carattere
;               con lo stesso codice di Codice Schermo.
;
;               I codici da 128 a 255, che non sono ASCII a 7 bit, non hanno
;               equivalente: la conversione è impossibile, con il Carry a 1 e .A
;               = $FF, e __BG_COL non viene modificato. Altrimenti la
;               conversione è sempre perfetta, con il Carry a 0.
;
; Input:
;   - .A (1 byte, CPU)       : Codice carattere in codifica ASCII a 7 bit
;
; Output:
;   - .A (1 byte, CPU)       : Codice del glifo in codifica ECM (da 0 a 63)
;                              ($FF se la conversione è impossibile)
;   - .X (1 byte, CPU)       : Codice carattere originale
;   - __BG_COL (1 byte, ZP)  : Selettore del colore di sfondo (da 0 a 3)
;   - Flag Carry (1 bit, .P) : 0 = conversione riuscita; 1 = conversione
;                              impossibile (.A = $FF)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, Zero, Carry), __BG_COL
; Dipendenze Esterne:   <io/chars_common.asm>
; ===========================================================================
```

3.1.10 ASC_ECM_C

```
; ============================================================================
; SUBROUTINE: ASC_ECM_C
; ----------------------------------------------------------------------------
; Descrizione:  Converte un codice dalla codifica ASCII a 7 bit alla pseudo-
;               codifica ECM, riconoscendo i codici di controllo. Come ASC_ECM,
;               ma ECM, come Codice Schermo, non ha codici di controllo: i
;               codici di controllo ASCII (da 0 a 31 e il DEL, 127) sono
;               impossibili, come lo sono i codici da 128 a 255, che non sono
;               ASCII a 7 bit.
;
;               Se la conversione è impossibile il Carry è a 1, .A vale $FF e
;               __BG_COL non viene modificato. Altrimenti il Carry è a 0.
;
; Input:
;   - .A (1 byte, CPU)       : Codice carattere in codifica ASCII a 7 bit
;
; Output:
;   - .A (1 byte, CPU)       : Codice del glifo in codifica ECM (da 0 a 63)
;                              ($FF se la conversione è impossibile)
;   - .X (1 byte, CPU)       : Codice carattere originale
;   - __BG_COL (1 byte, ZP)  : Selettore del colore di sfondo (da 0 a 3)
;   - Flag Carry (1 bit, .P) : 0 = conversione riuscita; 1 = conversione
;                              impossibile (.A = $FF)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, Zero, Carry), __BG_COL
; Dipendenze Esterne:   <io/chars_common.asm>
; ============================================================================
```

chars_ecm.asm

3.1.11 ECM_SCR

```
; ============================================================================
; SUBROUTINE: ECM_SCR
; ----------------------------------------------------------------------------
; Descrizione:  Converte un codice dalla pseudo-codifica ECM, cioè i codici come
;               si interpretano con il modo Colore di Sfondo Esteso attivo, alla
;               codifica Codice Schermo. In ingresso .A è il codice del glifo,
;               da 0 a 63, e __BG_COL il selettore del colore di sfondo, da 0 a
;               3: i due bit più significativi di .A e i sei più significativi
;               di __BG_COL sono ignorati. Il codice Schermo ha il glifo nei sei
;               bit meno significativi e il selettore nei due più significativi.
;               È la conversione inversa di SCR_ECM.
;
;               Nessuna informazione si perde e non ci sono eccezioni: il Carry
;               è sempre 0. Codice Schermo non ha codici di controllo: ECM_SCR_C
;               è identica a ECM_SCR.
;
; Input:
;   - .A (1 byte, CPU)       : Codice del glifo in codifica ECM (da 0 a 63)
;   - __BG_COL (1 byte, ZP)  : Selettore del colore di sfondo (da 0 a 3)
;
; Output:
;   - .A (1 byte, CPU)       : Codice carattere in codifica Codice Schermo
;   - .X (1 byte, CPU)       : Codice Schermo equivalente (glifo e sfondo)
;   - Flag Carry (1 bit, .P) : sempre 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, Zero, Carry)
; Dipendenze Esterne:   Nessuna
; ============================================================================
```

3.1.12 ECM_SCR_C

```
; ================================================================================
; SUBROUTINE: ECM_SCR_C
; --------------------------------------------------------------------------------
; Descrizione:  Converte un codice dalla pseudo-codifica ECM alla codifica
;               Codice Schermo, riconoscendo i codici di controllo. Codice
;               Schermo non ha codici di controllo: ECM_SCR_C è identica a
;               ECM_SCR (vedi ECM_SCR per la descrizione).
;
; Input:
;   - .A (1 byte, CPU)       : Codice del glifo in codifica ECM (da 0 a 63)
;   - __BG_COL (1 byte, ZP)  : Selettore del colore di sfondo (da 0 a 3)
;
; Output:
;   - .A (1 byte, CPU)       : Codice carattere in codifica Codice Schermo
;   - .X (1 byte, CPU)       : Codice Schermo equivalente (glifo e sfondo)
;   - Flag Carry (1 bit, .P) : sempre 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, Zero, Carry)
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

3.1.13 ECM_PET

```
; ===========================================================================
; SUBROUTINE: ECM_PET
; ---------------------------------------------------------------------------
; Descrizione:  Converte un codice dalla pseudo-codifica ECM alla codifica
;               PETSCII, senza produrre codici di controllo. In ingresso .A è il
;               codice del glifo, da 0 a 63, e __BG_COL il selettore del colore
;               di sfondo, da 0 a 3: i due bit più significativi di .A e i sei
;               più significativi di __BG_COL sono ignorati. Equivale a ECM_SCR
;               seguita da SCR_PET ed è la conversione inversa di PET_ECM.
;
;               BGCOL0 dà i codici da 32 a 95 e BGCOL1 i codici da 160 a 223,
;               quelli generati dalla tastiera. BGCOL2 e BGCOL3 sono i caratteri
;               inversi, che PETSCII non ha: si restituisce il carattere normale
;               con Carry a 1, che significa che il carattere in .A va stampato
;               in modalità inversa. Il glifo 30 con BGCOL1 o BGCOL3 è
;               l'aspetto che in ECM ha il pigreco (codice Schermo 94 o 222, di
;               cui il VIC-II ignora i due bit più significativi): diventa 255,
;               come in SCR_PET, e non 94 come con BGCOL0 o BGCOL2.
;
; Input:
;   - .A (1 byte, CPU)       : Codice del glifo in codifica ECM (da 0 a 63)
;   - __BG_COL (1 byte, ZP)  : Selettore del colore di sfondo (da 0 a 3)
;
; Output:
;   - .A (1 byte, CPU)       : Codice carattere in codifica PETSCII
;   - .X (1 byte, CPU)       : Codice Schermo equivalente (glifo e sfondo)
;   - Flag Carry (1 bit, .P) : 0 = conversione perfetta; 1 = il carattere va
;                              stampato in modalità inversa
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, Zero, Carry)
; Dipendenze Esterne:   <io/chars_common.asm>
; ===========================================================================
```

3.1.14 ECM_PET_C

```
; ============================================================================
; SUBROUTINE: ECM_PET_C
; ----------------------------------------------------------------------------
; Descrizione:  Converte un codice dalla pseudo-codifica ECM alla codifica
;               PETSCII, producendo codici di controllo. In ingresso .A è il
;               codice del glifo, da 0 a 63, e __BG_COL il selettore del colore
;               di sfondo, da 0 a 3: i due bit più significativi di .A e i sei
;               più significativi di __BG_COL sono ignorati. Equivale a
;               ECM_SCR_C seguita da SCR_PET_C. Come in ECM_PET, BGCOL0 dà i
;               codici da 32 a 95 e BGCOL1 i codici da 160 a 223.
;
;               I glifi da 0 a 31 con BGCOL2 e con BGCOL3 sono i caratteri
;               inversi che in modo Virgolette rappresentano i codici di
;               controllo da 0 a 31 e da 128 a 159, e diventano quei codici (il
;               glifo 30 con BGCOL3, che in ECM è l'aspetto del pigreco
;               inverso, diventa 158, il codice di YELLOW). I glifi da
;               32 a 63 con BGCOL2 e BGCOL3 diventano, come in ECM_PET, il
;               carattere normale da stampare in modalità inversa.
;
;               Il Carry a 0 dice che la conversione è perfetta. Con il Carry a
;               1 la conversione è imperfetta in uno di due modi, e si
;               distinguono con un test su .A: se i bit #6 e #5 di .A sono
;               entrambi zero (and #%01100000 dà zero), .A è un codice di
;               controllo; altrimenti .A è un carattere stampabile, che va
;               stampato in modalità inversa.
;
; Input:
;   - .A (1 byte, CPU)       : Codice del glifo in codifica ECM (da 0 a 63)
;   - __BG_COL (1 byte, ZP)  : Selettore del colore di sfondo (da 0 a 3)
;
; Output:
;   - .A (1 byte, CPU)       : Codice carattere in codifica PETSCII
;   - .X (1 byte, CPU)       : Codice Schermo equivalente (glifo e sfondo)
;   - Flag Carry (1 bit, .P) : 0 = conversione perfetta; 1 = conversione
;                              imperfetta: .A è un codice di controllo o un
;                              carattere da stampare in modalità inversa
;                              (vedi la Descrizione)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, Zero, Carry)
; Dipendenze Esterne:   <io/chars_common.asm>
; ============================================================================
```

3.1.15 ECM_ASC

```
; ===========================================================================
; SUBROUTINE: ECM_ASC
; ---------------------------------------------------------------------------
; Descrizione:  Converte un codice dalla pseudo-codifica ECM alla codifica ASCII
;               a 7 bit, senza produrre codici di controllo. In ingresso .A è il
;               codice del glifo, da 0 a 63, e __BG_COL il selettore del colore
;               di sfondo, da 0 a 3: i due bit più significativi di .A e i sei
;               più significativi di __BG_COL sono ignorati. Equivale a ECM_SCR
;               seguita da SCR_ASC.
;
;               Valgono le regole di SCR_ASC: i glifi con BGCOL0 e BGCOL1 si
;               convertono nel carattere ASCII che ha lo stesso aspetto, anche
;               con le sostituzioni di glifo (vedi [NOTA #03 in
;               <io/chars.asm>]). BGCOL2 e BGCOL3 sono i caratteri inversi, che
;               ASCII non ha: la conversione è impossibile, come per i grafici
;               che non somigliano a nessun carattere ASCII. In quel caso il
;               Carry è a 1 e .A vale $FF, che non è un codice ASCII valido.
;
; Input:
;   - .A (1 byte, CPU)       : Codice del glifo in codifica ECM (da 0 a 63)
;   - __BG_COL (1 byte, ZP)  : Selettore del colore di sfondo (da 0 a 3)
;
; Output:
;   - .A (1 byte, CPU)       : Codice carattere in codifica ASCII a 7 bit
;                              ($FF se la conversione è impossibile)
;   - .X (1 byte, CPU)       : Codice Schermo equivalente (glifo e sfondo)
;   - Flag Carry (1 bit, .P) : 0 = conversione riuscita; 1 = conversione
;                              impossibile, con .A = $FF
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, Zero, Carry)
; Dipendenze Esterne:   <io/chars_common.asm>
; ===========================================================================
```

3.1.16 ECM_ASC_C

```
; ===========================================================================
; SUBROUTINE: ECM_ASC_C
; ---------------------------------------------------------------------------
; Descrizione:  Converte un codice dalla pseudo-codifica ECM alla codifica ASCII
;               a 7 bit, producendo codici di controllo. In ingresso .A è il
;               codice del glifo, da 0 a 63, e __BG_COL il selettore del colore
;               di sfondo, da 0 a 3: i due bit più significativi di .A e i sei
;               più significativi di __BG_COL sono ignorati. Equivale a
;               ECM_SCR_C seguita da SCR_ASC_C. Come in ECM_ASC, ma i glifi da 0
;               a 31 con BGCOL2, che sono i caratteri inversi da 128 a 159 di
;               Codice Schermo, diventano i codici di controllo da 0 a 31, e il
;               glifo 38 con BGCOL1, che somiglia a una cancellazione, diventa
;               il DEL (127).
;
;               Il Carry a 0 dice che la conversione è riuscita. Con il Carry a
;               1 la conversione è impossibile o cambia la semantica, e si
;               distinguono con un test su .A: se vale $FF la conversione è
;               impossibile; altrimenti .A è un codice di controllo (da 0 a 31
;               oppure 127).
;
; Input:
;   - .A (1 byte, CPU)       : Codice del glifo in codifica ECM (da 0 a 63)
;   - __BG_COL (1 byte, ZP)  : Selettore del colore di sfondo (da 0 a 3)
;
; Output:
;   - .A (1 byte, CPU)       : Codice carattere in codifica ASCII a 7 bit
;                              ($FF se la conversione è impossibile)
;   - .X (1 byte, CPU)       : Codice Schermo equivalente (glifo e sfondo)
;   - Flag Carry (1 bit, .P) : 0 = conversione riuscita; 1 = conversione
;                              impossibile (.A = $FF) o codice di controllo
;                              (vedi la Descrizione)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, Zero, Carry)
; Dipendenze Esterne:   <io/chars_common.asm>
; ===========================================================================
```

chars_pet.asm

3.1.17 PET_SCR

```
; ============================================================================
; SUBROUTINE: PET_SCR
; ----------------------------------------------------------------------------
; Descrizione:  Converte un codice dalla codifica PETSCII alla codifica Codice
;               Schermo, senza distinguere i codici di controllo. La conversione
;               è "glifo per glifo": il carattere stampato con il codice di
;               arrivo ha lo stesso aspetto di quello di partenza.
;
;               I codici da 32 a 127 e da 160 a 255 sono caratteri
;               normali. Quelli da 96 a 127 e da 192 a 223 sono due
;               codici per lo stesso glifo, come lo sono quelli da 160
;               a 191 e da 224 a 255. Il pigreco ha due codici (126 e
;               255): il 255 va nel codice 94 come il 126.
;
;               I codici di controllo (da 0 a 31 e da 128 a 159) non hanno un
;               equivalente in Codice Schermo. Si restituisce il glifo che
;               mostrano in modo Virgolette, cioè il carattere inverso
;               (da 128 a 159 e da 192 a 223), e la conversione è perfetta.
;               Per distinguere i codici di controllo si usa PET_SCR_C.
;
; Input:
;   - .A (1 byte, CPU)       : Codice carattere in codifica PETSCII
;
; Output:
;   - .A (1 byte, CPU)       : Codice carattere in codifica Codice Schermo
;   - .X (1 byte, CPU)       : Codice carattere originale
;   - Flag Carry (1 bit, .P) : 0 = conversione perfetta (sempre); 1 = non
;                              si verifica
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, Zero, Carry)
; Dipendenze Esterne:   <io/chars_common.asm>
; ============================================================================
```

3.1.18 PET_SCR_C

```
; ============================================================================
; SUBROUTINE: PET_SCR_C
; ----------------------------------------------------------------------------
; Descrizione:  Converte un codice dalla codifica PETSCII alla codifica Codice
;               Schermo, riconoscendo i codici di controllo. Come PET_SCR, ma i
;               codici di controllo (da 0 a 31 e da 128 a 159) non hanno un
;               equivalente in Codice Schermo, che non ha codici di controllo:
;               la conversione è impossibile.
;
;               Il Carry a 0 dice che la conversione è perfetta. Il Carry a 1
;               dice che la conversione è impossibile, perché il codice è un
;               codice di controllo, e in questo caso .A vale $FF. Il valore di
;               .A non basta a distinguere i due casi, perché $FF è anche un
;               Codice Schermo valido (il pigreco inverso): si guarda il Carry.
;
; Input:
;   - .A (1 byte, CPU)       : Codice carattere in codifica PETSCII
;
; Output:
;   - .A (1 byte, CPU)       : Codice carattere in codifica Codice Schermo
;                              ($FF se la conversione è impossibile)
;   - .X (1 byte, CPU)       : Codice carattere originale
;   - Flag Carry (1 bit, .P) : 0 = conversione perfetta; 1 = conversione
;                              impossibile (codice di controllo)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, Zero, Carry)
; Dipendenze Esterne:   <io/chars_common.asm>
; ============================================================================
```

3.1.19 PET_ASC

```
; ================================================================================
; SUBROUTINE: PET_ASC
; --------------------------------------------------------------------------------
; Descrizione:  Converte un codice dalla codifica PETSCII alla codifica ASCII a
;               7 bit, senza produrre codici di controllo. La conversione è
;               "glifo per glifo", con i codici ASCII letti come li mostra il
;               set di caratteri #2: le lettere minuscole stanno in 65-90 di
;               PETSCII (e in 1-26 di Codice Schermo) e le maiuscole in 193-218.
;
;               I caratteri ASCII che il Commodore non ha sono sostituiti da
;               caratteri grafici: la routine li riconverte nel carattere ASCII
;               che sostituiscono (vedi [NOTA #03 in <io/chars.asm>]). I codici
;               duplicati di PETSCII (da 96 a 127 e da 192 a 223, da 160 a 191 e
;               da 224 a 255) danno lo stesso risultato.
;               risultato.
;
;               I codici di controllo da 0 a 31 sono trattati come i caratteri
;               di Codice Schermo con lo stesso codice: ASCII ha il glifo
;               corrispondente, ma PETSCII lo mostra inverso in modo
;               Virgolette. Il Carry è quindi a 1 e il carattere in .A va
;               stampato in modalità inversa. Non hanno equivalente, con il
;               Carry a 1 e .A = $FF (non è un codice ASCII valido), i codici da
;               128 a 159, i grafici che non somigliano a nessun carattere ASCII
;               e i controlli 28 e 31, i cui glifi (sterlina e freccia a
;               sinistra) ASCII non ha.
;
;               Il Carry a 1 ha dunque due significati, che si distinguono con
;               un test su .A: se vale $FF la conversione è impossibile;
;               altrimenti .A è un carattere da stampare in modalità inversa (il
;               codice originale in .X è minore di 32).
;
; Input:
;   - .A (1 byte, CPU)       : Codice carattere in codifica PETSCII
;
; Output:
;   - .A (1 byte, CPU)       : Codice carattere in codifica ASCII a 7 bit
;                              ($FF se la conversione è impossibile)
;   - .X (1 byte, CPU)       : Codice carattere originale
;   - Flag Carry (1 bit, .P) : 0 = conversione riuscita; 1 = conversione
;                              impossibile (.A = $FF) o carattere da
;                              stampare in modalità inversa
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, Zero, Carry)
; Dipendenze Esterne:   <io/chars_common.asm>
; ================================================================================
```

3.1.20 PET_ASC_C

```
; ============================================================================
; SUBROUTINE: PET_ASC_C
; ----------------------------------------------------------------------------
; Descrizione:  Converte un codice dalla codifica PETSCII alla codifica ASCII a
;               7 bit, riconoscendo i codici di controllo. Come PET_ASC, ma i
;               codici di controllo da 0 a 31 diventano i codici di controllo
;               ASCII con lo stesso codice, senza convertirne il significato
;               (che in molti casi è diverso), e la conversione è perfetta.
;
;               Il codice 166 e il suo duplicato 230, che somigliano a una
;               cancellazione, diventano il DEL (127), un codice di controllo:
;               la conversione è imperfetta, perché ASCII esegue un controllo
;               invece di mostrare il glifo di partenza. In PET_ASC sono
;               impossibili.
;
;               I codici da 128 a 159, i grafici che non somigliano a nessun
;               carattere ASCII e quelli da 160 a 191 e da 224 a 255 senza
;               equivalente non hanno una conversione: il Carry è a 1 e .A vale
;               $FF, che non è un codice ASCII valido.
;
;               Il Carry a 1 ha dunque due significati, che si distinguono con
;               un test su .A: se vale $FF la conversione è impossibile; se vale
;               127 è il codice di controllo DEL.
;
; Input:
;   - .A (1 byte, CPU)       : Codice carattere in codifica PETSCII
;
; Output:
;   - .A (1 byte, CPU)       : Codice carattere in codifica ASCII a 7 bit
;                              ($FF se la conversione è impossibile)
;   - .X (1 byte, CPU)       : Codice carattere originale
;   - Flag Carry (1 bit, .P) : 0 = conversione riuscita; 1 = conversione
;                              impossibile (.A = $FF) o codice di
;                              controllo DEL (.A = 127)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, Zero, Carry)
; Dipendenze Esterne:   <io/chars_common.asm>
; ============================================================================
```

3.1.21 PET_ECM

```
; ===========================================================================
; SUBROUTINE: PET_ECM
; ---------------------------------------------------------------------------
; Descrizione:  Converte un codice dalla codifica PETSCII alla pseudo-codifica
;               ECM, cioè i codici come si interpretano con il modo Colore di
;               Sfondo Esteso attivo. La conversione separa le due informazioni:
;               .A riceve il codice del glifo, da 0 a 63, e il selettore del
;               colore di sfondo, da 0 a 3, va in __BG_COL. È il risultato di
;               PET_SCR seguita da SCR_ECM.
;
;               Il colore di sfondo dipende solo dal blocco di partenza: BGCOL0
;               per i codici da 32 a 95, BGCOL1 per i codici da 96 a 127 e da
;               160 a 255 (i caratteri ottenuti con SHIFT e i loro duplicati),
;               BGCOL2 per i codici di controllo da 0 a 31 e BGCOL3 per quelli
;               da 128 a 159, che il Kernal mostra come caratteri inversi in
;               modo Virgolette. Il glifo è dato dai sei bit meno
;               significativi del codice Schermo: in ECM i grafici non esistono,
;               e i codici da 160 a 191 e da 224 a 255 hanno i glifi da 32 a 63.
;
;               Il pigreco ha due codici, 126 e 255. In ECM il VIC-II ignora i
;               due bit più significativi del codice Schermo e quindi mostra il
;               pigreco (codice Schermo 94) come la freccia in su, cioè il
;               glifo 30 con BGCOL1: è ciò che si vede sull'hardware reale, non
;               una sostituzione di glifo, e perciò non è una transcodifica nel
;               senso di <io/chars.asm>. Il 126 è nella tabella dei blocchi, il
;               255 è l'unica eccezione. Il Carry è sempre a 0, perché ECM è
;               l'interpretazione dei codici e la routine ne dà l'aspetto. La
;               modalità inversa non è nel codice PETSCII: chi stampa in
;               modalità inversa fa l'OR di __BG_COL con %10.
;
; Input:
;   - .A (1 byte, CPU)       : Codice carattere in codifica PETSCII
;
; Output:
;   - .A (1 byte, CPU)       : Codice del glifo in codifica ECM (da 0 a 63)
;   - .X (1 byte, CPU)       : Codice carattere originale
;   - __BG_COL (1 byte, ZP)  : Selettore del colore di sfondo (da 0 a 3)
;   - Flag Carry (1 bit, .P) : 0 = conversione perfetta (sempre); 1 = non
;                              si verifica
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, Zero, Carry), __BG_COL
; Dipendenze Esterne:   <io/chars_common.asm>
; ===========================================================================
```

3.1.22 PET_ECM_C

```
; ============================================================================
; SUBROUTINE: PET_ECM_C
; ----------------------------------------------------------------------------
; Descrizione:  Converte un codice dalla codifica PETSCII alla pseudo-codifica
;               ECM, riconoscendo i codici di controllo. Come PET_ECM, ma i
;               codici di controllo (da 0 a 31 e da 128 a 159) non hanno un
;               equivalente in ECM, che come Codice Schermo non ha codici di
;               controllo: la conversione è impossibile.
;
;               Il Carry a 0 dice che la conversione è perfetta. Il Carry a 1
;               dice che è impossibile, perché il codice è un codice di
;               controllo: .A vale $FF e __BG_COL non viene modificato. Il
;               valore di .A non basta a distinguere i due casi, perché $FF è
;               anche un codice ECM valido: si guarda il Carry.
;
; Input:
;   - .A (1 byte, CPU)       : Codice carattere in codifica PETSCII
;
; Output:
;   - .A (1 byte, CPU)       : Codice del glifo in codifica ECM (da 0 a 63)
;                              ($FF se la conversione è impossibile)
;   - .X (1 byte, CPU)       : Codice carattere originale
;   - __BG_COL (1 byte, ZP)  : Selettore del colore di sfondo (da 0 a 3)
;   - Flag Carry (1 bit, .P) : 0 = conversione perfetta; 1 = conversione
;                              impossibile (codice di controllo)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, Zero, Carry), __BG_COL
; Dipendenze Esterne:   <io/chars_common.asm>
; ============================================================================
```

chars_scr.asm

3.1.23 SCR_PET

```
; ============================================================================
; SUBROUTINE: SCR_PET
; ----------------------------------------------------------------------------
; Descrizione:  Converte un codice dalla codifica Codice Schermo alla codifica
;               PETSCII, senza produrre codici di controllo. La conversione è
;               "glifo per glifo": il carattere stampato con il codice di arrivo
;               ha lo stesso aspetto di quello di partenza.
;
;               I codici da 0 a 127 sono i caratteri normali. Quelli da 64 a 127
;               hanno in PETSCII due codici, e si restituisce quello generato
;               dalla tastiera (192-223 e 160-191).
;
;               I codici da 128 a 255 sono i caratteri inversi, che PETSCII non
;               ha: si restituisce il carattere normale con Carry a 1, che
;               significa che il carattere in .A va stampato in modalità
;               inversa.
;
;               Il pigreco è un'eccezione: il codice 94 e il codice 222 (il
;               pigreco inverso) diventano 255, il codice che genera la
;               tastiera.
;
; Input:
;   - .A (1 byte, CPU)       : Codice carattere in codifica Codice Schermo
;
; Output:
;   - .A (1 byte, CPU)       : Codice carattere in codifica PETSCII
;   - .X (1 byte, CPU)       : Codice carattere originale
;   - Flag Carry (1 bit, .P) : 0 = conversione perfetta; 1 = il carattere va
;                              stampato in modalità inversa
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, Zero, Carry)
; Dipendenze Esterne:   <io/chars_common.asm>
; ============================================================================
```

3.1.24 SCR_PET_C

```
; ============================================================================
; SUBROUTINE: SCR_PET_C
; ----------------------------------------------------------------------------
; Descrizione:  Converte un codice dalla codifica Codice Schermo alla codifica
;               PETSCII, producendo codici di controllo per i caratteri inversi
;               che li rappresentano. La conversione è "glifo per glifo", con i
;               codici di controllo mostrati in modo Virgolette.
;
;               I codici da 0 a 127 si convertono come in SCR_PET, compreso il
;               pigreco (94 diventa 255). I caratteri inversi da 128 a 159 e da
;               192 a 223 diventano i codici di controllo da 0 a 31 e da 128 a
;               159, che in modo Virgolette si vedono con quel glifo
;               inverso; il pigreco inverso (222) diventa quindi 158 (il codice
;               di YELLOW). L'alternativa, che è anche la scelta del Kernal
;               nella lettura dello schermo, sarebbe 255 con il Carry a 1, cioè
;               il pigreco da stampare in modalità inversa: non è adottata. I
;               caratteri inversi da 160 a 191 e da 224 a 255 diventano, come in
;               SCR_PET, il carattere normale da stampare in modalità inversa.
;
;               Il Carry a 0 dice che la conversione è perfetta. Con il Carry
;               a 1 la conversione è imperfetta in uno di due modi, e si
;               distinguono con un test su .A: se i bit #6 e #5 di .A sono
;               entrambi zero (and #%01100000 dà zero), .A è un codice di
;               controllo, che mostra il glifo inverso solo se viene stampato
;               in modo Virgolette e fuori dalle virgolette esegue il
;               controllo; altrimenti .A è un carattere stampabile, che va
;               stampato in modalità inversa. In ogni caso il bit #7 di .X
;               dice che il carattere originale era inverso.
;
; Input:
;   - .A (1 byte, CPU)       : Codice carattere in codifica Codice Schermo
;
; Output:
;   - .A (1 byte, CPU)       : Codice carattere in codifica PETSCII
;   - .X (1 byte, CPU)       : Codice carattere originale
;   - Flag Carry (1 bit, .P) : 0 = conversione perfetta; 1 = conversione
;                              imperfetta: .A è un codice di controllo o un
;                              carattere da stampare in modalità inversa (vedi
;                              la Descrizione)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, Zero, Carry)
; Dipendenze Esterne:   <io/chars_common.asm>
; ============================================================================
```

3.1.25 SCR_ASC

```
; ============================================================================
; SUBROUTINE: SCR_ASC
; ----------------------------------------------------------------------------
; Descrizione:  Converte un codice dalla codifica Codice Schermo alla codifica
;               ASCII a 7 bit, senza produrre codici di controllo. La
;               conversione è "glifo per glifo", con i codici letti come li
;               mostra il set di caratteri #2: da 1 a 26 le lettere minuscole,
;               da 65 a 90 le maiuscole, da 32 a 63 gli stessi caratteri che ha
;               ASCII con quei codici.
;
;               I caratteri ASCII che il Commodore non ha sono sostituiti da
;               caratteri grafici: la routine li riconverte nel carattere ASCII
;               che sostituiscono (vedi [NOTA #03 in <io/chars.asm>]).
;
;               Gli altri codici, cioè i caratteri inversi e i grafici che non
;               somigliano a nessun carattere ASCII, non hanno equivalente: il
;               Carry è a 1 e .A vale $FF, che non è un codice ASCII valido. Il
;               codice 102, che somiglia a una cancellazione, in questa variante
;               è impossibile, perché il suo equivalente sarebbe il DEL (127),
;               un codice di controllo: lo produce SCR_ASC_C.
;
; Input:
;   - .A (1 byte, CPU)       : Codice carattere in codifica Codice Schermo
;
; Output:
;   - .A (1 byte, CPU)       : Codice carattere in codifica ASCII a 7 bit
;   - .X (1 byte, CPU)       : Codice carattere originale
;   - Flag Carry (1 bit, .P) : 0 = conversione riuscita; 1 = conversione
;                              impossibile, con .A = $FF
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, Zero, Carry)
; Dipendenze Esterne:   <io/chars_common.asm>
; ============================================================================
```

3.1.26 SCR_ASC_C

```
; ============================================================================
; SUBROUTINE: SCR_ASC_C
; ----------------------------------------------------------------------------
; Descrizione:  Converte un codice dalla codifica Codice Schermo alla codifica
;               ASCII a 7 bit, producendo codici di controllo. Come SCR_ASC, con
;               in più due conversioni verso codici di controllo ASCII: i
;               caratteri inversi da 128 a 159 diventano i codici da 0 a 31, e
;               il codice 102, che somiglia a una cancellazione, diventa il DEL,
;               cioè il codice 127.
;
;               Il Carry a 0 dice che la conversione è riuscita, con la
;               sostituzione del glifo quando serve. Con il Carry a 1 la
;               conversione è impossibile o cambia la semantica, e si
;               distinguono con un test su .A: se vale $FF la conversione è
;               impossibile e non ha equivalente ASCII; altrimenti .A è un
;               codice di controllo (da 0 a 31 oppure 127), che esegue il
;               controllo invece di mostrare il glifo di partenza.
;
; Input:
;   - .A (1 byte, CPU)       : Codice carattere in codifica Codice Schermo
;
; Output:
;   - .A (1 byte, CPU)       : Codice carattere in codifica ASCII a 7 bit
;   - .X (1 byte, CPU)       : Codice carattere originale
;   - Flag Carry (1 bit, .P) : 0 = conversione riuscita; 1 = conversione
;                              impossibile (.A = $FF) o codice di controllo
;                              (vedi la Descrizione)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, Zero, Carry)
; Dipendenze Esterne:   <io/chars_common.asm>
; ============================================================================
```

3.1.27 SCR_ECM

```
; ============================================================================
; SUBROUTINE: SCR_ECM
; ----------------------------------------------------------------------------
; Descrizione:  Converte un codice dalla codifica Codice Schermo alla pseudo-
;               codifica ECM, cioè i codici come si interpretano con il modo
;               Colore di Sfondo Esteso attivo. In ECM i due bit più
;               significativi del codice scelgono il registro del colore di
;               sfondo (BGCOL0-BGCOL3) e i sei bit restanti il glifo, da 0 a 63.
;
;               La conversione separa le due informazioni: .A riceve il codice
;               del glifo, cioè il codice con i due bit più significativi
;               azzerati, e il selettore del colore di sfondo, da 0 a 3, va in
;               __BG_COL. Nessuna informazione si perde e non ci sono eccezioni:
;               il Carry è sempre 0.
;
;               Codice Schermo non ha codici di controllo: SCR_ECM_C è identica
;               a SCR_ECM.
;
; Input:
;   - .A (1 byte, CPU)       : Codice carattere in codifica Codice Schermo
;
; Output:
;   - .A (1 byte, CPU)       : Codice del glifo in codifica ECM (da 0 a 63)
;   - .X (1 byte, CPU)       : Codice carattere originale
;   - __BG_COL (1 byte, ZP)  : Selettore del colore di sfondo (da 0 a 3)
;   - Flag Carry (1 bit, .P) : sempre 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .P (Negative, Zero, Carry), __BG_COL
; Dipendenze Esterne:   Nessuna
; ============================================================================
```

3.1.28 SCR_ECM_C

```
; ===========================================================================
; SUBROUTINE: SCR_ECM_C
; ---------------------------------------------------------------------------
; Descrizione:  Converte un codice dalla codifica Codice Schermo alla pseudo-
;               codifica ECM, trattando i codici di controllo. Codice Schermo
;               non ha codici di controllo, quindi la routine è identica a
;               SCR_ECM e c'è solo per completare la famiglia di routine: la
;               conversione separa il codice del glifo (i sei bit meno
;               significativi, in .A) dal selettore del colore di sfondo (i due
;               bit più significativi, da 0 a 3, in __BG_COL). Nessuna
;               informazione si perde: il Carry è sempre 0.
;
; Input:
;   - .A (1 byte, CPU)       : Codice carattere in codifica Codice Schermo
;
; Output:
;   - .A (1 byte, CPU)       : Codice del glifo in codifica ECM (da 0 a 63)
;   - .X (1 byte, CPU)       : Codice carattere originale
;   - __BG_COL (1 byte, ZP)  : Selettore del colore di sfondo (da 0 a 3)
;   - Flag Carry (1 bit, .P) : sempre 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .P (Negative, Zero, Carry), __BG_COL
; Dipendenze Esterne:   Nessuna
; ===========================================================================
```

3.2 disk_tools.asm

3.2.1 Descrizione

Utilità per la gestione delle unità disco.

Utilità	Descrizione
DISK_STATUS	Legge lo stato dell'unità disco
DECODE_DISK_STATUS	Decodifica lo stato dell'unità disco
DISK_DIRECTORY	Stampa la directory del disco
DISK_COMMAND	Invia un comando all'unità disco

3.2.2 Note d'uso

Nessuna nota in questo modulo.

3.2.3 DISK_STATUS

```
; ================================================================================
; SUBROUTINE: DISK_STATUS
; --------------------------------------------------------------------------------
; Descrizione:  Legge il messaggio di stato dell'unità disco dal canale comandi
;               (file logico 15, indirizzo secondario 15) e lo scrive in STACK,
;               la base della pagina dello stack, terminato da $00: il carattere
;               di ritorno a capo ($0D) che chiude il messaggio è sostituito
;               dallo zero finale. Il messaggio occupa al massimo 256 byte ed è
;               sovrascritto a ogni chiamata, quindi il chiamante lo usa subito
;               (per esempio con STROUT). Se il canale non si apre il buffer non
;               viene scritto.
;
;               Il flag di Carry dice solo se il messaggio è stato letto, non se
;               il disco ha segnalato un errore: per estrarne il codice si usa
;               DECODE_DISK_STATUS.
;
; Input:
;   - .A (1 byte, CPU)             : Numero dell'unità disco
;
; Output:
;   - STACK (fino a 256 byte, RAM) : Messaggio di stato terminato da $00
;   - Flag Carry (1 bit, .P)       : 0 = messaggio letto; 1 = canale non aperto
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       STACK (fino a 256 byte)
; Dipendenze Esterne:   <c64/kernal.asm>
;                       <c64/symbols.asm>
; ================================================================================
```

3.2.4 DECODE_DISK_STATUS

```
; ============================================================================
; SUBROUTINE: DECODE_DISK_STATUS
; ----------------------------------------------------------------------------
; Descrizione:  Decodifica il messaggio di stato dell'unità disco contenuto in
;               STACK (scritto da DISK_STATUS) e restituisce il codice
;               dell'errore e gli altri due numeri del messaggio. Il messaggio
;               ha la forma "NN,MESSAGGIO,TT,SS", dove le virgole fanno parte
;               del messaggio e le virgolette no: NN è il codice dell'errore,
;               MESSAGGIO una stringa descrittiva, TT e SS di norma la traccia e
;               il settore in cui è avvenuto l'errore (in alcuni casi hanno un
;               altro significato). Senza errori il messaggio è "00,OK,00,00". I
;               tre numeri hanno due cifre e si restituiscono in binario, da 0 a
;               99. Il messaggio non viene alterato e non supera i 40 caratteri:
;               la ricerca della virgola che precede TT si ferma alla
;               quarantesima posizione.
;
;               Il controllo del formato è minimo: la decodifica si ferma al
;               primo carattere diverso da quello previsto e in quel caso .A, .X
;               e .Y sono indefiniti. Il flag di Carry dice solo se il messaggio
;               è stato decodificato, non se il disco ha segnalato un errore:
;               per saperlo il chiamante guarda .A (0 = nessun errore).
;
; Input:
;   - STACK (fino a 40 byte, RAM) : Messaggio di stato terminato da $00
;
; Output:
;   - .A (1 byte, CPU)            : Codice dell'errore (NN)
;   - .X (1 byte, CPU)            : Primo numero dopo il messaggio (TT)
;   - .Y (1 byte, CPU)            : Secondo numero dopo il messaggio (SS)
;   - Flag Carry (1 bit, .P)      : 0 = messaggio decodificato; 1 = formato
;                                   non valido
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   <c64/symbols.asm>
; ============================================================================
```

3.2.5 DISK_DIRECTORY

```
; ============================================================================
; SUBROUTINE: DISK_DIRECTORY
; ----------------------------------------------------------------------------
; Descrizione:  Stampa la directory del disco dell'unità indicata, una voce per
;               linea: la lunghezza in blocchi, uno spazio e il nome. Apre un
;               canale dati (file logico 88, indirizzo secondario 0), legge la
;               directory come un programma e stampa ogni linea con __PUT_CHAR.
;
;               I file da elencare si scelgono con un pattern: una stringa
;               terminata da $00 puntata da INDEX (per esempio "$0"). La
;               subroutine ne calcola la lunghezza e lo passa al sistema
;               operativo dell'unità senza altri controlli: un pattern nullo o
;               più lungo di 40 caratteri è un errore, e la subroutine esce
;               subito con il Carry a 1 senza stampare nulla. Un pattern non
;               valido per l'unità produce invece un errore dell'unità, trattato
;               come ogni altro.
;
;               Il flag di Carry è l'esito: se si verifica un errore dell'unità
;               la subroutine va a capo, stampa il messaggio di stato dell'unità
;               (letto con DISK_STATUS) ed esce con il Carry a 1. La stampa dei
;               numeri e del messaggio usa LINPRT e STROUT del BASIC, che
;               alterano proprie celle di lavoro (tra cui FAC): il loro
;               contenuto non è garantito dopo la chiamata.
;
;               Richiede __USE_DISK_STATUS = 1.
;
; Input:
;   - .A (1 byte, CPU)             : Numero dell'unità disco
;   - INDEX (2 byte, ZP)           : Puntatore al pattern, terminato da $00
;
; Output:
;   - Flag Carry (1 bit, .P)       : 0 = directory stampata; 1 = pattern errato,
;                                      canale non aperto o errore dell'unità
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       STACK (fino a 256 byte)
; Dipendenze Esterne:   <c64/kernal.asm>
;                       <c64/symbols.asm>
; ============================================================================
```

3.2.6 DISK_COMMAND

```
; ============================================================================
; SUBROUTINE: DISK_COMMAND
; ----------------------------------------------------------------------------
; Descrizione:  Invia un comando DOS all'unità disco indicata. Il comando è una
;               stringa terminata da $00 puntata da INDEX e non può essere vuoto
;               né superare i 40 caratteri: il DOS dell'unità tronca un comando
;               più lungo, con risultati imprevedibili. La subroutine rifiuta un
;               comando non valido senza inviarlo ed esce con il Carry a 1.
;
;               Il comando si invia aprendo il canale comandi (indirizzo
;               secondario 15) con la stringa come nome del file: l'OPEN è tutto
;               ciò che serve. Il file logico non può essere il 15, perché
;               DISK_STATUS apre per conto suo il file logico 15 e l'OPEN
;               fallirebbe se fosse già aperto: la subroutine usa il file logico
;               88, lo stesso di DISK_DIRECTORY, e lo chiude dopo la lettura
;               dello stato.
;
;               Il messaggio di stato con cui il DOS risponde al comando si
;               legge con DISK_STATUS e si scrive in STACK, terminato da $00:
;               occupa al massimo 256 byte e il chiamante lo usa subito (per
;               esempio con STROUT, o con DECODE_DISK_STATUS per ricavarne il
;               codice dell'errore). Il flag di Carry dice solo se il comando è
;               stato inviato e lo stato letto, non se l'unità ha segnalato un
;               errore. Se il canale comandi non si apre il comando non parte e
;               STACK non viene scritto. Se invece non si riesce a leggere lo
;               stato, il comando è comunque partito ma il Carry vale 1 e STACK
;               non viene scritto.
;
;               Richiede __USE_DISK_STATUS = 1.
;
; Input:
;   - .A (1 byte, CPU)             : Numero dell'unità disco
;   - INDEX (2 byte, ZP)           : Puntatore al comando, terminato da $00
;
; Output:
;   - STACK (fino a 256 byte, RAM) : Messaggio di stato terminato da $00, non
;                                    scritto se il comando non è inviato
;   - Flag Carry (1 bit, .P)       : 0 = comando inviato; 1 = errore (comando
;                                    vuoto o più lungo di 40 caratteri, canale
;                                    non aperto o stato non letto)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       STACK (fino a 256 byte)
; Dipendenze Esterne:   <c64/kernal.asm>
;                       <c64/symbols.asm>
; ============================================================================
```

3.3 keyboard_tools.asm

3.3.1 Descrizione

Lettura diretta della tastiera dalla matrice del CIA #1, senza passare dal Kernal. Deriva da quella del gioco Platoon, accorciata e con la serie storica integrata.

Utilità	Descrizione
KEYSCAN	Scansiona un tasto della matrice della tastiera

3.3.2 Note d'uso

```
; NOTA #01
; --------
; KEYSCAN non cerca quale tasto sia premuto e non restituisce il codice tastiera
; di un tasto: fa l'inverso. Riceve il codice tastiera di un tasto e verifica se
; quel tasto è stato appena premuto. Non è adatta a scoprire quale tasto sia
; premuto tra tutti, perché non scansiona l'intera matrice.
;
; KEYSCAN controlla un tasto per volta e va chiamata a ogni passaggio di
; lettura. Segnala una nuova pressione una sola volta: finché il tasto resta
; premuto, le chiamate successive non riportano niente di nuovo. Un codice
; tastiera non valido (da KBD_NO_KEY in su) è un errore, come il falso tasto.
; Con il Carry a 0, il flag di Negative dice se il tasto è premuto e il flag di
; Zero se la pressione è nuova. Gli esempi sono nelle note da [NOTA #03] in poi.
```

```
; NOTA #02
; --------
; La serie storica, la cella in pagina zero __KBD_HISTORY, è un solo byte,
; condiviso da tutti i tasti: finché il tasto resta premuto contiene la sola
; colonna, non la riga, e al rilascio viene azzerata. Il chiamante la azzera
; prima del primo uso. Quindi si controlla un tasto per volta: se nello stesso
; passaggio si scansionano tasti diversi, le chiamate si disturbano a vicenda
; e KEYSCAN segnala una nuova pressione a ogni passaggio finché uno dei tasti
; resta premuto. Due tasti della stessa colonna, poi, non si
; distinguono tra loro.
```

```
; NOTA #03
; --------
; Uso base: una scansione a ogni passaggio, per esempio dentro un gestore IRQ.
; Il Carry a 1 segnala un errore, il flag di Zero a 0 una nuova pressione.
; Esempio:
;
;   lda #KBD_SPACE                ; Tasto da scansionare.
;   jsr KEYSCAN
;   bcs .Exit_Handler             ; Falso tasto: ignora la lettura.
;
;   beq .Exit_Handler             ; Nessuna nuova pressione.
;
;   inc EXTCOL                    ; Nuova pressione: cambia il bordo.
;
; .Exit_Handler:
;   rts
```

```
; NOTA #04
; --------
; Debouncing. I contatti della tastiera rimbalzano per qualche millisecondo,
; sia premendo sia rilasciando il tasto. Il flag di Negative dice se il tasto
; è ancora premuto: una nuova pressione si conferma al passaggio successivo, e
; se il tasto non è più premuto era solo un rimbalzo. Lo snippet è per un
; gestore IRQ chiamato a ogni fotogramma, e _CANDIDATE è una cella di RAM del
; programma, azzerata all'avvio. Esempio:
;
;   lda #KBD_SPACE                ; Tasto da scansionare.
;   jsr KEYSCAN
;   bcs .Exit_Handler             ; Falso tasto: ignora la lettura.
;
;   bpl .Clear_Candidate          ; Il tasto non è premuto.
;
;   beq .Check_Candidate          ; Il tasto è tenuto.
;
;   lda #1                        ; Nuova pressione: la conferma
;   sta _CANDIDATE                ; arriva al prossimo passaggio.
;   rts
;
; .Clear_Candidate:
;   lda #0                        ; Rimbalzo o rilascio: annulla
;   sta _CANDIDATE                ; la pressione in attesa.
;   rts
;
; .Check_Candidate:
;   lda _CANDIDATE                ; C'è una pressione da confermare?
;
;   beq .Exit_Handler             ; No: il tasto è tenuto da prima.
;
;   lda #0                        ; Sì: la pressione è confermata.
;   sta _CANDIDATE
;
;   inc EXTCOL                    ; Cambia il bordo.
;
; .Exit_Handler:
;   rts
```

```
; NOTA #05
; --------
; Attesa della pressione e del rilascio. Il primo ciclo attende una nuova
; pressione, il secondo attende il rilascio: finché il tasto resta premuto il
; flag di Negative vale 1. Lo snippet è per il programma principale e non per
; un gestore IRQ, perché i cicli non cedono mai il controllo. Esempio:
;
; .Loop_Scan_Press:
;   lda #KBD_SPACE               ; REPEAT
;   jsr KEYSCAN
;   bcs .Loop_Scan_Press         ; Falso tasto: riprova.
;
; .Loop_Check_Press:
;   beq .Loop_Scan_Press         ; REPEAT UNTIL: nuova pressione
;
;   inc EXTCOL                   ; Tasto premuto: cambia il bordo.
;
; .Loop_Poll_Release:
;   lda #KBD_SPACE               ; REPEAT
;   jsr KEYSCAN
;   bcs .Loop_Poll_Release       ; Falso tasto: riprova.
;
; .Loop_Check_Release:
;   bmi .Loop_Poll_Release       ; REPEAT UNTIL: tasto non premuto
```

3.3.3 KEYSCAN

```
; ============================================================================
; SUBROUTINE: KEYSCAN
; ----------------------------------------------------------------------------
; Descrizione:  Verifica se un tasto, indicato dal suo codice tastiera, è stato
;               appena premuto. Legge la matrice della tastiera direttamente dal
;               CIA #1 e presuppone che i registri di direzione delle sue due
;               porte abbiano i valori predefiniti ($FF e $00).
;
;               Non cerca il tasto premuto e non ne restituisce il codice
;               tastiera: il codice tastiera è un dato in ingresso.
;
;               Un codice tastiera da KBD_NO_KEY in su non è valido: la
;               subroutine esce subito con il Carry a 1, senza toccare le porte.
;
;               Il codice tastiera è la posizione del tasto nella matrice, cioè
;               la riga per 8 più la colonna (KBD_*, da $00 a $3F). La serie
;               storica, __KBD_HISTORY, è un solo byte: contiene la maschera
;               della colonna finché il tasto resta premuto e vale zero dopo
;               il rilascio.
;
;               Il Carry segnala il falso tasto: la colonna risulta bassa
;               anche senza nessuna riga selezionata, quindi la causa è
;               l'interferenza del joystick e non la tastiera. Con il Carry
;               a 0 il flag di Negative dice se il tasto è premuto e il flag
;               di Zero se la pressione è nuova.
;
; Input:
;   - .A (1 byte, CPU)             : Codice tastiera del tasto da scansionare
;   - __KBD_HISTORY (1 byte, ZP)   : Serie storica, da azzerare prima del
;                                    primo uso
;
; Output:
;   - __KBD_HISTORY (1 byte, ZP)   : Serie storica aggiornata
;   - Flag Negative (1 bit, .P)    : 0 = tasto non premuto; 1 = tasto premuto
;                                    (nuova pressione o tenuto). Indefinito se
;                                    Carry = 1
;   - Flag Zero (1 bit, .P)        : 0 = nuova pressione; 1 = tasto non premuto
;                                    o ancora tenuto. Indefinito se Carry = 1
;   - Flag Carry (1 bit, .P)       : 0 = lettura valida; 1 = falso tasto o
;                                    codice tastiera non valido
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero, Carry), CIAPRA, CIAPRB,
;                       __KBD_HISTORY
; Dipendenze Esterne:   <c64/keyboard.asm>
;                       <c64/symbols.asm>
;                       <chip/cia.asm>
; ============================================================================
```

Libreria

math

4 Libreria math

Modulo	Descrizione
`math8.asm`	Macro per operazioni matematiche su valori a 8 bit.
`math16.asm`	Macro per operazioni matematiche su valori a 16 bit.
`math24.asm`	Macro per operazioni matematiche su valori a 24 bit.
`math32.asm`	Macro per operazioni matematiche su valori a 32 bit.
`math_div.asm`	Divisione intera senza segno e con segno per tutte le combinazioni di operandi da 8 a 32 bit. Il quoziente ha la lunghezza del dividendo e il resto quella del divisore, senza perdita di precisione.
`math_mult.asm`	Moltiplicazione intera senza segno e con segno per tutte le combinazioni di operandi da 8 a 32 bit. Il prodotto ha sempre la lunghezza esatta m+n bit, senza perdita di precisione.

4.1 math8.asm

4.1.1 Descrizione

Macro per operazioni matematiche su valori a 8 bit.

Utilità	Descrizione
`m_Neg`	Nega un valore a 8 bit
`m_Not`	NOT booleano di un valore a 8 bit
`m_AslS`	Scorrimento aritmetico a sinistra con segno
`m_AsrS`	Scorrimento aritmetico a destra con segno
`m_Rl8`	Rotazione a sinistra senza passare per il Carry
`m_Rr8`	Rotazione a destra senza passare per il Carry

4.1.2 Note d'uso

Nessuna nota in questo modulo.

4.1.3 m_Neg

```
; ================================================================================
; MACRO: m_Neg
; --------------------------------------------------------------------------------
; Descrizione:  Sottrae un valore a 8 bit da zero, secondo l'indirizzamento
;               scelto, e memorizza il risultato nella locazione desiderata.
;
;                 "IMM"  .op_ è un valore a 8 bit.
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .op_ (1 o 2 byte, Macro)  : Valore (IMM) o indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (1 byte, Macro)     : Riceve il risultato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, oVerflow, Zero, Carry), .res_
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.1.4 m_Not

```
; ================================================================================
; MACRO: m_Not
; --------------------------------------------------------------------------------
; Descrizione:  Inverte tutti i bit di un valore a 8 bit, secondo
;               l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata.
;
;                 "IMM"  .op_ è un valore a 8 bit.
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .op_ (1 o 2 byte, Macro)  : Valore (IMM) o indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (1 byte, Macro)     : Riceve il risultato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), .res_
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.1.5 m_AslS

```
; ================================================================================
; MACRO: m_AslS
; --------------------------------------------------------------------------------
; Descrizione:  Fa scorrere a sinistra di un bit un valore a 8 bit con segno,
;               secondo l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata. A differenza di un semplice ASL, il
;               cambiamento di segno dovuto all'uscita del bit più alto è
;               segnalato dal flag di oVerflow.
;
;                 "ACC"  L'operando e il risultato sono in .A.
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Con "ACC" la macro usa NUMWRK+0 come cella di appoggio.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "ACC", "ABS" oppure "ZP"
;   - .op_ (1 o 2 byte, Macro)  : Indirizzo dell'operando (ignorato con "ACC")
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato (ignorato con "ACC")
;   - .A (1 byte, CPU)          : Operando (solo con "ACC")
;
; Output:
;   - .A (1 byte, CPU)          : Risultato (solo con "ACC")
;   - .res_ (1 byte, Macro)     : Riceve il risultato (con "ABS" e "ZP")
;   - Flag oVerflow (1 bit, .P) : 0 = segno invariato; 1 = segno cambiato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, oVerflow, Zero, Carry),
;                       NUMWRK+0 (se .mode_ = "ACC"),
;                       .res_ (se .mode_ = "ABS" o "ZP")
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.1.6 m_AsrS

```
; ================================================================================
; MACRO: m_AsrS
; --------------------------------------------------------------------------------
; Descrizione:  Fa scorrere a destra di un bit un valore a 8 bit con segno,
;               secondo l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata. Il bit di segno si conserva e il bit
;               uscito finisce nel Carry.
;
;                 "ACC"  L'operando e il risultato sono in .A.
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "ACC", "ABS" oppure "ZP"
;   - .op_ (1 o 2 byte, Macro)  : Indirizzo dell'operando (ignorato con "ACC")
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato (ignorato con "ACC")
;   - .A (1 byte, CPU)          : Operando (solo con "ACC")
;
; Output:
;   - .A (1 byte, CPU)          : Risultato (solo con "ACC")
;   - .res_ (1 byte, Macro)     : Riceve il risultato (con "ABS" e "ZP")
;   - Flag Carry (1 bit, .P)    : 0 = bit uscito 0; 1 = bit uscito 1
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero, Carry),
;                       .res_ (se .mode_ = "ABS" o "ZP")
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.1.7 m_Rl8

```
; ================================================================================
; MACRO: m_Rl8
; --------------------------------------------------------------------------------
; Descrizione:  Ruota a sinistra di un bit un valore a 8 bit senza passare per
;               il Carry, secondo l'indirizzamento scelto, e memorizza il
;               risultato nella locazione desiderata. Il bit più alto finisce
;               nella posizione più bassa.
;
;                 "ACC"  L'operando e il risultato sono in .A.
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "ACC", "ABS" oppure "ZP"
;   - .op_ (1 o 2 byte, Macro)  : Indirizzo dell'operando (ignorato con "ACC")
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato (ignorato con "ACC")
;   - .A (1 byte, CPU)          : Operando (solo con "ACC")
;
; Output:
;   - .A (1 byte, CPU)          : Risultato (solo con "ACC")
;   - .res_ (1 byte, Macro)     : Riceve il risultato (con "ABS" e "ZP")
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero, Carry),
;                       .res_ (se .mode_ = "ABS" o "ZP")
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.1.8 m_Rr8

```
; ================================================================================
; MACRO: m_Rr8
; --------------------------------------------------------------------------------
; Descrizione:  Ruota a destra di un bit un valore a 8 bit senza passare per il
;               Carry, secondo l'indirizzamento scelto, e memorizza il risultato
;               nella locazione desiderata. Il bit più basso finisce nella
;               posizione più alta.
;
;                 "ACC"  L'operando e il risultato sono in .A.
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "ACC", "ABS" oppure "ZP"
;   - .op_ (1 o 2 byte, Macro)  : Indirizzo dell'operando (ignorato con "ACC")
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato (ignorato con "ACC")
;   - .A (1 byte, CPU)          : Operando (solo con "ACC")
;
; Output:
;   - .A (1 byte, CPU)          : Risultato (solo con "ACC")
;   - .res_ (1 byte, Macro)     : Riceve il risultato (con "ABS" e "ZP")
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero, Carry),
;                       .res_ (se .mode_ = "ABS" o "ZP")
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.2 math16.asm

4.2.1 Descrizione

Macro per operazioni matematiche su valori a 16 bit.

Utilità	Descrizione
`m_Asl16`	Scorrimento a sinistra di un valore a 16 bit
`m_Add16`	Somma due valori a 16 bit
`m_Sub16`	Sottrae due valori a 16 bit
`m_Neg16`	Nega un valore a 16 bit
`m_Not16`	NOT booleano di un valore a 16 bit
`m_And16`	AND booleano di due valori a 16 bit
`m_Ora16`	OR booleano di due valori a 16 bit
`m_Eor16`	EOR booleano di due valori a 16 bit
`m_AslS16`	Scorrimento a sinistra con segno, segnala V
`m_Lsr16`	Scorrimento logico a destra a 16 bit
`m_AsrS16`	Scorrimento aritmetico a destra con segno
`m_Rol16`	Rotazione a sinistra attraverso il Carry
`m_Rl16`	Rotazione a sinistra senza passare per il Carry
`m_Ror16`	Rotazione a destra attraverso il Carry
`m_Rr16`	Rotazione a destra senza passare per il Carry

4.2.2 Note d'uso

Nessuna nota in questo modulo.

4.2.3 m_Asl16

```
; ================================================================================
; MACRO: m_Asl16
; --------------------------------------------------------------------------------
; Descrizione:  Fa scorrere a sinistra di un bit un valore a 16 bit, secondo
;               l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata. È uno scorrimento logico: non si tiene
;               traccia di un eventuale overflow.
;
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "ABS" oppure "ZP"
;   - .op_ (1 o 2 byte, Macro)  : Indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (2 byte, Macro)     : Riceve il risultato a 16 bit
;   - Flag Carry (1 bit, .P)    : 0 = bit uscito 0; 1 = bit uscito 1
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A (se .op_ != .res_), .P (Negative, Zero, Carry),
;                       .res_+0 .. +1
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.2.4 m_Add16

```
; ================================================================================
; MACRO: m_Add16
; --------------------------------------------------------------------------------
; Descrizione:  Somma due valori a 16 bit, secondo l'indirizzamento scelto, e
;               memorizza il risultato nella locazione desiderata.
;
;                 "IMM"  Immediato: .add1_ è un indirizzo e .add2_ è un
;                        valore a 16 bit.
;                 "ABS"  Assoluto: gli operandi sono indirizzi
;                        assoluti.
;                 "ZP"   Pagina Zero: gli operandi sono indirizzi in
;                        pagina zero.
;
;               Se i due addendi coincidono ("ABS" e "ZP") il raddoppio è
;               eseguito da m_Asl16, che non imposta V.
;
;               Se .res_ coincide con il primo operando le copie sono omesse, e
;               N e Z non riflettono il risultato.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)    : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .add1_ (1 o 2 byte, Macro) : Indirizzo del primo addendo
;   - .add2_ (2 byte, Macro)     : Secondo addendo: valore (IMM) o indirizzo
;   - .res_ (1 o 2 byte, Macro)  : Indirizzo del risultato, assoluto o in pagina
;                                  zero
;
; Output:
;   - .res_ (2 byte, Macro)      : Riceve il risultato a 16 bit
;   - Flag Carry (1 bit, .P)     : 0 = riporto assente; 1 = riporto
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, oVerflow, Zero, Carry), .res_+0 .. +1
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.2.5 m_Sub16

```
; ================================================================================
; MACRO: m_Sub16
; --------------------------------------------------------------------------------
; Descrizione:  Sottrae il sottraendo dal minuendo, due valori a 16 bit, secondo
;               l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata.
;
;                 "IMM"  Immediato: .min_ è un indirizzo e .sub_ è un
;                        valore a 16 bit.
;                 "ABS"  Assoluto: gli operandi sono indirizzi
;                        assoluti.
;                 "ZP"   Pagina Zero: gli operandi sono indirizzi in
;                        pagina zero.
;
;               Se .res_ coincide con il minuendo e il sottraendo vale zero, la
;               copia è omessa, e N e Z non riflettono il risultato.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .min_ (1 o 2 byte, Macro) : Indirizzo del minuendo
;   - .sub_ (2 byte, Macro)     : Sottraendo: valore (IMM) o indirizzo
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (2 byte, Macro)     : Riceve il risultato a 16 bit
;   - Flag Carry (1 bit, .P)    : 0 = prestito; 1 = nessun prestito
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, oVerflow, Zero, Carry), .res_+0 .. +1
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.2.6 m_Neg16

```
; ================================================================================
; MACRO: m_Neg16
; --------------------------------------------------------------------------------
; Descrizione:  Sottrae un valore a 16 bit da zero, secondo l'indirizzamento
;               scelto, e memorizza il risultato nella locazione desiderata.
;
;                 "IMM"  .op_ è un valore a 16 bit.
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Con "IMM" il risultato è calcolato dall'assemblatore e caricato
;               da m_Mov16.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .op_ (2 byte, Macro)      : Valore (IMM) o indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (2 byte, Macro)     : Riceve il risultato a 16 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, oVerflow, Zero, Carry), .res_+0 .. +1
; Dipendenze Esterne:   <mem/mem16.asm>
; ================================================================================
```

4.2.7 m_Not16

```
; ================================================================================
; MACRO: m_Not16
; --------------------------------------------------------------------------------
; Descrizione:  Inverte tutti i bit di un valore a 16 bit, secondo
;               l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata.
;
;                 "IMM"  .op_ è un valore a 16 bit.
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Con "IMM" il risultato è calcolato dall'assemblatore e caricato
;               da m_Mov16.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .op_ (2 byte, Macro)      : Valore (IMM) o indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (2 byte, Macro)     : Riceve il risultato a 16 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), .res_+0 .. +1
; Dipendenze Esterne:   <mem/mem16.asm>
; ================================================================================
```

4.2.8 m_And16

```
; ==============================================================================
; MACRO: m_And16
; ------------------------------------------------------------------------------
; Descrizione:  Esegue un AND booleano tra due valori a 16 bit, secondo
;               l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata.
;
;                 "IMM"  Immediato: .op1_ è un indirizzo e .op2_ è un
;                        valore a 16 bit.
;                 "ABS"  Assoluto: gli operandi sono indirizzi
;                        assoluti.
;                 "ZP"   Pagina Zero: gli operandi sono indirizzi in
;                        pagina zero.
;
;               Con "IMM" i byte della costante pari a $00 o $FF sono trattati a
;               parte: azzerano il byte o lo lasciano com'è.
;
;               Se .res_ coincide con il primo operando le copie inutili sono
;               omesse, e N e Z non riflettono il risultato.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .op1_ (1 o 2 byte, Macro) : Indirizzo del primo operando
;   - .op2_ (2 byte, Macro)     : Secondo operando: valore (IMM) o indirizzo
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (2 byte, Macro)     : Riceve il risultato a 16 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), .res_+0 .. +1
; Dipendenze Esterne:   <mem/mem16.asm>
; ==============================================================================
```

4.2.9 m_Ora16

```
; ================================================================================
; MACRO: m_Ora16
; --------------------------------------------------------------------------------
; Descrizione:  Esegue un OR booleano tra due valori a 16 bit, secondo
;               l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata.
;
;                 "IMM"  Immediato: .op1_ è un indirizzo e .op2_ è un
;                        valore a 16 bit.
;                 "ABS"  Assoluto: gli operandi sono indirizzi
;                        assoluti.
;                 "ZP"   Pagina Zero: gli operandi sono indirizzi in
;                        pagina zero.
;
;               Con "IMM" i byte della costante pari a $00 o $FF sono trattati a
;               parte: lasciano il byte com'è o lo portano a $FF.
;
;               Se .res_ coincide con il primo operando le copie inutili sono
;               omesse, e N e Z non riflettono il risultato.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .op1_ (1 o 2 byte, Macro) : Indirizzo del primo operando
;   - .op2_ (2 byte, Macro)     : Secondo operando: valore (IMM) o indirizzo
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (2 byte, Macro)     : Riceve il risultato a 16 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), .res_+0 .. +1
; Dipendenze Esterne:   <mem/mem16.asm>
; ================================================================================
```

4.2.10 m_Eor16

```
; ================================================================================
; MACRO: m_Eor16
; --------------------------------------------------------------------------------
; Descrizione:  Esegue un EOR booleano tra due valori a 16 bit, secondo
;               l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata.
;
;                 "IMM"  Immediato: .op1_ è un indirizzo e .op2_ è un
;                        valore a 16 bit.
;                 "ABS"  Assoluto: gli operandi sono indirizzi
;                        assoluti.
;                 "ZP"   Pagina Zero: gli operandi sono indirizzi in
;                        pagina zero.
;
;               Con "IMM" i byte della costante pari a $00 sono trattati a
;               parte: lasciano il byte com'è.
;
;               Se .res_ coincide con il primo operando le copie inutili sono
;               omesse, e N e Z non riflettono il risultato.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .op1_ (1 o 2 byte, Macro) : Indirizzo del primo operando
;   - .op2_ (2 byte, Macro)     : Secondo operando: valore (IMM) o indirizzo
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (2 byte, Macro)     : Riceve il risultato a 16 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), .res_+0 .. +1
; Dipendenze Esterne:   <mem/mem16.asm>
; ================================================================================
```

4.2.11 m_AslS16

```
; ================================================================================
; MACRO: m_AslS16
; --------------------------------------------------------------------------------
; Descrizione:  Fa scorrere a sinistra di un bit un valore a 16 bit con segno,
;               secondo l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata. A differenza di m_Asl16 il cambiamento di
;               segno dovuto all'uscita del bit più alto è segnalato da V.
;
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "ABS" oppure "ZP"
;   - .op_ (1 o 2 byte, Macro)  : Indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (2 byte, Macro)     : Riceve il risultato a 16 bit
;   - Flag oVerflow (1 bit, .P) : 0 = segno invariato; 1 = segno cambiato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, oVerflow, Zero, Carry), .res_+0 .. +1
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.2.12 m_Lsr16

```
; ================================================================================
; MACRO: m_Lsr16
; --------------------------------------------------------------------------------
; Descrizione:  Fa scorrere a destra di un bit un valore a 16 bit, secondo
;               l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata. Il bit più alto del risultato vale zero.
;
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "ABS" oppure "ZP"
;   - .op_ (1 o 2 byte, Macro)  : Indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (2 byte, Macro)     : Riceve il risultato a 16 bit
;   - Flag Carry (1 bit, .P)    : 0 = bit uscito 0; 1 = bit uscito 1
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A (se .op_ != .res_), .P (Negative, Zero, Carry),
;                       .res_+0 .. +1
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.2.13 m_AsrS16

```
; ================================================================================
; MACRO: m_AsrS16
; --------------------------------------------------------------------------------
; Descrizione:  Fa scorrere a destra di un bit un valore a 16 bit con segno,
;               secondo l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata. Il bit di segno si conserva.
;
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "ABS" oppure "ZP"
;   - .op_ (1 o 2 byte, Macro)  : Indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (2 byte, Macro)     : Riceve il risultato a 16 bit
;   - Flag Carry (1 bit, .P)    : 0 = bit uscito 0; 1 = bit uscito 1
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero, Carry), .res_+0 .. +1
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.2.14 m_Rol16

```
; ================================================================================
; MACRO: m_Rol16
; --------------------------------------------------------------------------------
; Descrizione:  Ruota a sinistra di un bit un valore a 16 bit attraverso il
;               Carry, secondo l'indirizzamento scelto, e memorizza il risultato
;               nella locazione desiderata. Il bit più alto esce nel Carry.
;
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "ABS" oppure "ZP"
;   - .op_ (1 o 2 byte, Macro)  : Indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (2 byte, Macro)     : Riceve il risultato a 16 bit
;   - Flag Carry (1 bit, .P)    : 0 = bit uscito 0; 1 = bit uscito 1
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A (se .op_ != .res_), .P (Negative, Zero, Carry),
;                       .res_+0 .. +1
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.2.15 m_Rl16

```
; ================================================================================
; MACRO: m_Rl16
; --------------------------------------------------------------------------------
; Descrizione:  Ruota a sinistra di un bit un valore a 16 bit senza passare per
;               il Carry, secondo l'indirizzamento scelto, e memorizza il
;               risultato nella locazione desiderata. Il bit più alto finisce
;               nella posizione più bassa.
;
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "ABS" oppure "ZP"
;   - .op_ (1 o 2 byte, Macro)  : Indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (2 byte, Macro)     : Riceve il risultato a 16 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero, Carry), .res_+0 .. +1
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.2.16 m_Ror16

```
; ================================================================================
; MACRO: m_Ror16
; --------------------------------------------------------------------------------
; Descrizione:  Ruota a destra di un bit un valore a 16 bit attraverso il Carry,
;               secondo l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata. Il bit più basso esce nel Carry.
;
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "ABS" oppure "ZP"
;   - .op_ (1 o 2 byte, Macro)  : Indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (2 byte, Macro)     : Riceve il risultato a 16 bit
;   - Flag Carry (1 bit, .P)    : 0 = bit uscito 0; 1 = bit uscito 1
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A (se .op_ != .res_), .P (Negative, Zero, Carry),
;                       .res_+0 .. +1
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.2.17 m_Rr16

```
; ================================================================================
; MACRO: m_Rr16
; --------------------------------------------------------------------------------
; Descrizione:  Ruota a destra di un bit un valore a 16 bit senza passare per il
;               Carry, secondo l'indirizzamento scelto, e memorizza il risultato
;               nella locazione desiderata. Il bit più basso finisce nella
;               posizione più alta.
;
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "ABS" oppure "ZP"
;   - .op_ (1 o 2 byte, Macro)  : Indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (2 byte, Macro)     : Riceve il risultato a 16 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero, Carry), .res_+0 .. +1
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.3 math24.asm

4.3.1 Descrizione

Macro per operazioni matematiche su valori a 24 bit.

Utilità	Descrizione
`m_Asl24`	Scorrimento a sinistra di un valore a 24 bit
`m_Add24`	Somma due valori a 24 bit
`m_Sub24`	Sottrae due valori a 24 bit
`m_Neg24`	Nega un valore a 24 bit
`m_Not24`	NOT booleano di un valore a 24 bit
`m_And24`	AND booleano di due valori a 24 bit
`m_Ora24`	OR booleano di due valori a 24 bit
`m_Eor24`	EOR booleano di due valori a 24 bit
`m_AslS24`	Scorrimento a sinistra con segno, segnala V
`m_Lsr24`	Scorrimento logico a destra a 24 bit
`m_AsrS24`	Scorrimento aritmetico a destra con segno
`m_Rol24`	Rotazione a sinistra attraverso il Carry
`m_Rl24`	Rotazione a sinistra senza passare per il Carry
`m_Ror24`	Rotazione a destra attraverso il Carry
`m_Rr24`	Rotazione a destra senza passare per il Carry

4.3.2 Note d'uso

Nessuna nota in questo modulo.

4.3.3 m_Asl24

```
; ================================================================================
; MACRO: m_Asl24
; --------------------------------------------------------------------------------
; Descrizione:  Fa scorrere a sinistra di un bit un valore a 24 bit, secondo
;               l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata. È uno scorrimento logico: non si tiene
;               traccia di un eventuale overflow.
;
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "ABS" oppure "ZP"
;   - .op_ (1 o 2 byte, Macro)  : Indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (3 byte, Macro)     : Riceve il risultato a 24 bit
;   - Flag Carry (1 bit, .P)    : 0 = bit uscito 0; 1 = bit uscito 1
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A (se .op_ != .res_), .P (Negative, Zero, Carry),
;                       .res_+0 .. +2
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.3.4 m_Add24

```
; ================================================================================
; MACRO: m_Add24
; --------------------------------------------------------------------------------
; Descrizione:  Somma due valori a 24 bit, secondo l'indirizzamento scelto, e
;               memorizza il risultato nella locazione desiderata.
;
;                 "IMM"  Immediato: .add1_ è un indirizzo e .add2_ è un
;                        valore a 24 bit.
;                 "ABS"  Assoluto: gli operandi sono indirizzi
;                        assoluti.
;                 "ZP"   Pagina Zero: gli operandi sono indirizzi in
;                        pagina zero.
;
;               Se i due addendi coincidono ("ABS" e "ZP") il raddoppio è
;               eseguito da m_Asl24, che non imposta V.
;
;               Se .res_ coincide con il primo operando le copie sono omesse, e
;               N e Z non riflettono il risultato.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)    : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .add1_ (1 o 2 byte, Macro) : Indirizzo del primo addendo
;   - .add2_ (3 byte, Macro)     : Secondo addendo: valore (IMM) o indirizzo
;   - .res_ (1 o 2 byte, Macro)  : Indirizzo del risultato, assoluto o in pagina
;                                  zero
;
; Output:
;   - .res_ (3 byte, Macro)      : Riceve il risultato a 24 bit
;   - Flag Carry (1 bit, .P)     : 0 = riporto assente; 1 = riporto
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, oVerflow, Zero, Carry), .res_+0 .. +2
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.3.5 m_Sub24

```
; ==============================================================================
; MACRO: m_Sub24
; ------------------------------------------------------------------------------
; Descrizione:  Sottrae il sottraendo dal minuendo, due valori a 24 bit, secondo
;               l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata.
;
;                 "IMM"  Immediato: .min_ è un indirizzo e .sub_ è un
;                        valore a 24 bit.
;                 "ABS"  Assoluto: gli operandi sono indirizzi
;                        assoluti.
;                 "ZP"   Pagina Zero: gli operandi sono indirizzi in
;                        pagina zero.
;
;               Se .res_ coincide con il minuendo e il sottraendo vale zero, la
;               copia è omessa, e N e Z non riflettono il risultato.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .min_ (1 o 2 byte, Macro) : Indirizzo del minuendo
;   - .sub_ (3 byte, Macro)     : Sottraendo: valore (IMM) o indirizzo
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (3 byte, Macro)     : Riceve il risultato a 24 bit
;   - Flag Carry (1 bit, .P)    : 0 = prestito; 1 = nessun prestito
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, oVerflow, Zero, Carry), .res_+0 .. +2
; Dipendenze Esterne:   Nessuna
; ==============================================================================
```

4.3.6 m_Neg24

```
; ==============================================================================
; MACRO: m_Neg24
; ------------------------------------------------------------------------------
; Descrizione:  Sottrae un valore a 24 bit da zero, secondo l'indirizzamento
;               scelto, e memorizza il risultato nella locazione desiderata.
;
;                 "IMM"  .op_ è un valore a 24 bit.
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Con "IMM" il risultato è calcolato dall'assemblatore e caricato
;               da m_Mov24.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .op_ (3 byte, Macro)      : Valore (IMM) o indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (3 byte, Macro)     : Riceve il risultato a 24 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, oVerflow, Zero, Carry), .res_+0 .. +2
; Dipendenze Esterne:   <mem/mem24.asm>
; ==============================================================================
```

4.3.7 m_Not24

```
; ================================================================================
; MACRO: m_Not24
; --------------------------------------------------------------------------------
; Descrizione:  Inverte tutti i bit di un valore a 24 bit, secondo
;               l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata.
;
;                 "IMM"  .op_ è un valore a 24 bit.
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Con "IMM" il risultato è calcolato dall'assemblatore e caricato
;               da m_Mov24.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .op_ (3 byte, Macro)      : Valore (IMM) o indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (3 byte, Macro)     : Riceve il risultato a 24 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), .res_+0 .. +2
; Dipendenze Esterne:   <mem/mem24.asm>
; ================================================================================
```

4.3.8 m_And24

```
; ============================================================================
; MACRO: m_And24
; ----------------------------------------------------------------------------
; Descrizione:  Esegue un AND booleano tra due valori a 24 bit, secondo
;               l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata.
;
;                 "IMM"  Immediato: .op1_ è un indirizzo e .op2_ è un
;                        valore a 24 bit.
;                 "ABS"  Assoluto: gli operandi sono indirizzi
;                        assoluti.
;                 "ZP"   Pagina Zero: gli operandi sono indirizzi in
;                        pagina zero.
;
;               Con "IMM" i byte della costante pari a $00 o $FF sono trattati a
;               parte: azzerano il byte o lo lasciano com'è.
;
;               Se .res_ coincide con il primo operando le copie inutili sono
;               omesse, e N e Z non riflettono il risultato.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .op1_ (1 o 2 byte, Macro) : Indirizzo del primo operando
;   - .op2_ (3 byte, Macro)     : Secondo operando: valore (IMM) o indirizzo
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (3 byte, Macro)     : Riceve il risultato a 24 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), .res_+0 .. +2
; Dipendenze Esterne:   <mem/mem24.asm>
; ============================================================================
```

4.3.9 m_Ora24

```
; ==============================================================================
; MACRO: m_Ora24
; ------------------------------------------------------------------------------
; Descrizione:  Esegue un OR booleano tra due valori a 24 bit, secondo
;               l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata.
;
;                 "IMM"  Immediato: .op1_ è un indirizzo e .op2_ è un
;                        valore a 24 bit.
;                 "ABS"  Assoluto: gli operandi sono indirizzi
;                        assoluti.
;                 "ZP"   Pagina Zero: gli operandi sono indirizzi in
;                        pagina zero.
;
;               Con "IMM" i byte della costante pari a $00 o $FF sono trattati a
;               parte: lasciano il byte com'è o lo portano a $FF.
;
;               Se .res_ coincide con il primo operando le copie inutili sono
;               omesse, e N e Z non riflettono il risultato.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .op1_ (1 o 2 byte, Macro) : Indirizzo del primo operando
;   - .op2_ (3 byte, Macro)     : Secondo operando: valore (IMM) o indirizzo
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (3 byte, Macro)     : Riceve il risultato a 24 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), .res_+0 .. +2
; Dipendenze Esterne:   <mem/mem24.asm>
; ==============================================================================
```

4.3.10 m_Eor24

```
; ================================================================================
; MACRO: m_Eor24
; --------------------------------------------------------------------------------
; Descrizione:  Esegue un EOR booleano tra due valori a 24 bit, secondo
;               l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata.
;
;                 "IMM"  Immediato: .op1_ è un indirizzo e .op2_ è un
;                        valore a 24 bit.
;                 "ABS"  Assoluto: gli operandi sono indirizzi
;                        assoluti.
;                 "ZP"   Pagina Zero: gli operandi sono indirizzi in
;                        pagina zero.
;
;               Con "IMM" i byte della costante pari a $00 sono trattati a
;               parte: lasciano il byte com'è.
;
;               Se .res_ coincide con il primo operando le copie inutili sono
;               omesse, e N e Z non riflettono il risultato.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .op1_ (1 o 2 byte, Macro) : Indirizzo del primo operando
;   - .op2_ (3 byte, Macro)     : Secondo operando: valore (IMM) o indirizzo
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (3 byte, Macro)     : Riceve il risultato a 24 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), .res_+0 .. +2
; Dipendenze Esterne:   <mem/mem24.asm>
; ================================================================================
```

4.3.11 m_AslS24

```
; ================================================================================
; MACRO: m_AslS24
; --------------------------------------------------------------------------------
; Descrizione:  Fa scorrere a sinistra di un bit un valore a 24 bit con segno,
;               secondo l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata. A differenza di m_Asl24 il cambiamento di
;               segno dovuto all'uscita del bit più alto è segnalato da V.
;
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "ABS" oppure "ZP"
;   - .op_ (1 o 2 byte, Macro)  : Indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (3 byte, Macro)     : Riceve il risultato a 24 bit
;   - Flag oVerflow (1 bit, .P) : 0 = segno invariato; 1 = segno cambiato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, oVerflow, Zero, Carry), .res_+0 .. +2
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.3.12 m_Lsr24

```
; ================================================================================
; MACRO: m_Lsr24
; --------------------------------------------------------------------------------
; Descrizione:  Fa scorrere a destra di un bit un valore a 24 bit, secondo
;               l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata. Il bit più alto del risultato vale zero.
;
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "ABS" oppure "ZP"
;   - .op_ (1 o 2 byte, Macro)  : Indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (3 byte, Macro)     : Riceve il risultato a 24 bit
;   - Flag Carry (1 bit, .P)    : 0 = bit uscito 0; 1 = bit uscito 1
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A (se .op_ != .res_), .P (Negative, Zero, Carry),
;                       .res_+0 .. +2
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.3.13 m_AsrS24

```
; ================================================================================
; MACRO: m_AsrS24
; --------------------------------------------------------------------------------
; Descrizione:  Fa scorrere a destra di un bit un valore a 24 bit con segno,
;               secondo l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata. Il bit di segno si conserva.
;
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "ABS" oppure "ZP"
;   - .op_ (1 o 2 byte, Macro)  : Indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (3 byte, Macro)     : Riceve il risultato a 24 bit
;   - Flag Carry (1 bit, .P)    : 0 = bit uscito 0; 1 = bit uscito 1
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero, Carry), .res_+0 .. +2
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.3.14 m_Rol24

```
; ================================================================================
; MACRO: m_Rol24
; --------------------------------------------------------------------------------
; Descrizione:  Ruota a sinistra di un bit un valore a 24 bit attraverso il
;               Carry, secondo l'indirizzamento scelto, e memorizza il risultato
;               nella locazione desiderata. Il bit più alto esce nel Carry.
;
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "ABS" oppure "ZP"
;   - .op_ (1 o 2 byte, Macro)  : Indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (3 byte, Macro)     : Riceve il risultato a 24 bit
;   - Flag Carry (1 bit, .P)    : 0 = bit uscito 0; 1 = bit uscito 1
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A (se .op_ != .res_), .P (Negative, Zero, Carry),
;                       .res_+0 .. +2
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.3.15 m_Rl24

```
; ================================================================================
; MACRO: m_Rl24
; --------------------------------------------------------------------------------
; Descrizione:  Ruota a sinistra di un bit un valore a 24 bit senza passare per
;               il Carry, secondo l'indirizzamento scelto, e memorizza il
;               risultato nella locazione desiderata. Il bit più alto finisce
;               nella posizione più bassa.
;
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "ABS" oppure "ZP"
;   - .op_ (1 o 2 byte, Macro)  : Indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (3 byte, Macro)     : Riceve il risultato a 24 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero, Carry), .res_+0 .. +2
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.3.16 m_Ror24

```
; ================================================================================
; MACRO: m_Ror24
; --------------------------------------------------------------------------------
; Descrizione:  Ruota a destra di un bit un valore a 24 bit attraverso il Carry,
;               secondo l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata. Il bit più basso esce nel Carry.
;
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "ABS" oppure "ZP"
;   - .op_ (1 o 2 byte, Macro)  : Indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (3 byte, Macro)     : Riceve il risultato a 24 bit
;   - Flag Carry (1 bit, .P)    : 0 = bit uscito 0; 1 = bit uscito 1
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A (se .op_ != .res_), .P (Negative, Zero, Carry),
;                       .res_+0 .. +2
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.3.17 m_Rr24

```
; ================================================================================
; MACRO: m_Rr24
; --------------------------------------------------------------------------------
; Descrizione:  Ruota a destra di un bit un valore a 24 bit senza passare per il
;               Carry, secondo l'indirizzamento scelto, e memorizza il risultato
;               nella locazione desiderata. Il bit più basso finisce nella
;               posizione più alta.
;
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "ABS" oppure "ZP"
;   - .op_ (1 o 2 byte, Macro)  : Indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (3 byte, Macro)     : Riceve il risultato a 24 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero, Carry), .res_+0 .. +2
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.4 math32.asm

4.4.1 Descrizione

Macro per operazioni matematiche su valori a 32 bit.

Utilità	Descrizione
`m_Asl32`	Scorrimento a sinistra di un valore a 32 bit
`m_Add32`	Somma due valori a 32 bit
`m_Sub32`	Sottrae due valori a 32 bit
`m_Neg32`	Nega un valore a 32 bit
`m_Not32`	NOT booleano di un valore a 32 bit
`m_And32`	AND booleano di due valori a 32 bit
`m_Ora32`	OR booleano di due valori a 32 bit
`m_Eor32`	EOR booleano di due valori a 32 bit
`m_AslS32`	Scorrimento a sinistra con segno, segnala V
`m_Lsr32`	Scorrimento logico a destra a 32 bit
`m_AsrS32`	Scorrimento aritmetico a destra con segno
`m_Rol32`	Rotazione a sinistra attraverso il Carry
`m_Rl32`	Rotazione a sinistra senza passare per il Carry
`m_Ror32`	Rotazione a destra attraverso il Carry
`m_Rr32`	Rotazione a destra senza passare per il Carry

4.4.2 Note d'uso

Nessuna nota in questo modulo.

4.4.3 m_Asl32

```
; ================================================================================
; MACRO: m_Asl32
; --------------------------------------------------------------------------------
; Descrizione:  Fa scorrere a sinistra di un bit un valore a 32 bit, secondo
;               l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata. È uno scorrimento logico: non si tiene
;               traccia di un eventuale overflow.
;
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "ABS" oppure "ZP"
;   - .op_ (1 o 2 byte, Macro)  : Indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (4 byte, Macro)     : Riceve il risultato a 32 bit
;   - Flag Carry (1 bit, .P)    : 0 = bit uscito 0; 1 = bit uscito 1
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A (se .op_ != .res_), .P (Negative, Zero, Carry),
;                       .res_+0 .. +3
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.4.4 m_Add32

```
; ================================================================================
; MACRO: m_Add32
; --------------------------------------------------------------------------------
; Descrizione:  Somma due valori a 32 bit, secondo l'indirizzamento scelto, e
;               memorizza il risultato nella locazione desiderata.
;
;                 "IMM"  Immediato: .add1_ è un indirizzo e .add2_ è un
;                        valore a 32 bit.
;                 "ABS"  Assoluto: gli operandi sono indirizzi
;                        assoluti.
;                 "ZP"   Pagina Zero: gli operandi sono indirizzi in
;                        pagina zero.
;
;               Se i due addendi coincidono ("ABS" e "ZP") il raddoppio è
;               eseguito da m_Asl32, che non imposta V.
;
;               Se .res_ coincide con il primo operando le copie sono omesse, e
;               N e Z non riflettono il risultato.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)    : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .add1_ (1 o 2 byte, Macro) : Indirizzo del primo addendo
;   - .add2_ (4 byte, Macro)     : Secondo addendo: valore (IMM) o indirizzo
;   - .res_ (1 o 2 byte, Macro)  : Indirizzo del risultato, assoluto o in pagina
;                                  zero
;
; Output:
;   - .res_ (4 byte, Macro)      : Riceve il risultato a 32 bit
;   - Flag Carry (1 bit, .P)     : 0 = riporto assente; 1 = riporto
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, oVerflow, Zero, Carry), .res_+0 .. +3
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.4.5 m_Sub32

```
; ================================================================================
; MACRO: m_Sub32
; --------------------------------------------------------------------------------
; Descrizione:  Sottrae il sottraendo dal minuendo, due valori a 32 bit, secondo
;               l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata.
;
;                 "IMM"  Immediato: .min_ è un indirizzo e .sub_ è un
;                        valore a 32 bit.
;                 "ABS"  Assoluto: gli operandi sono indirizzi
;                        assoluti.
;                 "ZP"   Pagina Zero: gli operandi sono indirizzi in
;                        pagina zero.
;
;               Se .res_ coincide con il minuendo e il sottraendo vale zero, la
;               copia è omessa, e N e Z non riflettono il risultato.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .min_ (1 o 2 byte, Macro) : Indirizzo del minuendo
;   - .sub_ (4 byte, Macro)     : Sottraendo: valore (IMM) o indirizzo
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (4 byte, Macro)     : Riceve il risultato a 32 bit
;   - Flag Carry (1 bit, .P)    : 0 = prestito; 1 = nessun prestito
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, oVerflow, Zero, Carry), .res_+0 .. +3
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.4.6 m_Neg32

```
; ==============================================================================
; MACRO: m_Neg32
; ------------------------------------------------------------------------------
; Descrizione:  Sottrae un valore a 32 bit da zero, secondo l'indirizzamento
;               scelto, e memorizza il risultato nella locazione desiderata.
;
;                 "IMM"  .op_ è un valore a 32 bit.
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Con "IMM" il risultato è calcolato dall'assemblatore e caricato
;               da m_Mov32.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .op_ (4 byte, Macro)      : Valore (IMM) o indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (4 byte, Macro)     : Riceve il risultato a 32 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, oVerflow, Zero, Carry), .res_+0 .. +3
; Dipendenze Esterne:   <mem/mem32.asm>
; ==============================================================================
```

4.4.7 m_Not32

```
; ===========================================================================
; MACRO: m_Not32
; ---------------------------------------------------------------------------
; Descrizione:  Inverte tutti i bit di un valore a 32 bit, secondo
;               l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata.
;
;                 "IMM"  .op_ è un valore a 32 bit.
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Con "IMM" il risultato è calcolato dall'assemblatore e caricato
;               da m_Mov32.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .op_ (4 byte, Macro)      : Valore (IMM) o indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (4 byte, Macro)     : Riceve il risultato a 32 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), .res_+0 .. +3
; Dipendenze Esterne:   <mem/mem32.asm>
; ===========================================================================
```

4.4.8 m_And32

```
; ================================================================================
; MACRO: m_And32
; --------------------------------------------------------------------------------
; Descrizione:  Esegue un AND booleano tra due valori a 32 bit, secondo
;               l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata.
;
;                 "IMM"  Immediato: .op1_ è un indirizzo e .op2_ è un
;                        valore a 32 bit.
;                 "ABS"  Assoluto: gli operandi sono indirizzi
;                        assoluti.
;                 "ZP"   Pagina Zero: gli operandi sono indirizzi in
;                        pagina zero.
;
;               Con "IMM" i byte della costante pari a $00 o $FF sono trattati a
;               parte: azzerano il byte o lo lasciano com'è.
;
;               Se .res_ coincide con il primo operando le copie inutili sono
;               omesse, e N e Z non riflettono il risultato.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .op1_ (1 o 2 byte, Macro) : Indirizzo del primo operando
;   - .op2_ (4 byte, Macro)     : Secondo operando: valore (IMM) o indirizzo
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (4 byte, Macro)     : Riceve il risultato a 32 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), .res_+0 .. +3
; Dipendenze Esterne:   <mem/mem32.asm>
; ================================================================================
```

4.4.9 m_Ora32

```
; ================================================================================
; MACRO: m_Ora32
; --------------------------------------------------------------------------------
; Descrizione:  Esegue un OR booleano tra due valori a 32 bit, secondo
;               l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata.
;
;                 "IMM"  Immediato: .op1_ è un indirizzo e .op2_ è un
;                        valore a 32 bit.
;                 "ABS"  Assoluto: gli operandi sono indirizzi
;                        assoluti.
;                 "ZP"   Pagina Zero: gli operandi sono indirizzi in
;                        pagina zero.
;
;               Con "IMM" i byte della costante pari a $00 o $FF sono trattati a
;               parte: lasciano il byte com'è o lo portano a $FF.
;
;               Se .res_ coincide con il primo operando le copie inutili sono
;               omesse, e N e Z non riflettono il risultato.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .op1_ (1 o 2 byte, Macro) : Indirizzo del primo operando
;   - .op2_ (4 byte, Macro)     : Secondo operando: valore (IMM) o indirizzo
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (4 byte, Macro)     : Riceve il risultato a 32 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), .res_+0 .. +3
; Dipendenze Esterne:   <mem/mem32.asm>
; ================================================================================
```

4.4.10 m_Eor32

```
; ================================================================================
; MACRO: m_Eor32
; --------------------------------------------------------------------------------
; Descrizione:  Esegue un EOR booleano tra due valori a 32 bit, secondo
;               l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata.
;
;                 "IMM"  Immediato: .op1_ è un indirizzo e .op2_ è un
;                        valore a 32 bit.
;                 "ABS"  Assoluto: gli operandi sono indirizzi
;                        assoluti.
;                 "ZP"   Pagina Zero: gli operandi sono indirizzi in
;                        pagina zero.
;
;               Con "IMM" i byte della costante pari a $00 sono trattati a
;               parte: lasciano il byte com'è.
;
;               Se .res_ coincide con il primo operando le copie inutili sono
;               omesse, e N e Z non riflettono il risultato.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .op1_ (1 o 2 byte, Macro) : Indirizzo del primo operando
;   - .op2_ (4 byte, Macro)     : Secondo operando: valore (IMM) o indirizzo
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (4 byte, Macro)     : Riceve il risultato a 32 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero), .res_+0 .. +3
; Dipendenze Esterne:   <mem/mem32.asm>
; ================================================================================
```

4.4.11 m_AslS32

```
; ================================================================================
; MACRO: m_AslS32
; --------------------------------------------------------------------------------
; Descrizione:  Fa scorrere a sinistra di un bit un valore a 32 bit con segno,
;               secondo l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata. A differenza di m_Asl32 il cambiamento di
;               segno dovuto all'uscita del bit più alto è segnalato da V.
;
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "ABS" oppure "ZP"
;   - .op_ (1 o 2 byte, Macro)  : Indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (4 byte, Macro)     : Riceve il risultato a 32 bit
;   - Flag oVerflow (1 bit, .P) : 0 = segno invariato; 1 = segno cambiato
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, oVerflow, Zero, Carry), .res_+0 .. +3
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.4.12 m_Lsr32

```
; ================================================================================
; MACRO: m_Lsr32
; --------------------------------------------------------------------------------
; Descrizione:  Fa scorrere a destra di un bit un valore a 32 bit, secondo
;               l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata. Il bit più alto del risultato vale zero.
;
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "ABS" oppure "ZP"
;   - .op_ (1 o 2 byte, Macro)  : Indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (4 byte, Macro)     : Riceve il risultato a 32 bit
;   - Flag Carry (1 bit, .P)    : 0 = bit uscito 0; 1 = bit uscito 1
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A (se .op_ != .res_), .P (Negative, Zero, Carry),
;                       .res_+0 .. +3
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.4.13 m_AsrS32

```
; ================================================================================
; MACRO: m_AsrS32
; --------------------------------------------------------------------------------
; Descrizione:  Fa scorrere a destra di un bit un valore a 32 bit con segno,
;               secondo l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata. Il bit di segno si conserva.
;
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "ABS" oppure "ZP"
;   - .op_ (1 o 2 byte, Macro)  : Indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (4 byte, Macro)     : Riceve il risultato a 32 bit
;   - Flag Carry (1 bit, .P)    : 0 = bit uscito 0; 1 = bit uscito 1
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero, Carry), .res_+0 .. +3
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.4.14 m_Rol32

```
; ================================================================================
; MACRO: m_Rol32
; --------------------------------------------------------------------------------
; Descrizione:  Ruota a sinistra di un bit un valore a 32 bit attraverso il
;               Carry, secondo l'indirizzamento scelto, e memorizza il risultato
;               nella locazione desiderata. Il bit più alto esce nel Carry.
;
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "ABS" oppure "ZP"
;   - .op_ (1 o 2 byte, Macro)  : Indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (4 byte, Macro)     : Riceve il risultato a 32 bit
;   - Flag Carry (1 bit, .P)    : 0 = bit uscito 0; 1 = bit uscito 1
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A (se .op_ != .res_), .P (Negative, Zero, Carry),
;                       .res_+0 .. +3
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.4.15 m_Rl32

```
; ================================================================================
; MACRO: m_Rl32
; --------------------------------------------------------------------------------
; Descrizione:  Ruota a sinistra di un bit un valore a 32 bit senza passare per
;               il Carry, secondo l'indirizzamento scelto, e memorizza il
;               risultato nella locazione desiderata. Il bit più alto finisce
;               nella posizione più bassa.
;
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "ABS" oppure "ZP"
;   - .op_ (1 o 2 byte, Macro)  : Indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (4 byte, Macro)     : Riceve il risultato a 32 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero, Carry), .res_+0 .. +3
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.4.16 m_Ror32

```
; ================================================================================
; MACRO: m_Ror32
; --------------------------------------------------------------------------------
; Descrizione:  Ruota a destra di un bit un valore a 32 bit attraverso il Carry,
;               secondo l'indirizzamento scelto, e memorizza il risultato nella
;               locazione desiderata. Il bit più basso esce nel Carry.
;
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "ABS" oppure "ZP"
;   - .op_ (1 o 2 byte, Macro)  : Indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (4 byte, Macro)     : Riceve il risultato a 32 bit
;   - Flag Carry (1 bit, .P)    : 0 = bit uscito 0; 1 = bit uscito 1
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A (se .op_ != .res_), .P (Negative, Zero, Carry),
;                       .res_+0 .. +3
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.4.17 m_Rr32

```
; ================================================================================
; MACRO: m_Rr32
; --------------------------------------------------------------------------------
; Descrizione:  Ruota a destra di un bit un valore a 32 bit senza passare per il
;               Carry, secondo l'indirizzamento scelto, e memorizza il risultato
;               nella locazione desiderata. Il bit più basso finisce nella
;               posizione più alta.
;
;                 "ABS"  .op_ è un indirizzo assoluto.
;                 "ZP"   .op_ è un indirizzo in pagina zero.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "ABS" oppure "ZP"
;   - .op_ (1 o 2 byte, Macro)  : Indirizzo dell'operando
;   - .res_ (1 o 2 byte, Macro) : Indirizzo del risultato, assoluto o in pagina
;                                 zero
;
; Output:
;   - .res_ (4 byte, Macro)     : Riceve il risultato a 32 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero, Carry), .res_+0 .. +3
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.5 math_div.asm

4.5.1 Descrizione

Divisione intera senza segno e con segno per tutte le combinazioni di operandi da 8 a 32 bit. Il quoziente ha la lunghezza del dividendo e il resto quella del divisore, senza perdita di precisione.

Utilità	Descrizione
`m_Div_Negate`	Nega un valore a più byte in pagina zero
`m_Div_Roll`	Fa scorrere a sinistra un valore a più byte
`m_Div_Core_Byte`	Nucleo senza segno con divisore a 8 bit
`m_Div_Core_Multi`	Nucleo senza segno con divisore a più byte
`m_Div_Core_8x8`	Nucleo senza segno 8x8 bit
`m_Div_Core_16x8`	Nucleo senza segno 16x8 bit
`m_Div_Core_24x8`	Nucleo senza segno 24x8 bit
`m_Div_Core_32x8`	Nucleo senza segno 32x8 bit
`m_Div_Core_16x16`	Nucleo senza segno 16x16 bit
`m_Div_Core_24x16`	Nucleo senza segno 24x16 bit
`m_Div_Core_32x16`	Nucleo senza segno 32x16 bit
`m_Div_Core_24x24`	Nucleo senza segno 24x24 bit
`m_Div_Core_32x24`	Nucleo senza segno 32x24 bit
`m_Div_Core_32x32`	Nucleo senza segno 32x32 bit
`DIV_8X8`	Divisione 8x8 bit senza segno
`SDIV_8X8`	Divisione 8x8 bit con segno
`DIV_16X8`	Divisione 16x8 bit senza segno
`SDIV_16X8`	Divisione 16x8 bit con segno
`DIV_24X8`	Divisione 24x8 bit senza segno
`SDIV_24X8`	Divisione 24x8 bit con segno
`DIV_32X8`	Divisione 32x8 bit senza segno
`SDIV_32X8`	Divisione 32x8 bit con segno
`DIV_16X16`	Divisione 16x16 bit senza segno
`SDIV_16X16`	Divisione 16x16 bit con segno
`DIV_24X16`	Divisione 24x16 bit senza segno
`SDIV_24X16`	Divisione 24x16 bit con segno
`DIV_32X16`	Divisione 32x16 bit senza segno
`SDIV_32X16`	Divisione 32x16 bit con segno
`DIV_24X24`	Divisione 24x24 bit senza segno
`SDIV_24X24`	Divisione 24x24 bit con segno
`DIV_32X24`	Divisione 32x24 bit senza segno

Utilità	Descrizione
SDIV_32X24	Divisione 32x24 bit con segno
DIV_32X32	Divisione 32x32 bit senza segno
SDIV_32X32	Divisione 32x32 bit con segno

4.5.2 Note d'uso

```
; NOTA #01
; --------
; Il dividendo ha m byte e il divisore n byte, con m maggiore o uguale a n.
; Il quoziente ha la lunghezza del dividendo e il resto quella del divisore,
; quindi nessuna informazione va persa. I valori a più byte si scambiano con i
; registri dal byte più basso al più alto: byte 0 in .A, byte 1 in .X e byte 2
; in .Y (cioè la terna .YXA, come nell'ordine di memoria). I byte del
; quoziente oltre il terzo si leggono da DIV_QUOT+3.
;
; Il resto si legge sempre da DIV_REM. Quando il resto ha un solo byte ed è
; rimasto un registro libero, la routine lo restituisce anche in quel registro.
;
;   Routine  Dividendo   Divisore    Quoziente        Resto
;   -------  ----------  ----------  ---------------  ------------
;   8x8      .A          .X          .A               .X e DIV_REM
;   16x8     DIV_1 (ZP)  .A          .A, .X           .Y e DIV_REM
;   24x8     DIV_1 (ZP)  .A          .A, .X, .Y       DIV_REM
;   32x8     DIV_1 (ZP)  .A          .A, .X, .Y (*)   DIV_REM
;   16x16    DIV_1 (ZP)  .A, .X      .A, .X           DIV_REM
;   24x16    DIV_1 (ZP)  .A, .X      .A, .X, .Y       DIV_REM
;   32x16    DIV_1 (ZP)  .A, .X      .A, .X, .Y (*)   DIV_REM
;   24x24    DIV_1 (ZP)  .A, .X, .Y  .A, .X, .Y       DIV_REM
;   32x24    DIV_1 (ZP)  .A, .X, .Y  .A, .X, .Y (*)   DIV_REM
;   32x32    DIV_1 (ZP)  DIV_2 (ZP)  .A, .X, .Y (*)   DIV_REM
;
;   (*) Il byte 3 del quoziente si legge da DIV_QUOT+3.
;
; Per le versioni con segno gli operandi sono in complemento a 2. Il resto ha il
; segno del dividendo e il quoziente si tronca verso zero, come nella divisione
; intera dei linguaggi di alto livello.
```

```
; NOTA #02
; --------
; Le routine lavorano in pagina zero sui 12 byte da NUMWRK a NUMWRK+11, cioè
; NUMWRK più i primi due byte di FAC, che sono liberi finché non si usa la
; matematica in virgola mobile del BASIC.
;
;   DIV_1     NUMWRK+0 .. NUMWRK+3     Dividendo (primo operando)
;   DIV_QUOT  NUMWRK+0 .. NUMWRK+3     Quoziente, coincide con DIV_1
;   DIV_2     NUMWRK+4 .. NUMWRK+7     Divisore (secondo operando)
;   DIV_REM   NUMWRK+8 .. NUMWRK+11    Resto
;
; DIV_QUOT coincide con DIV_1: il dividendo viene consumato bit dopo bit e al
; suo posto entrano i bit del quoziente. Per questo il dividendo non si
; conserva. Nelle versioni con segno anche il divisore diventa positivo.
;
; La stessa area è condivisa con ZPL_1 e ZPL_2 (puntatori utente a 24/32 bit)
; e con le routine di moltiplicazione: non si possono usare contemporaneamente.
```

```
; NOTA #03
; --------
; Si elabora un bit del dividendo alla volta, dal più alto. Il resto parziale
; scorre a sinistra e riceve il bit; se è maggiore o uguale al divisore si
; sottrae e il bit di quoziente vale 1, altrimenti vale 0 e il resto non si
; tocca: non esiste nessun ripristino. Il bit di quoziente è il Carry lasciato
; dal confronto ed entra dal basso nel dividendo alla rotazione successiva.
; Dividendo e quoziente condividono gli stessi byte e il quoziente non richiede
; nessun passo a parte.
;
; Con un divisore di un byte il resto sta in .A per tutto il tempo. Con più
; byte sta in DIV_REM e il confronto parte dal byte più alto: si ferma appena
; l'esito è noto, cioè di solito dopo un solo byte.
;
; I cicli medi con operandi casuali comprendono il caricamento dei registri
; e l'rts, ma non la jsr. Il divisore è casuale e non nullo, in parte più
; corto dell'operando.
;
;   Routine  Senza segno  Con segno
;   8x8            200       267
;   16x8           451       528
;   24x8           783       869
;   32x8          1195      1289
;   16x16          686       768
;   24x16         1188      1279
;   32x16         1775      1876
;   24x24         1335      1431
;   32x24         1991      2095
;   32x32         2152      2244
```

```
; NOTA #04
; --------
; Divisione per zero: la routine esce con il Carry a 1 e .A uguale a
; ERR_DIVISION_BY_ZERO, senza toccare il quoziente e il resto.
;
; Un dividendo nullo non ha un percorso a parte: il ciclo lo elabora come ogni
; altro valore e dà quoziente e resto uguali a zero.
;
; Overflow nelle versioni con segno: l'unico caso è il dividendo più negativo
; diviso per -1, per esempio -128 / -1 a 8 bit. Il quoziente corretto (+128)
; non entra in 8 bit con il segno: la routine restituisce il quoziente
; troncato ai bit bassi ($80, che letto con il segno vale -128), pone il flag
; di oVerflow a 1 e lascia il Carry a 0. Il bit perso è il bit di segno del
; quoziente vero, che qui vale sempre 0: chi vuole il risultato esatto lo
; ricostruisce con un byte in più a zero. In tutti gli altri casi il flag di
; oVerflow vale 0.
```

4.5.3 m_Div_Negate

```
; ================================================================================
; MACRO: m_Div_Negate
; --------------------------------------------------------------------------------
; Descrizione:  Nega sul posto un valore a più byte in pagina zero, cioè lo
;               sostituisce con il suo complemento a 2. Serve per rendere
;               positivi dividendo e divisore e per correggere il segno di
;               quoziente e resto.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .address_ (1 byte, Macro)   : Indirizzo in pagina zero del byte meno
;                                   significativo
;   - .bytes_ (1 byte, Macro)     : Numero di byte del valore (da 1 a 4)
;
; Output:
;   - .address_ (1 .. 4 byte, ZP) : Il valore negato, nelle stesse locazioni
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, oVerflow, Zero, Carry),
;                       .address_+0 .. .address_+.bytes_-1
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.5.4 m_Div_Roll

```
; ================================================================================
; MACRO: m_Div_Roll
; --------------------------------------------------------------------------------
; Descrizione:  Fa scorrere a sinistra di un bit un valore a più byte in pagina
;               zero. Il Carry entra nel bit meno significativo e il bit più
;               alto del valore esce nel Carry.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .address_ (1 byte, Macro)   : Indirizzo in pagina zero del byte meno
;                                   significativo
;   - .bytes_ (1 byte, Macro)     : Numero di byte del valore (da 1 a 4)
;   - Flag Carry (1 bit, .P)      : 0 = entra uno 0; 1 = entra un 1
;
; Output:
;   - .address_ (1 .. 4 byte, ZP) : Il valore scorso di un bit
;   - Flag Carry (1 bit, .P)      : 0 = esce uno 0; 1 = esce un 1
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .P (Negative, oVerflow, Zero, Carry),
;                       .address_+0 .. .address_+.bytes_-1
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.5.5 m_Div_Core_Byte

```
; ================================================================================
; MACRO: m_Div_Core_Byte
; --------------------------------------------------------------------------------
; Descrizione:  Divisione senza segno con un divisore a 8 bit e un dividendo da
;               1 a 4 byte. Per ogni bit del dividendo, dal più alto, il resto
;               parziale scorre a sinistra e riceve il bit; se è maggiore o
;               uguale al divisore si sottrae e il bit di quoziente vale 1. Quel
;               bit entra nel dividendo alla rotazione seguente, quindi il
;               quoziente prende il posto del dividendo.
;
;               Il resto parziale sta in .A per tutta l'elaborazione e si scrive
;               in memoria una sola volta, alla fine. Il divisore deve essere
;               diverso da zero: la macro non lo controlla.
;
; Input:
;   - .dvd_bytes_ (1 byte, Macro) : Byte del dividendo (da 1 a 4)
;   - DIV_1 (1 .. 4 byte, ZP)     : Dividendo (senza segno)
;   - DIV_2+0 (1 byte, ZP)        : Divisore (senza segno, diverso da zero)
;
; Output:
;   - DIV_QUOT (1 .. 4 byte, ZP)  : Quoziente
;   - DIV_REM+0 (1 byte, ZP)      : Resto
;   - .A (1 byte, CPU)            : Resto (uguale a DIV_REM+0)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0 .. DIV_QUOT+.dvd_bytes_-1, DIV_REM+0
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.5.6 m_Div_Core_Multi

```
; ================================================================================
; MACRO: m_Div_Core_Multi
; --------------------------------------------------------------------------------
; Descrizione:  Divisione senza segno con un divisore da 2 a 4 byte e un
;               dividendo da 1 a 4 byte. Per ogni bit del dividendo, dal più
;               alto, il resto parziale scorre a sinistra e riceve il bit; se è
;               maggiore o uguale al divisore si sottrae e il bit di quoziente
;               vale 1. Quel bit entra nel dividendo alla rotazione seguente,
;               quindi il quoziente prende il posto del dividendo.
;
;               Il confronto parte dal byte più alto e si ferma appena l'esito è
;               noto. Se il resto parziale esce dai suoi byte si sottrae senza
;               confrontare. Il divisore deve essere diverso da zero: la macro
;               non lo controlla.
;
; Input:
;   - .dvd_bytes_ (1 byte, Macro) : Byte del dividendo (da 1 a 4)
;   - .dvs_bytes_ (1 byte, Macro) : Byte del divisore (da 2 a 4)
;   - DIV_1 (1 .. 4 byte, ZP)     : Dividendo (senza segno)
;   - DIV_2 (2 .. 4 byte, ZP)     : Divisore (senza segno, diverso da zero)
;
; Output:
;   - DIV_QUOT (1 .. 4 byte, ZP)  : Quoziente
;   - DIV_REM (2 .. 4 byte, ZP)   : Resto
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0 .. DIV_QUOT+.dvd_bytes_-1,
;                       DIV_REM+0 .. DIV_REM+.dvs_bytes_-1
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.5.7 m_Div_Core_8x8

```
; ================================================================================
; MACRO: m_Div_Core_8x8
; --------------------------------------------------------------------------------
; Descrizione:  Divisione senza segno tra due valori a 8 bit con il nucleo
;               generico m_Div_Core_Multi.
;
;               Il divisore deve essere diverso da zero: la macro non lo
;               controlla.
;
; Input:
;   - DIV_1+0 (1 byte, ZP)    : Dividendo (senza segno)
;   - DIV_2+0 (1 byte, ZP)    : Divisore (senza segno, diverso da zero)
;
; Output:
;   - .A (1 byte, CPU)        : Resto (uguale a DIV_REM+0)
;   - DIV_QUOT+0 (1 byte, ZP) : Quoziente a 8 bit
;   - DIV_REM+0 (1 byte, ZP)  : Resto a 8 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0, DIV_REM+0
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.5.8 m_Div_Core_16x8

```
; ================================================================================
; MACRO: m_Div_Core_16x8
; --------------------------------------------------------------------------------
; Descrizione:  Divisione senza segno tra un valore a 16 bit (dividendo) e uno a
;               8 bit (divisore) con il nucleo generico m_Div_Core_Byte.
;
;               Il divisore deve essere diverso da zero: la macro non lo
;               controlla.
;
; Input:
;   - DIV_1+0 .. DIV_1+1 (2 byte, ZP)       : Dividendo (senza segno)
;   - DIV_2+0 (1 byte, ZP)                  : Divisore (senza segno, diverso da
;                                             zero)
;
; Output:
;   - .A (1 byte, CPU)                      : Resto (uguale a DIV_REM+0)
;   - DIV_QUOT+0 .. DIV_QUOT+1 (2 byte, ZP) : Quoziente a 16 bit
;   - DIV_REM+0 (1 byte, ZP)                : Resto a 8 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0 .. DIV_QUOT+1, DIV_REM+0
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.5.9 m_Div_Core_24x8

```
; ============================================================================
; MACRO: m_Div_Core_24x8
; ----------------------------------------------------------------------------
; Descrizione:  Divisione senza segno tra un valore a 24 bit (dividendo) e uno a
;               8 bit (divisore) con il nucleo generico m_Div_Core_Byte.
;
;               Il divisore deve essere diverso da zero: la macro non lo
;               controlla.
;
; Input:
;   - DIV_1+0 .. DIV_1+2 (3 byte, ZP)       : Dividendo (senza segno)
;   - DIV_2+0 (1 byte, ZP)                  : Divisore (senza segno, diverso da
;                                             zero)
;
; Output:
;   - .A (1 byte, CPU)                      : Resto (uguale a DIV_REM+0)
;   - DIV_QUOT+0 .. DIV_QUOT+2 (3 byte, ZP) : Quoziente a 24 bit
;   - DIV_REM+0 (1 byte, ZP)                : Resto a 8 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0 .. DIV_QUOT+2, DIV_REM+0
; Dipendenze Esterne:   <c64/symbols.asm>
; ============================================================================
```

4.5.10 m_Div_Core_32x8

```
; ============================================================================
; MACRO: m_Div_Core_32x8
; ----------------------------------------------------------------------------
; Descrizione:  Divisione senza segno tra un valore a 32 bit (dividendo) e uno a
;               8 bit (divisore) con il nucleo generico m_Div_Core_Byte.
;
;               Il divisore deve essere diverso da zero: la macro non lo
;               controlla.
;
; Input:
;   - DIV_1+0 .. DIV_1+3 (4 byte, ZP)       : Dividendo (senza segno)
;   - DIV_2+0 (1 byte, ZP)                  : Divisore (senza segno, diverso da
;                                             zero)
;
; Output:
;   - .A (1 byte, CPU)                      : Resto (uguale a DIV_REM+0)
;   - DIV_QUOT+0 .. DIV_QUOT+3 (4 byte, ZP) : Quoziente a 32 bit
;   - DIV_REM+0 (1 byte, ZP)                : Resto a 8 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0 .. DIV_QUOT+3, DIV_REM+0
; Dipendenze Esterne:   <c64/symbols.asm>
; ============================================================================
```

4.5.11 m_Div_Core_16x16

```
; ================================================================================
; MACRO: m_Div_Core_16x16
; --------------------------------------------------------------------------------
; Descrizione:  Divisione senza segno tra due valori a 16 bit con il nucleo
;               generico m_Div_Core_Multi.
;
;               Il divisore deve essere diverso da zero: la macro non lo
;               controlla.
;
; Input:
;   - DIV_1+0 .. DIV_1+1 (2 byte, ZP)       : Dividendo (senza segno)
;   - DIV_2+0 .. DIV_2+1 (2 byte, ZP)       : Divisore (senza segno, diverso da
;                                             zero)
;
; Output:
;   - DIV_QUOT+0 .. DIV_QUOT+1 (2 byte, ZP) : Quoziente a 16 bit
;   - DIV_REM+0 .. DIV_REM+1 (2 byte, ZP)   : Resto a 16 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0 .. DIV_QUOT+1, DIV_REM+0 .. DIV_REM+1
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.5.12 m_Div_Core_24x16

```
; ================================================================================
; MACRO: m_Div_Core_24x16
; --------------------------------------------------------------------------------
; Descrizione:  Divisione senza segno tra un valore a 24 bit (dividendo) e uno a
;               16 bit (divisore) con il nucleo generico m_Div_Core_Multi.
;
;               Il divisore deve essere diverso da zero: la macro non lo
;               controlla.
;
; Input:
;   - DIV_1+0 .. DIV_1+2 (3 byte, ZP)       : Dividendo (senza segno)
;   - DIV_2+0 .. DIV_2+1 (2 byte, ZP)       : Divisore (senza segno, diverso da
;                                             zero)
;
; Output:
;   - DIV_QUOT+0 .. DIV_QUOT+2 (3 byte, ZP) : Quoziente a 24 bit
;   - DIV_REM+0 .. DIV_REM+1 (2 byte, ZP)   : Resto a 16 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0 .. DIV_QUOT+2, DIV_REM+0 .. DIV_REM+1
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.5.13 m_Div_Core_32x16

```
; ================================================================================
; MACRO: m_Div_Core_32x16
; --------------------------------------------------------------------------------
; Descrizione:  Divisione senza segno tra un valore a 32 bit (dividendo) e uno a
;               16 bit (divisore) con il nucleo generico m_Div_Core_Multi.
;
;               Il divisore deve essere diverso da zero: la macro non lo
;               controlla.
;
; Input:
;   - DIV_1+0 .. DIV_1+3 (4 byte, ZP)       : Dividendo (senza segno)
;   - DIV_2+0 .. DIV_2+1 (2 byte, ZP)       : Divisore (senza segno, diverso da
;                                             zero)
;
; Output:
;   - DIV_QUOT+0 .. DIV_QUOT+3 (4 byte, ZP) : Quoziente a 32 bit
;   - DIV_REM+0 .. DIV_REM+1 (2 byte, ZP)   : Resto a 16 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0 .. DIV_QUOT+3, DIV_REM+0 .. DIV_REM+1
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.5.14 m_Div_Core_24x24

```
; ================================================================================
; MACRO: m_Div_Core_24x24
; --------------------------------------------------------------------------------
; Descrizione:  Divisione senza segno tra due valori a 24 bit con il nucleo
;               generico m_Div_Core_Multi.
;
;               Il divisore deve essere diverso da zero: la macro non lo
;               controlla.
;
; Input:
;   - DIV_1+0 .. DIV_1+2 (3 byte, ZP)       : Dividendo (senza segno)
;   - DIV_2+0 .. DIV_2+2 (3 byte, ZP)       : Divisore (senza segno, diverso da
;                                             zero)
;
; Output:
;   - DIV_QUOT+0 .. DIV_QUOT+2 (3 byte, ZP) : Quoziente a 24 bit
;   - DIV_REM+0 .. DIV_REM+2 (3 byte, ZP)   : Resto a 24 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0 .. DIV_QUOT+2, DIV_REM+0 .. DIV_REM+2
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.5.15 m_Div_Core_32x24

```
; ============================================================================
; MACRO: m_Div_Core_32x24
; ----------------------------------------------------------------------------
; Descrizione:  Divisione senza segno tra un valore a 32 bit (dividendo) e uno a
;               24 bit (divisore) con il nucleo generico m_Div_Core_Multi.
;
;               Il divisore deve essere diverso da zero: la macro non lo
;               controlla.
;
; Input:
;   - DIV_1+0 .. DIV_1+3 (4 byte, ZP)       : Dividendo (senza segno)
;   - DIV_2+0 .. DIV_2+2 (3 byte, ZP)       : Divisore (senza segno, diverso da
;                                             zero)
;
; Output:
;   - DIV_QUOT+0 .. DIV_QUOT+3 (4 byte, ZP) : Quoziente a 32 bit
;   - DIV_REM+0 .. DIV_REM+2 (3 byte, ZP)   : Resto a 24 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0 .. DIV_QUOT+3, DIV_REM+0 .. DIV_REM+2
; Dipendenze Esterne:   <c64/symbols.asm>
; ============================================================================
```

4.5.16 m_Div_Core_32x32

```
; ================================================================================
; MACRO: m_Div_Core_32x32
; --------------------------------------------------------------------------------
; Descrizione:  Divisione senza segno tra due valori a 32 bit con il nucleo
;               generico m_Div_Core_Multi.
;
;               Il divisore deve essere diverso da zero: la macro non lo
;               controlla.
;
; Input:
;   - DIV_1+0 .. DIV_1+3 (4 byte, ZP)       : Dividendo (senza segno)
;   - DIV_2+0 .. DIV_2+3 (4 byte, ZP)       : Divisore (senza segno, diverso da
;                                             zero)
;
; Output:
;   - DIV_QUOT+0 .. DIV_QUOT+3 (4 byte, ZP) : Quoziente a 32 bit
;   - DIV_REM+0 .. DIV_REM+3 (4 byte, ZP)   : Resto a 32 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0 .. DIV_QUOT+3, DIV_REM+0 .. DIV_REM+3
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.5.17 DIV_8X8

```
; ============================================================================
; SUBROUTINE: DIV_8X8
; ----------------------------------------------------------------------------
; Descrizione:  Divide due interi a 8 bit, senza segno. Il quoziente ha 8 bit e
;               il resto 8 bit.
;
;               Se il divisore è zero esce con il Carry a 1 e
;               ERR_DIVISION_BY_ZERO in .A, senza toccare quoziente e resto.
;
; Input:
;   - .A (1 byte, CPU)        : Dividendo
;   - .X (1 byte, CPU)        : Divisore
;
; Output:
;   - .A (1 byte, CPU)        : Byte 0 del quoziente, o ERR_DIVISION_BY_ZERO se
;                               C = 1
;   - .X (1 byte, CPU)        : Resto (uguale a DIV_REM+0)
;   - DIV_QUOT+0 (1 byte, ZP) : Quoziente a 8 bit
;   - DIV_REM+0 (1 byte, ZP)  : Resto a 8 bit
;   - Flag Carry (1 bit, .P)  : 0 = eseguita; 1 = divisore nullo
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0, DIV_2+0, DIV_REM+0
; Dipendenze Esterne:   <c64/errors.asm>
;                       <c64/symbols.asm>
; ============================================================================
```

4.5.18 SDIV_8X8

```
; ============================================================================
; SUBROUTINE: SDIV_8X8
; ----------------------------------------------------------------------------
; Descrizione:  Divide due interi a 8 bit, con segno. Il quoziente ha 8 bit e il
;               resto 8 bit.
;
;               Rende positivi dividendo e divisore, esegue il nucleo senza
;               segno e corregge i segni: il quoziente è negativo se i segni
;               degli operandi sono diversi e il resto ha il segno del
;               dividendo.
;
;               L'unico overflow possibile è -2^7 diviso -1: il quoziente esce
;               troncato ai 8 bit bassi e il flag di oVerflow vale 1.
;
;               Se il divisore è zero esce con il Carry a 1 e
;               ERR_DIVISION_BY_ZERO in .A, senza toccare quoziente e resto.
;
; Input:
;   - .A (1 byte, CPU)          : Dividendo con segno
;   - .X (1 byte, CPU)          : Divisore con segno
;
; Output:
;   - .A (1 byte, CPU)          : Byte 0 del quoziente, o ERR_DIVISION_BY_ZERO
;                                 se C = 1
;   - .X (1 byte, CPU)          : Resto (uguale a DIV_REM+0)
;   - DIV_QUOT+0 (1 byte, ZP)   : Quoziente a 8 bit con segno
;   - DIV_REM+0 (1 byte, ZP)    : Resto a 8 bit con segno
;   - Flag oVerflow (1 bit, .P) : 0 = nessuno; 1 = overflow
;   - Flag Carry (1 bit, .P)    : 0 = eseguita; 1 = divisore nullo
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0, DIV_2+0, DIV_REM+0
; Dipendenze Esterne:   <c64/errors.asm>
;                       <c64/symbols.asm>
; ============================================================================
```

4.5.19 DIV_16X8

```
; ============================================================================
; SUBROUTINE: DIV_16X8
; ----------------------------------------------------------------------------
; Descrizione:  Divide un intero a 16 bit per uno a 8 bit, senza segno. Il
;               quoziente ha 16 bit e il resto 8 bit.
;
;               Se il divisore è zero esce con il Carry a 1 e
;               ERR_DIVISION_BY_ZERO in .A, senza toccare quoziente e resto.
;
; Input:
;   - DIV_1+0 .. DIV_1+1 (2 byte, ZP)       : Dividendo
;   - .A (1 byte, CPU)                      : Divisore
;
; Output:
;   - .A (1 byte, CPU)                      : Byte 0 del quoziente, o
;                                             ERR_DIVISION_BY_ZERO se C = 1
;   - .X (1 byte, CPU)                      : Byte 1 del quoziente
;   - .Y (1 byte, CPU)                      : Resto (uguale a DIV_REM+0)
;   - DIV_QUOT+0 .. DIV_QUOT+1 (2 byte, ZP) : Quoziente a 16 bit
;   - DIV_REM+0 (1 byte, ZP)                : Resto a 8 bit
;   - Flag Carry (1 bit, .P)                : 0 = eseguita; 1 = divisore nullo
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0 .. DIV_QUOT+1, DIV_2+0, DIV_REM+0
; Dipendenze Esterne:   <c64/errors.asm>
;                       <c64/symbols.asm>
; ============================================================================
```

4.5.20 SDIV_16X8

```
; ================================================================================
; SUBROUTINE: SDIV_16X8
; --------------------------------------------------------------------------------
; Descrizione:  Divide un intero a 16 bit per uno a 8 bit, con segno. Il
;               quoziente ha 16 bit e il resto 8 bit.
;
;               Rende positivi dividendo e divisore, esegue il nucleo senza
;               segno e corregge i segni: il quoziente è negativo se i segni
;               degli operandi sono diversi e il resto ha il segno del
;               dividendo.
;
;               L'unico overflow possibile è -2^15 diviso -1: il quoziente esce
;               troncato ai 16 bit bassi e il flag di oVerflow vale 1.
;
;               Se il divisore è zero esce con il Carry a 1 e
;               ERR_DIVISION_BY_ZERO in .A, senza toccare quoziente e resto.
;
; Input:
;   - DIV_1+0 .. DIV_1+1 (2 byte, ZP)       : Dividendo con segno
;   - .A (1 byte, CPU)                      : Divisore con segno
;
; Output:
;   - .A (1 byte, CPU)                      : Byte 0 del quoziente, o
;                                             ERR_DIVISION_BY_ZERO se C = 1
;   - .X (1 byte, CPU)                      : Byte 1 del quoziente
;   - .Y (1 byte, CPU)                      : Resto (uguale a DIV_REM+0)
;   - DIV_QUOT+0 .. DIV_QUOT+1 (2 byte, ZP) : Quoziente a 16 bit con segno
;   - DIV_REM+0 (1 byte, ZP)                : Resto a 8 bit con segno
;   - Flag oVerflow (1 bit, .P)             : 0 = nessuno; 1 = overflow
;   - Flag Carry (1 bit, .P)                : 0 = eseguita; 1 = divisore nullo
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0 .. DIV_QUOT+1, DIV_2+0, DIV_REM+0
; Dipendenze Esterne:   <c64/errors.asm>
;                       <c64/symbols.asm>
; ================================================================================
```

4.5.21 DIV_24X8

```
; ================================================================================
; SUBROUTINE: DIV_24X8
; --------------------------------------------------------------------------------
; Descrizione:  Divide un intero a 24 bit per uno a 8 bit, senza segno. Il
;               quoziente ha 24 bit e il resto 8 bit.
;
;               Se il divisore è zero esce con il Carry a 1 e
;               ERR_DIVISION_BY_ZERO in .A, senza toccare quoziente e resto.
;
; Input:
;   - DIV_1+0 .. DIV_1+2 (3 byte, ZP)       : Dividendo
;   - .A (1 byte, CPU)                      : Divisore
;
; Output:
;   - .A (1 byte, CPU)                      : Byte 0 del quoziente, o
;                                             ERR_DIVISION_BY_ZERO se C = 1
;   - .X (1 byte, CPU)                      : Byte 1 del quoziente
;   - .Y (1 byte, CPU)                      : Byte 2 del quoziente
;   - DIV_QUOT+0 .. DIV_QUOT+2 (3 byte, ZP) : Quoziente a 24 bit
;   - DIV_REM+0 (1 byte, ZP)                : Resto a 8 bit
;   - Flag Carry (1 bit, .P)                : 0 = eseguita; 1 = divisore nullo
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0 .. DIV_QUOT+2, DIV_2+0, DIV_REM+0
; Dipendenze Esterne:   <c64/errors.asm>
;                       <c64/symbols.asm>
; ================================================================================
```

4.5.22 SDIV_24X8

```
; ============================================================================
; SUBROUTINE: SDIV_24X8
; ----------------------------------------------------------------------------
; Descrizione:  Divide un intero a 24 bit per uno a 8 bit, con segno. Il
;               quoziente ha 24 bit e il resto 8 bit.
;
;               Rende positivi dividendo e divisore, esegue il nucleo senza
;               segno e corregge i segni: il quoziente è negativo se i segni
;               degli operandi sono diversi e il resto ha il segno del
;               dividendo.
;
;               L'unico overflow possibile è -2^23 diviso -1: il quoziente esce
;               troncato ai 24 bit bassi e il flag di oVerflow vale 1.
;
;               Se il divisore è zero esce con il Carry a 1 e
;               ERR_DIVISION_BY_ZERO in .A, senza toccare quoziente e resto.
;
; Input:
;   - DIV_1+0 .. DIV_1+2 (3 byte, ZP)       : Dividendo con segno
;   - .A (1 byte, CPU)                      : Divisore con segno
;
; Output:
;   - .A (1 byte, CPU)                      : Byte 0 del quoziente, o
;                                             ERR_DIVISION_BY_ZERO se C = 1
;   - .X (1 byte, CPU)                      : Byte 1 del quoziente
;   - .Y (1 byte, CPU)                      : Byte 2 del quoziente
;   - DIV_QUOT+0 .. DIV_QUOT+2 (3 byte, ZP) : Quoziente a 24 bit con segno
;   - DIV_REM+0 (1 byte, ZP)                : Resto a 8 bit con segno
;   - Flag oVerflow (1 bit, .P)             : 0 = nessuno; 1 = overflow
;   - Flag Carry (1 bit, .P)                : 0 = eseguita; 1 = divisore nullo
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0 .. DIV_QUOT+2, DIV_2+0, DIV_REM+0
; Dipendenze Esterne:   <c64/errors.asm>
;                       <c64/symbols.asm>
; ============================================================================
```

4.5.23 DIV_32X8

```
; ================================================================================
; SUBROUTINE: DIV_32X8
; --------------------------------------------------------------------------------
; Descrizione:  Divide un intero a 32 bit per uno a 8 bit, senza segno. Il
;               quoziente ha 32 bit e il resto 8 bit.
;
;               Se il divisore è zero esce con il Carry a 1 e
;               ERR_DIVISION_BY_ZERO in .A, senza toccare quoziente e resto.
;
; Input:
;   - DIV_1+0 .. DIV_1+3 (4 byte, ZP)       : Dividendo
;   - .A (1 byte, CPU)                      : Divisore
;
; Output:
;   - .A (1 byte, CPU)                      : Byte 0 del quoziente, o
;                                             ERR_DIVISION_BY_ZERO se C = 1
;   - .X (1 byte, CPU)                      : Byte 1 del quoziente
;   - .Y (1 byte, CPU)                      : Byte 2 del quoziente
;   - DIV_QUOT+0 .. DIV_QUOT+3 (4 byte, ZP) : Quoziente a 32 bit
;   - DIV_REM+0 (1 byte, ZP)                : Resto a 8 bit
;   - Flag Carry (1 bit, .P)                : 0 = eseguita; 1 = divisore nullo
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0 .. DIV_QUOT+3, DIV_2+0, DIV_REM+0
; Dipendenze Esterne:   <c64/errors.asm>
;                       <c64/symbols.asm>
; ================================================================================
```

4.5.24 SDIV_32X8

```
; ================================================================================
; SUBROUTINE: SDIV_32X8
; --------------------------------------------------------------------------------
; Descrizione:  Divide un intero a 32 bit per uno a 8 bit, con segno. Il
;               quoziente ha 32 bit e il resto 8 bit.
;
;               Rende positivi dividendo e divisore, esegue il nucleo senza
;               segno e corregge i segni: il quoziente è negativo se i segni
;               degli operandi sono diversi e il resto ha il segno del
;               dividendo.
;
;               L'unico overflow possibile è -2^31 diviso -1: il quoziente esce
;               troncato ai 32 bit bassi e il flag di oVerflow vale 1.
;
;               Se il divisore è zero esce con il Carry a 1 e
;               ERR_DIVISION_BY_ZERO in .A, senza toccare quoziente e resto.
;
; Input:
;   - DIV_1+0 .. DIV_1+3 (4 byte, ZP)       : Dividendo con segno
;   - .A (1 byte, CPU)                      : Divisore con segno
;
; Output:
;   - .A (1 byte, CPU)                      : Byte 0 del quoziente, o
;                                             ERR_DIVISION_BY_ZERO se C = 1
;   - .X (1 byte, CPU)                      : Byte 1 del quoziente
;   - .Y (1 byte, CPU)                      : Byte 2 del quoziente
;   - DIV_QUOT+0 .. DIV_QUOT+3 (4 byte, ZP) : Quoziente a 32 bit con segno
;   - DIV_REM+0 (1 byte, ZP)                : Resto a 8 bit con segno
;   - Flag oVerflow (1 bit, .P)             : 0 = nessuno; 1 = overflow
;   - Flag Carry (1 bit, .P)                : 0 = eseguita; 1 = divisore nullo
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0 .. DIV_QUOT+3, DIV_2+0, DIV_REM+0
; Dipendenze Esterne:   <c64/errors.asm>
;                       <c64/symbols.asm>
; ================================================================================
```

4.5.25 DIV_16X16

```
; ================================================================================
; SUBROUTINE: DIV_16X16
; --------------------------------------------------------------------------------
; Descrizione:  Divide due interi a 16 bit, senza segno. Il quoziente ha 16 bit
;               e il resto 16 bit.
;
;               Se il divisore è zero esce con il Carry a 1 e
;               ERR_DIVISION_BY_ZERO in .A, senza toccare quoziente e resto.
;
; Input:
;   - DIV_1+0 .. DIV_1+1 (2 byte, ZP)       : Dividendo
;   - .A (1 byte, CPU)                      : Divisore, byte 0
;   - .X (1 byte, CPU)                      : Divisore, byte 1
;
; Output:
;   - .A (1 byte, CPU)                      : Byte 0 del quoziente, o
;                                             ERR_DIVISION_BY_ZERO se C = 1
;   - .X (1 byte, CPU)                      : Byte 1 del quoziente
;   - DIV_QUOT+0 .. DIV_QUOT+1 (2 byte, ZP) : Quoziente a 16 bit
;   - DIV_REM+0 .. DIV_REM+1 (2 byte, ZP)   : Resto a 16 bit
;   - Flag Carry (1 bit, .P)                : 0 = eseguita; 1 = divisore nullo
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0 .. DIV_QUOT+1, DIV_2+0 .. DIV_2+1,
;                       DIV_REM+0 .. DIV_REM+1
; Dipendenze Esterne:   <c64/errors.asm>
;                       <c64/symbols.asm>
; ================================================================================
```

4.5.26 SDIV_16X16

```
; ================================================================================
; SUBROUTINE: SDIV_16X16
; --------------------------------------------------------------------------------
; Descrizione:  Divide due interi a 16 bit, con segno. Il quoziente ha 16 bit e
;               il resto 16 bit.
;
;               Rende positivi dividendo e divisore, esegue il nucleo senza
;               segno e corregge i segni: il quoziente è negativo se i segni
;               degli operandi sono diversi e il resto ha il segno del
;               dividendo.
;
;               L'unico overflow possibile è -2^15 diviso -1: il quoziente esce
;               troncato ai 16 bit bassi e il flag di oVerflow vale 1.
;
;               Se il divisore è zero esce con il Carry a 1 e
;               ERR_DIVISION_BY_ZERO in .A, senza toccare quoziente e resto.
;
; Input:
;   - DIV_1+0 .. DIV_1+1 (2 byte, ZP)       : Dividendo con segno
;   - .A (1 byte, CPU)                      : Divisore con segno, byte 0
;   - .X (1 byte, CPU)                      : Divisore con segno, byte 1
;
; Output:
;   - .A (1 byte, CPU)                      : Byte 0 del quoziente, o
;                                             ERR_DIVISION_BY_ZERO se C = 1
;   - .X (1 byte, CPU)                      : Byte 1 del quoziente
;   - DIV_QUOT+0 .. DIV_QUOT+1 (2 byte, ZP) : Quoziente a 16 bit con segno
;   - DIV_REM+0 .. DIV_REM+1 (2 byte, ZP)   : Resto a 16 bit con segno
;   - Flag oVerflow (1 bit, .P)             : 0 = nessuno; 1 = overflow
;   - Flag Carry (1 bit, .P)                : 0 = eseguita; 1 = divisore nullo
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0 .. DIV_QUOT+1, DIV_2+0 .. DIV_2+1,
;                       DIV_REM+0 .. DIV_REM+1
; Dipendenze Esterne:   <c64/errors.asm>
;                       <c64/symbols.asm>
; ================================================================================
```

4.5.27 DIV_24X16

```
; ================================================================================
; SUBROUTINE: DIV_24X16
; --------------------------------------------------------------------------------
; Descrizione:  Divide un intero a 24 bit per uno a 16 bit, senza segno. Il
;               quoziente ha 24 bit e il resto 16 bit.
;
;               Se il divisore è zero esce con il Carry a 1 e
;               ERR_DIVISION_BY_ZERO in .A, senza toccare quoziente e resto.
;
; Input:
;   - DIV_1+0 .. DIV_1+2 (3 byte, ZP)       : Dividendo
;   - .A (1 byte, CPU)                      : Divisore, byte 0
;   - .X (1 byte, CPU)                      : Divisore, byte 1
;
; Output:
;   - .A (1 byte, CPU)                      : Byte 0 del quoziente, o
;                                             ERR_DIVISION_BY_ZERO se C = 1
;   - .X (1 byte, CPU)                      : Byte 1 del quoziente
;   - .Y (1 byte, CPU)                      : Byte 2 del quoziente
;   - DIV_QUOT+0 .. DIV_QUOT+2 (3 byte, ZP) : Quoziente a 24 bit
;   - DIV_REM+0 .. DIV_REM+1 (2 byte, ZP)   : Resto a 16 bit
;   - Flag Carry (1 bit, .P)                : 0 = eseguita; 1 = divisore nullo
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0 .. DIV_QUOT+2, DIV_2+0 .. DIV_2+1,
;                       DIV_REM+0 .. DIV_REM+1
; Dipendenze Esterne:   <c64/errors.asm>
;                       <c64/symbols.asm>
; ================================================================================
```

4.5.28 SDIV_24X16

```
; ============================================================================
; SUBROUTINE: SDIV_24X16
; ----------------------------------------------------------------------------
; Descrizione:  Divide un intero a 24 bit per uno a 16 bit, con segno. Il
;               quoziente ha 24 bit e il resto 16 bit.
;
;               Rende positivi dividendo e divisore, esegue il nucleo senza
;               segno e corregge i segni: il quoziente è negativo se i segni
;               degli operandi sono diversi e il resto ha il segno del
;               dividendo.
;
;               L'unico overflow possibile è -2^23 diviso -1: il quoziente esce
;               troncato ai 24 bit bassi e il flag di oVerflow vale 1.
;
;               Se il divisore è zero esce con il Carry a 1 e
;               ERR_DIVISION_BY_ZERO in .A, senza toccare quoziente e resto.
;
; Input:
;   - DIV_1+0 .. DIV_1+2 (3 byte, ZP)       : Dividendo con segno
;   - .A (1 byte, CPU)                      : Divisore con segno, byte 0
;   - .X (1 byte, CPU)                      : Divisore con segno, byte 1
;
; Output:
;   - .A (1 byte, CPU)                      : Byte 0 del quoziente, o
;                                             ERR_DIVISION_BY_ZERO se C = 1
;   - .X (1 byte, CPU)                      : Byte 1 del quoziente
;   - .Y (1 byte, CPU)                      : Byte 2 del quoziente
;   - DIV_QUOT+0 .. DIV_QUOT+2 (3 byte, ZP) : Quoziente a 24 bit con segno
;   - DIV_REM+0 .. DIV_REM+1 (2 byte, ZP)   : Resto a 16 bit con segno
;   - Flag oVerflow (1 bit, .P)             : 0 = nessuno; 1 = overflow
;   - Flag Carry (1 bit, .P)                : 0 = eseguita; 1 = divisore nullo
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0 .. DIV_QUOT+2, DIV_2+0 .. DIV_2+1,
;                       DIV_REM+0 .. DIV_REM+1
; Dipendenze Esterne:   <c64/errors.asm>
;                       <c64/symbols.asm>
; ============================================================================
```

4.5.29 DIV_32X16

```
; ================================================================================
; SUBROUTINE: DIV_32X16
; --------------------------------------------------------------------------------
; Descrizione:  Divide un intero a 32 bit per uno a 16 bit, senza segno. Il
;               quoziente ha 32 bit e il resto 16 bit.
;
;               Se il divisore è zero esce con il Carry a 1 e
;               ERR_DIVISION_BY_ZERO in .A, senza toccare quoziente e resto.
;
; Input:
;   - DIV_1+0 .. DIV_1+3 (4 byte, ZP)       : Dividendo
;   - .A (1 byte, CPU)                      : Divisore, byte 0
;   - .X (1 byte, CPU)                      : Divisore, byte 1
;
; Output:
;   - .A (1 byte, CPU)                      : Byte 0 del quoziente, o
;                                             ERR_DIVISION_BY_ZERO se C = 1
;   - .X (1 byte, CPU)                      : Byte 1 del quoziente
;   - .Y (1 byte, CPU)                      : Byte 2 del quoziente
;   - DIV_QUOT+0 .. DIV_QUOT+3 (4 byte, ZP) : Quoziente a 32 bit
;   - DIV_REM+0 .. DIV_REM+1 (2 byte, ZP)   : Resto a 16 bit
;   - Flag Carry (1 bit, .P)                : 0 = eseguita; 1 = divisore nullo
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0 .. DIV_QUOT+3, DIV_2+0 .. DIV_2+1,
;                       DIV_REM+0 .. DIV_REM+1
; Dipendenze Esterne:   <c64/errors.asm>
;                       <c64/symbols.asm>
; ================================================================================
```

4.5.30 SDIV_32X16

```
; ================================================================================
; SUBROUTINE: SDIV_32X16
; --------------------------------------------------------------------------------
; Descrizione:  Divide un intero a 32 bit per uno a 16 bit, con segno. Il
;               quoziente ha 32 bit e il resto 16 bit.
;
;               Rende positivi dividendo e divisore, esegue il nucleo senza
;               segno e corregge i segni: il quoziente è negativo se i segni
;               degli operandi sono diversi e il resto ha il segno del
;               dividendo.
;
;               L'unico overflow possibile è -2^31 diviso -1: il quoziente esce
;               troncato ai 32 bit bassi e il flag di oVerflow vale 1.
;
;               Se il divisore è zero esce con il Carry a 1 e
;               ERR_DIVISION_BY_ZERO in .A, senza toccare quoziente e resto.
;
; Input:
;   - DIV_1+0 .. DIV_1+3 (4 byte, ZP)       : Dividendo con segno
;   - .A (1 byte, CPU)                      : Divisore con segno, byte 0
;   - .X (1 byte, CPU)                      : Divisore con segno, byte 1
;
; Output:
;   - .A (1 byte, CPU)                      : Byte 0 del quoziente, o
;                                             ERR_DIVISION_BY_ZERO se C = 1
;   - .X (1 byte, CPU)                      : Byte 1 del quoziente
;   - .Y (1 byte, CPU)                      : Byte 2 del quoziente
;   - DIV_QUOT+0 .. DIV_QUOT+3 (4 byte, ZP) : Quoziente a 32 bit con segno
;   - DIV_REM+0 .. DIV_REM+1 (2 byte, ZP)   : Resto a 16 bit con segno
;   - Flag oVerflow (1 bit, .P)             : 0 = nessuno; 1 = overflow
;   - Flag Carry (1 bit, .P)                : 0 = eseguita; 1 = divisore nullo
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0 .. DIV_QUOT+3, DIV_2+0 .. DIV_2+1,
;                       DIV_REM+0 .. DIV_REM+1
; Dipendenze Esterne:   <c64/errors.asm>
;                       <c64/symbols.asm>
; ================================================================================
```

4.5.31 DIV_24X24

```
; ============================================================================
; SUBROUTINE: DIV_24X24
; ----------------------------------------------------------------------------
; Descrizione:  Divide due interi a 24 bit, senza segno. Il quoziente ha 24 bit
;               e il resto 24 bit.
;
;               Se il divisore è zero esce con il Carry a 1 e
;               ERR_DIVISION_BY_ZERO in .A, senza toccare quoziente e resto.
;
; Input:
;   - DIV_1+0 .. DIV_1+2 (3 byte, ZP)       : Dividendo
;   - .A (1 byte, CPU)                      : Divisore, byte 0
;   - .X (1 byte, CPU)                      : Divisore, byte 1
;   - .Y (1 byte, CPU)                      : Divisore, byte 2
;
; Output:
;   - .A (1 byte, CPU)                      : Byte 0 del quoziente, o
;                                             ERR_DIVISION_BY_ZERO se C = 1
;   - .X (1 byte, CPU)                      : Byte 1 del quoziente
;   - .Y (1 byte, CPU)                      : Byte 2 del quoziente
;   - DIV_QUOT+0 .. DIV_QUOT+2 (3 byte, ZP) : Quoziente a 24 bit
;   - DIV_REM+0 .. DIV_REM+2 (3 byte, ZP)   : Resto a 24 bit
;   - Flag Carry (1 bit, .P)                : 0 = eseguita; 1 = divisore nullo
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0 .. DIV_QUOT+2, DIV_2+0 .. DIV_2+2,
;                       DIV_REM+0 .. DIV_REM+2
; Dipendenze Esterne:   <c64/errors.asm>
;                       <c64/symbols.asm>
; ============================================================================
```

4.5.32 SDIV_24X24

```
; ============================================================================
; SUBROUTINE: SDIV_24X24
; ----------------------------------------------------------------------------
; Descrizione:  Divide due interi a 24 bit, con segno. Il quoziente ha 24 bit e
;               il resto 24 bit.
;
;               Rende positivi dividendo e divisore, esegue il nucleo senza
;               segno e corregge i segni: il quoziente è negativo se i segni
;               degli operandi sono diversi e il resto ha il segno del
;               dividendo.
;
;               L'unico overflow possibile è -2^23 diviso -1: il quoziente esce
;               troncato ai 24 bit bassi e il flag di oVerflow vale 1.
;
;               Se il divisore è zero esce con il Carry a 1 e
;               ERR_DIVISION_BY_ZERO in .A, senza toccare quoziente e resto.
;
; Input:
;   - DIV_1+0 .. DIV_1+2 (3 byte, ZP)       : Dividendo con segno
;   - .A (1 byte, CPU)                      : Divisore con segno, byte 0
;   - .X (1 byte, CPU)                      : Divisore con segno, byte 1
;   - .Y (1 byte, CPU)                      : Divisore con segno, byte 2
;
; Output:
;   - .A (1 byte, CPU)                      : Byte 0 del quoziente, o
;                                             ERR_DIVISION_BY_ZERO se C = 1
;   - .X (1 byte, CPU)                      : Byte 1 del quoziente
;   - .Y (1 byte, CPU)                      : Byte 2 del quoziente
;   - DIV_QUOT+0 .. DIV_QUOT+2 (3 byte, ZP) : Quoziente a 24 bit con segno
;   - DIV_REM+0 .. DIV_REM+2 (3 byte, ZP)   : Resto a 24 bit con segno
;   - Flag oVerflow (1 bit, .P)             : 0 = nessuno; 1 = overflow
;   - Flag Carry (1 bit, .P)                : 0 = eseguita; 1 = divisore nullo
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0 .. DIV_QUOT+2, DIV_2+0 .. DIV_2+2,
;                       DIV_REM+0 .. DIV_REM+2
; Dipendenze Esterne:   <c64/errors.asm>
;                       <c64/symbols.asm>
; ============================================================================
```

4.5.33 DIV_32X24

```
; ============================================================================
; SUBROUTINE: DIV_32X24
; ----------------------------------------------------------------------------
; Descrizione:  Divide un intero a 32 bit per uno a 24 bit, senza segno. Il
;               quoziente ha 32 bit e il resto 24 bit.
;
;               Se il divisore è zero esce con il Carry a 1 e
;               ERR_DIVISION_BY_ZERO in .A, senza toccare quoziente e resto.
;
; Input:
;   - DIV_1+0 .. DIV_1+3 (4 byte, ZP)       : Dividendo
;   - .A (1 byte, CPU)                      : Divisore, byte 0
;   - .X (1 byte, CPU)                      : Divisore, byte 1
;   - .Y (1 byte, CPU)                      : Divisore, byte 2
;
; Output:
;   - .A (1 byte, CPU)                      : Byte 0 del quoziente, o
;                                             ERR_DIVISION_BY_ZERO se C = 1
;   - .X (1 byte, CPU)                      : Byte 1 del quoziente
;   - .Y (1 byte, CPU)                      : Byte 2 del quoziente
;   - DIV_QUOT+0 .. DIV_QUOT+3 (4 byte, ZP) : Quoziente a 32 bit
;   - DIV_REM+0 .. DIV_REM+2 (3 byte, ZP)   : Resto a 24 bit
;   - Flag Carry (1 bit, .P)                : 0 = eseguita; 1 = divisore nullo
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0 .. DIV_QUOT+3, DIV_2+0 .. DIV_2+2,
;                       DIV_REM+0 .. DIV_REM+2
; Dipendenze Esterne:   <c64/errors.asm>
;                       <c64/symbols.asm>
; ============================================================================
```

4.5.34 SDIV_32X24

```
; ============================================================================
; SUBROUTINE: SDIV_32X24
; ----------------------------------------------------------------------------
; Descrizione:  Divide un intero a 32 bit per uno a 24 bit, con segno. Il
;               quoziente ha 32 bit e il resto 24 bit.
;
;               Rende positivi dividendo e divisore, esegue il nucleo senza
;               segno e corregge i segni: il quoziente è negativo se i segni
;               degli operandi sono diversi e il resto ha il segno del
;               dividendo.
;
;               L'unico overflow possibile è -2^31 diviso -1: il quoziente esce
;               troncato ai 32 bit bassi e il flag di oVerflow vale 1.
;
;               Se il divisore è zero esce con il Carry a 1 e
;               ERR_DIVISION_BY_ZERO in .A, senza toccare quoziente e resto.
;
; Input:
;   - DIV_1+0 .. DIV_1+3 (4 byte, ZP)       : Dividendo con segno
;   - .A (1 byte, CPU)                      : Divisore con segno, byte 0
;   - .X (1 byte, CPU)                      : Divisore con segno, byte 1
;   - .Y (1 byte, CPU)                      : Divisore con segno, byte 2
;
; Output:
;   - .A (1 byte, CPU)                      : Byte 0 del quoziente, o
;                                             ERR_DIVISION_BY_ZERO se C = 1
;   - .X (1 byte, CPU)                      : Byte 1 del quoziente
;   - .Y (1 byte, CPU)                      : Byte 2 del quoziente
;   - DIV_QUOT+0 .. DIV_QUOT+3 (4 byte, ZP) : Quoziente a 32 bit con segno
;   - DIV_REM+0 .. DIV_REM+2 (3 byte, ZP)   : Resto a 24 bit con segno
;   - Flag oVerflow (1 bit, .P)             : 0 = nessuno; 1 = overflow
;   - Flag Carry (1 bit, .P)                : 0 = eseguita; 1 = divisore nullo
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0 .. DIV_QUOT+3, DIV_2+0 .. DIV_2+2,
;                       DIV_REM+0 .. DIV_REM+2
; Dipendenze Esterne:   <c64/errors.asm>
;                       <c64/symbols.asm>
; ============================================================================
```

4.5.35 DIV_32X32

```
; ============================================================================
; SUBROUTINE: DIV_32X32
; ----------------------------------------------------------------------------
; Descrizione:  Divide due interi a 32 bit, senza segno. Il quoziente ha 32 bit
;               e il resto 32 bit.
;
;               Se il divisore è zero esce con il Carry a 1 e
;               ERR_DIVISION_BY_ZERO in .A, senza toccare quoziente e resto.
;
; Input:
;   - DIV_1+0 .. DIV_1+3 (4 byte, ZP)       : Dividendo
;   - DIV_2+0 .. DIV_2+3 (4 byte, ZP)       : Divisore
;
; Output:
;   - .A (1 byte, CPU)                      : Byte 0 del quoziente, o
;                                             ERR_DIVISION_BY_ZERO se C = 1
;   - .X (1 byte, CPU)                      : Byte 1 del quoziente
;   - .Y (1 byte, CPU)                      : Byte 2 del quoziente
;   - DIV_QUOT+0 .. DIV_QUOT+3 (4 byte, ZP) : Quoziente a 32 bit
;   - DIV_REM+0 .. DIV_REM+3 (4 byte, ZP)   : Resto a 32 bit
;   - Flag Carry (1 bit, .P)                : 0 = eseguita; 1 = divisore nullo
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0 .. DIV_QUOT+3, DIV_REM+0 .. DIV_REM+3
; Dipendenze Esterne:   <c64/errors.asm>
;                       <c64/symbols.asm>
; ============================================================================
```

4.5.36 SDIV_32X32

```
; ================================================================================
; SUBROUTINE: SDIV_32X32
; --------------------------------------------------------------------------------
; Descrizione:  Divide due interi a 32 bit, con segno. Il quoziente ha 32 bit e
;               il resto 32 bit.
;
;               Rende positivi dividendo e divisore, esegue il nucleo senza
;               segno e corregge i segni: il quoziente è negativo se i segni
;               degli operandi sono diversi e il resto ha il segno del
;               dividendo.
;
;               L'unico overflow possibile è -2^31 diviso -1: il quoziente esce
;               troncato ai 32 bit bassi e il flag di oVerflow vale 1.
;
;               Se il divisore è zero esce con il Carry a 1 e
;               ERR_DIVISION_BY_ZERO in .A, senza toccare quoziente e resto.
;
; Input:
;   - DIV_1+0 .. DIV_1+3 (4 byte, ZP)       : Dividendo con segno
;   - DIV_2+0 .. DIV_2+3 (4 byte, ZP)       : Divisore con segno
;
; Output:
;   - .A (1 byte, CPU)                      : Byte 0 del quoziente, o
;                                             ERR_DIVISION_BY_ZERO se C = 1
;   - .X (1 byte, CPU)                      : Byte 1 del quoziente
;   - .Y (1 byte, CPU)                      : Byte 2 del quoziente
;   - DIV_QUOT+0 .. DIV_QUOT+3 (4 byte, ZP) : Quoziente a 32 bit con segno
;   - DIV_REM+0 .. DIV_REM+3 (4 byte, ZP)   : Resto a 32 bit con segno
;   - Flag oVerflow (1 bit, .P)             : 0 = nessuno; 1 = overflow
;   - Flag Carry (1 bit, .P)                : 0 = eseguita; 1 = divisore nullo
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       DIV_QUOT+0 .. DIV_QUOT+3, DIV_2+0 .. DIV_2+3,
;                       DIV_REM+0 .. DIV_REM+3
; Dipendenze Esterne:   <c64/errors.asm>
;                       <c64/symbols.asm>
; ================================================================================
```

4.6 math_mult.asm

4.6.1 Descrizione

Moltiplicazione intera senza segno e con segno per tutte le combinazioni di operandi da 8 a 32 bit. Il prodotto ha sempre la lunghezza esatta m+n bit, senza perdita di precisione.

Utilità	Descrizione
m_Mul_Negate	Nega un valore a più byte in pagina zero
m_Mul_Stage	Elabora un byte del moltiplicatore
m_Mul_Core	Nucleo generico senza segno
m_Mul_Core_8x8	Nucleo senza segno 8x8 bit
m_Mul_Core_16x8	Nucleo senza segno 16x8 bit
m_Mul_Core_24x8	Nucleo senza segno 24x8 bit
m_Mul_Core_32x8	Nucleo senza segno 32x8 bit
m_Mul_Core_16x16	Nucleo senza segno 16x16 bit
m_Mul_Core_24x16	Nucleo senza segno 24x16 bit
m_Mul_Core_32x16	Nucleo senza segno 32x16 bit
m_Mul_Core_24x24	Nucleo senza segno 24x24 bit
m_Mul_Core_32x24	Nucleo senza segno 32x24 bit
m_Mul_Core_32x32	Nucleo senza segno 32x32 bit
MUL_8X8	Prodotto 8x8 bit senza segno
SMUL_8X8	Prodotto 8x8 bit con segno
MUL_16X8	Prodotto 16x8 bit senza segno
SMUL_16X8	Prodotto 16x8 bit con segno
MUL_24X8	Prodotto 24x8 bit senza segno
SMUL_24X8	Prodotto 24x8 bit con segno
MUL_32X8	Prodotto 32x8 bit senza segno
SMUL_32X8	Prodotto 32x8 bit con segno
MUL_16X16	Prodotto 16x16 bit senza segno
SMUL_16X16	Prodotto 16x16 bit con segno
MUL_24X16	Prodotto 24x16 bit senza segno
SMUL_24X16	Prodotto 24x16 bit con segno
MUL_32X16	Prodotto 32x16 bit senza segno
SMUL_32X16	Prodotto 32x16 bit con segno
MUL_24X24	Prodotto 24x24 bit senza segno
SMUL_24X24	Prodotto 24x24 bit con segno
MUL_32X24	Prodotto 32x24 bit senza segno

Utilità	Descrizione
SMUL_32X24	Prodotto 32x24 bit con segno
MUL_32X32	Prodotto 32x32 bit senza segno
SMUL_32X32	Prodotto 32x32 bit con segno

4.6.2 Note d'uso

```
; NOTA #01
; --------
; Il primo operando è il più lungo e fa da moltiplicando; il secondo è il
; moltiplicatore. Le routine con operandi in registri usano, dal byte più
; basso al più alto, .A, .X e .Y (cioè la terna .YXA, come nell'ordine di
; memoria). I byte del prodotto oltre il terzo non entrano nei registri: si
; leggono da MUL_RES+3 in poi.
;
;   Routine  1º operando    2º operando    Registri in uscita
;   -------  -------------  -------------  ------------------
;   8x8      .A             .X             .A, .X
;   16x8     .A basso,      .Y             .A, .X, .Y
;            .X alto
;   24x8     MUL_1 (ZP)     .A             .A, .X, .Y
;   32x8     MUL_1 (ZP)     .A             .A, .X, .Y
;   16x16    MUL_1 (ZP)     .A basso,      .A, .X, .Y
;                           .X alto
;   24x16    MUL_1 (ZP)     .A basso,      .A, .X, .Y
;                           .X alto
;   32x16    MUL_1 (ZP)     .A basso,      .A, .X, .Y
;                           .X alto
;   24x24    MUL_1 (ZP)     .A, .X, .Y     .A, .X, .Y
;   32x24    MUL_1 (ZP)     .A, .X, .Y     .A, .X, .Y
;   32x32    MUL_1 (ZP)     MUL_2 (ZP)     .A, .X, .Y
;
; Per le versioni con segno gli operandi sono in complemento a 2 e il prodotto
; ha il segno corretto. L'input non si conserva: il moltiplicatore viene
; consumato e, nelle versioni con segno, gli operandi diventano positivi.
```

```
; NOTA #02
; --------
; Le routine lavorano in pagina zero sui 12 byte da NUMWRK a NUMWRK+11, cioè
; NUMWRK più i primi due byte di FAC, che sono liberi finché non si usa la
; matematica in virgola mobile del BASIC.
;
;   MUL_1    NUMWRK+0 .. NUMWRK+3     Moltiplicando (primo operando)
;   MUL_2    NUMWRK+4 .. NUMWRK+7     Moltiplicatore (secondo operando)
;   MUL_RES  NUMWRK+4 .. NUMWRK+11    Prodotto, da 2 a 8 byte
;
; MUL_2 e MUL_RES coincidono: il moltiplicatore occupa i byte bassi del
; prodotto e viene consumato mentre i byte definitivi prendono il suo posto,
; bit dopo bit. I byte alti di MUL_RES ospitano l'accumulatore. Per questo il
; moltiplicatore non si conserva e MUL_2 e MUL_RES non si usano insieme.
;
; La stessa area è condivisa con ZPL_1 e ZPL_2 (puntatori utente a 24/32 bit)
; e con le routine di divisione: non si possono usare contemporaneamente.
```

```
; NOTA #03
; --------
; Si elabora un byte del moltiplicatore alla volta, con 8 iterazioni di
; shift-and-add: se il bit meno significativo vale 1 si somma il moltiplicando
; all'accumulatore alto, poi tutto scorre di un bit a destra. Il moltiplicando
; non si sposta mai e ogni iterazione fa scorrere solo l'accumulatore e il
; byte in esame del moltiplicatore, non l'intero prodotto. Il byte più alto
; dell'accumulatore resta in .A per tutta l'elaborazione e si scrive in
; memoria una sola volta, alla fine.
;
; I cicli medi con operandi casuali comprendono il caricamento dei registri
; e l'rts, ma non la jsr.
;
;   Routine  Senza segno  Con segno
;   8x8             167       214
;   16x8            270       315
;   24x8            336       390
;   32x8            415       466
;   16x16           496       555
;   24x16           663       700
;   32x16           818       864
;   24x24           982      1007
;   32x24          1222      1234
;   32x32          1582      1624
```

4.6.3 m_Mul_Negate

```
; ================================================================================
; MACRO: m_Mul_Negate
; --------------------------------------------------------------------------------
; Descrizione:  Nega sul posto un valore a più byte in pagina zero, cioè lo
;               sostituisce con il suo complemento a 2. Serve per rendere
;               positivi gli operandi e per correggere il segno del prodotto.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .address_ (1 byte, Macro)   : Indirizzo in pagina zero del byte meno
;                                   significativo
;   - .bytes_ (1 byte, Macro)     : Numero di byte del valore (da 1 a 8)
;
; Output:
;   - .address_ (1 .. 8 byte, ZP) : Il valore negato, nelle stesse locazioni
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, oVerflow, Zero, Carry),
;                       .address_+0 .. .address_+.bytes_-1
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

4.6.4 m_Mul_Stage

```
; ================================================================================
; MACRO: m_Mul_Stage
; --------------------------------------------------------------------------------
; Descrizione:  Elabora un byte del moltiplicatore con 8 iterazioni di
;               shift-and-add. Ogni iterazione somma il moltiplicando
;               all'accumulatore alto se il bit in esame vale 1, poi fa scorrere
;               a destra l'accumulatore e il byte in esame. I bit esaminati
;               escono dal byte del moltiplicatore e al loro posto entrano i bit
;               bassi dell'accumulatore, che formano un byte definitivo del
;               prodotto finale.
;
;               Se il moltiplicando ha un solo byte l'accumulatore alto sta
;               tutto in .A. Altrimenti il byte più alto dell'accumulatore sta
;               in .A e gli altri nei byte di MUL_RES che seguono quelli del
;               moltiplicatore; il byte più alto si scrive in memoria solo alla
;               fine di m_Mul_Core.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .stage_ (1 byte, Macro)        : Indice del byte del moltiplicatore (0 =
;                                      il meno significativo)
;   - .mcand_bytes_ (1 byte, Macro)  : Byte del moltiplicando (da 1 a 4)
;   - .mplier_bytes_ (1 byte, Macro) : Byte del moltiplicatore (da 1 a 4)
;   - MUL_1 (1 .. 4 byte, ZP)        : Moltiplicando
;   - MUL_RES (2 .. 8 byte, ZP)      : Moltiplicatore e accumulatore alto
;   - .A (1 byte, CPU)               : Byte più alto dell'accumulatore
;
; Output:
;   - MUL_RES (2 .. 8 byte, ZP)      : Il byte .stage_ diventa un byte
;                                      definitivo del prodotto
;   - .A (1 byte, CPU)               : Byte più alto dell'accumulatore
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y (se .mcand_bytes_ > 1),
;                       .P (Negative, oVerflow, Zero, Carry),
;                      MUL_RES+0 .. MUL_RES+6
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.6.5 m_Mul_Core

```
; ================================================================================
; MACRO: m_Mul_Core
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplica due valori senza segno con l'algoritmo shift-and-add
;               elaborando un byte del moltiplicatore alla volta (m_Mul_Stage).
;               Il moltiplicatore sta nei byte bassi di MUL_RES, che a fine
;               operazione contengono il prodotto di .mcand_bytes_ +
;               .mplier_bytes_ byte.
;
;               Conviene usare come moltiplicando l'operando più lungo, perché
;               il costo cresce con il numero di byte del moltiplicatore.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mcand_bytes_ (1 byte, Macro)  : Byte del moltiplicando (da 1 a 4)
;   - .mplier_bytes_ (1 byte, Macro) : Byte del moltiplicatore (da 1 a 4)
;   - MUL_1 (1 .. 4 byte, ZP)        : Moltiplicando (senza segno)
;   - MUL_2 (1 .. 4 byte, ZP)        : Moltiplicatore (senza segno)
;
; Output:
;   - MUL_RES (2 .. 8 byte, ZP)      : Prodotto della moltiplicazione
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y (se .mcand_bytes_ > 1),
;                       .P (Negative, oVerflow, Zero, Carry),
;                      MUL_RES+0 .. MUL_RES+7
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.6.6 m_Mul_Core_8x8

```
; ================================================================================
; MACRO: m_Mul_Core_8x8
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplicazione senza segno tra due valori a 8 bit con il nucleo
;               generico m_Mul_Core.
;
; Input:
;   - MUL_1+0 (1 byte, ZP)               : Moltiplicando
;   - MUL_2+0 (1 byte, ZP)               : Moltiplicatore
;
; Output:
;   - MUL_RES+0 .. MUL_RES+1 (2 byte, ZP) : Prodotto a 16 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .P (Negative, oVerflow, Zero, Carry),
;                       MUL_RES+0 .. MUL_RES+1
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.6.7 m_Mul_Core_16x8

```
; ================================================================================
; MACRO: m_Mul_Core_16x8
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplicazione senza segno tra un valore a 16 bit
;               (moltiplicando) e uno a 8 bit (moltiplicatore) con il nucleo
;               generico m_Mul_Core.
;
; Input:
;   - MUL_1+0 .. MUL_1+1 (2 byte, ZP)     : Moltiplicando
;   - MUL_2+0 (1 byte, ZP)                : Moltiplicatore
;
; Output:
;   - MUL_RES+0 .. MUL_RES+2 (3 byte, ZP) : Prodotto a 24 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       MUL_RES+0 .. MUL_RES+2
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.6.8 m_Mul_Core_24x8

```
; ================================================================================
; MACRO: m_Mul_Core_24x8
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplicazione senza segno tra un valore a 24 bit
;               (moltiplicando) e uno a 8 bit (moltiplicatore) con il nucleo
;               generico m_Mul_Core.
;
; Input:
;   - MUL_1+0 .. MUL_1+2 (3 byte, ZP)     : Moltiplicando
;   - MUL_2+0 (1 byte, ZP)                : Moltiplicatore
;
; Output:
;   - MUL_RES+0 .. MUL_RES+3 (4 byte, ZP) : Prodotto a 32 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       MUL_RES+0 .. MUL_RES+3
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.6.9 m_Mul_Core_32x8

```
; ================================================================================
; MACRO: m_Mul_Core_32x8
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplicazione senza segno tra un valore a 32 bit
;               (moltiplicando) e uno a 8 bit (moltiplicatore) con il nucleo
;               generico m_Mul_Core.
;
; Input:
;   - MUL_1+0 .. MUL_1+3 (4 byte, ZP)     : Moltiplicando
;   - MUL_2+0 (1 byte, ZP)                : Moltiplicatore
;
; Output:
;   - MUL_RES+0 .. MUL_RES+4 (5 byte, ZP) : Prodotto a 40 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       MUL_RES+0 .. MUL_RES+4
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.6.10 m_Mul_Core_16x16

```
; ================================================================================
; MACRO: m_Mul_Core_16x16
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplicazione senza segno tra due valori a 16 bit con il
;               nucleo generico m_Mul_Core.
;
; Input:
;   - MUL_1+0 .. MUL_1+1 (2 byte, ZP)     : Moltiplicando
;   - MUL_2+0 .. MUL_2+1 (2 byte, ZP)     : Moltiplicatore
;
; Output:
;   - MUL_RES+0 .. MUL_RES+3 (4 byte, ZP) : Prodotto a 32 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       MUL_RES+0 .. MUL_RES+3
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.6.11 m_Mul_Core_24x16

```
; ================================================================================
; MACRO: m_Mul_Core_24x16
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplicazione senza segno tra un valore a 24 bit
;               (moltiplicando) e uno a 16 bit (moltiplicatore) con il nucleo
;               generico m_Mul_Core.
;
; Input:
;   - MUL_1+0 .. MUL_1+2 (3 byte, ZP)     : Moltiplicando
;   - MUL_2+0 .. MUL_2+1 (2 byte, ZP)     : Moltiplicatore
;
; Output:
;   - MUL_RES+0 .. MUL_RES+4 (5 byte, ZP) : Prodotto a 40 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       MUL_RES+0 .. MUL_RES+4
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.6.12 m_Mul_Core_32x16

```
; ================================================================================
; MACRO: m_Mul_Core_32x16
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplicazione senza segno tra un valore a 32 bit
;               (moltiplicando) e uno a 16 bit (moltiplicatore) con il nucleo
;               generico m_Mul_Core.
;
; Input:
;   - MUL_1+0 .. MUL_1+3 (4 byte, ZP)     : Moltiplicando
;   - MUL_2+0 .. MUL_2+1 (2 byte, ZP)     : Moltiplicatore
;
; Output:
;   - MUL_RES+0 .. MUL_RES+5 (6 byte, ZP) : Prodotto a 48 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       MUL_RES+0 .. MUL_RES+5
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.6.13 m_Mul_Core_24x24

```
; ================================================================================
; MACRO: m_Mul_Core_24x24
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplicazione senza segno tra due valori a 24 bit con il
;               nucleo generico m_Mul_Core.
;
; Input:
;   - MUL_1+0 .. MUL_1+2 (3 byte, ZP)     : Moltiplicando
;   - MUL_2+0 .. MUL_2+2 (3 byte, ZP)     : Moltiplicatore
;
; Output:
;   - MUL_RES+0 .. MUL_RES+5 (6 byte, ZP) : Prodotto a 48 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       MUL_RES+0 .. MUL_RES+5
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.6.14 m_Mul_Core_32x24

```
; ================================================================================
; MACRO: m_Mul_Core_32x24
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplicazione senza segno tra un valore a 32 bit
;               (moltiplicando) e uno a 24 bit (moltiplicatore) con il nucleo
;               generico m_Mul_Core.
;
; Input:
;   - MUL_1+0 .. MUL_1+3 (4 byte, ZP)     : Moltiplicando
;   - MUL_2+0 .. MUL_2+2 (3 byte, ZP)     : Moltiplicatore
;
; Output:
;   - MUL_RES+0 .. MUL_RES+6 (7 byte, ZP) : Prodotto a 56 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       MUL_RES+0 .. MUL_RES+6
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.6.15 m_Mul_Core_32x32

```
; ================================================================================
; MACRO: m_Mul_Core_32x32
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplicazione senza segno tra due valori a 32 bit con il
;               nucleo generico m_Mul_Core.
;
; Input:
;   - MUL_1+0 .. MUL_1+3 (4 byte, ZP)     : Moltiplicando
;   - MUL_2+0 .. MUL_2+3 (4 byte, ZP)     : Moltiplicatore
;
; Output:
;   - MUL_RES+0 .. MUL_RES+7 (8 byte, ZP) : Prodotto a 64 bit
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       MUL_RES+0 .. MUL_RES+7
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.6.16 MUL_8X8

```
; ================================================================================
; SUBROUTINE: MUL_8X8
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplica due interi a 8 bit, senza segno.
;
; Input:
;   - .A (1 byte, CPU)                    : Primo operando (moltiplicando)
;   - .X (1 byte, CPU)                    : Secondo operando (moltiplicatore)
;
; Output:
;   - .A (1 byte, CPU)                    : Byte 0 del prodotto
;   - .X (1 byte, CPU)                    : Byte 1 del prodotto
;   - MUL_RES+0 .. MUL_RES+1 (2 byte, ZP) : Prodotto a 16 bit
;   - Flag Carry (1 bit, .P)              : sempre 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .P (Negative, oVerflow, Zero, Carry), MUL_1+0,
;                       MUL_RES+0 .. MUL_RES+1
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.6.17 SMUL_8X8

```
; ================================================================================
; SUBROUTINE: SMUL_8X8
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplica due interi a 8 bit, con segno.
;
;               Rende positivi gli operandi, esegue il nucleo senza segno e, se
;               i segni erano diversi, nega il prodotto.
;
; Input:
;   - .A (1 byte, CPU)                   : Primo operando con segno
;                                          (moltiplicando)
;   - .X (1 byte, CPU)                   : Secondo operando con segno
;                                          (moltiplicatore)
;
; Output:
;   - .A (1 byte, CPU)                   : Byte 0 del prodotto
;   - .X (1 byte, CPU)                   : Byte 1 del prodotto
;   - MUL_RES+0 .. MUL_RES+1 (2 byte, ZP) : Prodotto a 16 bit con segno
;   - Flag Carry (1 bit, .P)             : sempre 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .P (Negative, oVerflow, Zero, Carry), MUL_1+0,
;                       MUL_RES+0 .. MUL_RES+1
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.6.18 MUL_16X8

```
; ================================================================================
; SUBROUTINE: MUL_16X8
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplica un intero a 16 bit per uno a 8 bit, senza segno.
;
; Input:
;   - .A (1 byte, CPU)                   : Primo operando, byte basso
;   - .X (1 byte, CPU)                   : Primo operando, byte alto
;   - .Y (1 byte, CPU)                   : Secondo operando (moltiplicatore)
;
; Output:
;   - .A (1 byte, CPU)                   : Byte 0 del prodotto
;   - .X (1 byte, CPU)                   : Byte 1 del prodotto
;   - .Y (1 byte, CPU)                   : Byte 2 del prodotto
;   - MUL_RES+0 .. MUL_RES+2 (3 byte, ZP) : Prodotto a 24 bit
;   - Flag Carry (1 bit, .P)             : sempre 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       MUL_1+0 .. MUL_1+1, MUL_RES+0 .. MUL_RES+2
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.6.19 SMUL_16X8

```
; ============================================================================
; SUBROUTINE: SMUL_16X8
; ----------------------------------------------------------------------------
; Descrizione:  Moltiplica un intero a 16 bit per uno a 8 bit, con segno.
;
;               Rende positivi gli operandi, esegue il nucleo senza segno e, se
;               i segni erano diversi, nega il prodotto.
;
; Input:
;   - .A (1 byte, CPU)                    : Primo operando con segno, byte basso
;   - .X (1 byte, CPU)                    : Primo operando con segno, byte alto
;   - .Y (1 byte, CPU)                    : Secondo operando con segno
;                                           (moltiplicatore)
;
; Output:
;   - .A (1 byte, CPU)                    : Byte 0 del prodotto
;   - .X (1 byte, CPU)                    : Byte 1 del prodotto
;   - .Y (1 byte, CPU)                    : Byte 2 del prodotto
;   - MUL_RES+0 .. MUL_RES+2 (3 byte, ZP) : Prodotto a 24 bit con segno
;   - Flag Carry (1 bit, .P)              : sempre 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       MUL_1+0 .. MUL_1+1, MUL_RES+0 .. MUL_RES+2
; Dipendenze Esterne:   <c64/symbols.asm>
; ============================================================================
```

4.6.20 MUL_24X8

```
; ================================================================================
; SUBROUTINE: MUL_24X8
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplica un intero a 24 bit per uno a 8 bit, senza segno.
;
; Input:
;   - MUL_1+0 .. MUL_1+2 (3 byte, ZP)     : Primo operando (moltiplicando)
;   - .A (1 byte, CPU)                    : Secondo operando (moltiplicatore)
;
; Output:
;   - .A (1 byte, CPU)                    : Byte 0 del prodotto
;   - .X (1 byte, CPU)                    : Byte 1 del prodotto
;   - .Y (1 byte, CPU)                    : Byte 2 del prodotto
;   - MUL_RES+0 .. MUL_RES+3 (4 byte, ZP) : Prodotto a 32 bit
;   - Flag Carry (1 bit, .P)              : sempre 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       MUL_RES+0 .. MUL_RES+3
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.6.21 SMUL_24X8

```
; ================================================================================
; SUBROUTINE: SMUL_24X8
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplica un intero a 24 bit per uno a 8 bit, con segno.
;
;               Rende positivi gli operandi, esegue il nucleo senza segno e, se
;               i segni erano diversi, nega il prodotto.
;
; Input:
;   - MUL_1+0 .. MUL_1+2 (3 byte, ZP)     : Primo operando con segno
;                                           (moltiplicando)
;   - .A (1 byte, CPU)                    : Secondo operando con segno
;                                           (moltiplicatore)
;
; Output:
;   - .A (1 byte, CPU)                    : Byte 0 del prodotto
;   - .X (1 byte, CPU)                    : Byte 1 del prodotto
;   - .Y (1 byte, CPU)                    : Byte 2 del prodotto
;   - MUL_RES+0 .. MUL_RES+3 (4 byte, ZP) : Prodotto a 32 bit con segno
;   - Flag Carry (1 bit, .P)              : sempre 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       MUL_1+0 .. MUL_1+2, MUL_RES+0 .. MUL_RES+3
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.6.22 MUL_32X8

```
; ================================================================================
; SUBROUTINE: MUL_32X8
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplica un intero a 32 bit per uno a 8 bit, senza segno.
;
; Input:
;   - MUL_1+0 .. MUL_1+3 (4 byte, ZP)     : Primo operando (moltiplicando)
;   - .A (1 byte, CPU)                    : Secondo operando (moltiplicatore)
;
; Output:
;   - .A (1 byte, CPU)                    : Byte 0 del prodotto
;   - .X (1 byte, CPU)                    : Byte 1 del prodotto
;   - .Y (1 byte, CPU)                    : Byte 2 del prodotto
;   - MUL_RES+0 .. MUL_RES+4 (5 byte, ZP) : Prodotto a 40 bit
;   - Flag Carry (1 bit, .P)              : sempre 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       MUL_RES+0 .. MUL_RES+4
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.6.23 SMUL_32X8

```
; ================================================================================
; SUBROUTINE: SMUL_32X8
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplica un intero a 32 bit per uno a 8 bit, con segno.
;
;               Rende positivi gli operandi, esegue il nucleo senza segno e, se
;               i segni erano diversi, nega il prodotto.
;
; Input:
;   - MUL_1+0 .. MUL_1+3 (4 byte, ZP)     : Primo operando con segno
;                                           (moltiplicando)
;   - .A (1 byte, CPU)                    : Secondo operando con segno
;                                           (moltiplicatore)
;
; Output:
;   - .A (1 byte, CPU)                    : Byte 0 del prodotto
;   - .X (1 byte, CPU)                    : Byte 1 del prodotto
;   - .Y (1 byte, CPU)                    : Byte 2 del prodotto
;   - MUL_RES+0 .. MUL_RES+4 (5 byte, ZP) : Prodotto a 40 bit con segno
;   - Flag Carry (1 bit, .P)              : sempre 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       MUL_1+0 .. MUL_1+3, MUL_RES+0 .. MUL_RES+4
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.6.24 MUL_16X16

```
; ================================================================================
; SUBROUTINE: MUL_16X16
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplica due interi a 16 bit, senza segno.
;
; Input:
;   - MUL_1+0 .. MUL_1+1 (2 byte, ZP)     : Primo operando (moltiplicando)
;   - .A (1 byte, CPU)                    : Secondo operando, byte 0
;   - .X (1 byte, CPU)                    : Secondo operando, byte 1
;
; Output:
;   - .A (1 byte, CPU)                    : Byte 0 del prodotto
;   - .X (1 byte, CPU)                    : Byte 1 del prodotto
;   - .Y (1 byte, CPU)                    : Byte 2 del prodotto
;   - MUL_RES+0 .. MUL_RES+3 (4 byte, ZP) : Prodotto a 32 bit
;   - Flag Carry (1 bit, .P)              : sempre 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       MUL_RES+0 .. MUL_RES+3
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.6.25 SMUL_16X16

```
; ================================================================================
; SUBROUTINE: SMUL_16X16
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplica due interi a 16 bit, con segno.
;
;               Rende positivi gli operandi, esegue il nucleo senza segno e, se
;               i segni erano diversi, nega il prodotto.
;
; Input:
;   - MUL_1+0 .. MUL_1+1 (2 byte, ZP)     : Primo operando con segno
;                                           (moltiplicando)
;   - .A (1 byte, CPU)                    : Secondo operando con segno, byte 0
;   - .X (1 byte, CPU)                    : Secondo operando con segno, byte 1
;
; Output:
;   - .A (1 byte, CPU)                    : Byte 0 del prodotto
;   - .X (1 byte, CPU)                    : Byte 1 del prodotto
;   - .Y (1 byte, CPU)                    : Byte 2 del prodotto
;   - MUL_RES+0 .. MUL_RES+3 (4 byte, ZP) : Prodotto a 32 bit con segno
;   - Flag Carry (1 bit, .P)              : sempre 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       MUL_1+0 .. MUL_1+1, MUL_RES+0 .. MUL_RES+3
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.6.26 MUL_24X16

```
; ================================================================================
; SUBROUTINE: MUL_24X16
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplica un intero a 24 bit per uno a 16 bit, senza segno.
;
; Input:
;   - MUL_1+0 .. MUL_1+2 (3 byte, ZP)     : Primo operando (moltiplicando)
;   - .A (1 byte, CPU)                    : Secondo operando, byte 0
;   - .X (1 byte, CPU)                    : Secondo operando, byte 1
;
; Output:
;   - .A (1 byte, CPU)                    : Byte 0 del prodotto
;   - .X (1 byte, CPU)                    : Byte 1 del prodotto
;   - .Y (1 byte, CPU)                    : Byte 2 del prodotto
;   - MUL_RES+0 .. MUL_RES+4 (5 byte, ZP) : Prodotto a 40 bit
;   - Flag Carry (1 bit, .P)              : sempre 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       MUL_RES+0 .. MUL_RES+4
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.6.27 SMUL_24X16

```
; ================================================================================
; SUBROUTINE: SMUL_24X16
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplica un intero a 24 bit per uno a 16 bit, con segno.
;
;               Rende positivi gli operandi, esegue il nucleo senza segno e, se
;               i segni erano diversi, nega il prodotto.
;
; Input:
;   - MUL_1+0 .. MUL_1+2 (3 byte, ZP)     : Primo operando con segno
;                                           (moltiplicando)
;   - .A (1 byte, CPU)                    : Secondo operando con segno, byte 0
;   - .X (1 byte, CPU)                    : Secondo operando con segno, byte 1
;
; Output:
;   - .A (1 byte, CPU)                    : Byte 0 del prodotto
;   - .X (1 byte, CPU)                    : Byte 1 del prodotto
;   - .Y (1 byte, CPU)                    : Byte 2 del prodotto
;   - MUL_RES+0 .. MUL_RES+4 (5 byte, ZP) : Prodotto a 40 bit con segno
;   - Flag Carry (1 bit, .P)              : sempre 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       MUL_1+0 .. MUL_1+2, MUL_RES+0 .. MUL_RES+4
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.6.28 MUL_32X16

```
; ================================================================================
; SUBROUTINE: MUL_32X16
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplica un intero a 32 bit per uno a 16 bit, senza segno.
;
; Input:
;   - MUL_1+0 .. MUL_1+3 (4 byte, ZP)     : Primo operando (moltiplicando)
;   - .A (1 byte, CPU)                    : Secondo operando, byte 0
;   - .X (1 byte, CPU)                    : Secondo operando, byte 1
;
; Output:
;   - .A (1 byte, CPU)                    : Byte 0 del prodotto
;   - .X (1 byte, CPU)                    : Byte 1 del prodotto
;   - .Y (1 byte, CPU)                    : Byte 2 del prodotto
;   - MUL_RES+0 .. MUL_RES+5 (6 byte, ZP) : Prodotto a 48 bit
;   - Flag Carry (1 bit, .P)              : sempre 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       MUL_RES+0 .. MUL_RES+5
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.6.29 SMUL_32X16

```
; ================================================================================
; SUBROUTINE: SMUL_32X16
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplica un intero a 32 bit per uno a 16 bit, con segno.
;
;               Rende positivi gli operandi, esegue il nucleo senza segno e, se
;               i segni erano diversi, nega il prodotto.
;
; Input:
;   - MUL_1+0 .. MUL_1+3 (4 byte, ZP)     : Primo operando con segno
;                                           (moltiplicando)
;   - .A (1 byte, CPU)                    : Secondo operando con segno, byte 0
;   - .X (1 byte, CPU)                    : Secondo operando con segno, byte 1
;
; Output:
;   - .A (1 byte, CPU)                    : Byte 0 del prodotto
;   - .X (1 byte, CPU)                    : Byte 1 del prodotto
;   - .Y (1 byte, CPU)                    : Byte 2 del prodotto
;   - MUL_RES+0 .. MUL_RES+5 (6 byte, ZP) : Prodotto a 48 bit con segno
;   - Flag Carry (1 bit, .P)              : sempre 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       MUL_1+0 .. MUL_1+3, MUL_RES+0 .. MUL_RES+5
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.6.30 MUL_24X24

```
; ================================================================================
; SUBROUTINE: MUL_24X24
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplica due interi a 24 bit, senza segno.
;
; Input:
;   - MUL_1+0 .. MUL_1+2 (3 byte, ZP)     : Primo operando (moltiplicando)
;   - .A (1 byte, CPU)                    : Secondo operando, byte 0
;   - .X (1 byte, CPU)                    : Secondo operando, byte 1
;   - .Y (1 byte, CPU)                    : Secondo operando, byte 2
;
; Output:
;   - .A (1 byte, CPU)                    : Byte 0 del prodotto
;   - .X (1 byte, CPU)                    : Byte 1 del prodotto
;   - .Y (1 byte, CPU)                    : Byte 2 del prodotto
;   - MUL_RES+0 .. MUL_RES+5 (6 byte, ZP) : Prodotto a 48 bit
;   - Flag Carry (1 bit, .P)              : sempre 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       MUL_RES+0 .. MUL_RES+5
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.6.31 SMUL_24X24

```
; ================================================================================
; SUBROUTINE: SMUL_24X24
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplica due interi a 24 bit, con segno.
;
;               Rende positivi gli operandi, esegue il nucleo senza segno e, se
;               i segni erano diversi, nega il prodotto.
;
; Input:
;   - MUL_1+0 .. MUL_1+2 (3 byte, ZP)     : Primo operando con segno
;                                           (moltiplicando)
;   - .A (1 byte, CPU)                    : Secondo operando con segno, byte 0
;   - .X (1 byte, CPU)                    : Secondo operando con segno, byte 1
;   - .Y (1 byte, CPU)                    : Secondo operando con segno, byte 2
;
; Output:
;   - .A (1 byte, CPU)                    : Byte 0 del prodotto
;   - .X (1 byte, CPU)                    : Byte 1 del prodotto
;   - .Y (1 byte, CPU)                    : Byte 2 del prodotto
;   - MUL_RES+0 .. MUL_RES+5 (6 byte, ZP) : Prodotto a 48 bit con segno
;   - Flag Carry (1 bit, .P)              : sempre 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       MUL_1+0 .. MUL_1+2, MUL_RES+0 .. MUL_RES+5
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.6.32 MUL_32X24

```
; ================================================================================
; SUBROUTINE: MUL_32X24
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplica un intero a 32 bit per uno a 24 bit, senza segno.
;
; Input:
;   - MUL_1+0 .. MUL_1+3 (4 byte, ZP)     : Primo operando (moltiplicando)
;   - .A (1 byte, CPU)                    : Secondo operando, byte 0
;   - .X (1 byte, CPU)                    : Secondo operando, byte 1
;   - .Y (1 byte, CPU)                    : Secondo operando, byte 2
;
; Output:
;   - .A (1 byte, CPU)                    : Byte 0 del prodotto
;   - .X (1 byte, CPU)                    : Byte 1 del prodotto
;   - .Y (1 byte, CPU)                    : Byte 2 del prodotto
;   - MUL_RES+0 .. MUL_RES+6 (7 byte, ZP) : Prodotto a 56 bit
;   - Flag Carry (1 bit, .P)              : sempre 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       MUL_RES+0 .. MUL_RES+6
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.6.33 SMUL_32X24

```
; ================================================================================
; SUBROUTINE: SMUL_32X24
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplica un intero a 32 bit per uno a 24 bit, con segno.
;
;               Rende positivi gli operandi, esegue il nucleo senza segno e, se
;               i segni erano diversi, nega il prodotto.
;
; Input:
;   - MUL_1+0 .. MUL_1+3 (4 byte, ZP)     : Primo operando con segno
;                                           (moltiplicando)
;   - .A (1 byte, CPU)                    : Secondo operando con segno, byte 0
;   - .X (1 byte, CPU)                    : Secondo operando con segno, byte 1
;   - .Y (1 byte, CPU)                    : Secondo operando con segno, byte 2
;
; Output:
;   - .A (1 byte, CPU)                    : Byte 0 del prodotto
;   - .X (1 byte, CPU)                    : Byte 1 del prodotto
;   - .Y (1 byte, CPU)                    : Byte 2 del prodotto
;   - MUL_RES+0 .. MUL_RES+6 (7 byte, ZP) : Prodotto a 56 bit con segno
;   - Flag Carry (1 bit, .P)              : sempre 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       MUL_1+0 .. MUL_1+3, MUL_RES+0 .. MUL_RES+6
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.6.34 MUL_32X32

```
; ================================================================================
; SUBROUTINE: MUL_32X32
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplica due interi a 32 bit, senza segno.
;
; Input:
;   - MUL_1+0 .. MUL_1+3 (4 byte, ZP)     : Primo operando (moltiplicando)
;   - MUL_2+0 .. MUL_2+3 (4 byte, ZP)     : Secondo operando (moltiplicatore)
;
; Output:
;   - .A (1 byte, CPU)                    : Byte 0 del prodotto
;   - .X (1 byte, CPU)                    : Byte 1 del prodotto
;   - .Y (1 byte, CPU)                    : Byte 2 del prodotto
;   - MUL_RES+0 .. MUL_RES+7 (8 byte, ZP) : Prodotto a 64 bit
;   - Flag Carry (1 bit, .P)              : sempre 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       MUL_RES+0 .. MUL_RES+7
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

4.6.35 SMUL_32X32

```
; ================================================================================
; SUBROUTINE: SMUL_32X32
; --------------------------------------------------------------------------------
; Descrizione:  Moltiplica due interi a 32 bit, con segno.
;
;               Rende positivi gli operandi, esegue il nucleo senza segno e, se
;               i segni erano diversi, nega il prodotto.
;
; Input:
;   - MUL_1+0 .. MUL_1+3 (4 byte, ZP)     : Primo operando con segno
;                                           (moltiplicando)
;   - MUL_2+0 .. MUL_2+3 (4 byte, ZP)     : Secondo operando con segno
;                                           (moltiplicatore)
;
; Output:
;   - .A (1 byte, CPU)                    : Byte 0 del prodotto
;   - .X (1 byte, CPU)                    : Byte 1 del prodotto
;   - .Y (1 byte, CPU)                    : Byte 2 del prodotto
;   - MUL_RES+0 .. MUL_RES+7 (8 byte, ZP) : Prodotto a 64 bit con segno
;   - Flag Carry (1 bit, .P)              : sempre 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry),
;                       MUL_1+0 .. MUL_1+3, MUL_RES+0 .. MUL_RES+7
; Dipendenze Esterne:   <c64/symbols.asm>
; ================================================================================
```

Libreria

mem

5 Libreria mem

Modulo	Descrizione
`mem16.asm`	Macro per la manipolazione di valori a 16 bit.
`mem24.asm`	Macro per la manipolazione di valori a 24 bit.
`mem32.asm`	Macro per la manipolazione di valori a 32 bit.

5.1 mem16.asm

5.1.1 Descrizione

Macro per la manipolazione di valori a 16 bit.

Utilità	Descrizione
`m_Mov16`	Sposta un valore o un puntatore a 16 bit
`m_Swap16`	Scambia due puntatori a 16 bit
`m_CmpU16`	Confronta due valori a 16 bit senza segno
`m_CmpS16`	Confronta due valori a 16 bit con segno
`m_Inc16`	Incrementa un contatore a 16 bit
`m_Dec16`	Decrementa un contatore a 16 bit

5.1.2 Note d'uso

Nessuna nota in questo modulo.

5.1.3 m_Mov16

```
; ================================================================================
; MACRO: m_Mov16
; --------------------------------------------------------------------------------
; Descrizione:  Copia un valore costante o un puntatore a 16 bit dalla sorgente
;               alla destinazione, secondo l'indirizzamento scelto.
;
;                 "IMM"  Immediato: .src_ è un valore a 16 bit e .dest_ è un
;                        indirizzo o una coppia di registri.
;                 "ABS"  Assoluto: .src_ e .dest_ sono indirizzi assoluti; uno
;                        dei due può essere una coppia di registri.
;                 "ZP"   Pagina Zero: .src_ e .dest_ sono indirizzi in pagina
;                        zero; uno dei due può essere una coppia di registri.
;
;               Le coppie di registri si scrivono ".<hi><lo>" (byte alto e byte
;               basso): .AX, .AY, .XY, .XA, .YA e .YX.
;
;               Con "IMM" una costante con i due byte uguali (per esempio 0,
;               $FFFF o $4242) usa una sequenza più breve. Con "ABS" e "ZP", da
;               indirizzo a indirizzo, la copia procede nel verso che non altera
;               il sorgente se le due aree si sovrappongono.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro) : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .src_ (2 byte, Macro)   : Valore (IMM), indirizzo o coppia di registri
;   - .dest_ (2 byte, Macro)  : Indirizzo o coppia di registri
;
; Output:
;   - .dest_ (2 byte, Macro)  : Riceve il valore di .src_
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, Zero)
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

5.1.4 m_Swap16

```
; ================================================================================
; MACRO: m_Swap16
; --------------------------------------------------------------------------------
; Descrizione:  Scambia il valore di due puntatori a 16 bit usando i registri.
;
; Input:
;   - .addr1_ (2 byte, Macro) : Indirizzo del primo valore
;   - .addr2_ (2 byte, Macro) : Indirizzo del secondo valore
;
; Output:
;   - .addr1_ (2 byte, Macro) : Riceve il valore di .addr2_
;   - .addr2_ (2 byte, Macro) : Riceve il valore di .addr1_
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .X, .Y, .P (Negative, Zero)
; Dipendenze Esterne:   <system/standard.asm>
; ================================================================================
```

5.1.5 m_CmpU16

```
; ============================================================================
; MACRO: m_CmpU16
; ----------------------------------------------------------------------------
; Descrizione:  Confronta due valori a 16 bit senza segno, secondo
;               l'indirizzamento scelto. Il risultato è nei flag, come con
;               l'istruzione CMP.
;
;                 "IMM"  Immediato: .op1_ è un indirizzo o una coppia di
;                        registri, .op2_ è un valore a 16 bit.
;                 "ABS"  Assoluto: gli operandi sono indirizzi assoluti; .op1_
;                        può essere una coppia di registri.
;                 "ZP"   Pagina Zero: gli operandi sono indirizzi in pagina
;                        zero; .op1_ può essere una coppia di registri.
;
;               Le coppie di registri si scrivono ".<hi><lo>" (byte alto e byte
;               basso): .AX, .AY, .XY, .XA, .YA e .YX. Con una coppia di
;               registri come .op1_ nessun registro viene alterato.
;
;               Con "IMM" e .op1_ indirizzo, i confronti con 0 e con $FFFF usano
;               una sequenza più breve.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro) : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .op1_ (2 byte, Macro)   : Primo operando: indirizzo o coppia di registri
;   - .op2_ (2 byte, Macro)   : Secondo operando: valore (IMM) o indirizzo
;
; Output:
;   - Flag Zero (1 bit, .P)   : 0 = .op1_ diverso da .op2_; 1 = uguali
;   - Flag Carry (1 bit, .P)  : 0 = .op1_ < .op2_; 1 = .op1_ >= .op2_ (senza
;                               segno)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero, Carry)
; Dipendenze Esterne:   Nessuna
; ============================================================================
```

5.1.6 m_CmpS16

```
; ==============================================================================
; MACRO: m_CmpS16
; ------------------------------------------------------------------------------
; Descrizione:  Confronta due valori a 16 bit con segno, secondo
;               l'indirizzamento scelto. Il flag di Carry si comporta come con
;               l'istruzione CMP (senza segno); con segno vale N xor V = 1 se
;               .op1_ < .op2_.
;
;                 "IMM"  Immediato: .op1_ è un indirizzo o una coppia di
;                        registri, .op2_ è un valore a 16 bit.
;                 "ABS"  Assoluto: gli operandi sono indirizzi assoluti; .op1_
;                        può essere una coppia di registri.
;                 "ZP"   Pagina Zero: gli operandi sono indirizzi in pagina
;                        zero; .op1_ può essere una coppia di registri.
;
;               Le coppie di registri si scrivono ".<hi><lo>" (byte alto e byte
;               basso): .AX, .AY, .XY, .XA, .YA e .YX.
;
;               Con "IMM" e .op1_ indirizzo o coppia di registri, il confronto
;               con 0 usa una sequenza più breve a parità di flag. Il flag di
;               Zero riflette solo il byte alto del risultato: per verificare
;               l'uguaglianza si usa m_CmpU16.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .op1_ (2 byte, Macro)     : Primo operando: indirizzo o coppia di registri
;   - .op2_ (2 byte, Macro)     : Secondo operando: valore (IMM) o indirizzo
;
; Output:
;   - Flag Negative (1 bit, .P) : 0 = bit #7 a 0; 1 = bit #7 a 1
;   - Flag oVerflow (1 bit, .P) : 0 = nessun overflow con segno; 1 = overflow
;                                 con segno
;   - Flag Zero (1 bit, .P)     : 0 = byte alto diverso da 0; 1 = uguale a 0
;   - Flag Carry (1 bit, .P)    : 0 = .op1_ < .op2_; 1 = .op1_ >= .op2_ (senza
;                                 segno)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   Nessuna
; ==============================================================================
```

5.1.7 m_Inc16

```
; ================================================================================
; MACRO: m_Inc16
; --------------------------------------------------------------------------------
; Descrizione:  Incrementa di 1 un contatore a 16 bit contenuto in un puntatore
;               o in una coppia di registri.
;
;               Le coppie di registri si scrivono ".<hi><lo>" (byte alto e byte
;               basso): .AX, .AY, .XY, .XA, .YA e .YX.
;
;               Con un indirizzo non viene alterato nessun registro.
;
;               I flag Carry, Negative e oVerflow in uscita sono indefiniti.
;
;               L'operando deve essere definito prima dell'invocazione, perché
;               la macro lo confronta per scegliere il ramo.
;
; Input:
;   - .addr_ (2 byte, Macro)  : Contatore: indirizzo o coppia di registri
;
; Output:
;   - .addr_ (2 byte, Macro)  : Contatore incrementato (da $FFFF a $0000)
;   - Flag Zero (1 bit, .P)   : 0 = risultato diverso da 0; 1 = risultato 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

5.1.8 m_Dec16

```
; ============================================================================
; MACRO: m_Dec16
; ----------------------------------------------------------------------------
; Descrizione:  Decrementa di 1 un contatore a 16 bit contenuto in un puntatore
;               o in una coppia di registri.
;
;               Le coppie di registri si scrivono ".<hi><lo>" (byte alto e byte
;               basso): .AX, .AY, .XY, .XA, .YA e .YX.
;
;               Con un indirizzo viene alterato .A; con una coppia di
;               registri solo i due registri della coppia.
;
;               I flag Carry, Negative e oVerflow in uscita sono indefiniti.
;
;               L'operando deve essere definito prima dell'invocazione, perché
;               la macro lo confronta per scegliere il ramo.
;
; Input:
;   - .addr_ (2 byte, Macro)  : Contatore: indirizzo o coppia di registri
;
; Output:
;   - .addr_ (2 byte, Macro)  : Contatore decrementato (da $0000 a $FFFF)
;   - Flag Zero (1 bit, .P)   : 0 = byte basso diverso da 0; 1 = byte basso 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   Nessuna
; ============================================================================
```

5.2 mem24.asm

5.2.1 Descrizione

Macro per la manipolazione di valori a 24 bit.

Utilità	Descrizione
`m_Mov24`	Sposta un valore o un puntatore a 24 bit
`m_Swap24`	Scambia due puntatori a 24 bit
`m_CmpU24`	Confronta due valori a 24 bit senza segno
`m_CmpS24`	Confronta due valori a 24 bit con segno
`m_Inc24`	Incrementa un contatore a 24 bit
`m_Dec24`	Decrementa un contatore a 24 bit

5.2.2 Note d'uso

Nessuna nota in questo modulo.

5.2.3 m_Mov24

```
; ================================================================================
; MACRO: m_Mov24
; --------------------------------------------------------------------------------
; Descrizione:  Copia un valore costante o un puntatore a 24 bit dalla sorgente
;               alla destinazione, secondo l'indirizzamento scelto.
;
;                 "IMM"  Immediato: .src_ è un valore a 24 bit e .dest_ è un
;                        indirizzo o una terna di registri.
;                 "ABS"  Assoluto: .src_ e .dest_ sono indirizzi assoluti; uno
;                        dei due può essere una terna di registri.
;                 "ZP"   Pagina Zero: .src_ e .dest_ sono indirizzi in pagina
;                        zero; uno dei due può essere una terna di registri.
;
;               Le terne di registri si scrivono ".<hi><mid><lo>" (byte alto,
;               intermedio e basso): .AXY e .YXA.
;
;               Con "IMM" una costante con i tre byte uguali (per esempio 0,
;               $FFFFFF o $424242) usa una sequenza più breve. Con "ABS" e "ZP",
;               da indirizzo a indirizzo, la copia procede nel verso che non
;               altera il sorgente se le due aree si sovrappongono.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro) : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .src_ (3 byte, Macro)   : Valore (IMM), indirizzo o terna di registri
;   - .dest_ (3 byte, Macro)  : Indirizzo o terna di registri
;
; Output:
;   - .dest_ (3 byte, Macro)  : Riceve il valore di .src_
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, Zero)
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

5.2.4 m_Swap24

```
; ================================================================================
; MACRO: m_Swap24
; --------------------------------------------------------------------------------
; Descrizione:  Scambia il valore di due puntatori a 24 bit usando i registri.
;
; Input:
;   - .addr1_ (3 byte, Macro) : Indirizzo del primo valore
;   - .addr2_ (3 byte, Macro) : Indirizzo del secondo valore
;
; Output:
;   - .addr1_ (3 byte, Macro) : Riceve il valore di .addr2_
;   - .addr2_ (3 byte, Macro) : Riceve il valore di .addr1_
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .X, .Y, .P (Negative, Zero)
; Dipendenze Esterne:   <system/standard.asm>
; ================================================================================
```

5.2.5 m_CmpU24

```
; ================================================================================
; MACRO: m_CmpU24
; --------------------------------------------------------------------------------
; Descrizione:  Confronta due valori a 24 bit senza segno, secondo
;               l'indirizzamento scelto. Il risultato è nei flag, come con
;               l'istruzione CMP.
;
;                 "IMM"  Immediato: .op1_ è un indirizzo o una terna di
;                        registri, .op2_ è un valore a 24 bit.
;                 "ABS"  Assoluto: gli operandi sono indirizzi assoluti; .op1_
;                        può essere una terna di registri.
;                 "ZP"   Pagina Zero: gli operandi sono indirizzi in pagina
;                        zero; .op1_ può essere una terna di registri.
;
;               Le terne di registri si scrivono ".<hi><mid><lo>" (byte alto,
;               intermedio e basso): .AXY e .YXA. Con una terna di registri come
;               .op1_ nessun registro viene alterato.
;
;               Con "IMM" e .op1_ indirizzo, i confronti con 0 e con $FFFFFF
;               usano una sequenza più breve.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro) : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .op1_ (3 byte, Macro)   : Primo operando: indirizzo o terna di registri
;   - .op2_ (3 byte, Macro)   : Secondo operando: valore (IMM) o indirizzo
;
; Output:
;   - Flag Zero (1 bit, .P)   : 0 = .op1_ diverso da .op2_; 1 = uguali
;   - Flag Carry (1 bit, .P)  : 0 = .op1_ < .op2_; 1 = .op1_ >= .op2_ (senza
;                               segno)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero, Carry)
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

5.2.6 m_CmpS24

```
; ==============================================================================
; MACRO: m_CmpS24
; ------------------------------------------------------------------------------
; Descrizione:  Confronta due valori a 24 bit con segno, secondo
;               l'indirizzamento scelto. Il flag di Carry si comporta come con
;               l'istruzione CMP (senza segno); con segno vale N xor V = 1 se
;               .op1_ < .op2_.
;
;                 "IMM"  Immediato: .op1_ è un indirizzo o una terna di
;                        registri, .op2_ è un valore a 24 bit.
;                 "ABS"  Assoluto: gli operandi sono indirizzi assoluti; .op1_
;                        può essere una terna di registri.
;                 "ZP"   Pagina Zero: gli operandi sono indirizzi in pagina
;                        zero; .op1_ può essere una terna di registri.
;
;               Le terne di registri si scrivono ".<hi><mid><lo>" (byte alto,
;               intermedio e basso): .AXY e .YXA. Con .AXY come .op1_ la macro
;               usa temporaneamente un byte dello stack e lo rilascia prima di
;               terminare.
;
;               Con "IMM" il confronto con 0 (indirizzo o terna di registri) e
;               con $FFFFFF (indirizzo o .AXY) usa una sequenza più breve a
;               parità di flag. Il flag di Zero riflette solo il byte alto del
;               risultato: per verificare l'uguaglianza si usa m_CmpU24.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .op1_ (3 byte, Macro)     : Primo operando: indirizzo o terna di registri
;   - .op2_ (3 byte, Macro)     : Secondo operando: valore (IMM) o indirizzo
;
; Output:
;   - Flag Negative (1 bit, .P) : 0 = bit #7 a 0; 1 = bit #7 a 1
;   - Flag oVerflow (1 bit, .P) : 0 = nessun overflow con segno; 1 = overflow
;                                 con segno
;   - Flag Zero (1 bit, .P)     : 0 = byte alto diverso da 0; 1 = uguale a 0
;   - Flag Carry (1 bit, .P)    : 0 = .op1_ < .op2_; 1 = .op1_ >= .op2_ (senza
;                                 segno)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   Nessuna
; ==============================================================================
```

5.2.7 m_Inc24

```
; ================================================================================
; MACRO: m_Inc24
; --------------------------------------------------------------------------------
; Descrizione:  Incrementa di 1 un contatore a 24 bit contenuto in un puntatore
;               o in una terna di registri.
;
;               Le terne di registri si scrivono ".<hi><mid><lo>" (byte alto,
;               intermedio e basso): .AXY e .YXA. Con un indirizzo non viene
;               alterato nessun registro.
;
;               I flag Carry, Negative e oVerflow in uscita sono indefiniti.
;
;               L'operando deve essere definito prima dell'invocazione, perché
;               la macro lo confronta per scegliere il ramo.
;
; Input:
;   - .addr_ (3 byte, Macro) : Contatore: indirizzo o terna di registri
;
; Output:
;   - .addr_ (3 byte, Macro) : Contatore incrementato (da $FFFFFF a $000000)
;   - Flag Zero (1 bit, .P)  : 0 = risultato diverso da 0; 1 = risultato 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

5.2.8 m_Dec24

```
; ==============================================================================
; MACRO: m_Dec24
; ------------------------------------------------------------------------------
; Descrizione:  Decrementa di 1 un contatore a 24 bit contenuto in un puntatore
;               o in una terna di registri.
;
;               Le terne di registri si scrivono ".<hi><mid><lo>" (byte alto,
;               intermedio e basso): .AXY e .YXA. Con un indirizzo viene
;               alterato .A; con una terna di registri solo i tre registri della
;               terna.
;
;               I flag Carry, Negative e oVerflow in uscita sono indefiniti.
;
;               L'operando deve essere definito prima dell'invocazione, perché
;               la macro lo confronta per scegliere il ramo.
;
; Input:
;   - .addr_ (3 byte, Macro) : Contatore: indirizzo o terna di registri
;
; Output:
;   - .addr_ (3 byte, Macro) : Contatore decrementato (da $000000 a $FFFFFF)
;   - Flag Zero (1 bit, .P)  : 0 = byte basso diverso da 0; 1 = byte basso 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .X, .Y, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   Nessuna
; ==============================================================================
```

5.3 mem32.asm

5.3.1 Descrizione

Macro per la manipolazione di valori a 32 bit.

Utilità	Descrizione
`m_Mov32`	Sposta un valore o un puntatore a 32 bit
`m_Swap32`	Scambia due puntatori a 32 bit
`m_CmpU32`	Confronta due valori a 32 bit senza segno
`m_CmpS32`	Confronta due valori a 32 bit con segno
`m_Inc32`	Incrementa un contatore a 32 bit
`m_Dec32`	Decrementa un contatore a 32 bit

5.3.2 Note d'uso

Nessuna nota in questo modulo.

5.3.3 m_Mov32

```
; ================================================================================
; MACRO: m_Mov32
; --------------------------------------------------------------------------------
; Descrizione:  Copia un valore costante o un puntatore a 32 bit dalla sorgente
;               alla destinazione, secondo l'indirizzamento scelto. A 32 bit non
;               sono previsti gruppi di registri: gli operandi sono indirizzi.
;
;                 "IMM"  Immediato: .src_ è un valore a 32 bit e .dest_ è un
;                        indirizzo.
;                 "ABS"  Assoluto: .src_ e .dest_ sono indirizzi assoluti.
;                 "ZP"   Pagina Zero: .src_ e .dest_ sono indirizzi in pagina
;                        zero.
;
;               Con "IMM" una costante con i quattro byte uguali (per esempio 0,
;               $FFFFFFFF o $42424242) usa una sequenza più breve. Con "ABS" e
;               "ZP" la copia procede nel verso che non altera il sorgente se le
;               due aree si sovrappongono.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro) : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .src_ (4 byte, Macro)   : Valore (IMM) o indirizzo
;   - .dest_ (4 byte, Macro)  : Indirizzo
;
; Output:
;   - .dest_ (4 byte, Macro)  : Riceve il valore di .src_
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero)
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

5.3.4 m_Swap32

```
; ================================================================================
; MACRO: m_Swap32
; --------------------------------------------------------------------------------
; Descrizione:  Scambia il valore di due puntatori a 32 bit usando i registri.
;
; Input:
;   - .addr1_ (4 byte, Macro) : Indirizzo del primo valore
;   - .addr2_ (4 byte, Macro) : Indirizzo del secondo valore
;
; Output:
;   - .addr1_ (4 byte, Macro) : Riceve il valore di .addr2_
;   - .addr2_ (4 byte, Macro) : Riceve il valore di .addr1_
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .X, .Y, .P (Negative, Zero)
; Dipendenze Esterne:   <system/standard.asm>
; ================================================================================
```

5.3.5 m_CmpU32

```
; ================================================================================
; MACRO: m_CmpU32
; --------------------------------------------------------------------------------
; Descrizione:  Confronta due valori a 32 bit senza segno, secondo
;               l'indirizzamento scelto. Il risultato è nei flag, come con
;               l'istruzione CMP. A 32 bit non sono previsti gruppi di registri:
;               gli operandi sono indirizzi.
;
;                 "IMM"  Immediato: .op1_ è un indirizzo, .op2_ è un valore a 32
;                        bit.
;                 "ABS"  Assoluto: gli operandi sono indirizzi assoluti.
;                 "ZP"   Pagina Zero: gli operandi sono indirizzi in pagina
;                        zero.
;
;               Con "IMM", i confronti con 0 e con $FFFFFFFF usano una sequenza
;               più breve.
;
;               Gli operandi devono essere definiti prima dell'invocazione,
;               perché la macro li confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro) : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .op1_ (4 byte, Macro)   : Primo operando: indirizzo
;   - .op2_ (4 byte, Macro)   : Secondo operando: valore (IMM) o indirizzo
;
; Output:
;   - Flag Zero (1 bit, .P)   : 0 = .op1_ diverso da .op2_; 1 = uguali
;   - Flag Carry (1 bit, .P)  : 0 = .op1_ < .op2_; 1 = .op1_ >= .op2_ (senza
;                               segno)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero, Carry)
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

5.3.6 m_CmpS32

```
; ================================================================================
; MACRO: m_CmpS32
; --------------------------------------------------------------------------------
; Descrizione:  Confronta due valori a 32 bit con segno, secondo
;               l'indirizzamento scelto. Il flag di Carry si comporta come con
;               l'istruzione CMP (senza segno); con segno vale N xor V = 1 se
;               .op1_ < .op2_. A 32 bit non sono previsti gruppi di registri:
;               gli operandi sono indirizzi.
;
;                 "IMM"  Immediato: .op1_ è un indirizzo, .op2_ è un valore a 32
;                        bit.
;                 "ABS"  Assoluto: gli operandi sono indirizzi assoluti.
;                 "ZP"   Pagina Zero: gli operandi sono indirizzi in pagina
;                        zero.
;
;               Con "IMM", il confronto con 0 e con $FFFFFFFF usa una sequenza
;               più breve a parità di flag. Il flag di Zero riflette solo il
;               byte alto del risultato: per verificare l'uguaglianza si usa
;               m_CmpU32.
;
;               Con "IMM", .op2_ deve essere definito prima dell'invocazione,
;               perché la macro lo confronta per scegliere il ramo.
;
; Input:
;   - .mode_ (stringa, Macro)   : Indirizzamento: "IMM", "ABS" oppure "ZP"
;   - .op1_ (4 byte, Macro)     : Primo operando: indirizzo
;   - .op2_ (4 byte, Macro)     : Secondo operando: valore (IMM) o indirizzo
;
; Output:
;   - Flag Negative (1 bit, .P) : 0 = bit #7 a 0; 1 = bit #7 a 1
;   - Flag oVerflow (1 bit, .P) : 0 = nessun overflow con segno; 1 = overflow
;                                 con segno
;   - Flag Zero (1 bit, .P)     : 0 = byte alto diverso da 0; 1 = uguale a 0
;   - Flag Carry (1 bit, .P)    : 0 = .op1_ < .op2_; 1 = .op1_ >= .op2_ (senza
;                                 segno)
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, oVerflow, Zero, Carry)
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

5.3.7 m_Inc32

```
; ================================================================================
; MACRO: m_Inc32
; --------------------------------------------------------------------------------
; Descrizione:  Incrementa di 1 un contatore a 32 bit contenuto in un puntatore.
;               Non viene alterato nessun registro.
;
;               Il flag Negative in uscita è indefinito.
;
; Input:
;   - .addr_ (4 byte, Macro) : Contatore: indirizzo
;
; Output:
;   - .addr_ (4 byte, Macro) : Contatore incrementato (da $FFFFFFFF a $00000000)
;   - Flag Zero (1 bit, .P)  : 0 = risultato diverso da 0; 1 = risultato 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .P (Negative, Zero)
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

5.3.8 m_Dec32

```
; ================================================================================
; MACRO: m_Dec32
; --------------------------------------------------------------------------------
; Descrizione:  Decrementa di 1 un contatore a 32 bit contenuto in un puntatore.
;               Viene alterato .A.
;
;               Il flag Negative in uscita è indefinito.
;
; Input:
;   - .addr_ (4 byte, Macro) : Contatore: indirizzo
;
; Output:
;   - .addr_ (4 byte, Macro) : Contatore decrementato (da $00000000 a $FFFFFFFF)
;   - Flag Zero (1 bit, .P)  : 0 = byte basso diverso da 0; 1 = byte basso 0
;
; Variazione Stack (.S):
;   - Stabile
;
; Registri Modificati:  .A, .P (Negative, Zero)
; Dipendenze Esterne:   Nessuna
; ================================================================================
```

Libreria

c64

6 Libreria c64

Modulo	Descrizione
`errors.asm`	Codici dei messaggi d'errore del BASIC 2.0.
`kernal.asm`	Tabella dei salti del Kernal e codici d'errore del Kernal.
`keyboard.asm`	Codici della matrice della tastiera. Con __C128_C64_MODE = 1 aggiunge il supporto per i registri extra usati nella versione del chip installata sul C128.
`symbols.asm`	Mappa di memoria del Commodore 64: pagina zero, BASIC, Kernal, vettori di sistema e ROM.

6.1 errors.asm

Codici dei messaggi d'errore del BASIC 2.0.

Etichetta	Valore	Descrizione
ERR_TOO_MANY_FILES	$01	
ERR_FILE_OPEN	$02	
ERR_FILE_NOT_OPEN	$03	
ERR_FILE_NOT_FOUND	$04	
ERR_DEVICE_NOT_PRESENT	$05	
ERR_NOT_INPUT_FILE	$06	
ERR_NOT_OUTPUT_FILE	$07	
ERR_MISSING_FILENAME	$08	
ERR_ILLEGAL_DEVICE_NUMBER	$09	
ERR_NEXT_WITHOUT_FOR	$0A	
ERR_SYNTAX	$0B	
ERR_RETURN_WITHOUT_GOSUB	$0C	
ERR_OUT_OF_DATA	$0D	
ERR_ILLEGAL_QUANTITY	$0E	
ERR_OVERFLOW	$0F	
ERR_OUT_OF_MEMORY	$10	
ERR_UNDEFD_STATEMENT	$11	
ERR_BAD_SUBSCRIPT	$12	
ERR_REDIMD_ARRAY	$13	
ERR_DIVISION_BY_ZERO	$14	
ERR_ILLEGAL_DIRECT	$15	
ERR_TYPE_MISMATCH	$16	
ERR_STRING_TOO_LONG	$17	
ERR_FILE_DATA	$18	
ERR_FORMULA_TOO_COMPLEX	$19	
ERR_CANT_CONTINUE	$1A	
ERR_UNDEFD_FUNCTION	$1B	
ERR_VERIFY	$1C	
ERR_LOAD	$1D	
ERR_BREAK	$1E	
ERR_READY	$80	

6.2 kernal.asm

Tabella dei salti del Kernal e codici d'errore del Kernal.

Tabella salti del Kernal

Etichetta	Valore	Descrizione
CINT	$FF81	Inizializza il chip VIC-II e l'editor di schermo (altera .A, .X e .Y)
IOINIT	$FF84	Inizializza i dispositivi I/O (altera .A, .X e .Y)
RAMTAS	$FF87	Esegui un test della RAM (altera .A, .X e .Y)
RESTOR	$FF8A	Ripristina i vettori di interruzione e di sistema (altera .A, .X e .Y)
VECTOR	$FF8D	Copia tutti i vettori del Kernal da/verso la RAM (altera .A, .X e .Y)
SETMSG	$FF90	Controlla l'output dei messaggi di sistema (altera .A)
SECOND	$FF93	Imposta l'indirizzo secondario per LISTEN (altera .A)
TKSA	$FF96	Imposta l'indirizzo secondario per TALK (altera .A)
MEMTOP	$FF99	Imposta l'indirizzo finale della memoria BASIC (altera .X e .Y)
MEMBOT	$FF9C	Imposta l'indirizzo iniziale della memoria BASIC (altera .X e .Y)
SCNKEY	$FF9F	Scansiona la tastiera (altera .A, .X e .Y)
SETTMO	$FFA2	Imposta il flag di timeout della scheda d'interfaccia IEEE (altera .A)
ACPTR	$FFA5	Leggi un byte dal bus seriale (altera .A e .X)
CIOUT	$FFA8	Trasmetti un byte sul bus seriale
UNTLK	$FFAB	Disconnetti tutti i dispositivi sul bus seriale che sono in modalità di trasmissione (altera .A)
UNLSN	$FFAE	Disconnetti tutti i dispositivi sul bus seriale che sono in modalità di ricezione (altera .A)
LISTEN	$FFB1	Poni un dispositivo seriale in ricezione (altera .A)
TALK	$FFB4	Poni un dispositivo seriale in trasmissione (altera .A)
READST	$FFB7	Leggi lo stato di un dispositivo I/O (altera .A)
SETLFS	$FFBA	Imposta un file logico (altera .A, .X e .Y)
SETNAM	$FFBD	Imposta il nome del file (altera .A, .X e .Y)
OPEN	$FFC0	Apri un file logico (altera .A, .X e .Y)
CLOSE	$FFC3	Chiudi un file logico (altera .A, .X e .Y)
CHKIN	$FFC6	Apri un canale in ingresso (altera .A e .X)
CHKOUT	$FFC9	Apri un canale in uscita (altera .A e .X)
CLRCHN	$FFCC	Resetta tutti i canali di I/O (altera .A e .X)
CHRIN	$FFCF	Leggi un carattere dal canale di ingresso (altera .A e .X)
CHROUT	$FFD2	Scrivi un carattere nel canale di uscita (altera .A)
LOAD	$FFD5	Carica byte da un dispositivo (altera .A, .X e .Y)
SAVE	$FFD8	Salva byte su un dispositivo (altera .A, .X e .Y)
SETTIM	$FFDB	Imposta il clock di sistema (altera .A, .X e .Y)

Etichetta	Valore	Descrizione
RDTIM	$FFDE	Leggi l'orologio di sistema (altera .A, .X e .Y)
STOP	$FFE1	Leggi lo stato del tasto RUN/STOP (altera .A)
GETIN	$FFE4	Leggi un carattere dalla tastiera (altera .A, .X e .Y)
CLALL	$FFE7	Chiudi tutti i files (altera .A e .X)
UDTIM	$FFEA	Aggiorna il clock di sistema (altera .A e .X)
SCREEN	$FFED	Recupera le dimensioni dello schermo (altera .X e .Y)
PLOT	$FFF0	Posiziona il cursore (altera .A, .X e .Y)
IOBASE	$FFF3	Ricava l'indirizzo di base della memoria I/O (altera .X e .Y)

Codici d'errore del Kernal

Etichetta	Valore	Descrizione
K_ERR_BREAK	$00	
K_ERR_TOO_MANY_FILES	$01	
K_ERR_FILE_OPEN	$02	
K_ERR_FILE_NOT_OPEN	$03	
K_ERR_FILE_NOT_FOUND	$04	
K_ERR_DEVICE_NOT_PRESENT	$05	
K_ERR_NOT_INPUT_FILE	$06	
K_ERR_NOT_OUTPUT_FILE	$07	
K_ERR_MISSING_FILENAME	$08	
K_ERR_ILLEGAL_DEVICE_NUMBER	$09	
K_ERR_MOVED_RS232_BUFFER	$F0	

6.3 keyboard.asm

Codici della matrice della tastiera. Con __C128_C64_MODE = 1 aggiunge il supporto per i registri extra usati nella versione del chip installata sul C128.

Etichetta	Valore	Descrizione
KBD_INST_DEL	$00	
KBD_RETURN	$01	
KBD_CURSOR_RIGHT	$02	
KBD_F7	$03	
KBD_F1	$04	
KBD_F3	$05	
KBD_F5	$06	
KBD_CURSOR_DOWN	$07	
KBD_3	$08	
KBD_W	$09	
KBD_A	$0A	
KBD_4	$0B	
KBD_Z	$0C	
KBD_S	$0D	
KBD_E	$0E	
KBD_LEFT_SHIFT	$0F	
KBD_5	$10	
KBD_R	$11	
KBD_D	$12	
KBD_6	$13	
KBD_C	$14	
KBD_F	$15	
KBD_T	$16	
KBD_X	$17	
KBD_7	$18	
KBD_Y	$19	
KBD_G	$1A	
KBD_8	$1B	
KBD_B	$1C	
KBD_H	$1D	
KBD_U	$1E	
KBD_V	$1F	
KBD_9	$20	
KBD_I	$21	
KBD_J	$22	

Etichetta	Valore	Descrizione
KBD_0	$23	
KBD_M	$24	
KBD_K	$25	
KBD_O	$26	
KBD_N	$27	
KBD_PLUS	$28	
KBD_P	$29	
KBD_L	$2A	
KBD_MINUS	$2B	
KBD_FULL_STOP	$2C	
KBD_COLON	$2D	
KBD_AT_SIGN	$2E	
KBD_COMMA	$2F	
KBD_POUND	$30	
KBD_ASTERISK	$31	
KBD_SEMICOLON	$32	
KBD_CLR_HOME	$33	
KBD_RIGHT_SHIFT	$34	
KBD_EQUALS	$35	
KBD_UP_ARROW	$36	
KBD_SLASH	$37	
KBD_1	$38	
KBD_LEFT_ARROW	$39	
KBD_CONTROL	$3A	
KBD_2	$3B	
KBD_SPACE	$3C	
KBD_COMMODORE	$3D	
KBD_Q	$3E	
KBD_RUN_STOP	$3F	
KBD_HELP	$40	
KBD_KEYPAD_8	$41	
KBD_KEYPAD_5	$42	
KBD_TAB	$43	
KBD_KEYPAD_2	$44	
KBD_KEYPAD_4	$45	
KBD_KEYPAD_7	$46	
KBD_KEYPAD_1	$47	
KBD_ESC	$48	
KBD_KEYPAD_PLUS	$49	

Etichetta	Valore	Descrizione
KBD_KEYPAD_MINUS	$4A	
KBD_LINE_FEED	$4B	
KBD_ENTER	$4C	
KBD_KEYPAD_6	$4D	
KBD_KEYPAD_9	$4E	
KBD_KEYPAD_3	$4F	
KBD_ALT	$50	
KBD_KEYPAD_0	$51	
KBD_KEYPAD_DOT	$52	
KBD_UP	$53	
KBD_DOWN	$54	
KBD_LEFT	$55	
KBD_RIGHT	$56	
KBD_NO_SCROLL	$57	
KBD_NO_KEY	$58	
KBD_NO_KEY	$40	

6.4 symbols.asm

Mappa di memoria del Commodore 64: pagina zero, BASIC, Kernal, vettori di sistema e ROM.

Pagina 0

Etichetta	Valore	Descrizione
D6510	$00	Registro direzione dati del chip 6510
R6510	$01	Registro porte dati del chip 6510
UNUSED	$02	Locazione non usata #1

Pagina 0 - BASIC

Etichetta	Valore	Descrizione
VERCK	$0A	Flag usato dal BASIC per differenziare LOAD (0) da VERIFY (1)
VALTYP	$0D	Flag usato dal BASIC per determinare se la variabile è stringa ($FF) o numerica (0)
CHANNL	$13	Canale di I/O attualmente in uso
TEMPST	$19	Stack per i descrittori delle stringhe temporanee (da TEMPST+0 a TEMPST+8)
INDEX	$22	Puntatore temporaneo per il BASIC #1
INDEX2	$24	Puntatore temporaneo per il BASIC #2
RESHO	$26	Area di lavoro per moltiplicazioni in virgola mobile (da RESHO+0 a RESHO+4)
TXTTAB	$2B	Puntatore all'inizio dell'area BASIC
VARTAB	$2D	Puntatore all'inizio dell'area Variabili
ARYTAB	$2F	Puntatore all'inizio dell'area Matrici
STREND	$31	Puntatore alla fine dell'area Matrici (+1) ed inizio della memoria libera
FRETOP	$33	Puntatore alla fine dell'area Stringhe
MEMSIZ	$37	Puntatore alla fine dell'area BASIC
JMPER	$54	Istruzione JMP usata dalla subroutine di valutazione delle funzioni matematiche
NUMWRK	$57	Area di lavoro utilizzata da varie subroutine numeriche del BASIC (da NUMWRK+0 a NUMWRK+9)
FAC	$61	Accumulatore in virgola mobile #1
FACEXP	FAC	Accumulatore in virgola mobile #1: Esponente
FACHO	FAC+$01	Accumulatore in virgola mobile #1: Mantissa (byte più significativo)
FACMOH	FAC+$02	Accumulatore in virgola mobile #1: Mantissa (secondo byte più significativo)
FACMO	FAC+$03	Accumulatore in virgola mobile #1: Mantissa (terzo byte più significativo)

Etichetta	Valore	Descrizione
FACLO	FAC+$04	Accumulatore in virgola mobile #1: Mantissa (byte meno significativo)
FACSGN	FAC+$05	Accumulatore in virgola mobile #1: Segno
ARG	$69	Accumulatore in virgola mobile #2
ARGEXP	ARG	Accumulatore in virgola mobile #2: Esponente
ARGHO	ARG+$01	Accumulatore in virgola mobile #2: Mantissa (byte più significativo)
ARGMOH	ARG+$02	Accumulatore in virgola mobile #2: Mantissa (secondo byte più significativo)
ARGMO	ARG+$03	Accumulatore in virgola mobile #2: Mantissa (terzo byte più significativo)
ARGLO	ARG+$04	Accumulatore in virgola mobile #2: Mantissa (byte meno significativo)
ARGSGN	ARG+$05	Accumulatore in virgola mobile #2: Segno
ARISGN	$6F	Comparazione dei segni degli accumulatori in virgola mobile: $00=segni uguali, $FF=segni differenti
FACOV	$70	Byte di precisione extra per l'accumulatore in virgola mobile #1
CHRGET	$73	Leggi il prossimo carattere nel testo di un programma BASIC
CHRGOT	$79	Rileggi l'ultimo carattere nel testo di un programma BASIC
TXTPTR	$7A	Puntatore al carattere attualmente letto
RNDX	$8B	Seme del generatore di numeri casuali del BASIC (da RNDX+0 a RNDX+4)

Pagina 0 - KERNAL

Etichetta	Valore	Descrizione
STATUS	$90	Stato I/O del Kernal
STKEY	$91	Flag: Premuto tasto STOP?
TIME	$A0	Orologio jiffy (da TIME+0 a TIME+2)
LA	$B8	Numero di file in uso
SA	$B9	Indirizzo secondario I/O in uso
FA	$BA	Numero di dispositivo I/O in uso
LSTX	$C5	Ultimo tasto premuto (tastiera hardware)
NDX	$C6	Numero di caratteri nel buffer della tastiera
RVS	$C7	Attivazione modalità REVERSE
SFDX	$CB	Tasto attualmente premuto (tastiera hardware)
BLNSW	$CC	Abilita il lampeggio del cursore
BLNCT	$CD	Temporizzatore del lampeggio del cursore
BLNON	$CF	Ultimo lampeggio del cursore era on oppure off?
PNTR	$D3	Posizione orizzontale del cursore (linea logica)

Etichetta	Valore	Descrizione
QTSW	$D4	Modo virgolette attivo?
TBLX	$D6	Posizione verticale del cursore
INSRT	$D8	Modo INSERT attivo?

Puntatori utente in pagina 0

Etichetta	Valore	Descrizione
ZP_1	$FB	Puntatore utente in pagina 0 #1
ZP_2	$FD	Puntatore utente in pagina 0 #2
ZP_3	$F7	Puntatore utente in pagina 0 #3 (in realtà è RIBUF - sicuro da usare se non si utilizza l'interfaccia RS-232)
ZP_4	$F9	Puntatore utente in pagina 0 #4 (in realtà è ROBUF - sicuro da usare se non si utilizza l'interfaccia RS-232)
ZP_5	$A3	Puntatore utente in pagina 0 #5 (sicuro da usare se non si utilizza il registratore a cassette o l'I/O seriale)
ZPL_1	NUMWRK	Puntatore utente a 24/32 bit in pagina 0 #1
ZPL_2	NUMWRK+4	Puntatore utente a 24/32 bit in pagina 0 #2

Pagina 1

Etichetta	Valore	Descrizione
STACK	$0100	Stack

Pagina 2

Etichetta	Valore	Descrizione
BUF	$0200	Buffer di input del BASIC
KEYD	$0277	Buffer di tastiera
COLOR	$0286	Colore attuale del testo
HIBASE	$0288	Pagina iniziale dell'area schermo
XMAX	$0289	Massima dimensione del buffer di tastiera
RPTFLG	$028A	Flag per la ripetizione automatica dei caratteri digitati
KOUNT	$028B	Ritardo tra ripetizioni automatiche dei caratteri digitati
DELAY	$028C	Ritardo per la prima ripetizione automatica di caratteri digitati
SHFLAG	$028D	Flag digitazione tasti Shift/CTRL/Commodore
LSTSHF	$028E	Ultima combinazione tasti Shift/CTRL/Commodore premuta
KEYLOG	$028F	Vettore alla subroutine di inizializzazione della tavola di decodifica tastiera

Etichetta	Valore	Descrizione
MODE	$0291	Abilita (0) o disabilita (128) la combinazione Shift+Commodore
PALFLG	$02A6	Se <>0 allora l'output video è PAL
FREMEM	$02A7	Memoria libera (da FREMEM+0 a FREMEM+88)

Pagina 3

Etichetta	Valore	Descrizione
IERROR	$0300	Vettore alla subroutine di stampa messaggi d'errore
IMAIN	$0302	Vettore al loop principale del BASIC
ICRNCH	$0304	Vettore alla subroutine di tokenizzazione
IQPLOP	$0306	Vettore alla subroutine di de-tokenizzazione
IGONE	$0308	Vettore alla subroutine di esecuzione prossima istruzione BASIC
IEVAL	$030A	Vettore alla subroutine di valutazione espressione numerica
SAREG	$030C	Area di memorizzazione del registro .A
SXREG	$030D	Area di memorizzazione del registro .X
SYREG	$030E	Area di memorizzazione del registro .Y
SPREG	$030F	Area di memorizzazione del registro .P
USRPOK	$0310	Punto d'ingresso per la funzione USR()
USRADD	$0311	Indirizzo funzione USR()
UNUSE2	$0313	Locazione non usata #2
CINV	$0314	Vettore alla subroutine di interrupt IRQ
CBINV	$0316	Vettore alla subroutine di interrupt BRK
NMINV	$0318	Vettore alla subroutine di interrupt NMI
FREME2	$0334	Memoria libera (da FREME2+0 a FREME2+203)
TMPPG1	$0334	Memoria libera (da TMPPG1+0 a TMPPG1+7)
TBUFFR	$033C	Buffer di I/O cassetta (sicuro da usare fino a TBUFFER+191 se non si usa il registratore a cassette)
TMPPG2	$03FC	Memoria libera (da TMPPG2+0 a TMPPG2+3)

Pagina 4

Etichetta	Valore	Descrizione
VICSCN	$0400	Indirizzo di base della memoria schermo all'accensione

Pagina 8

Etichetta	Valore	Descrizione
BASTXT	$0801	Indirizzo di base dell'area BASIC all'accensione

Banchi di memoria cartucce

Etichetta	Valore	Descrizione
CRT_LO	$8000	Banco di memoria per cartucce #1
CRT_HI	$E000	Banco di memoria per cartucce #2

BASIC

Etichetta	Valore	Descrizione
BASROM	$A000	Indirizzo di base della ROM del BASIC
ERROR	$A437	Gestore degli errori del BASIC
READY	$A474	Esce nel BASIC stampando "READY."
NEWSTT	$A7AE	Prepara la prossima istruzione per l'esecuzione
GONE	$A7E4	Decodifica ed esegui la prossima istruzione
STROUT	$AB1E	Stampa la stringa puntata da .A/.Y
FRMNUM	$AD8A	Valuta un'espressione numerica
FRMEVL	$AD9E	Valuta un'espressione numerica oppure stringa
PIVAL	$AEA8	Valore della costante π in virgola mobile
NOTOP	$AED4	FAC = NOT FAC
CHKCLS	$AEF7	Verifica la presenza di una parentesi chiusa e poi saltala
CHKOPN	$AEFA	Verifica la presenza di una parentesi aperta e poi saltala
CHKCOM	$AEFD	Verifica la presenza di una virgola e poi saltala
SYNCHR	$AEFF	Verifica la presenza del carattere in .A e poi saltalo
OROP	$AFE6	FAC = ARG OR FAC
ANDOP	$AFE9	FAC = ARG AND FAC
N32768	$B1A5	Valore della costante -32 768 in virgola mobile
GIVFAY	$B1AA	Converti un numero in virgola mobile in un intero con segno in .A/.Y
AYINT	$B1BF	Converti un numero in virgola mobile in un intero con segno in FACMO/FACLO
FCERR	$B248	Esci con l'errore "?ILLEGAL QUANTITY"
GIVAYF	$B391	Converti intero a 16 bit con segno in numero a virgola mobile
STRLIT	$B487	Prepara i puntatori per una stringa in memoria
FRESTR	$B6A3	Controlla che FAC punti ad una stringa e poi...
FREFAC	$B6A6	...Dealloca una stringa temporanea
GTBYTC	$B79B	Leggi il prossimo carattere dallo stream di input...
GETBYT	$B79E	...poi convertilo in un intero tra 0 e 255 e mettilo in .X
GETNUM	$B7EB	Estrai un parametro a 16 bit ed uno ad 8 bit separati da una virgola dal testo del BASIC
COMBYT	$B7F1	Controlla la presenza di una virgola nello stream di input ed estrai il seguente valore ad 8 bit
GETADR	$B7F7	Converti un numero in virgola mobile in intero tra 0 e 65 535

Etichetta	Valore	Descrizione
FADDH	$B849	Somma 0.5 al numero contenuto in FAC
FSUB	$B850	FAC = MEM - FAC
FSUBT	$B853	FAC = ARG - FAC
FADD	$B867	FAC = MEM + FAC
FADDT	$B86A	FAC = ARG + FAC
FONE	$B9BC	Valore della costante 1 in virgola mobile
FSQR2	$B9DB	Valore della costante SQR(2) in virgola mobile
FLOG2	$B9E5	Valore della costante LOG(2) in virgola mobile
LOG	$B9EA	FAC = LOG(FAC)
FMULT	$BA28	FAC = MEM * FAC
FMULTT	$BA2B	FAC = ARG * FAC
CONUPK	$BA8C	Carica ARG con un numero float presente all'indirizzo .A/.Y
TENC	$BAF9	Valore della costante 10 in virgola mobile
FDIV	$BB0F	FAC = MEM / FAC
FDIVT	$BB12	FAC = ARG / FAC
MOVFM	$BBA2	Carica FAC con un numero float presente in memoria
MOV2F	$BBC7	Copia FAC in memoria
MOVMF	$BBD4	Arrotonda FAC e copialo in memoria
MOVFA	$BBFC	Copia ARG in FAC
MOVAF	$BC0C	Copia FAC in ARG (con arrotondamento)
MOVEF	$BC0F	Copia FAC in ARG (senza arrotondamento)
ROUND	$BC1B	Arrotonda FAC a seconda del valore di FACOV
SIGN	$BC2B	Restituisci il segno di FAC in .A
SGN	$BC39	Sostituisci FAC con il suo segno
FLOATB	$BC4F	Normalizza FAC
FCOMP	$BC5B	Confronta FAC con un numero float presente in memoria
QINT	$BC9B	Converte FAC in un intero di 4 byte con segno
INT	$BCCC	Tronca FAC ad un intero
FIN	$BCF3	Converti una stringa ASCII in un numero in virgola mobile (FAC)
LINPRT	$BDCD	Stampa un numero decimale contenuto in .A/.X
FOUT	$BDDD	Converti FAC in una stringa ASCII
FHALF	$BF11	Valore della costante 0.5 in virgola mobile
SQR	$BF71	FAC = SQR(FAC)
FPWRT	$BF7B	FAC = ARG ^ FAC
EXP	$BFED	FAC = EXP(FAC)

Generatore di caratteri

Etichetta	Valore	Descrizione
CHRGEN	$D000	Indirizzo di base del generatore di caratteri
CASEU	CHRGEN+$0000	Indirizzo di base del set di caratteri maiuscolo/grafica (128 caratteri)

Etichetta	Valore	Descrizione
CASEURV	CHRGEN+$0400	Indirizzo di base del set di caratteri maiuscolo/grafica (128 caratteri, reverse)
CASEL	CHRGEN+$0800	Indirizzo di base del set di caratteri minuscolo/maiuscolo (128 caratteri)
CASELRV	CHRGEN+$0C00	Indirizzo di base del set di caratteri minuscolo/maiuscolo (128 caratteri, reverse)

Pagina 216

Etichetta	Valore	Descrizione
COLRAM	$D800	Indirizzo di base della memoria colore

Kernal

Etichetta	Valore	Descrizione
KRNROM	$E000	Indirizzo di base della ROM del Kernal
POLY1	$E043	Calcola un polinomio dispari di grado n in FAC
POLY2	$E059	Calcola un polinomio generico di grado n in FAC
RND	$E097	Genera il prossimo numero casuale e mettilo in FAC
EREXIT	$E0F9	Converti una condizione d'errore del Kernal in una condizione d'errore del BASIC
SLPARA	$E1D4	Decodifica i parametri di LOAD/SAVE/VERIFY dallo stream di input e prepara l'I/O
COS	$E264	FAC = COS(FAC)
SIN	$E26B	FAC = SIN(FAC)
TAN	$E2B4	FAC = TAN(FAC)
PI2	$E2E0	Valore della costante π/2 in virgola mobile
TWOPI	$E2E5	Valore della costante 2π in virgola mobile
FR4	$E2EA	Valore della costante 0.25 in virgola mobile
ATN	$E30E	FAC = ATN(FAC)
CLRSCR	$E544	Inizializza lo schermo
IRQHND	$EA31	Gestore standard delle interruzioni (IRQ)
IRQMIN	$EA81	Gestore minimale delle interruzioni
RESET	$FCE2	Reset del sistema
NMIHND	$FE43	Gestore standard delle interruzioni non mascherabili (NMI)
BRKHND	$FE66	Gestore standard delle interruzioni software (BRK)
IRQROUT	$FF48	Instradatore delle interruzioni IRQ/BRK

Vettori hardware del 6510

Etichetta	Valore	Descrizione
NMIVEC	$FFFA	Vettore NMI
RESVEC	$FFFC	Vettore Reset
IRBVEC	$FFFE	Vettore IRQ/BRK

Libreria

chip

7 Libreria chip

Modulo	Descrizione
`cia.asm`	Registri dei due chip CIA.
`rec.asm`	Registri del RAM Expansion Controller (REC).
`sid.asm`	Registri del Sound Interface Device (SID).
`vdc.asm`	Registri e codici colore del Video Display Controller (VDC). Con __C128_DCR = 1 aggiunge il supporto per il registro extra usato nella versione del chip installata sul C128DCR.
`vic.asm`	Registri e codici colore del chip VIC.
`vic_ii.asm`	Registri e codici colore del chip VIC-II. Con __C128_C64_MODE = 1 aggiunge il supporto per i registri extra usati nella versione del chip installata sul C128.

7.1 cia.asm

Registri dei due chip CIA.

Complex Interface Adapter (CIA) #1

Etichetta	Valore	Descrizione
CIA_1	$DC00	Indirizzo di base del chip CIA #1
CIAPRA	CIA_1+$00	Registro di controllo porte dati A
CIAPRB	CIA_1+$01	Registro di controllo porte dati B
CIDDRA	CIA_1+$02	Registro direzione dati A
CIDDRB	CIA_1+$03	Registro direzione dati B
TIMALO	CIA_1+$04	Timer A (byte basso)
TIMAHI	CIA_1+$05	Timer A (byte alto)
TIMBLO	CIA_1+$06	Timer B (byte basso)
TIMBHI	CIA_1+$07	Timer B (byte alto)
TODTEN	CIA_1+$08	Orologio Time-Of-Day (decimi di secondo)
TODSEC	CIA_1+$09	Orologio Time-Of-Day (secondi)
TODMIN	CIA_1+$0A	Orologio Time-Of-Day (minuti)
TODHRS	CIA_1+$0B	Orologio Time-Of-Day (ore)
CIASDR	CIA_1+$0C	Porta dati seriale
CIAICR	CIA_1+$0D	Registro di controllo interrupt
CIACRA	CIA_1+$0E	Registro di controllo A
CIACRB	CIA_1+$0F	Registro di controllo B

Complex Interface Adapter (CIA) #2

Etichetta	Valore	Descrizione
CIA_2	$DD00	Indirizzo di base del chip CIA #2
CI2PRA	CIA_2+$00	Registro di controllo porte dati A
CI2PRB	CIA_2+$01	Registro di controllo porte dati B
C2DDRA	CIA_2+$02	Registro direzione dati A
C2DDRB	CIA_2+$03	Registro direzione dati B
TI2ALO	CIA_2+$04	Timer A (byte basso)
TI2AHI	CIA_2+$05	Timer A (byte alto)
TI2BLO	CIA_2+$06	Timer B (byte basso)
TI2BHI	CIA_2+$07	Timer B (byte alto)
TO2TEN	CIA_2+$08	Orologio Time-Of-Day (decimi di secondo)
TO2SEC	CIA_2+$09	Orologio Time-Of-Day (secondi)
TO2MIN	CIA_2+$0A	Orologio Time-Of-Day (minuti)

Etichetta	Valore	Descrizione
TO2HRS	CIA_2+$0B	Orologio Time-Of-Day (ore)
CI2SDR	CIA_2+$0C	Porta dati seriale
CI2ICR	CIA_2+$0D	Registro di controllo interrupt
CI2CRA	CIA_2+$0E	Registro di controllo A
CI2CRB	CIA_2+$0F	Registro di controllo B

7.2 rec.asm

Registri del RAM Expansion Controller (REC).

Etichetta	Valore	Descrizione
REC	$DF00	Indirizzo di base del chip REC
DMAST	REC+$00	Registro di stato del chip REC
DMACMD	REC+$01	Registro di comando del chip REC
DMAADL	REC+$02	Indirizzo di base della memoria di sistema (byte basso)
DMAADH	REC+$03	Indirizzo di base della memoria di sistema (byte alto)
DMALO	REC+$04	Indirizzo di base della memoria di espansione (byte basso)
DMAHI	REC+$05	Indirizzo di base della memoria di espansione (byte alto)
DMABNK	REC+$06	Selettore del banco di memoria di espansione
DMADAL	REC+$07	Numero di byte da trasferire (byte basso)
DMADAH	REC+$08	Numero di byte da trasferire (byte alto)
DMASUM	REC+$09	Registro di controllo delle interruzioni
DMAVER	REC+$0A	Registro di controllo degli indirizzamenti

7.3 sid.asm

Registri del Sound Interface Device (SID).

Etichetta	Valore	Descrizione
SID	$D400	Indirizzo di base del chip SID
FRELO1	SID+$00	Controllo frequenza voce #1 (byte basso)
FREHI1	SID+$01	Controllo frequenza voce #1 (byte alto)
PWLO1	SID+$02	Larghezza onda quadra voce #1 (byte basso)
PWHI1	SID+$03	Larghezza onda quadra voce #1 (byte alto)
VCREG1	SID+$04	Registro di controllo voce #1
ATDCY1	SID+$05	Registro Attack/Delay voce #1
SUREL1	SID+$06	Registro Sustain/Release voce #1
FRELO2	SID+$07	Controllo frequenza voce #2 (byte basso)
FREHI2	SID+$08	Controllo frequenza voce #2 (byte alto)
PWLO2	SID+$09	Larghezza onda quadra voce #2 (byte basso)
PWHI2	SID+$0A	Larghezza onda quadra voce #2 (byte alto)
VCREG2	SID+$0B	Registro di controllo voce #2
ATDCY2	SID+$0C	Registro Attack/Delay voce #2
SUREL2	SID+$0D	Registro Sustain/Release voce #2
FRELO3	SID+$0E	Controllo frequenza voce #3 (byte basso)
FREHI3	SID+$0F	Controllo frequenza voce #3 (byte alto)
PWLO3	SID+$10	Larghezza onda quadra voce #3 (byte basso)
PWHI3	SID+$11	Larghezza onda quadra voce #3 (byte alto)
VCREG3	SID+$12	Registro di controllo voce #3
ATDCY3	SID+$13	Registro Attack/Delay voce #3
SUREL3	SID+$14	Registro Sustain/Release voce #3
CUTLO	SID+$15	Frequenza di taglio del filtro (byte basso)
CUTHI	SID+$16	Frequenza di taglio del filtro (byte alto)
RESON	SID+$17	Registro di controllo risonanza dei filtri
SIGVOL	SID+$18	Volume e registro di selezione dei filtri
POTX	SID+$19	Posizione paddle #1 (o #3)
POTY	SID+$1A	Posizione paddle #2 (o #4)
RANDOM	SID+$1B	Uscita oscillatore #3 / generatore di numeri casuali
ENV3	SID+$1C	Uscita generatore di inviluppo #3

7.4 vdc.asm

Registri e codici colore del Video Display Controller (VDC). Con __C128_DCR = 1 aggiunge il supporto per il registro extra usato nella versione del chip installata sul C128DCR.

Etichetta	Valore	Descrizione
VDC	$D600	Indirizzo di base del chip VDC
VDCADR	VDC+$00	Registro di indirizzamento / stato
VDCDAT	VDC+$01	Registro dati

Registri interni del chip VDC

Etichetta	Valore	Descrizione
VDCHZT	0	Numero totale di colonne
VDCHZD	1	Numero visualizzato di colonne
VDCHZS	2	Posizione di sincronismo orizzontale
VDCSYW	3	Larghezza dei sincronismi orizzontale e verticale
VDCVET	4	Numero totale di righe
VDCVEA	5	Controllo fine della posizione verticale
VDCVED	6	Numero visualizzato di righe
VDCVES	7	Posizione di sincronismo verticale
VDCILC	8	Registro di controllo modalità interlacciata
VDCCTV	9	Numero di linee di scansione per carattere
VDCCSS	10	Registro di controllo del cursore
VDCCES	11	Linea finale di scansione del cursore
VDCDSH	12	Indirizzo di base della memoria schermo (alto)
VDCDSL	13	Indirizzo di base della memoria schermo (basso)
VDCCPH	14	Indirizzo della posizione del cursore (alto)
VDCCPL	15	Indirizzo della posizione del cursore (basso)
VDCLPV	16	Posizione verticale della penna ottica
VDCLPH	17	Posizione orizzontale della penna ottica
VDCUAH	18	Puntatore alla memoria video (alto)
VDCUAL	19	Puntatore alla memoria video (basso)
VDCATH	20	Indirizzo di base della memoria attributi (alto)
VDCATL	21	Indirizzo di base della memoria attributi (basso)
VDCHCP	22	Dimensione orizzontale dei caratteri
VDCVCP	23	Dimensione verticale dei caratteri
VDCMVS	24	Controllo fine dello scorrimento verticale
VDCMHS	25	Controllo fine dello scorrimento orizzontale

Etichetta	Valore	Descrizione
VDCCOL	26	Colore di sfondo e dei caratteri
VDCRIN	27	Incremento indirizzo per riga
VDCCSR	28	Indirizzo del set di caratteri
VDCUDL	29	Posizione della linea di sottolineatura
VDCWDC	30	Numero di byte (-1) per le operazioni di copia o scrittura
VDCDTA	31	Selettore di lettura / scrittura della memoria
VDCBLH	32	Indirizzo sorgente per la copia di blocchi di memoria (alto)
VDCBLL	33	Indirizzo sorgente per la copia di blocchi di memoria (basso)
VDCDEB	34	Posizione iniziale blanking orizzontale
VDCDEE	35	Posizione finale blanking orizzontale
VDCDRM	36	Numero di cicli di rinfresco memoria per linea di scansione
VDCSYN	37	Polarità dei sincronismi orizzontale e verticale

Codici colore del chip VDC

Etichetta	Valore	Descrizione
VDC_BLACK	$00	Codice VDC del colore nero
VDC_DARK_GREY	$01	Codice VDC del colore grigio scuro
VDC_DARK_BLUE	$02	Codice VDC del colore blu scuro
VDC_LIGHT_BLUE	$03	Codice VDC del colore blu chiaro
VDC_DARK_GREEN	$04	Codice VDC del colore verde scuro
VDC_LIGHT_GREEN	$05	Codice VDC del colore verde chiaro
VDC_DARK_CYAN	$06	Codice VDC del colore ciano scuro
VDC_LIGHT_CYAN	$07	Codice VDC del colore ciano chiaro
VDC_DARK_RED	$08	Codice VDC del colore rosso scuro
VDC_LIGHT_RED	$09	Codice VDC del colore rosso chiaro
VDC_DARK_PURPLE	$0A	Codice VDC del colore viola scuro
VDC_LIGHT_PURPLE	$0B	Codice VDC del colore viola chiaro
VDC_DARK_YELLOW	$0C	Codice VDC del colore giallo scuro
VDC_LIGHT_YELLOW	$0D	Codice VDC del colore giallo chiaro
VDC_LIGHT_GREY	$0E	Codice VDC del colore grigio chiaro
VDC_WHITE	$0F	Codice VDC del colore bianco
VDC_TRANSPARENT	$FF	Valore fittizio usato per indicare che il registro colore non va alterato

7.5 vic.asm

Registri e codici colore del chip VIC.

Etichetta	Valore	Descrizione
VIC	$9000	Indirizzo base del chip VIC
VICCRO	VIC+$00	Centraggio orizzontale dello schermo e modo interlacciato
VICCRI	VIC+$01	Centraggio verticale dello schermo
VICCR2	VIC+$02	Colonne visualizzate / Indirizzo schermo (bit #9)
VICCR3	VIC+$03	Righe visualizzate / Posizione raster (bit #0)
VICCR4	VIC+$04	Posizione raster (bit #8-#1)
VICCR5	VIC+$05	Indirizzo schermo (bit #13-#10) / Indirizzo mappa caratteri (bit #13-#10)
VICCR6	VIC+$06	Penna ottica (ascissa)
VICCR7	VIC+$07	Penna ottica (ordinata)
VICCR8	VIC+$08	Paddle X
VICCR9	VIC+$09	Paddle Y
VICCRA	VIC+$0A	Frequenza oscillatore #1 (basso)
VICCRB	VIC+$0B	Frequenza oscillatore #2 (alto)
VICCRC	VIC+$0C	Frequenza oscillatore #3 (soprano)
VICCRD	VIC+$0D	Frequenza oscillatore #4 (rumore)
VICCRE	VIC+$0E	Colore ausiliario / Volume
VICCRF	VIC+$0F	Colore sfondo / bordo / modalità inversa

Codici colore del chip VIC

Etichetta	Valore	Descrizione
VIC_BLACK	$00	Codice VIC del colore nero
VIC_WHITE	$01	Codice VIC del colore bianco
VIC_RED	$02	Codice VIC del colore rosso
VIC_CYAN	$03	Codice VIC del colore ciano
VIC_PURPLE	$04	Codice VIC del colore viola
VIC_GREEN	$05	Codice VIC del colore verde
VIC_BLUE	$06	Codice VIC del colore blu
VIC_YELLOW	$07	Codice VIC del colore giallo
VIC_ORANGE	$08	Codice VIC del colore arancione
VIC_LIGHT_ORANGE	$09	Codice VIC del colore arancione chiaro
VIC_PINK	$0A	Codice VIC del colore rosa
VIC_LIGHT_CYAN	$0B	Codice VIC del colore ciano chiaro
VIC_LIGHT_PURPLE	$0C	Codice VIC del colore viola chiaro

Etichetta	Valore	Descrizione
VIC_LIGHT_GREEN	$0D	Codice VIC del colore verde chiaro
VIC_LIGHT_BLUE	$0E	Codice VIC del colore azzurro
VIC_LIGHT_YELLOW	$0F	Codice VIC del colore giallo chiaro
VIC_TRANSPARENT	$FF	Valore fittizio usato per indicare che il registro colore non va alterato

7.6 vic_ii.asm

Registri e codici colore del chip VIC-II. Con __C128_C64_MODE = 1 aggiunge il supporto per i registri extra usati nella versione del chip installata sul C128.

Etichetta	Valore	Descrizione
VIC	$D000	Indirizzo base del chip VIC-II
SP0X	VIC+$00	Posizione orizzontale sprite #0
SP0Y	VIC+$01	Posizione verticale sprite #0
SP1X	VIC+$02	Posizione orizzontale sprite #1
SP1Y	VIC+$03	Posizione verticale sprite #1
SP2X	VIC+$04	Posizione orizzontale sprite #2
SP2Y	VIC+$05	Posizione verticale sprite #2
SP3X	VIC+$06	Posizione orizzontale sprite #3
SP3Y	VIC+$07	Posizione verticale sprite #3
SP4X	VIC+$08	Posizione orizzontale sprite #4
SP4Y	VIC+$09	Posizione verticale sprite #4
SP5X	VIC+$0A	Posizione orizzontale sprite #5
SP5Y	VIC+$0B	Posizione verticale sprite #5
SP6X	VIC+$0C	Posizione orizzontale sprite #6
SP6Y	VIC+$0D	Posizione verticale sprite #6
SP7X	VIC+$0E	Posizione orizzontale sprite #7
SP7Y	VIC+$0F	Posizione verticale sprite #7
MSIGX	VIC+$10	MSB delle posizioni orizzontali degli sprite
SCROLY	VIC+$11	Registro di controllo e scorrimento verticale fine
RASTER	VIC+$12	Linea di raster attuale
LPENX	VIC+$13	Posizione orizzontale della penna ottica
LPENY	VIC+$14	Posizione verticale della penna ottica
SPENA	VIC+$15	Abilitazione sprite
SCROLX	VIC+$16	Registro di controllo e scorrimento orizzontale fine
YXPAND	VIC+$17	Espansione sprite in direzione verticale
VMCSB	VIC+$18	Registro di controllo della memoria video
VICIRQ	VIC+$19	Registro dei flag di interrupt
IRQMSK	VIC+$1A	Registro di maschera IRQ
SPBGPR	VIC+$1B	Priorità sprite/caratteri
SPMC	VIC+$1C	Abilitazione sprite multicolore
XXPAND	VIC+$1D	Espansione sprite in direzione orizzontale
SPSPCL	VIC+$1E	Collisione sprite/sprite
SPBGCL	VIC+$1F	Collisione sprite/caratteri

Etichetta	Valore	Descrizione
EXTCOL	VIC+$20	Colore del bordo
BGCOL0	VIC+$21	Colore dello sfondo
BGCOL1	VIC+$22	Colore ausiliare di sfondo #1
BGCOL2	VIC+$23	Colore ausiliare di sfondo #2
BGCOL3	VIC+$24	Colore ausiliare di sfondo #3
SPMC0	VIC+$25	Colore ausiliare degli sprite #1
SPMC1	VIC+$26	Colore ausiliare degli sprite #2
SP0COL	VIC+$27	Colore dello sprite #0
SP1COL	VIC+$28	Colore dello sprite #1
SP2COL	VIC+$29	Colore dello sprite #2
SP3COL	VIC+$2A	Colore dello sprite #3
SP4COL	VIC+$2B	Colore dello sprite #4
SP5COL	VIC+$2C	Colore dello sprite #5
SP6COL	VIC+$2D	Colore dello sprite #6
SP7COL	VIC+$2E	Colore dello sprite #7
XSCAN	VIC+$2F	Registro di scansione dei tasti supplementari
CLKRATE	VIC+$30	Registro di controllo della frequenza di clock

Codici colore del chip VIC-II

Etichetta	Valore	Descrizione
VIC_BLACK	$00	Codice VIC del colore nero
VIC_WHITE	$01	Codice VIC del colore bianco
VIC_RED	$02	Codice VIC del colore rosso
VIC_CYAN	$03	Codice VIC del colore ciano
VIC_PURPLE	$04	Codice VIC del colore viola
VIC_GREEN	$05	Codice VIC del colore verde
VIC_BLUE	$06	Codice VIC del colore blu
VIC_YELLOW	$07	Codice VIC del colore giallo
VIC_ORANGE	$08	Codice VIC del colore arancione
VIC_BROWN	$09	Codice VIC del colore marrone
VIC_LIGHT_RED	$0A	Codice VIC del colore rosso chiaro
VIC_DARK_GREY	$0B	Codice VIC del colore grigio scuro
VIC_MEDIUM_GREY	$0C	Codice VIC del colore grigio medio
VIC_LIGHT_GREEN	$0D	Codice VIC del colore verde chiaro
VIC_LIGHT_BLUE	$0E	Codice VIC del colore azzurro
VIC_LIGHT_GREY	$0F	Codice VIC del colore grigio chiaro

Codici colore primario dei caratteri in modalità multicolore

Etichetta	Valore	Descrizione
VIC_MC_BLACK	$00 | $08	Colore primario nero
VIC_MC_WHITE	$01 | $08	Colore primario bianco
VIC_MC_RED	$02 | $08	Colore primario rosso
VIC_MC_CYAN	$03 | $08	Colore primario ciano
VIC_MC_PURPLE	$04 | $08	Colore primario viola
VIC_MC_GREEN	$05 | $08	Colore primario verde
VIC_MC_BLUE	$06 | $08	Colore primario blu
VIC_MC_YELLOW	$07 | $08	Colore primario giallo
VIC_TRANSPARENT	$FF	Valore fittizio usato per indicare che il registro colore non va alterato

Codici colore per la selezione del colore di sfondo nella stampa in modo ECM

Etichetta	Valore	Descrizione
VIC_BG0	$00	Colore di sfondo #0
VIC_BG1	$01	Colore di sfondo #1
VIC_BG2	$02	Colore di sfondo #2
VIC_BG3	$03	Colore di sfondo #3

Codici PETSCII per la selezione del colore di sfondo nella stampa in modo ECM

Etichetta	Valore	Descrizione
ECM_BG0	$01	Colore di sfondo #0
ECM_BG1	$02	Colore di sfondo #1
ECM_BG2	$03	Colore di sfondo #2
ECM_BG3	$04	Colore di sfondo #3